科爱传播
KE'AI COMMUNICATIONS

21世纪信息安全大系

Windows取证分析

（附DVD工具包）

【美】Harlan Carvey 著

王智慧 崔孝晨 陆道宏 译

Windows Forensic Analysis

科学出版社

(TP-4073.0101)

SYNGRESS®

ELSEVIER
原版引进

Windows取证分析（附DVD工具包）

Windows Forensic Analysis

本书的写作源于实战的需要，主要关注Windows取证分析这一技术领域，主要讨论了Windows系统开机和关机的不同时刻对证据数据收集和分析的技术问题，重点阐述了Windows的内存分析、注册表分析、文件分析、可执行文件分析，以及Rootkits等内容。本书不仅为取证分析人员、调查人员和应急响应人员提供参考，也可为政府和公司的调查人员、司法官员及对Windows取证分析感兴趣的读者提供参考和帮助。

本书特色

- 结合实践经验，详解Windows取证分析的技术问题。
- 提供大量利用Perl脚本编写的程序，可以在多种操作系统运行。
- 随书附送DVD光盘，包含很多常用工具、脚本文件，以及视频等有价值的信息。

电话：010-64006871
网址：www.kbooks.cn

ISBN 978-7-03-023308-0

9 787030 233080 >

定价：45.00 元
（含DVD光盘）

销售分类建议：计算机／信息安全
本版本只限于在中华人民共和国境内销售

Windows Forensic Analysis
Windows 取证分析

〔美〕 Harlan Carvey 著

王智慧 崔孝晨 陆道宏 译

科学出版社

北京

图字：01-2008-2323 号

This is a translated version of
Windows Forensic Analysis
Harlan Carvey
Copyright © 2007 Elsevier Inc.
ISBN 10：1-59749-156-X
ISBN 13：978-1-59749-156-3

All rights reserved.

No part of this publication may be reproduced or transmitted in any form or by any means，electronic or mechanical，including photocopy，recording，or any information storage and retrieval system，without permission in writing from the publisher.

AUTHORIZED EDITION FOR SALE IN P. R. CHINA ONLY
本版本只限于在中华人民共和国境内销售

图书在版编目(CIP)数据

Windows 取证分析/（美）Harlan Carvey 著；王智慧，崔孝晨，陆道宏译. —北京：科学出版社，2009
ISBN 978-7-03-023308-0

Ⅰ. W… Ⅱ. ①卡…②王…③崔…④陆… Ⅲ. 窗口软件，Windows-安全技术 Ⅳ. TP316. 7

中国版本图书馆 CIP 数据核字（2008）第 169820 号

责任编辑：田慎鹏 霍志国/责任校对：张怡君
责任印制：钱玉芬/封面设计：耕者设计工作室

科学出版社 出版
北京东黄城根北街 16 号
邮政编码：100717
http://www.sciencep.com

新蕾印刷厂 印刷
科学出版社发行 各地新华书店经销

*

2009 年 1 月第 一 版 开本：787×1092 1/16
2009 年 1 月第一次印刷 印张：15
印数：1—4 000 字数：320 000

定价：45. 00元（含DVD光盘）
（如有印装质量问题，我社负责调换）

作者致谢

首先要感谢上帝赐给我生命中的一切，对于这一切我将永远感激。

感谢我生命中的爱和光——Terri，以及她美丽的女儿 Kylie。她们的耐心和理解是对我写作这第二本书的极大支持。我知道在我进入思考、字斟句酌的时候冷落了她们。和我这样的呆子生活在一起对这两个美丽的女人来说并不容易。

同时要感谢很多人对本书的贡献，无论大小，对本书的形成都是必不可少的。Jesse Kornblum 需要特别感谢，他不仅是本书的技术编辑，还是一个很好的朋友，并且他提供了很多建议，落实到书中。也要感谢 Jesse 在计算机取证领域的贡献，他的作品包括 FRED 磁盘工具、哈希工具等，还有很多计算机取证领域的论文。感谢 Cory Altheide，是他提供了跟踪 USB 设备在 Windows 系统中遗留的痕迹的想法。感谢 Andreas Schuster 在计算机取证领域当前和未来的贡献，包括 Windows 内存分析领域。其他为本书带来帮助的人包括 Aaron Walters，FATKit 工具的作者之一；通用动力（General Dynamics）高级信息系统的 Bill Harback；GSG Systems 公司的总裁 George M. Garner；密尔沃基警察局 Richard F. Mcquown 侦探；格温特郡警察局高科技犯罪调查科 Jon Evans 侦探；以及计算机取证分析专家 Don Lewis。

作者简介

Harlan Carvey（CISSP），同时也是《Windows 取证和事件恢复》（*Windows Forensics and Incident Recovery*）一书的作者。Harlan Carvey 是硅谷北部和大都会地区的计算机取证与应急响应顾问，现在他为全美所有地区的客户提供紧急事件响应和计算机取证服务。Harlan 的专业领域集中在 Windows 2000 及其后续平台的的应急响应、注册表和内存分析、以及事后的计算机取证分析。Harlan 曾作为专职的安全工程师提供漏洞评估和渗透测试服务。Harlan 也为联邦政府部门提供应急响应和计算机取证服务。

Harlan 曾获得弗吉利亚军事学院（Virginia Military Institute）电子工程学士学位和拉瓦尔研究生学院（Naval Postgraduate School）电子工程硕士学位

Harlan 在此对他的妻子——Terri，在本书写作过程中的支持、耐心和幽默表示感谢。

技术编辑

Jesse D. Kornblum 是 ManTech SMA 首席计算机取证工程师。现在为情报部门人员开发新的计算机取证工具和技术。Jesse 居住于华盛顿特区，是数个领域研究的开拓者，包括自动应急响应、模糊哈希和 Windows 内存分析。而且，他独立开发了几个广为使用的计算机取证工具，例如 md5deep 和 foremost。Jesse 现在是《数字调查》期刊的编委，并且为 ForensicsWiki 项目贡献了很多内容。Jesse 曾作为特别调查员为美国空军提供计算机犯罪调查服务。

Dave Kleiman（CAS，CCE，CIFI，CISM，CISSP，ISSAP，ISSMP，MCSE），从 1990 年开始就在信息安全领域工作。现在，他创建了 SecurityBreachResponse. com 网站，也是 Securit-e-Doc 公司的首席信息安全官。在此之前，他是 Intelliswitch 公司的技术运营副总裁，对国际通信和因特网进行监控。Dave 是一个安全专家，作为前佛罗里达州认证的司法官员，他擅长计算机取证调查、应急响应、入侵分析、安全审计和安全网络架构。Dave 为网络专家写过几本 Windows 方面的安全安装和配置指南，他开发的 Windows 操作系统防护工具——S-Lok（www. s-doc. com/products/slok. asp），比 NSA、NIST 和微软的通用安全指南更加好用。

Dave 参与了《微软 Log Parser 工具包》（*Microsoft Log Parser Toolkit*. Syngress Publishing，ISBN：1-932266-52-6）一书的写作。他经常参加国家安全会议的演讲，同时为很多安全相关的时事通讯、网站和网络论坛写稿。Dave 参加了数个安全相关的组织，包括国际反恐和安全专家协会（IACSP）、国际计算机取证分析员组织（ISFCE）、信息系统审计和控制协会（ISACA）、高科技犯罪调查协会（HTCIA）、网络和系统专家协会（NaSPA）、认证的欺诈检查人员协会（ACFE）、反恐委员会（ATAB）和 ASIS 国际。他也是 FBI InfraGard 高级会员、区域主席和国际信息系统取证协会（IIS-FA）的教育主管。

技术评审

Troy Larson 是微软网络安全小组的高级取证工程师，他的工作就是分析微软最新的技术并保持取证实践技术的先进性。Troy 是针对 Windows 和 Office 取证问题的讲师，当前专注于开发 Vista 和 Office2007 的取证技术。在加入微软取证小组之前，Troy 工作于 Ernst & Young 国际取证实践和事务公司。Troy 是华盛顿州 Bar 组织会员，毕业于加州大学伯克利分校并获得法律学士学位。

译者简介

王智慧 副研究员，1995 年毕业于中国科技大学化学物理系，2006 年毕业于清华大学电子工程系，获硕士学位。现就职于北京华夏物证鉴定中心，从事声像资料及电子物证鉴定，是中国合格评定国家认可委员会（CNAS）批准的授权签字人。

崔孝晨 2000 年获华东政法学院法学学士学位，2006 年毕业于复旦大学计算机科学与技术系（第二本科）。现就职于上海市公安局信安处，从事计算机取证和恶意软件分析工作。

陆道宏 1995 年毕业于华东理工大学计算机与科学系，获硕士学位，长期从事信息安全与计算机取证研究工作。2004 年合伙创建上海盘石数码信息技术有限公司，领导开发了盘石计算机取证系统（SafeImager）、盘石介质取证分析系统（SafeAnalyzer）、盘石手机取证分析系统（SafeMobile）等系列计算机取证专业工具。

序

自从1991年在美国召开的国际计算机专家会议上首次提出“计算机取证（Computer Forensic)”术语以来，随着互联网的普及和计算机犯罪案件的增多，计算机取证一直是信息安全领域中的热门话题，国际上几乎每年都要召开以计算机取证为主题的学术会议，用于计算机取证的技术工具逐渐发展成为信息安全产业的一个特殊种类，甚至很快也有了诸如“国际计算机证据组织（IOCE)”之类的“非政府”组织。自然，近年有关计算机取证的专业书籍也出版不少，其中涉及基础理论、取证技术和取证工具的专著先后被译介到国内，这些工作对国内计算机取证的研究和实践都起到了十分重要的推动作用。

因而，当王智慧先生将他与几位朋友的译著《Windows取证分析》的清样送来之后，我是怀着几分欣喜，利用两个周末的余暇认真奉读的。所以如此，一是因为智慧本人一直致力于，也乐于、长于计算机取证工作，在理论功底和实践经验上颇有见地并屡建奇功，能入他法眼的书籍自有不凡之处；二是因为对Windows的取证是现实生活中最常见，也最复杂的一项工作。据智慧介绍，原书作者Harlan Carvey曾在美国军方长期从事信息安全工作，有过十多年的取证工作经验，并且有过《Windows取证和事件恢复》一书面世，颇受业内人士推崇。有着这样的专业背景，加上今年岁末难得的晴天暖阳，我的两个周末是在愉快地阅读中度过的。

掩卷之时，正值满月初升。回味本书，觉得有三点特别的感受不得不说：一是知识内容上的创新。初看书名，很容易会先入为主地把它看作作者旧作《Windows取证和事件恢复》的翻版。其实不然，正如作者在前言中声明的，他不想将此书写成旧作的续集，细读本书中对开机取证、注册表分析的深入解析和在文件分析、内存分析、Rootkits分析方面的独到体会，你会相信作者说到做到了。二是技术工具上的创意。本书介绍了不少现实可用的取证工具，这些工具能帮助读者更好地理解和体会作者在书中提出和阐述的概念，而且这些工具并非对Encase之类的商业产品的简单罗列，而是精选了不少当下热门的“活的”网上新宠，尤其可喜的是，有的工具就是作者本人的杰作，用自己的看家本领说事，应该算是有创意。三是实践经验上的创见。由于作者在此领域有十多年的工作经验，本书中列举的实例很多是自身实践的积累，不少实例是在其他同类书籍中找不到的。例如结合英国布莱尔政府Word格式文档的信息泄露案例，作者介绍的与“对象链接嵌入技术（OLE）流”相关的信息痕迹与恢复经验，以及反复强调的可重复性和取证数据的自动化处理等体会，也都是其他著述鲜有涉猎的。

当然，如果要说本书不够过瘾的地方，可能要算第6章“可执行文件分析”部分，这本是当前恶意软件（间谍软件）作恶的重要环节。由于涉及逆向工程技术，作者的意见是可以就此另写一本书，因而分析不深。另外，从理论上讲，本书对分布式取证、自动化取证的后处理、数据挖掘和包括交换文件、休眠文件等在内的内存分析等热点问题尚可再深入一些。考虑到本书专为实践需要而写，面面俱到反倒有些苛求和不妥。

有过几次译书的经历，我知道翻译专业书籍的艰辛，尤其是信息安全这样的全新领域，技术发展快，概念创新多，要把一些特定的概念术语和技术细节译好，绝非易事。值得高兴的是，几位年轻朋友在本书的翻译中下了不少功夫，如将 live response 转意为“开机取证”，就没有拘泥于原文，但却传达了原意，用心良苦，可见一斑。

我把奉读清样后的感受写下来，算是对同行朋友们开拓视野、勇于冒险、敢于干译书这种“费力不讨好”的公益事业的一种支持和鼓励。一孔之见，权充为序。

吴世忠

二 OO 八年十二月十三日

于昆明湖畔

前　言

本书的写作目的来自实战需要。很多计算机取证专家在调查中过度依赖取证工具得出的结果，并不了解信息来自哪里或是怎样创建和推导出来的。“任天堂式取证”（加载获取的镜像，按一个键就能生成结果）的时代已经过去了。作为分析和检验人员，不能再希望取证调查是这种方式。网络犯罪手段和形式越来越复杂，调查人员需要理解系统中存在哪些证据要素，这些要素是怎样产生和修改的。只有掌握了这些知识，我们才能理解一些要素确实是证据所在，而且现在有很多关于反取证的演讲和论文资料，在一些主流的会议上都有关于针对取证分析专家培训和工具的反取证技术议题，一些技术正使得取证更加困难。本书旨在加深对取证信息的深入理解，不仅列举了调查人员从开机系统及获取的镜像中能够得到的信息，也提供了定位更多取证信息要素的方法。

本书的主要意图在于对取证社群的反馈，从这个社群中我得到了很多。我在信息安全领域从业超过 10 年（之前是在军方从事物理和通信安全方面的工作），期间遇到过很多厉害的人，也做过很多有趣的事，得到过很多人的帮助，这使我逐渐深入到安全和取证研究的领域。我的一些研究成果在不同的会议上发表，这又带来更多的问题和讨论，推动研究的进一步发展。知识的不断交流和讨论带动知识的发展，从而提高了整个研究领域的水平。

本书主要讨论 Windows 系统开机和关机的不同时刻对证据数据收集和分析的技术问题，但是不可能涉及每个方面，还有很多方面值得研究和深入。如果本书能够带动读者对 Windows 系统更加深入的调查和分析，就达到目的了。

目标读者

本书关注 Windows 取证分析这一技术领域，该领域相对来说比较狭窄，推荐正在从事 Windows 取证分析工作或者对这方面感兴趣的人阅读本书。本书可以作为一个有用的参考书，一些读者可能会觉得本书技术太过深奥、超过了个人的理解力，但我的感觉是取证需要逐步积累，可以将本书中的内容作为探寻和研究的起点。我开始写本书的时候，就没有想写成我的第一本书《Windows 取证和事件恢复》的续，而是要在 Windows 取证领域提供一些更加基础、更加全面的参考资源，不仅为了我自己，也为其他在计算机取证领域工作的人们。

本书写作的过程中，我的目标是为取证分析人员、调查人员和应急响应人员提供一个参考资源。取证人员就不说了，对于面临应急响应压力的系统管理员来说，就是要消除他们的疑问：“我应该怎么做?”。在这个前提下，本书逐步消除一些当前应急响应的误解，例如，擦除硬盘、重装系统等。有时给系统打补丁也不能解决问题，可能导致再次入侵，所以必须找到问题的根源。

本书可以为公司和政府的调查人员、司法官员和顾问提供参考和帮助，同时也可以

为在学校开发或参加计算机取证学位学习的研究人员提供帮助。

贯穿全书，调查人员、第一响应人员、分析员和管理员这几种称呼是可以交换使用的。因为在不同的场景下一个人可能变换不同的角色。一些情况下，调查人员会与公司的管理员一起工作，使用域管理员账户来收集数据。有时管理员陪同调查人员或应急响应人员使用管理员级别的用户账户来调查被入侵的系统。不要被这些名词混淆了，大部分情况下都是同样的意思。

通读本书时注意几件事请。首先，有很多 Perl 脚本编写的程序。选择 Perl 脚本并非一些神奇的原因：只是 Perl 脚本非常灵活而强大。我喜欢修改代码直接运行而不需要重新编译。说到编译，如果你从来没有用过 Perl 或是对 Perl 不熟悉，也不用担心。除了极少数例外，书中的大部分 Perl 脚本都同时提供了使用 Perl2Exe 程序编译的、可以在 Windows 系统中独立运行的可执行程序版本。这些执行程序可以在不安装 Perl 的情况下直接运行（书中使用的 Perl 版本可以从 ActiveState. com 或其他地方免费下载）。Perl 的另外一个特色是可以跨平台，随书附赠的光盘中提供的从二进制文件中提取数据的 Perl 脚本基本上都可以跨平台运行。光盘中编译好的独立执行程序只能在 Windows 平台运行，但是 Perl 脚本可以在 Windows，Linux，甚至 Mac OS X 平台运行。很多光盘中的脚本都在 Linux 的 Perl 环境中测试并成功运行。因而，分析人员并不局限于使用某一特定平台。一些脚本需要安装附加的模块，可以通过 Perl 包管理程序（PPM）来安装。PPM 是 ActiveState 发行的 Perl 程序的一部分，可以用于 Windows，Linux，Mac OS X 和一些其他平台。另一个使用 Perl 的实用效果是自动化。我发现，数据提取、二进制分析等很多工作会不断地重复，手工提取总是会犯一些错误。如果提取和分析任务可以使用 Perl 自动化处理，就可以写好代码，这样在做同样任务的时候就不会发生错误。如果流程已经就绪，对流程的调整还是很简单的——我发现只有在不知道该做什么的时候，调整才是特别困难的。

本书使用的取证分析程序是来自 Technology Pathways 的 ProDiscover 应急响应版。感谢 Chris Brown 的慷慨，我从版本 3.0 的时候就开始用 ProDsicover，而且极其简单易用。ProDiscover 分析 Windows 系统的镜像非常好，有很多易用而强大的功能。Chris 和 Alex Augustin 对问题的反馈和更新也很及时，与 Ted Augusting 在会议上交谈时，发现他在取证领域具有深厚的知识背景。ProDiscover 本身是个很好的分析平台，应急响应版本则在现场响应方面简单实用，可以方便地收集易失数据。同样，以我的观点来看，Chris 选择 Perl 作为 ProDiscover 的脚本非常明智。调查员可以使用 Perl 脚本（ProScripts）扩展对镜像的分析功能（例如，搜索、数据提取和细微的数据分析）。随书附赠的光盘中包含我写的几个常用的分析脚本（注意，ProScripts 是 Perl 脚本，而不是使用 Perl2Exe 编译的程序，在和 ProDiscover 一起使用的时候必须使用脚本的方式）。

本书内容组织

本书共分为 7 章。

第 1 章　开机取证：数据收集

本章解决从开机系统中收集易变数据的基本问题。由于网络犯罪的复杂化、存储容量的不断增长等原因，开机取证得到了更多的关注。开机取证不仅吸引了取证顾问（例

如，我）的注意，而且政府部门的专业人员也开始在调查过程中采用该技术从开机系统中收集易变数据。本章列举了收集易变数据的技术和工具，同时介绍了取证服务器项目（Forensic Server Project）。

第 2 章　开机取证：数据分析

将数据获取和数据分析分开是因为这是两个问题。很多情况下，需要获取的数据并不改变，例如，在特定时间得到系统活动的快照。这时候，数据的解释就对案件很重要了。同样，有时候我们到达现场，发现初始事件报告只是对系统活动特征的描述，而不触及问题根本所在。在开机取证过程中，怎样对收集的数据进行分析、如何找到想要的信息，常常依赖于案件性质，欺诈、入侵、恶意软件的案件各不相同。本章展现了一个对开机取证收集到的数据进行相关分析的框架，从而使得对系统行为的刻画和问题根本原因的识别变得更加简单易懂。

第 3 章　Windows 内存分析

自从 2005 年夏天 Windows 内存分析技术被正式介绍以来，对这一课题的研究已真正启动。过去，在开机系统中获取的物理内存数据只用来搜索字符串（例如，密码、IP 地址和电子邮件地址），然后数据就进入存档了。不幸的是，以这种方式发现的信息很少提供上下文环境。感谢 DFRWS 2005 内存分析挑战带来的研究成果，获取的内存可以使用更细的粒度被识别和调查，甚至可以提取执行文件的镜像。本章讨论获取和分析内存镜像的问题，同时也包括从开机系统中提取特定进程的内存数据。

第 4 章　注册表分析

Windows 注册表维护了大量（可以说过多的）关于系统状态的信息。很多情况下，注册表可以视为日志文件，因为注册表也以特定方式记录了时间戳。不过，由于数据记录存储方面的原因，使用 ASCII 或者 unicode 字符串搜索并不能得到很有价值的信息。本章将揭秘注册表结构，使读者能从二进制存储及删除空间中找到注册表数据单元。随后将用很大篇幅讨论注册表中不同的数据项（注册表中键/值），描述这些键值对调查的价值，同时讨论了用来从获取的镜像中提取信息的很多工具。

第 5 章　文件分析

很多调查人员忽略了 Windows 系统维护的一些列的日志文件，这些日志文件通常对记录的项目都有时间戳信息，而且一些文件中的时间戳信息可以作为对事件进行时间分析的一部分。很多时间戳是由应用程序维护，并不立即呈现。本章详细讨论各种不同的文件、文件格式和文件的元数据，并且包括提取信息的工具。

第 6 章　可执行文件分析

可执行文件是文件分析中一个特例。因为需要在不同版本的 Windows 系统中运行，可执行文件一般具有已知、一致的文档结构。不过，恶意软件的作者找到了混淆文件结构的方法，使得恶意软件更加难以（虽然不是不可能）分析。通过理解这些文件的格式及常规文件的结构，分析人员可以在调查中识别哪些文件是合法的、可疑文件对 Windows 系统的影响，以及恶意软件具有的因素。

第 7 章　Rootkits 及其检测

最后一章讨论系统级木马（Rootkits）的问题，希望对这一充满神秘色彩的恶意软件的讨论能给管理员、现场响应人员和分析人员（记住这可能是同一个人）提供定位和

识别 Rootkits 的有力武器。Rootkits 的使用在迅速增长，不仅网络犯罪分子在使用，一些“合法”的商业应用程序也在用。对 Rootkits 及其检测技术的理解是在 Windows 系统领域工作的最难点，本章详述对这一领域进行调查的各种技术。

光盘内容

本书附赠的光盘包括很多有价值的信息和工具。所有的工具都按照章节组织到不同的目录中，包括每章的代码和样例程序（第 7 章除外）。另外，有一个附加的目录包括一些不属于任何一章的几个工具，主要是我在实际工作中开发的，希望能有所帮助。

所有光盘上的工具都是 Perl 脚本。不过几乎所有的 Perl 脚本都已“编译”成简单易用的 Windows 独立执行程序。Perl 脚本自身是平台独立的，可以在 Windows，Linux，甚至 Mac OS X 操作系统运行。编译的 Windows 执行程序是为了方便未安装 Perl 的用户使用。有几章还包括 ProScripts，这是专为 Technology Pathways 公司（www. techpathways. com）的 ProDiscover 取证分析工具编写的 Perl 脚本。

另外，有几章目录中包括了样例文件，读者可以通过这些文件熟悉工具的用途。尤其是第 5 章提供了不少文件。使用工具解释其用途是一回事，但是真要使用它来提取信息就是另外一回事了。通过样例文件，读者可以在任何地方（例如，飞机上）通过笔记本使用这些工具，而不需要自己获取文件。

最后，光盘中还包括了几个视频文件。这些视频文件（. wmv）对书中谈到的工具进行演示，涉及的工具也在光盘上。过去，我通过写附录来解释取证服务器项目的安装和使用，但是发现听录音、看视频比看书更加有效。

光盘中提供的所有工具仅供“参考”，作者对其使用不做任何担保。除了 bonus 目录中的工具之外，所有工具在书中都有描述。因而，读者通过本书可以了解工具的用途。所有的 Perl 脚本编译的 exe 工具都提供了源代码脚本，全部基于命令行，双击运行不会产生有效的输出。大部分工具需要文件名（包括全路径）作为命令行参数。如果使用中有问题，可以看 Perl 脚本的源码，或者通过“-h”参数来显示帮助信息。如果还有问题，可以发送电子邮件到 keydet89@yahoo. com，附加简洁、完整的问题描述，我会看问题出在哪里。

由于授权的问题，其他人所写的第三方工具没有包括在光盘中，但在书中提供了工具的链接。

非常感谢！希望你能觉得本书有用并喜欢它。有任何问题，请和我联系。

——Harlan Carvey

目　录

前言

第 1 章　开机取证：数据收集 …… 1

引言 …… 2

开机取证（Live Response） …… 2

诺卡德交换原理 …… 3

易变信息的次序 …… 5

何时进行开机取证 …… 5

收集什么数据 …… 7

系统时间 …… 8

当前登录用户 …… 9

打开的文件 …… 10

网络信息（缓存的 NetBIOS 名字列表） …… 11

网络连接 …… 12

进程信息 …… 13

进程到端口的映射 …… 18

进程内存 …… 20

网络状态 …… 20

剪贴板内容 …… 22

服务/驱动信息 …… 23

命令行历史 …… 24

映射的驱动器 …… 25

共享 …… 25

非易变信息 …… 26

注册表设置 …… 26

事件日志 …… 29

设备和其他信息 …… 29

有关怎样挑选工具 …… 29

开机取证方法 …… 30

本地开机取证方法 …… 31

远程取证方法 …… 32

混合方法 …… 33

小结 …… 36

参考资料 …… 36

快速解决方案 …… 38

常见问题 …… 39
第 2 章 开机取证：数据分析 …… 41
引言 …… 42
数据分析 …… 42
案例一 …… 43
案例二 …… 46
敏捷分析 …… 49
扩大范围 …… 51
应对 …… 52
防范 …… 53
小结 …… 54
参考资料 …… 54
快速解决方案 …… 55
常见问题 …… 55
第 3 章 Windows 内存分析 …… 57
引言 …… 58
内存分析简史 …… 58
获取物理内存镜像 …… 59
基于硬件的方案 …… 59
利用火线接口 …… 60
崩溃转储 …… 60
利用虚拟机 …… 62
休眠文件 …… 63
DD …… 63
分析物理内存镜像 …… 65
进程基础 …… 65
分析内存镜像 …… 67
分析进程内存 …… 72
提取进程可执行文件镜像 …… 73
内存镜像分析和页交换文件 …… 76
根据内存镜像判断操作系统类型 …… 77
分析内存池 …… 78
获取进程内存 …… 79
小结 …… 80
参考资料 …… 80
快速解决方案 …… 81
常见问题 …… 82
第 4 章 注册表分析 …… 85
引言 …… 86

注册表内部结构 …… 86
配置单元文件内的注册表结构 …… 88
注册表作为日志文件 …… 92
监视注册表变化 …… 93
注册表分析 …… 94
系统信息 …… 95
自动启动位置 …… 98
枚举注册表自动启动位置 …… 105
USB 移动存储设备 …… 105
Mounted Devices …… 109
查找用户 …… 111
追踪用户活动 …… 114
Windows XP 系统还原点 …… 122
小结 …… 125
光盘内容 …… 126
参考资料 …… 126
快速解决方案 …… 127
常见问题 …… 128
第 5 章　文件分析 …… 131
引言 …… 132
事件日志 …… 132
理解事件 …… 132
事件日志文件格式 …… 135
事件日志头部 …… 136
事件记录结构 …… 137
Vista 事件日志 …… 140
IIS 日志 …… 141
因特网浏览器历史 …… 143
其他日志文件 …… 144
回收站 …… 151
系统还原点 …… 153
Prefetch 文件 …… 154
快捷方式文件 …… 155
文件元数据 …… 156
Word 文档 …… 158
PDF 文档 …… 162
图像文件 …… 163
文件特征分析 …… 163
NTFS 分支数据流 …… 164

其他分析方法 …… 170
小结 …… 172
参考资料 …… 173
快速解决方案 …… 174
常见问题 …… 175
第 6 章 可执行文件分析 …… 177
引言 …… 178
静态分析 …… 178
记录文件信息 …… 178
分析可执行文件 …… 180
动态分析 …… 196
测试环境 …… 196
一次性系统 …… 197
工具 …… 198
流程 …… 201
小结 …… 203
参考资料 …… 204
快速解决方案 …… 205
常见问题 …… 206
第 7 章 Rootkits 及其检测 …… 207
引言 …… 208
Rootkits …… 208
Rootkit 检测 …… 212
开机检测 …… 212
GMER …… 214
Helios …… 215
MS Strider GhostBuster …… 215
F-Secure BlackLight …… 216
Sophos Anti-Rootkit …… 217
AntiRootkit.com …… 218
后期检测 …… 218
预防 …… 219
小结 …… 220
参考资料 …… 220
快速解决方案 …… 221
常见问题 …… 222

第 1 章
开机取证：数据收集

本章主要内容：

- ■ 开机取证
- ■ 收集什么数据
- ■ 非易变信息
- ■开机取证方法

- √ 小结
- √ 快速解决方案
- √ 常见问题

引言

传统调查过程中，传统的方法是现场调查人员直接关闭计算机，然后取得系统硬盘的按位镜像。这种方法简单实用，但是在当前很多实际的调查过程中却不再适用。调查人员和应急响应人员发现有些案例中面临的问题，不能通过硬盘镜像找到答案。例如，在一些通过即时聊天过程诱骗的儿童失踪案中，如何最好地处理证据就是这样一种情形，这种情形在与一些司法官员的交流中也得到了证实。

该问题并不是仅困扰司法人员。很多案件中，最佳的证据和信息源存在于计算机内存中（网络连接、即时聊天客户端的内容、即时聊天进程的内存数据等），因为一些即时聊天程序客户端并不自动保存聊天记录信息。另外一些案件中，调查人员需要知道系统中是否有木马或者恶意程序的运行，敏感的数据文件是否从该系统中被复制，在系统运行的时候，到底发生了哪些行为。技术支持人员通过 IDS 或者防火墙发现异常/问题流量的时候，经常直接关闭产生这些流量数据的系统，而不管流量数据产生的原因。类似这些情况，调查人员都需要进行开机取证（live response）[①] ——在系统运行的时候收集数据。开机取证措施也会有一些负面影响，本章将讨论这些问题。

开机取证（Live Response）

一些时候，关闭系统并获取硬盘镜像并不可行。在电子商务持续发展的今天，由系统关机引起的业务丢失可能导致每分钟数千美金的损失，并且一些公司在服务品质协议（SLAs）中保证“5 个 9”的在线时间——也就是说，公司保证维护时间外的系统在线运行时间要达到 99.999%。根据系统配置的不同，关机并离线获取硬盘镜像可能需要数小时。

信息高速公路中已经不仅仅是追逐娱乐和无聊的地方。很多系统的犯罪在其中发生，其行为也变得越来越复杂和尖端。有专门潜入计算机系统窃取个人信息（口令、个人文件、税单等信息）的恶意程序，并且有些程序的代码仅仅存在于内存中，从不写盘。这样，当系统关机时，所有的程序证据就消失了。

2006 年 4 月，希捷发布了第一个 750GB 硬盘（更多信息参见网页：www.seagate.com/cda/newsinfo/newsroom/releases/article/0,1121,3153,00.html）。如果一个 RAID 系统由 5～8 块这样的硬盘组成，则镜像操作可能需要 6TB 的存储量。镜像这些硬盘需要花费多少时间呢？一般来说，获取并校验一个 80GB 的硬盘要 4 小时以上。如果我们仅对一个进程的信息感兴趣，需要镜像整个系统吗？

一些案件中，在进入传统的计算机取证调查过程（例如，镜像）之前，需要收集一些系统的在线信息。在线信息的最大特征是易变，也就是说，在关掉电源之后，在线信息就会消失。在线信息通常存在于物理内存——RAM 中，包括进程、网络连接、剪贴板数据等。这些信息描述了调查员到达现场、站在计算机之前时系统的状态。作为调查人员，需要做的就是快速获取并分析这些数据，对事件的特性和范围做出判断。当系统

---

① live respones 字面含义是现场响应，但是这里的现场指正在运行的计算机内的现场，为避免歧义，本书将其一律翻译为开机取证。live 在本书中有时也翻译为在线——译注。

电源关闭、准备镜像时，在线信息就再也没有了。

一些工具和技术为从开机系统中收集易变信息提供了可能性，通过提取系统的在线信息提供了系统状态更好的总体描述。这就是“开机取证”的范畴：访问在线、运行的系统并收集易变（或者非易变）信息。

另外一个可能和开机取证引起混淆的术语就是开机获取（live acquisition）。开机取证收集系统的易变信息；开机获取指获取硬盘数据，也就是在系统运行的时候获取硬盘的镜像。本章首先讨论开机取证的工具、技术和方法，包括开机取证收集什么信息及怎样来收集，第 2 章将讨论怎样分析这些数据。之后，将分析开机获取的方案，对获取镜像的分析是本书后续章节的内容。

在开始讨论开机取证的工具和行为之前，需要明确两个概念：诺卡德交换原理和易变数据变化的次序。这些概念是本章的基础。

诺卡德交换原理

开机取证时，调查人员和现场响应人员必须牢记一个重要原则：无论是作为用户还是调查员，当与在线系统交互时，都会对系统造成改变。时间的流逝、进程运行、数据的保存或删除、网络连接或终止等活动会造成系统的改变。即使没有任何交互，运行的系统也处于不断变化中。当调查人员在系统中运行指令收集易变数据或其他信息时，当然也会造成系统的改变。运行程序会导致数据被加载到物理内存中，此时其他运行程序使用的内存数据就可能被写到交换文件中。调查人员使用网络来发送收集到的信息，则会创建新的网络连接。所有这些改变都可以被收集，并且可以被诺卡德交换原理所解释。

20 世纪初，爱德蒙·诺卡德博士在取证科学和犯罪现场重构领域的工作经验形成了诺卡德交换原理。本质上来说，诺卡德原理认为当两个对象接触时，物质会在这两个对象之间产生交换或者传送。如果看过“犯罪现场调查”电视节目，就会发现犯罪现场调查人员经常会说起可能的“传送”。在汽车撞击的时候会发生“传送”，同时尸体上也会发现携带一些明显不属于现场、来自于“传送”的物质。

这个简单的原理也适用于数字领域。例如，两台计算机通过网络通信时，信息会在两者之间交换，一台计算机中的信息会出现在另外一台计算机的内存、日志中（参见下面“工具和陷阱”中的例子：诺卡德原理和 netcat）。一个可移动的存储设备连接到 Windows 计算机时，设备信息会保存到这台计算机上。调查人员和运行的系统进行交互时，运行程序并复制数据会导致系统的改变。这些改变可能是临时的（进程内存、网络连接），也可能是永久的（日志文件、注册表项）。

工具和陷阱

诺卡德原理和 netcat

一个简单的工具——netcat，可以用来演示诺卡德原理[1]。对于不熟悉 netcat 工具的读者，netcat 工具就是一个可以通过网络连接读取/写入数据的方便易用的工具。

本例中需要 3 个工具：netcat（nc.exe），pmdump.exe[2]和 strings.exe[3]（也可以使用 BinText）。可以在一个系统中演示这个例子，当然最好是使用两个系统。如果是使用一个系统，则创建两个目录，每个目录中保存一个 netcat 程序。使用如下的命令行指令启动 netcat 程序，进入侦听模式：

```
C:\test>nc -L -d -p 8080 -e cmd.exe
```

这个命令告诉 netcat 使用附加模式侦听端口 8080，当连接成功时，启动命令行程序。输入命令并按 Enter 键后，打开任务管理器（Task Manager），可以看到所创建的进程 ID（PID）。此时，打开同一系统的另外一个命令行窗口或者在另外一个系统中开启命令行窗口。输入如下命令连接到前一命令所创建的 netcat 侦听进程：

```
C:\test2>nc <IPaddress> 8080
```

这个命令告诉 netcat 使用客户模式连接到该 IP 地址的端口 8080，也就是侦听程序所在的地址和端口。如果试验中仅使用了一个系统，使用 127.0.0.1 作为 IP 地址。连接成功后会出现常见的命令行提示符，显示操作系统的版本和授权信息。在该提示符下输入几个简单的指令，如 dir 或者其他命令，使得信息通过这个连接进行传送。在 netcat 侦听进程所在的计算机上打开另外一个命令行窗口，使用 pmdump.exe（后续章节还会有介绍）命令来获取侦听进程的内存数据：

```
C:\test>pmdump <PID> netcat1.log
```

这个指令会获取 PID 进程的内存数据（包括进程的物理内存和交换文件中的数据）并保存到 netcat1.log 文件中。如果感兴趣，也可以保存客户端进程的内存数据。现在进程的内存数据保存到文件中了，可以退出两边的进程。使用 strings.exe 工具从侦听进程的内存文件中提取字符串，或者使用 BinText 程序打开文件，文件中可以看到客户端程序所在的 IP 地址。对客户端程序的进程内存进行分析可以显示侦听程序所在的系统的信息。这个例子很好地演示了数字世界中的诺卡德原理。

用来收集信息的程序也可能对运行系统产生另外的影响。例如，程序可能会读取一些注册表键，这些注册表键的地址会读取到内存中。Windows XP 系统使用预读（prefetching）技术提高应用程序的启动效率，如果调查人员运行用户使用过的程序，该应用的 prefetch 文件的修改和最后访问时间（也包括文件内容）都会发生变化。如果调查人员使用的程序未曾运行过，Prefetch 目录中会创建一个新的 prefetch 文件（假设系统未超过 128 个“.pf”文件的上限，关于这点，本书的后续章节会详细论述）。

调查人员必须了解这些改变，而且必须记录这些动作的改变才能解释它们对系统的影响。例如，调查人员必须能够确定 Windows Prefetch 目录中哪些“.pf”文件是调查过程中产生或改变的，哪些是用户行为的结果。这点对注册表键值同样适用，与 Windows XP 的应用程序预读能力类似，调查活动对系统注册表键也会产生影响。一些注册表键会创建，同时一些注册表键的最后写入（LastWrite）时间会改变。一些注册表键的修改并非调查工具直接产生的，而是来自于 shell 程序（例如，Windows Explorer），这是因为系统一直处于运行状态。

通过测试和对调查工具的深入了解，可以使我们明确记录并解释系统中发现的内容哪些是调查行为产生的，哪些来自于用户行为或者攻击行为。

易变信息的次序

易变信息存在于运行系统的内存中，其中特定类型的易变信息变化更加频繁。也就是说，运行系统中的一些易变信息比其他信息的生命周期更加短暂。例如，网络连接过期不用时会存在数分钟。浏览特定站点或者构建一些网络连接，使用 netstat. exe 命令观察连接情况。然后关闭使用的客户应用程序，网络连接情况会随时间变化，然后逐渐从 netstat. exe 的输出中消失。系统时间随时变化，剪贴板中的数据在内容改变或者关机之前保持不变。系统服务（UNIX 中的系统进程）类的特殊进程会长时间的运行。反之一些进程的生命周期则短得多，完成任务后就从内存中消失了。这表明需要首先收集变化频繁的信息，更持久的易变信息可以晚一点获取。

RFC 3227（证据收集和存档指南，www. faqs. org/frcs/rfc3227）是这方面很好的指南。RFC 3227 发布于 2002 年 2 月，今天仍然保持其实用性，因为一些核心指南并不随技术的改变而发生变化。RFC 3227 定义了准确收集系统信息的原则；记录详细的日志信息；记录 UTC、本地时间和系统时间的差异；最小改变数据的可能性。在讨论开机取证的时候，必须牢记这些原则。

提 示

RFC 3227 指出必须记录系统时间和 UTC 时间的差异，同时留下详细的案件日志，以便在数年后还能解释并证实当时的行为。

RFC 3227 的 2.1 节为“易变信息的次序”，很有意思。该节按照从快到慢变化的次序列出了多种易变信息。那些具有更快易变特性的数据，例如，网络连接的状态，要首先收集。具有较慢易变特性的信息，例如，系统的物理配置等，可以随后收集。通过这个指南，可以得到详细的指导：收集系统的何种信息，从哪里获取信息，使用什么工具，怎样收集信息会对“目标”系统的影响最小。

何时进行开机取证

对调查人员和现场响应人员来说，也许“何时需要开机取证”这个问题更加重要。今天的大部分案例（例如，刑事和民事案件、公司内部的调查），并没有明确定义何时进行开机取证。事实上，大部分情况下开机取证及易变信息并不重要。是否进行开机取证依实际情形而定，环境（调查人员的意图、公司策略、法律要素）和问题的本质都会对这个问题产生影响。

我们来看几个实际案例。一位系统管理员报告说存在一些异常网络流量，管理员收到了 IDS 的报警信息，并且从防火墙日志中也发现了可疑的和报警相关的日志。她说内部网的一个特定系统似乎在发送异常的网络数据流，已经获取了 IDS 报警日志和网络日志，这时需要系统主管来决定是否需要获取详细的网络数据流。在获取网络数据流

信息之后，怎样与特定的系统/主机联系起来？这很简单，不是么？毕竟系统的IP地址是确定的（获取的流量数据中有源和目标的IP地址），如果获取了以太网数据包，还有MAC地址。但是怎样将网络流量和运行系统中的一个特定用户或者进程联系起来呢？

如果需要明确指出可疑流量的来源（哪个进程），就必须在关机之前收集运行进程和网络连接的信息。其他开机取证中收集的信息可以显示哪些用户远程登录到系统中，通过网络登录还是后门程序，或者仅仅是一个系统调度任务创建的进程。

还有其他情形需要或者必须进行开机取证吗？“木马辩护”怎么样？在这样的案件中，恶意行为被辩护方归结于木马或者后门。2002年10月，案件中发现朱利安·格林的系统有非法的图片信息（报告中说超过了170张），取证分析人员发现他的系统中有数个木马，这些木马在浏览器打开的时候会访问非法网站，最终格林被证明无罪。

第二年，安荣·卡菲里声称别人控制了他的计算机并发起了对其他系统的入侵，而不是他自己。卡菲里的辩护认为虽然在取证分析的过程中并未发现木马，但木马还是可能存在的。这样的辩护使他逃过了责罚。

这样的案例使我们明白了在获取系统时同时获取系统进程和网络连接信息是何等重要，尤其是调查人员到达现场时恶意程序可能正在运行。这些信息可以说明，是否系统存在不寻常的进程，是否有他人在控制系统并上传数据或者对系统发动攻击。

如前所述，另外一个开机取证的原因就是在一些案件中系统不能被关机。大型的关键应用主机，例如，电子商务系统中使用的主机，每分钟的离线时间流失的订单可能导致成百上千元的损失。获取硬盘镜像的过程（这类系统总是使用多个硬盘，并且做了RAID）需要很长的时间，这些系统的离线必须有足够的理由。这样做不是仅仅需要系统管理员向IT经理来证实，而是需要CEO向董事会来证实了。

另一个需要考虑的因素是法律要求的告知。这起始于加利福尼亚州SB 1386法案，由于被安全入侵导致敏感信息丢失的公司必须告知客户，这样客户才能采取必要的措施，避免特定数据被盗可能导致的损失[4]。本书写作的时候，其他州开始跟随加利福尼亚州的做法，甚至有形成联邦告知法的趋势。这意味着保存和处理用户敏感信息的公司在处理入侵事件的时候不能保持沉默，这些公司必须明确知晓安全入侵是否导致敏感数据被盗。毕竟，通知用户他们的信息被其他不明人物知晓（可能是多方知晓）会对公司业务造成严重的影响。客户可能会终止服务并告知家人和朋友。新的客户会和竞争对手签约。当前和未来收入减少会改变公司的形象甚至可能导致破产。所以公司不会在怀疑的情况下告知客户的，会在确定客户信息丢失的情况下才会最终告知，不是吗？

例如，一个“匿名”黑客声称通过互联网入侵了某组织，并且得到了用户姓名、社会安全号、信用卡数据及其他很多信息。该组织的高级管理人员需要确切知道事实是否如此，黑客是如何得到这些数据的。这样，调查人员就需要进行开机取证，收集系统的易变信息，如运行的进程、网络连接等。开机取证还可以调查仅在内存中存在而不在硬盘上写入任何执行代码的恶意软件。

收集什么数据

本节介绍运行的系统中有哪些易变数据，开机取证使用怎样的工具获取信息。

开机取证时，物理内存（RAM）中的数据可能是需要最先获取的。根据诺卡德交换原理，最先获取内存数据，可以使得我们对内存的影响最小。之后，获取其他易变数据的工具会不断加载到内存中（获取内存的工具也是如此），改变内存数据的内容。然而，对内存数据的获取和分析还是一个很新的研究领域（开始于 2005 年夏天），值得用一章来讨论（见第 3 章）。

本章需要讨论的易变数据类型如下：

- 系统时间
- 当前登录用户
- 打开的文件
- 网络信息
- 网络连接
- 进程信息
- 进程到端口的映射
- 进程内存
- 网络状态
- 剪贴板内容
- 服务/驱动信息
- 命令行历史
- 映射的驱动器
- 共享

对每种易变数据类型，将讨论在 Windows 下使用哪些工具来获取该数据，而且本章讨论的工具以命令行形式为主。

开机取证过程中，命令行工具（CLI）比图形界面（GUI）的调查工具有特殊的优势。比较容易想到的优势包括命令行工具占用更小的内存，对内存的影响会比较小。命令行工具依赖的动态链接库（DLLs）更少，对系统的影响也比较少。但是，每个工具都是不同的，到底会产生哪些影响，必须对使用的工具进行深入地测试。

警　告

不能仅对工具运行的“内存足迹”做出假设，没有彻底地分析和测试（参见第 6 章），不会知道一个执行工具会在系统中留下哪些痕迹。

使用命令行工具的主要原因在于命令行工具往往比较简洁，注重一个简单、特定功能的完成，因而比较容易通过批处理或脚本工具进行自动化。命令行工具可以通过批处

理和脚本语言结合在一起，运行结果大部分通过控制台（或标准输出，STDOUT）输出，可以重定向到文件和网络套接字。GUI 工具的输出结果基本上保存为文件，通过保存、另存为等文件菜单项操作，很难自动化。

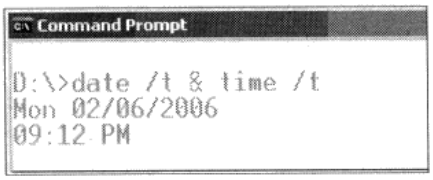


图 1.1　显示 Windows XP 的系统日期和时间

大部分 GUI 工具的开发人员并不需要考虑应急响应或取证的需求。开机取证要求对系统的影响最小（随后还要进行镜像和分析操作），所以要避免写文件，并且在获取数据的时候越快越好。

当然，GUI 工具并非完全不能用于开机取证过程，如果发现一个 GUI 工具可以很好地完成取证工作，那么就采用，毕竟获取数据是排在第一位的。

不管使用哪个工具，之前都要检查许可协议。一些工具可以自由使用，另外一些工具在企业环境中使用就要收费。阅读许可协议可以避免可能发生的麻烦。

系统时间

事件调查过程中，首先要获取的信息包括系统时间。系统时间为以后获取的数据信息构建了时间上下文环境，并且会为系统事件时间线的正确分析提供帮助。显示系统时间最常见的方法如图 1.1 所示。

背景注释

获取系统时间

调查员可以使用如下的 Perl 脚本获取系统时间：

```
printlocatime(time)."\n";
```

该脚本显示的是本地时间信息，本地时间已经考虑了时区、夏时制等系统设置。Perl 脚本也可以使用 GMT 时间显示：

```
printgmtime(time)."\n";
```

本书附送的光盘中包含一个 systime.pl 的 Perl 脚本文件，该脚本演示了怎样使用 Windows API 获取系统时间信息。Systime.exe 文件是通过 Perl2Exe 编译的独立可执行程序。

另外一个获取系统时间信息的方法是使用 WMI 工具访问 Win32_OperationSystem 类，并且显示 LocalDateTime 值。

系统时间之外，系统上线运行时间（Uptime，也就是开机到现在的运行时间）信息也很重要，可以为调查提供另外一个线索。例如，比较系统运行时间和进程运行时间可以有效估计系统被入侵的可能时间（进程信息的获取在后面讨论）。

此外，调查人员在记录系统时间的同时也要记录真实时间，也可称为“墙上时间”。

同时记录两者才可以确定系统时间是否正确。通过两者之间的比对才可得到系统日志文件中记录的事件真实发生时间。这点在组合多个信息源进行的时间戳分析中尤其重要。

另一个时间相关的重要信息就是计算机的时区设置。Windows 系统中，如果文件保存在 NTFS 分区，文件时间保存为 UTC 时间（等同于格林尼治时间，GMT 时间）[5]。如果文件保存在 FAT 分区，文件时间却是本地系统时间。时区设置对系统的后期分析非常重要，在进行远程开机取证过程中则更加重要。

当前登录用户

调查过程中，有时需要知道系统的当前登录用户是谁，包括本地登录用户（通过控制台或键盘登录）和远程登录用户（通过 net use 命令或共享）。登录用户为收集的其他系统信息提供了上下文线索，例如，运行进程的用户上下文、文件宿主、文件最后访问时间等。该信息也可以和安全日志进行相关分析，尤其是特殊审计开关打开的时候。

Psloggedon

也许确定当前登录用户最著名的工具就是 psloggedon. exe[6]。该命令不但可以显示本地登录的用户，同时也可以显示远程登录的用户名，例如，通过共享登录的用户。图 1. 2 是 psloggedon. exe 程序的输出结果。这个例子通过 Windows XP 登录到 Windows 2000 系统（Petra），然后在 Windows 2000 中运行命令。

```
Users logged on locally:
     10/13/2006 2:34:30 PM     PETRA\Administrator

Users logged on via resource shares:
     10/13/2006 2:36:26 PM     (null)\ADMINISTRATOR
```

图 1. 2　Windows 2000 中 Psloggedon. exe 的输出结果

Net Sessions

本命令不仅可以查看通过远程登录访问系统的用户名，还可以显示使用的 IP 地址、客户端名称等信息。图 1. 3 显示的是 Windows 2003 系统中执行 net sessions 命令的结果。

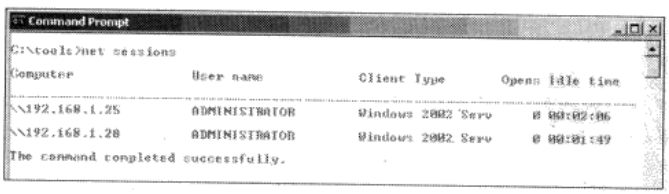


图 1. 3　Windows 2003 中 net sessions 命令的输出结果

图 1. 3 中显示了两个通过管理员账户登录 Windows 2003 系统的信息，其 IP 地址、客户端（Windows XP）信息也有，两个客户端都没有打开文件，连接的时间也不长（通过 Idle time 列可以看出）。

Logonsessions

这是一个 SysInternals. com 提供的命令行程序，可以列出系统的活动登录会话信息[7]。图 1.4 显示的是在 Windows XP 系统中执行 logonsessions. exe 命令的部分输出（系统名称为 Ender）。

```
[6] Logon session 00000000:000478c7:
    User name:    ENDER\Harlan
    Auth package: NTLM
    Logon type:   Interactive
    Session:      0
    Sid:          S-1-5-21-1606980848-308236825-682003330-1004
    Logon time:   2/6/2006 4:19:42 PM
    Logon server: ENDER
    DNS Domain:
    UPN:
      304: C:\WINDOWS\system32\wscntfy.exe
     1372: C:\WINDOWS\Explorer.EXE
     1828: C:\Program Files\CyberLink\PowerDVD\DVDLauncher.exe
     1252: C:\WINDOWS\BCMSMMSG.exe
      892: C:\Program Files\Synaptics\SynTP\SynTPLpr.exe
      788: C:\Program Files\Synaptics\SynTP\SynTPEnh.exe
     1000: C:\WINDOWS\System32\spool\drivers\w32x86\3\hpztsb07.exe
     1016: C:\Program Files\Viewpoint\Viewpoint Manager\ViewMgr.exe
     1932: C:\Program Files\QuickTime\qttask.exe
      108: C:\Program Files\Real\RealPlayer\RealPlay.exe
      552: C:\Program Files\Mozilla Firefox\firefox.exe
```

图 1.4　Windows XP 中 logonsessions 命令的输出结果

从图 1.4 可以看出，Logonsessions. exe 比其他命令提供了更多的信息。例如，登录使用的验证信息（也许知道客户端使用了 Kerberos 登录而不是 LanManager 对调查人员很重要）、登录的类型、活动进程等。

提 示

随书附赠的光盘目录“\ch1\dat”目录中有 logonsess. txt 文件，其中保存了如图 1.3 所示的在 Windows 2003 系统中运行 logonsessions 命令的输出结果。

另外一个好用的工具是来自 Somarsoft. com 的免费工具 netusers. exe。通过 netusers. exe 的-local 和-history 选项，可以获取所有本地用户最后登录时间的报告。最后登录时间由注册表维护，特定注册表项的信息在第 4 章讨论。Netusers. exe 可以在线获取这个信息。

要牢记这些工具并不能显示是否有人通过后门登录到系统。类似 SubSeven[8]之类的后门和木马允许用户使用原始 TCP 连接“登录”，绕过 Windows 安全认证机制。所以这些连接不会通过 psloggedon. exe 之类的工具显示出来。不过有了这些工具的输出，可以对比以后得到的用户列表，看哪些用户没有在这边显示。这样就可以用来发现隐藏的信息，甚至从未被发现的功能机制。

打开的文件

如果调查人员通过 psloggedon. exe 程序找到有用户远程登录到系统中，那么同时也要了解该用户打开了什么文件。远程登录的用户总是要执行一些命令或者打开文件。公司环境中一般允许用户通过共享查看图片、歌曲等共享文件。安全防护差的 Windows 系统，

例如，没有设置管理员口令的 Windows 2000 系统，一旦连接到因特网就会被“访问”，文件被搜索和复制。net file 命令、psfile. exe[9]、openfiles. exe[10]（Windows XP 专业版、Windows 2003 的本地命令）等工具可以显示系统中被远程打开的文件。

网络信息（缓存的 NetBIOS 名字列表）

入侵者在获得系统访问权限之后，有时想要知道网络中还有哪些其他系统可以通过被入侵的系统访问。在调查中经常会碰到这类场景，形式各不相同。有时系统中创建了批处理文件，有时入侵者通过 SQL 注入执行 net view 命令（使用浏览器，通过 Web 服务器、数据库服务器等给系统发命令）。如果通过 NetBIOS[11]通信和其他系统建立了连接（和登录系统或者通过共享连接类似），系统将会维护一个连接过的系统名字列表。通过查看缓存的名字列表，调查人员可以知道哪些系统已经受到影响。

来看一个例子，我的家庭“网络”是由一台笔记本电脑和几个以独立系统运行的 Vmware 会话组成的虚拟网络。为了演示缓存的 NetBIOS 名字信息，我启动一个 Windows 2000 会话并登录查看 DHCP 获得的 IP 地址。然后回到宿主操作系统（Windows XP Pro SP2），启动命令行界面，输入 **nbstat -A 192. 168. 1. 22**，查看“远程”系统的名字列表。再用 **nbstat -c** 查看宿主操作系统中缓存的 NetBIOS 名字列表。结果显示在图 1. 5 中。

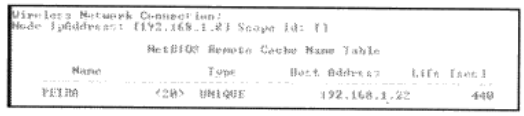


图 1. 5　NetBIOS 名字列表缓存

也许读者这时会想：“这又怎么样？有什么用呢？”换位思考，如果自己是攻击者，已经取得了一个系统的访问权限，那么我还会尽可能的获得其他系统的权限。为达到这一目标，就需要查看网络中还有什么系统，是否有漏洞，并且尽可能找到容易入侵的目标。如果要扫描漏洞，可能会惊动其他人，并且扫描需要上传工具到被入侵的机器中，这也可能会惊动用户。这时候可以使用 nbstat. exe 来定位具有潜在漏洞的系统。例如，图 1. 6 显示了输出 NetBIOS 缓存信息的结果。

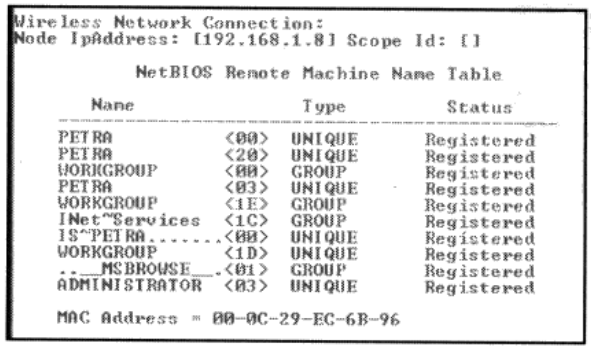


图 1. 6　nbstat -A 192. 168. 1. 22 的输出结果

从图 1.6 的输出结果可以看到管理员登录在系统中，系统中有 IIS Web 服务器运行。渗透测试或者攻击者会利用 NetBIOS 名字列表中的信息来进一步攻击系统，或者定位其他具有漏洞的系统。Microsoft 知识库（MS KB）中编号为 163 409[12] 和 119 495[13]的两篇文章提供了关于名字列表的更多信息。

网络连接

一旦有报告说发生了安全事件，调查人员就应该收集针对被影响系统的网络连接信息。随着时间的消逝，连接信息会慢慢过期，时间越久，丢失的信息越多。调查人员在接触系统以后，通过初始的查看，可以确定攻击者是否还登录在系统中。或者可能发现一个蠕虫、IRC 僵尸程序对外的连接，可能它们正在搜索其他入侵目标、正在进行自身的升级或者连接到一个入侵控制中心。连接信息可以提供重要的线索并为其他信息提供佐证。并非每一个系统都安装了防火墙，尤其是防火墙也不会记录成功的进出连接信息。也不是所有的系统都有一个类似 Port Report[14] 的应用，用来记录网络连接信息。调查人员必须快速反应，在有限的时间内高效地收集需要的信息。

我曾参与数个案件调查，案件中客户获取了系统的镜像文件。他们问："是否有敏感信息被复制?"没有基于网络的连接信息，答案只能是不知道。另外一些案件中，网络连接信息大大缩短了调查时间，尤其是初始报警的信息和被入侵主机没有任何关系的时候。例举一个特殊的情形，对系统镜像的时间线分析发现在入侵者通过后门访问系统的时候，两个不同的管理员也在访问系统修补两个不同的问题，这时如果有进出系统的网络连接信息，对定位入侵就会有极大的帮助。

Netstat

Netstat 也许是 Windows 系统中收集网络连接信息最有名的工具了。这个命令行工具简单而直观地显示了 TCP 和 UDP 连接的状态、网络流量统计等信息。Netstat. exe 是 Windows 的本地命令，是随操作系统一起发布提供的。

Netstat 最常用的方式是使用*-ano* 选项开关，该选项设定程序显示 TCP/UDP 网络连接、侦听端口和使用网络连接的进程标识（PIDs）信息。图 1.7 显示 *netstat-ano* 命令的输出。

```
C:\Netstat -ano

Active Connections

  Proto  Local Address          Foreign Address        State           PID
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING       1044
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING       4
  TCP    127.0.0.1:1026         0.0.0.0:0              LISTENING       1992
  TCP    127.0.0.1:1031         127.0.0.1:1032         ESTABLISHED     2536
  TCP    127.0.0.1:1032         127.0.0.1:1031         ESTABLISHED     2536
  TCP    192.168.1.25:139       0.0.0.0:0              LISTENING       4
  TCP    192.168.1.25:1310      216.239.51.104:80      ESTABLISHED     2536
  TCP    192.168.1.25:1323      209.18.34.43:80        ESTABLISHED     2536
  TCP    192.168.1.25:1326      209.18.34.41:80        ESTABLISHED     2536
  TCP    192.168.1.25:1327      209.18.34.41:80        ESTABLISHED     2536
  TCP    192.168.37.1:139       0.0.0.0:0              LISTENING       4
  TCP    192.168.206.1:139      0.0.0.0:0              LISTENING       4
```

图 1.7　Windows XP 中 netstat -ano 命令输出的部分信息

提 示

一般情况下，Windows 2000 对 netstat. exe-o 选项没有输出。在 Microsoft 知识库 907 980 号文章中提到，应用一个补丁程序可以使得 Windows 2000 下的 netstat. exe 列出进程的 PID 信息。

图 1.7 中，*netstat -ano* 命令显示活动的网络连接信息，并且有每个连接的状态，使用该端口的进程标识（PIDs）。通过 netstat 的输出可以寻找非正常的连接。例如，用户系统中一个高端口的客户进程连接到远程的端口 80 就不正常，通常连接到端口 80 的应该是网络浏览器进程。不过，调查人员很容易忽略该信息。在我调查的案件中就发现过一个使用 wget. exe 连接到远程端口 80 下载恶意软件和黑客程序的实例。不经过仔细地研究和推敲，该连接可能被当成合法的网络浏览。

提 示

Microsoft 知识库文章 137 984 号[15]已经比较古老了，其中提供了 netstat. exe 输出中连接状态的描述。

netstat 的-r 选项将会显示路由表信息，并且可以看出哪些是系统的永久路由。这为调查人员提供了很有用的信息，系统管理员也可用来对系统排错。我曾经发现过一个业务系统被用来传输文件到远程主机，该过程只在设置了永久路由的系统中起作用。因为永久路由定义了特定数据流量通过 VPN 连接传输，而不是通过系统的正常路由。我在排错过程中发现了这个非正常的情况，通过建立永久路由激活该功能并告知了系统管理员。系统管理员也慢慢回忆起这个设置，并且最终解决了问题。

进程信息

调查人员对可能的被入侵系统的运行进程信息总是很关注，当然有时也会忽略。通过任务管理器（Task Manager）查看进程信息时，可以看到每个进程的一些信息。不过，很多需要收集的信息并不能通过任务管理器看到，例如：

- 可执行文件的全路径
- 启动进程时的命令行参数信息
- 进程运行的时间
- 进程运行的安全/用户上下文环境
- 进程加载了哪些模块
- 进程的内存数据内容

任务管理器提供了一些信息，但不是全部。例如，一些恶意软件使用 svchost. exe 名称，这是一个 Windows 系统中的合法进程名（参见对 svchost 的帮助信息）。合法进程的可执行镜像位于 System32 目录中并被 Windows 文件保护机制（WFP，参见 WFP

的帮助信息）所保护。WFP 运行过程中会保护系统进程不被修改或替换，如果有恶意修改/替换行为发生，WFP 会从缓存中替换回一个好的文件，并且在系统日志中记录该信息。

为什么需要这些信息？如果在任务管理器中查看进程信息，我们怎么知道哪个进程是“可疑”的？一个简单的方法就是查看可执行文件的全路径（那些不在 C:\Windows\System32目录下的 svchost. exe 的运行进程就是可疑的）。另外，使用*-L -d -p 80 -e cmd. exe* 参数启动的 inetinfo. exe 进程也是可疑的，该命令行显示这是一个 netcat 后门应用。很多恶意软件使用合法文件名来伪装自己。例如，W32/Nachi[16] 蠕虫就在 C:\Windows\System32\Wins 目录中保存了一个 TFTP 程序，并且取名为svchost. exe。当这个程序运行时，通过任务管理器不能将它与合法的 svchost. exe 区分开来。

你知道吗?

文件保护机制

Windows 文件保护机制简称 WFP，在 Windows 2000 时加入到系统中，Windows XP 和 Vista 亦应用该机制。简而言之，WFP 保护关键的系统文件不被意外的修改或删除。假设系统被入侵，而 WFP 机制仍然有效，如果尝试修改或删除 WFP 保护的文件，WFP 保护机制则被激活，使用缓存中完好的文件替换回来。同时系统日志将记录一个 ID 为 64 001[17] 的条目。

Microsoft 知识库 222 193 号文章[18] 描述了 Windows 文件保护机制，并且对与 WFP 相关的注册表键进行了深入描述和分析。

现在一起来了解可以获得进程详细信息的工具。

Tlist

Tlist. exe 包括在 Microsoft 调试工具中（MS Debugging Tools）[19] 可以显示运行进程的很多信息。例如，-v 参数可以显示会话标识、PID、进程名称、相关服务、命令行参数等，如下所示：

```
0 344 svchost. exe Svcs: LmHosts, SSDPSRV, WebClient
    Command Line: C:\WINDOWS\System32\svchost. exe -k LocalService
```

其他参数会单独显示这些信息。-c 参数显示运行进程的命令行信息（类似于 cmdline. exe[20]）。-s 参数显示相关的服务信息（如果没有服务，则显示窗口标题）。-t 参数显示任务树，列出每个进程的父进程，如下所示：

```
System (4)
  smss. exe (628)
    csrss. exe (772)
    winlogon. exe (1056)
      services. exe (1100)
```

```
svchost.exe (1296)
svchost.exe (1344)
svchost.exe (1688)
  wscntfy.exe (1184)
```

使用-m 参数，Tlist.exe 还可以用来查找拥有特定模块的进程。例如，wsock32.dll 提供网络相关的功能，是 Windows32 位套接字动态库，如需列出所有加载该模块的进程，使用如下的命令：

```
D:\tools>tlist -m wsock32.dll
```

上面的命令返回每个进程的 PID 和名称，例如：

```
WSOCK32.dll -1688 svchost.exe
wsock32.dll -344 svchost.exe
WSOCK32.dll -1992 alg.exe
WSOCK32.dll -1956 explorer.exe          Program Manager
wsock32.dll 452 ViewMgr.exe             AXTimer
WSOCK32.dll 480 realplay.exe
```

Tasklist

Tasklist.exe 是 Windows XP 专业版和 Windows 2003 的本地应用（但不包括在 Windows XP 家庭版中），可以用来替代 tlist.exe[21]。两者的差别非常小，大部分是命名和选项参数的不同。

Tasklist.exe 提供输出表格、CSV、列表等不同格式的选项。*/v* 选项参数提供进程的最多信息，包括程序名称（没有全路径）、PID、进程名、会话数量、进程状态、用户名、窗口标题。使用*/svc* 参数也可以列出进程相关的服务信息。

Pslist

Pslist.exe 显示系统中运行进程的基本信息，包括每个进程已经运行的时间（用户模式和核心模式）[22]。*-x* 选项显示线程及进程使用的内存详细信息。*-t* 选项参数显示类似 tlist.exe 的任务树。不过，pslist 没有提供执行进程的路径信息、命令行信息，也没有提供用户信息。

Listdlls

Listdlls.exe 显示进程使用的动态链接库[23]，同时还会显示加载模块镜像的全路径及内存中加载的动态链接库（DLLs）版本与磁盘上镜像版本的不同。这个信息对调查人员非常有用，因为每个程序都会加载或者“引入”特定的动态链接库。动态链接库提供了实际可用的代码，所以应用开发人员在开发新的应用时不必每次实现常用的函数。每个动态链接库包含一些特定的函数，通过输出表显示出来，调用程序通过输入表来显示动态链接库及调用的函数。调查人员使用适当的工具就可以知道程序加载、访问哪些动态链接库。不过，也有一些程序可以调用输入表之外的动态链接库，例如，IE 可以

加载工具栏和浏览器帮助对象，这些代码都通过动态链接库运行。间谍软件、木马程序和后门程序使用动态链接库注入技术把自己加载到运行程序的内存空间中，使得它们在运行的同时不会显示在进程列表中，因为这时它们是其他进程的一部分。这与子进程不同（可以通过 tlist. exe -t 参数输出显示），此时执行的恶意软件没有自己的进程标识（PID）。

Listdlls. exe 会显示执行程序的全路径信息，参见如下的部分输出结果：

```
svchost.exe pid: 1292
Command line: C:\WINDOWS\system32\svchost -k DcomLaunch
```

与 tlist. exe 类似，listdlls. exe 还可以显示加载特定动态链接库的进程（使用-*d dllname* 参数）。这在需要分析特定动态链接库时比较有用，通过这个命令可以查看其他使用该动态链接库的进程。

Handle

Handle. exe 显示系统中进程打开的句柄[24]，包括打开的文件、端口、注册表项、线程等。这对确定运行进程访问哪些资源特别有用。图 1.8 显示了 Windows XP SP2 上运行 handle. exe 的部分输出。

```
svchost.exe pid: 1288 NT AUTHORITY\SYSTEM
    C: File           C:\WINDOWS\system32
   64: File           C:\WINDOWS\WinSxS\x86_Microsoft.Windows.Comm
5b64144ccf1df_6.0.2600.2180_x-ww_a84f1ff9
   B8: File           C:\WINDOWS\Sti_Trace.log
  168: File           C:\WINDOWS\wiaservc.log
  1C4: File           C:\WINDOWS\wiadebug.log
  1D8: File           C:\WINDOWS\Sti_Trace.log
```

图 1.8　Handle. exe 的部分输出

图 1.8 显示了 svchost. exe 打开的一些句柄——本例中有几个 Windows 目录下的日志文件。在写本章的时候，我发现 winword. exe 打开的一个句柄包括 Word 文档的全路径。

Handle. exe 有几个可用的选项参数，例如，*-a* 显示所有的句柄，*-u* 显示某用户的句柄。

工具和陷阱

进程和 WMI

光盘中“\ch1 \code”目录下的 Perl 脚本 proc. pl 显示了怎样利用 Win32_Process[25]类和 WMI 来获取进程信息。脚本文件和编译好的独立 proc. exe（使用 Perl2Exe 编译，光盘中也有）显示了进程标识（PID）、进程名称、用户、父进程标识、启动进程的命令行、执行文件的全路径及进程的服务相关信息。

脚本和程序可以获取本地信息，也可以获取远程系统的信息，直接执行 proc. exe 就可以取得本地系统的信息。要取远程系统的信息，参数如下：

```
C:\tools>proc <system> <user> <password>
```

一个执行的例子如下：

```
C:\tools>proc WebSvr Administrator password
```

Proc.exe 执行结果的部分输出如下：

```
PID             :668
Name            :spoolsv.exe
User            :NTAUTHORITY\SYSTEM
ParentPID       :1100[services.exe]
CmdLine         :C:\WINDOWS\system32\spoolsv.exe
Exe             :C:\WINDOWS\system32\spoolsv.exe
Services        :Spooler
```

脚本很容易定制修改输出为另外的格式，例如，使用逗号来分割，这样方便用电子表格软件来打开分析。

光盘中相同目录下的 procmon.pl（及编译好的“procmon.exe”）演示了怎样使用 WMI 来监控本地系统中进程的创建。

从命令行启动 procmon.exe，运行过程中将会报告新建进程的 PID、用户、执行程序的路径及进程启动的命令行，如下所示：

```
PID      USER       PROCESS
---- ---------- ----------------
3208 Harlan C:\WINDOWS\system32\cmd.exe ("C:\WINDOWS\system32\cmd.exe")
1768      Harlan     C:\WINDOWS\system32\ping.exe(ping192.168.1.1)
3100      Harlan     C:\WINDOWS\system32\sol.exe(sol)
```

类似 procmon.exe 这样的工具，可以记录进程的创建信息，对监控应用安装过程中的进程创建及恶意软件的分析特别有用。

现在已经比较清楚了，调查过程中还没有哪个工具可以搜集所有的进程相关信息。可以采用一个工具来快速查看（tlist.exe 或者 tasklist.exe 比较好），当然也可以使用多个工具。例如，使用 pslist -x 和 listdlls.exe，也可以运行 handle.exe，依赖于调查所需要的信息详细程度。在本章的后续部分和第 3 章还有更详细的讨论，尤其是这些数据的相关分析。

工具和陷阱

Svchost

Svchost 是 Windows 2000，XP，2003 系统中的常见进程。在任务管理器中会出现多次，在默认的 Windows 2000 系统中至少出现 2 次，在 XP 系统中出现 5 次，而在 2003 系统中则要出现 7 次。Svchost.exe 的每个实例运行一到多个服务，这点可以通过

使用 XP 和 2003 系统中的 *tasklist/svc* 命令或者 2000 系统中的 *tlist -s* 命令看到。

Microsoft 知识库文章序号为 314 056[26]提供了 XP 系统中 svchost. exe 的更多信息，序号为 250 320[27]的文章则提供了 2000 系统中 svchost. exe 的类似信息。

基本上来说，svchost. exe 是提供从 dll 中运行服务的基本进程。Svchost 的每个实例可以运行一个或多个服务。系统启动的时候，svchost 读取注册表键以获取需要运行的服务组信息：

```
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows
NT\CurrentVersion\Svchost
```

有几个木马和后门程序尝试使用 svchost. exe 名称复制到被入侵的系统中。Backdoor. XTS[28]和 Backdoor. Litmus[29]就是这样的恶意程序，它们使用 svchost. exe 名称来隐藏自己，因为大部分系统管理员和调查人员认为任务管理其中有多个 svchost. exe 进程并非异常。不过，Windows 系统中将伪造的 svchost. exe 程序复制到 system32 目录并不明智，因为 WFP 的文件保护机制会起作用。

提 示

Pulist. exe[30]是一个资源包工具，可以同时列出运行系统中每个进程的信息和进程相关的用户信息。

进程到端口的映射

系统中存在打开的网络连接的时候，一定是有进程在使用这个连接。也就是说，每一个网络连接和开放的端口都有进程相关联。有几个工具可以用来调查和获取这种进程到端口的映射信息。

Netstat

Windows XP 和 Windows 2003 中，netstat. exe 程序提供-*o* 选项来显示网络连接对应的进程标识（PID）。取得这个信息之后（参见前面介绍的 *netstat -ano* 命令），必须使用类似 tlist. exe 或 tasklist. exe 命令来和输出中的 PID 相关联，确定使用该连接的进程名称（或其他信息），如图 1.9 所示。

Windows XP SP2 新增了一个-*b* 选项，用来“显示创建每个连接或侦听端口的执行程序名称”。这个选项也包括在 Windows 2003 SP1 中，并且可以提供使用特定端口的进程的更多信息。某些情况下，输出中还会显示进程使用的一些模块（DLLs）。

提 示

另外一个特别有用的工具是 Microsoft 提供的 tcpvcon. exe[31]。它的输出与 netstat. exe 类似，其中包含了连接的状态和对方信息。Windows XP 和 2003 中，tcpvcon. exe 还提供连接的进程标识（PID）和执行命令的程序路径。

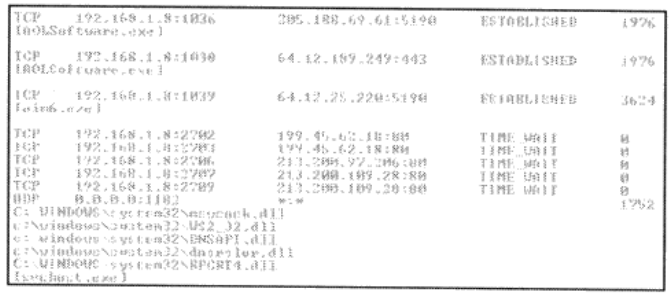


图 1.9　Windows XP SP2 系统中 netstat -anop 命令的部分输出

Fport

Fport.exe 一直是 Windows 系统中用来获取进程到端口映射的首选工具之一[32]。它的输出简明易懂，不过该工具需要管理员账户运行，这在使用不具有管理员权限的普通账户进行调查时会是个问题。

Openports

Openports.exe 也许是 Windows 系统中获取进程到端口映射信息的最好的工具[33]。该工具允许输出为多种格式，包括 netstat 风格、fport 风格和 CSV 格式，并且不需要管理员权限来执行。使用-*fport* 选项，openports.exe 可以输出为 fport 风格，显示进程标识、进程名称、端口号、协议（TCP/UDP）和执行程序的路径，如下所示：

```
4       SYSTEM      -> 139     TCP   SYSTEM
4       SYSTEM      -> 139     TCP   SYSTEM
168     alg         -> 1025    TCP   C:\WINDOWS\System32\alg.exe
1340    svchost     -> 135     TCP   C:\WINDOWS\system32\svchost.exe
1568    firefox     -> 1037    TCP   C:\Program Files\Mozilla Firefox\firefox.exe
1568    firefox     -> 1038    TCP   C:\Program Files\Mozilla Firefox\firefox.exe
```

提示

如果调查的是 Windows 2000 系统，并且不具有前面提到的可以让netstat.exe列出网络连接进程标识的补丁，一个很好的选择就是使用 openports.exe -netstat 选项。

每个项目输出为一行，可以很容易地使用自动工具来进行分析（见第 2 章）。

使用-netstat 选项，openports.exe 的输出类似 netstat.exe，只显示了进程标识信息，而没有执行程序的路径。要获取每个网络连接的完整视图，需要将 openports.exe 和 netstat.exe 的输出进行相关分析，这部分内容在第 2 章中讨论。

在具有管理员权限的情况下，而且系统中同时存在 fport.exe（2.0 版）和 openports.exe（1.0 版）。Openports.exe 的输出显示了连接进程的执行程序路径，而 fport.exe的输出中并不包含这一信息，此时使用哪个工具呢？我的建议是读者在确定

使用哪个工具之前进行测试，而且如果是在公司环境中使用这些工具，需要阅读每个工具的授权协议（记住，在使用每个第三方工具之前都要阅读授权协议）。

也有人会选择使用端口扫描工具（例如，nmap[34]）获得远程主机的开放端口信息，以确定系统是否被入侵。扫描会发现处于侦听模式的开放端口，侦听的服务可能是认证服务、Web 服务、FTP 服务，但也可能是后门。如果发现系统中有端口开放，但是不能通过 netstat 或其他工具显示该端口，进程到端口映射程序也不能显示，那就可以确定系统有异常。这时还要再次验证扫描结果以确认扫描的是正确的系统。如果问题还存在，那么大致可以确定存在一个内核级的木马，也就是 Rootkits（第 7 章介绍更多这方面的内容）。

进程内存

开机系统会有一系列的运行进程，本质上任何一个进程都可能具有恶意性。当系统中的进程运行时，进程名基本上会和执行程序的文件名一致。以 Windows 为例，文件可以任意命名。坏人不会将恶意代码起一个容易辨认的名字，如“坏东西. exe”（bad-stuff. exe）。他们经常会起一个不会引起注意的名字，或者会使用一个 Windows 正常程序的名字来伪装。

一旦发现并确定了可疑进程，调查员就需要知道该进程的更多信息。这可以通过获取该进程的内存来得到。有几个工具可以来完成这项任务，如前所述，对于内存收集和讨论的更多内容请参见本书第 3 章。

网络状态

对调查人员来说，系统中网卡（NICs）连接的状态也很重要。例如，现在的笔记本都带内置的无线网卡，调查中可能需要确认系统是否连接到无线网络，分配的 IP 地址是多少。在系统获取之前查询网卡的状态对以后的调查会很有帮助。

Ipconfig

Ipconfig. exe 是 Windows 的本地应用[35]，可以用来显示网卡信息及其状态。该命令最有用的参数是/all，该参数显示系统中网卡的网络配置，包括网卡状态、是否使用 DHCP、当前 IP 地址等。

调查中会发现网卡信息很重要，因为可能会分析网络流量数据。系统的 IP 地址可能会在运行的某一时刻更改，而且很多基于 Web 的电子邮件服务（例如，Yahoo 邮件）会在撰写的邮件头中记录系统的 IP 地址。我参加过一个特别的调查，一个前雇员总是写一些无聊的邮件（不是骚扰邮件）到公司邮箱。通过查看邮件头可以查到发送邮件的地址。有几封邮件来自当地的一个复印店，其他邮件来自一个公共图书馆。在政府人员和复印店管理员的无私帮助下，我们缩小了目标范围。通过对公共图书馆排查，发现对象使用的系统位于二楼。对象在得知我们掌握的这些信息后惊呆了，也停止了发送恶意邮件。如果这个人没有被解雇，而是从他的系统来发送 Yahoo 邮件，我们也能确定他的地址。

Promiscdetect 和 Promqry

有时被入侵的系统会安装网络嗅探器来获取网络流量，例如，获取其他系统的登录信息，或者用来得到网络拓扑和其他系统的服务信息。一些恶意软件本身就具有嗅探功能，或者通过随后的下载安装这一功能。如要通过网卡获取网络流量数据，必须将网卡设置为“混杂（promiscuous）”模式。一些管理员和调查人员不会看到这一信息，因为网卡处于“混杂”模式并没有明显的标志。没有系统任务栏图标显示，从系统控制面版的设置中也不能清楚地看到。

通过工具可以看到网卡处于混杂模式，其中一个工具就是 promiscdetect. exe[36]。另外一个是 promqry. exe[37]，由 Tim Rains 编写。两个工具的差异在于 promqry. exe 可以用来检测远程系统，管理员通过这一工具可以扫描管理域中的系统，查找可能的嗅探行为。

工具和陷阱

混杂模式

光盘中的“\ch2 \code”目录下 Perl 脚本程序 ndis. pl 使用 WMI 机制来确定网卡设置。它可以用来确定网卡是否处于混杂模式，是否可以从网络中嗅探数据。

相同目录下的 ndis. exe 文件是这个脚本独立执行程序版本，如果系统中没有安装 Perl 程序，可以使用它。

图 1. 10 显示的是 ndis. exe 程序的部分输出信息。

```
Dell Wireless WLAN 1350 WLAN Mini
            NDIS_PACKET_TYPE_MULTICAST
            NDIS_PACKET_TYPE_DIRECTED
        --> NDIS_PACKET_TYPE_PROMISCUOUS <--
            NDIS_PACKET_TYPE_BROADCAST

VMware Virtual Ethernet Adapter for VMnet8
            NDIS_PACKET_TYPE_MULTICAST
            NDIS_PACKET_TYPE_DIRECTED
            NDIS_PACKET_TYPE_BROADCAST

Broadcom 440x 10/100 Integrated Controller
            NDIS_PACKET_TYPE_MULTICAST
            NDIS_PACKET_TYPE_DIRECTED
            NDIS_PACKET_TYPE_BROADCAST
```

图 1. 10　Windows XP 下 ndis. exe 程序的部分输出

图 1. 10 是在 WireShark[38]（以前版本称为 Ethereal）程序启动后显示的信息，该程序正在通过无线网卡嗅探数据。高亮部分清晰地显示出无线网卡处于混杂模式。Perl 脚本和独立程序只可以在本地系统运行，对程序进行修改同样会适用于远程系统，类似 promqry. exe。

此类工具的另外一个用途是确定在线系统中哪个网卡是活跃的通信界面。老式Toshiba Tecra 8100笔记本需要通过PCMCIA卡连接到网线网络，现在新出的笔记本都有内置的无线网络功能。从外观上看不出有外界连接，也看不到灯在闪烁，不像使用RJ-45的以太网连接，所有网络功能都是内置的。如果张三进入会议室，打开笔记本，他是在记笔记还是在浏览网页或是在发送电子邮件呢？无线接入使用越来越普遍，不仅因为现在的接入点不断增多，也是因为笔记本内置的原因。

无线接入点可以用来连接到组织网络中，也可以被一些人用来窃取网络中的信息。我曾经处理过这样一个问题，公司公关部的人想要在开会时使用笔记本访问因特网，但是他没有联系IT部门，也没有联系我（安全管理员）。他打开笔记本，发现了我们的无线接入点，由于采用了WEP加密和IP地址过滤，所以并未取得访问权限。因为没有和我们联系，而且正在会议中，他决定连接到检测到的一个开放的连接点，该连接点是隔壁公司使用的，没有任何安全防护。当他建立连接后，等于在我们的信息网络中开了一个口，绕过了所有安全防护机制，没有防火墙，也没有防病毒检测机制。此时，很难说哪种情况更加具有破坏性——使用他的连接作为跳板对本公司信息网络的感染，还是隔壁公司的信息网络被入侵引起的法律问题。

调查过程中，获取系统活动网络接口信息总是不错的。这不仅为其他易变信息提供了基础，也会为后期分析提供帮助。

剪贴板内容

剪贴板是可以暂时存放数据的内存区域，其中的数据可以再次使用。大部分Windows应用通过菜单栏的Edit选项提供这一功能。单击Edit会出现一个下拉菜单，其中有剪切（Cut）、复制（Copy）和粘贴（Paste）操作选项。Microsoft的Word 2003包括Office剪贴板选项

剪贴板常用来方便地移动数据——在文档之间或在Windows的应用之间。用户选择文本或数据，选择复制功能，然后选择粘贴功能将数据插入到其他地方。剪切功能从原有的文档中移除数据，数据进入剪贴板中。

人们在打开电脑的时候不会意识到复制的数据会保存在剪贴板中。当人们编辑具有敏感信息的文档时，客户的个人信息需要加入到文档中。用户定位、选择并复制信息到剪贴板，然后粘贴到文档中。只要机器保持运行，用户没有退出，剪贴板没有被其他数据所替代，则数据会一直保存在剪贴板中。

尝试一下，唤醒你的计算机，打开文本编辑器或者一个新的Word文档，简单地使用Ctrl+V组合键将当前剪贴板上的内容粘贴到文档中。再试一下另外一台计算机。你可能会看到Web网址、聊天信息片段、口令或者文档的整段文本内容。剪贴板上的数据并不是直接可见，但是会一直在那里，并且会是问题所在——Microsoft知识库中有一篇文章就叫“怎样阻止网站获取Windows剪贴板中的数据”[39]。

剪贴板中发现的数据也会对案件有用，例如，信息或资产窃取、欺诈和骚扰案，其中的数据会成为线索。有时在剪贴板中还会发现图片或整段的文章内容。

Pclip.exe[40]是一个命令行程序，可以用来获取剪贴板中的内容。类似pclip.exe的命令行程序很容易用于脚本和批处理程序，自动收集信息。

工具和陷阱

剪贴板内容

Perl 提供了一个简单的 API 界面，用于访问剪贴板内容。下面的脚本使用字符串方式打印剪贴板内容：

```
usestrict;
useWin32::Clipboard;
print"Clipboardcontents = ".Win32::Clipboard()->Get()."\n";
```

如果想使用 WIN32::Clipboard 的更多功能，请参见该模块的文档。

服务/驱动信息

根据注册表中的配置，在系统启动的时候服务和驱动也会自动启动。大部分用户不会看到系统中作为进程运行的这些服务，因为进程中并没有服务的明显标志（通过任务管理器可以看到进程的运行，但不知是何服务），但是服务肯定是在运行。不是所有的服务都是通过用户或系统管理员安装。一些恶意软件会将自己安装为服务，甚至是系统驱动。

工具和陷阱

服务信息

光盘目录“\ch1 \code”目录中的 svc.pl 脚本使用 WMI（访问 Win32 _ Service 类[41]）来获取服务相关的信息，可以从本地获取，也可以从远程获取。Svc.exe 是使用 Perl2Exe 编译的独立可执行程序。

脚本和独立程序都可以显示服务的如下信息：

- 服务名称
- 服务的显示名称
- 启动名称
- 服务描述字符串
- 服务的进程标识（可以用来做服务和进程之间的映射）
- 服务的执行程序路径
- 服务的启动模式
- 服务的当前状态
- 服务状态
- 服务类型（核心驱动、共享进程等）
- 标签 ID，惟一标识，用来确定在加载顺序组中的服务启动顺序

图 1.11 是本工具显示的一个样本信息。

```
Name    : UMWdf
Display : Windows User Mode Driver Framework
Start   : NT AUTHORITY\LocalService
Desc    : Enables Windows user mode drivers.
PID     : 1660
Path    : C:\WINDOWS\system32\wdfmgr.exe
Mode    : Auto
State   : Running
Status  : OK
Type    : Own Process
TagID   : 0
```

图 1.11 Windows XP 上执行 svc.exe 的一个样本输出

可以修改 Perl 脚本和可执行程序以得到不同的输出格式，例如，可以输出为逗号分割的 CSV 格式，这样可以很方便地使用电子表格程序打开和分析。

命令行历史

假设一个计算机调查现场，系统正开着且可以看到屏幕上有几个命令行窗口。这种情况下，线索可能就在用户输入的命令中，如 ftp 或 ping。如果用户输入了 cls 命令来清除窗口，就不能使用滚动条来查看刚刚输入的命令。不过，可以使用 doskey /history 命令来显示前面输入的命令情况，如下所示：

```
D:\tools>doskey /history
move proc.exe d:\awl2\ch2\code
perl2exe -small d:\awl2\ch2\code\proc.pl
move proc.exe d:\awl2\ch2\code
y
cd \awl2\ch2\code
proc
cd \perl2exe
perl2exe -small d:\awl2\ch2\code\procmon.pl procmon
move procmon.exe d:\awl2\ch2\code
cd d:\awl2\ch2\code
procmon
cd \tools
openports -fport
openports -netstat
cls
doskey /history
cd \tools
dir prom*
promqry
dir prom*
promqry
```

例如，一次在西海岸教“应急响应”课程。午间休息时，我“入侵”了学生的机器。在几台机器上，我特意打开了命令行窗口且输入了几个命令，然后输入 *cls* 清除窗口内容。学生回来时，我注意到房间后排的一个学生立刻关闭了屏幕上的命令行窗口，而其他几位没有关闭窗口的同学则像设计中的那样，通过命令行窗口找到了入侵的线索。不过，我得承认在实际的调查过程中还没有用过这个命令。直到现在，我没有碰到过现场运行的计算机命令行窗口是打开的。但这并不意味着以后不会发生。

映射的驱动器

调查过程中，也许需要掌握系统中映射的驱动器或共享来自哪里。映射可能由用户创建，可能出自不良意图（例如，用户得到了管理员口令并访问公司的资源）。更进一步，也许从文件系统或注册表中不能发现这些映射的共享连接信息，不过这些驱动器映射动态信息还是可以与前面获取的网络连接信息相关联。

图 1.12 显示的是 di.exe 程序的输入，di 的意思是驱动器信息（drive info），这个程序可以在随书附赠的光盘中找到。

图 1.12 中 di.exe 的输出来自 Windows XP 家庭版系统，系统中驱动器映射则是网络上 Windows 2003 服务器的共享 C$。

```
D:\awl2\ch2\code>di
Drive    Type         File System  Path                 Free Space
-----    ----         -----------  ----                 ----------
C:\      Fixed        NTFS                              1.15  GB
D:\      Fixed        NTFS                              8.18  GB
E:\      Fixed        NTFS                              5.19  GB
F:\      CD ROM                                         0.0B
G:\      Removable    Fat32                             974.45 MB
Z:\      Network      NTFS         \\192.168.1.71\c$    2.96  GB
```

图 1.12　di.exe 的输出

注意，di.exe 的输出还包括可移除驱动器的信息（驱动器 G:），是一个连接的 USB 闪存，这部分内容在第 4 章中还会有讨论。

共享

除了调查系统中使用的网络资源之外，还需要获取系统中共享给网络的资源。系统中共享的信息可以通过 HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services\lanmanserver\Shares 得到，也可以在开机取证时运行类似于 share.exe 的命令行工具得到。光盘中有这个程序，同时也有这个程序的 Perl 源代码。Share.exe 的部分输出如下所示：

```
Name -> SharedDocs
Type -> Disk Drive
Path -> C:\DOCUMENTS AND SETTINGS\ALL USERS\DOCUMENTS
Status -> OK
```

非易变信息

开机取证过程中，并不局限于易变信息。有时也需要收集一些重启后还存在的永久信息，例如，注册表键的内容和文件。因为调查员需要快速得到一些信息用于分析，或者是由于入侵者还登录在系统上。这样的案子中，调查员可能让机器一直运行并在线，用于追踪攻击者（或僵尸网络），同时他也需要保存特定的信息，以防信息被修改或删除。

系统启动时，一些内容会被更改，例如，驱动器映射或者被映射的信息、启动的服务、安装的应用等。这些更改在启动后不会再存在，因而有必要记录在调查文档中。

注册表设置

一些注册表键值和设置会影响随后的取证分析与调查。虽然这些设置本身是非易变信息，它们还是会影响调查员的处理过程和决定，甚至会影响是否继续调查的决定。

有几个工具可用来收集注册表信息。我个人倾向于写一个 Perl 脚本，用来获取注册表的特定键值及其子健的值。Reg. exe 是一个命令行工具，可以访问和管理注册表，它包括在 Windows 2000 支持工具中，并且是 Windows XP 和 2003 的本地工具。

ClearPageFileAtShutdown

这个特殊的注册表值告诉操作系统关机时清除页交换文件。Windows 使用虚拟内存架构，一些进程使用的内存内容会被交换出去，位于交换文件中。系统关闭时，交换文件中的信息会在硬盘上保持不变，其中可能会有解密的密码、聊天会话信息和其他字符串信息，为调查提供重要的线索。然而，如果关机时清除了交换文件，这些潜在的线索信息就很难再提取。

Microsoft 知识库中有关于这个注册表键值的论述，分别针对 Windows 2000[42] 和 XP[43] 系统。

DisableLastAccess

Windows 可以禁用文件最后访问时间的修改。这会提高系统性能[44]，尤其是对使用频繁的文件服务器。对普通的工作站或者类似的日常使用的台式机、笔记本不会有明显的性能提升。Windows 2003 中，将下列键值设为 1：

```
HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\FileSystem\Disablelastacess
```

根据 Microsoft 的 Windows 2003 性能调整指南文档[45]，这个键值默认并不存在，必须由用户创建。

Windows XP 和 2003 系统中，可以用 fsutil 命令查询激活该设置。例如，查询设置

使用这个命令：

```
C:\>fsutil behavior query disablelastaccess
```

如果设置了该注册表键值，尤其是在系统调查过程之前，则对最后访问时间的分析就没有任何价值了。

警　告

DisableLastAccess 功能在 Vista 系统中是默认激活的。在应急响应和取证调查过程中记住这点。编写本书的时候，关于该问题的取证和分析还在发展过程中。

自动启动

注册表中有几个键用来定义自动启动的位置，文件系统中也有。通过自动启动的定义，可以指定应用程序自动启动，而不需用户参与。一些位置定义程序在操作系统启动的时候自动启动，或者是在用户登录的时候启动，另外一些定义程序在用户进行特定操作的时候启动。如果定义成用户执行特定操作的时候启动，此时用户并不知道有其他程序启动了。

这些位置都定义在注册表中，找到这些位置似乎很难。但是好消息是这样的位置在注册表中是有限的，即使数量有不少。在这里不会列出所有的注册表键，我把它留在后续的章节——注册表分析中来专门讨论。不过，要在现场响应的时候收集这方面的信息。这里介绍两种方法。一种是使用类似 reg. exe 的工具（前面提到过）来提取特定注册表的键值。第二种方法是使用 AutoRuns[46] 这样的自动工具来提取。AutoRuns 的作者（Mark Russinovich 和 Bryce Cogswell，现在是 Microsoft 的职员）在工具中维护了一个很好的键值列表。一些情况下，我首先在工具中发现添加了新的位置，然后才在恶意软件中看到有利用。AutoRuns 有图形界面和命令行两种模式，功能完全相同。例如，可以使用-m 的命令行参数版本隐藏 Microsoft 签名的启动项（启动项的可执行文件由厂商签名），-v 参数验证数字签名。

AutoRuns 也可以检查文件系统中的自动启动区域，例如，调度任务（Scheduled Tasks）。一些管理员使用调度任务来获得超过管理员级别的提升权限（例如，SYSTEM级别），用以查看注册表的特殊部分。得到管理员权限的攻击者也可以做同样的事来进一步提升权限。

注册表中另外一个有价值的区域是保护存储区域（Protected Storage area，参见后面的进一步帮助）。保护存储区域中的数据以加密方式存储。如果已经获取了系统镜像，那么使用 AccessData 的 FTK 可以解密并恢复这部分信息。不过，开机取证时收集这部分信息更加简单，尤其是在时间紧急且这部分信息特别重要的时候。

背景注释

保护存储（Protected Storage）

保护存储区域是用于保存用户敏感信息的内存区域。系统关闭后，信息则以加密方式保存在注册表中，用户登录时，则位于内存中。Windows 将口令和自动完成的表单信息保存在其中，以备后续使用。开机时保护存储区域的内容可以通过类似 pstoreview. exe[47] 之类的工具查看，或者使用 Portected Storage Explored[48] 工具。

这部分信息在调查站点访问的案件，或者牵涉 HotMail，MSN 口令的案子中特别有用。

保护存储区域的内容对坏人也很有用。我曾发现感染了 IRC 僵尸蠕虫（恶意软件的一种，安装后会与 IRC 频道连接等待指令，频道管理员可以发送一个指令命令上千个僵尸机器运行）在得到指令后把保护存储区域的信息发送给坏人。2006 年 2 月 19 日，Brian Krebs 在华盛顿邮报杂志发表一篇文章[49]，这篇文章中写到黑客通过发送一个指令（pstore）来获取所有被感染系统的保护区域的信息，包括 PayPal、美国银行、eBay、花旗银行账户的用户名口令，甚至包括军方和联邦政府的邮件账号。

保护存储服务中的内容通过内嵌于 IE 浏览器的自动完成机制来实现[50]。自动完成的设置可以参见图 1.13，单击 IE 的“工具”菜单，然后选择“Internet 选项”的“内容”选项，然后单击“自动完成”按钮。自动完成功能激活时，被感染的系统如果使用 IE 来访问在线商店或在线银行的账户时，账号信息就会被 Brian文章中的黑客所得到。

图 1.13　IE 6.0“自动完成设置”对话框

PassView[51]和 Protected Storage Explorer 采用图形界面来访问保护存储信息，pstoreview. exe[53]是一个命令行工具，可以获取相同的信息。在调查过程中可能需要这部分信息，尤其是处理的案件中包括使用口令对 Web 站点的访问等行为时。

事件日志

事件日志文件必然存在于文件系统中，而且其内容可能改变。事实上，根据审计策略的配置情况，事件日志会快速发生改变。

依赖于“对象”系统中审计策略的配置情况，以及在现场响应时的操作，调任人员的活动也可能被记录在事件日志中。例如，如果调查员决定从远程来访问系统收集信息（系统在另外一个大楼或城市，调查员不能亲临现场，而需要获取一些信息），那么如果设置了正确的审计策略，则远程登录的每次情况都会记录在安全事件日志中。如果由调查动作生成的日志足够多，则可能覆盖调查所需要的其他有价值的日志信息。Psloglist. exe[54]和 dumpevt. exe[55]工具可以用来获取日志记录数据，或者直接将系统中的 .evt文件复制出来（这依赖于调查所使用的账户权限和访问级别，第 5 章中有详细的讨论）。

这时读者可能会问：“好吧，有了这些工具，我在进行现场调查取证，那么到底需要收集哪些信息?”标准答案是看情况。也许这不是读者想要的答案，但这正是因为每个现场处理的事件都各不相同，所以要获取的信息也必然不同。

现场中最有用的易变信息总是与所处理的现场事件类型有关。例如，如果是远程入侵或木马后门相关的事件，则进程、网络连接、进程到端口映射信息（也许还包括一些注册表键值）最重要。不过如果是公司环境中的雇员盗窃了公司的信息或违反了公司的使用策略（AUP），那么系统连接的存储设备信息、Web 浏览日志信息、剪贴板数据就会对调查更有价值。

这个问题的关键是调查员必须知道现场有哪些信息，怎样提取及使用这些信息。当我们开始考虑不同的事件类型及需要什么样的信息时，可以看到在对不同的事件进行处理时，工具和调查中感兴趣的数据会有一些重叠。虽然不能开发出一个针对各种不同调查现场的“万能”批处理文件，但可以开发一些小的批处理文件（或者应用于 Forensic Server Project、WFT 的配置文件）。这样就可以针对不同现场提取相应的信息。

设备和其他信息

现场中可能还需要收集系统中其他非易变信息并记录在文档中，例如，系统中安装的硬盘信息。Di. pl 是一个 Perl 脚本，使用 WMI 机制获取系统中的逻辑驱动器信息（C:\，D:\等），包括安装的固定驱动器、可移动存储和远程共享信息。Sr. pl 可以获取 Windows XP 中的系统还原点信息（系统还原信息在第 4 章和第 5 章都有涉及）。

来自 Microsoft 的 DevCon[56]可以列出 Windows 系统中连接的设备信息。这是一个代替设备管理器的命令行工具，可以显示可用的设备类以及连接设备的状态。

有关怎样挑选工具

本章以及其他章节会提到用来完成特定任务的各种不同的工具。本书并不想成为工

具列表大全，而且这也不可能实现。本书的主要意图是到哪里查找信息及怎样来收集数据信息，有时仅仅为了说明信息是存在的。

在收集开机系统的信息时，经常必须使用可用的 API 与操作系统交互。不同的工具使用不同的 API 调用收集相同的信息。

如果知道工具是怎样收集信息的则很有好处。执行文件使用了什么 API？访问了哪个动态链接库？数据怎样显示，以及不同工具收集的数据本质上有什么区别？

对工具进行测试以确定其对运行系统的影响。这些工具会在系统中留下痕迹么？如果有，到底是怎样的痕迹？确信将这些痕迹记录在文档中，因为文档可以用来识别（和记录）执行的步骤，减小使用工具的影响，调整这些工具的使用。例如，Windows XP 有程序预读机制，这意味着当调查员运行工具时，有关应用的一些信息（例如，代码页）会记录在位于 WINDIR\Prefetch 目录的一个. pf 文件中。这个目录有 128 个. pf 文件的上限。如果在执行现场调查时该目录中不足 128 个. pf 文件，则执行工具的影响仅限于 Prefetch 目录中会增加这些工具的. pf 文件。在一些特殊情况下，这会引起问题。打个比方，调查时使用了 nc. exe（netcat）工具。如果系统中有人已经使用过 nc. exe 工具，则使用相同的工具名称会覆盖已有的 nc. exe 的. pf 文件。这样就会破坏潜在的证据（例如，文件 MAC 时间戳信息、文件中的数据及执行文件的路径等）。

对使用的工具进行测试和验证是一项费力的工作。毕竟，谁会对每一个工具进行严格的测试呢？调查员也可以参考一些提供这类信息的站点，虽然这些信息不是用来为应急响应和计算机取证准备的。不过，如果已经建立了现场响应的框架（工具、流程等），测试并不一定很艰难，可以做一些简单的工作来对工具进行测试和记录。这类似于分析和测试可疑的恶意程序，第 6 章对恶意程序分析进行详细讨论。对工具的测试和记录包括静态和动态两种，静态测试包括记录工具的惟一标识信息，例如：

- 工具从哪里来的（URL）
- 文件大小
- 使用已知算法得到的文件加密哈希值
- 文件中提取的信息，例如，PE 头、文件版本和输入/输出表等

这些信息可以很容易使用命令行和 Perl 脚本工具得到，整个收集过程可以自动完成（可以将信息记录在数据库、电子表格或文本文件中）。

动态测试包括在运行工具时监控并记录系统发生的更改。类似 InControl 的快照比较工具在动态测试时非常有用，RegMon[57] 和 FileMon[58] 也是。RegMon 和 FileMOn 不仅可以监控工具运行过程中访问了哪些注册表项和文件，还可以监控创建和修改操作。也可以用类似 Wireshark[59] 的抓包程序来监控工具运行时系统进出的网络流量，尤其对从其他动态库中导入了网络功能的工具，动态测试步骤必不可少。

开机取证方法

根据现场情况的不同，执行开机取证操作时，获取系统数据的正确方法和流程会发生变化。作为顾问和紧急响应人员，我发现最好的方式就是对当前手中的工具箱和其他可能的工具（考虑购买软件、许可、其他费用和限制问题）能有一个完整地理解，然后

决定当前情况下能做什么。Windows 开机取证有两种基本的方法：本地和远程。

本地开机取证方法

本地取证意味着坐在系统的控制台前，使用键盘来输入命令，信息也保存在本地硬盘、移动介质（闪存、USB 硬盘）或映射到本地的网络共享驱动器中。这种情况非常普遍，取证人员能立即到达现场，使用光盘或闪存中的工具对系统进行直接物理访问。从数个本地系统中收集数据要比通过网络连接或者无线网络要快。使用正确数量的外部存储设备和访问权限，现场响应人员可以快速高效地收集必需的信息。为使现场工作更加高效，响应人员可以将工具保存在光盘上，编写具有弹性的批处理文件和脚本（例如，USB 连接的存储器具有不同的盘符、Windows 安装在不同的驱动器中等）。

最简单的方法就是将机本地取证实现并贯穿在批处理文件中。批处理文件和脚本的好处是不需要一遍遍地输入相同的命令（总是发生相同的错误），批处理文件只要写一遍就可以无限次的自动运行。一个简单的开机取证使用的批处理文件例子如下所示：

```
tlist.exe -c > %1\tlist-c.log
tlist.exe -t > %1\tlist-t.log
tlist.exe -s > %1\tlist-s.log
openports.exe -fport > %1\openports-fport.log
netstat.exe -ano > %1\netstat-ano.log
```

3 个工具、5 个简单的命令。将这个文件保存为 local.bat 并放到光盘上，并且将相关工具也复制到相同位置。

同时也要将每个操作系统可信的命令行处理程序放在光盘上。在启动批处理文件之前，观察系统并看看有什么网络驱动器，或者插入一个闪存盘看它使用的是哪个盘符（假设为 F:\），然后运行如下批处理文件（此处 D:\是光盘驱动器盘符）：

```
D:\>local.bat F:
```

批处理文件执行完成后，闪存中会有 5 个文件。当然，根据现场获取数据的需要，批处理文件中还可以加入其他命令。

有几个免费可用的本地响应工具包，包括应急响应获取报告（Incident Response Collection Report，IRCR）（第二版）[60]和 Windows 取证工具包（Windows Forensics Toolkit，WFT）[61]，WFT 的作者是 Monty McDougal。虽然两个工具包的实现和输出有所不同，但其基本功能是一致的：用 Windows 批处理文件控制运行外部可执行程序，本地保存输出结果。WFT 可以保存原始数据并可以将命令输出保存为 HTML 报告。

另外一个本地取证的方法是使用 Windows API 将需要的功能集成到一个应用程序中，Agile 风险管理公司的 Nigilant32[62]就是这样的工具。Ngigilant32 使用与外部工具相同的 Windows API 收集系统易变信息（如图 1.14 所示），并且加入了文件系统检查和物理内存导出功能。

很多人拥有批处理风格的工具包。当我到客户公司或者参加会议时，我会和其他有趣的人交谈，比较相互之间的工具。一些人有我的第一本书《Windows 取证和事件恢复》，并且有其中第 5 章列出的工具，或者看过其他工具并集成到工具包中。奇怪的

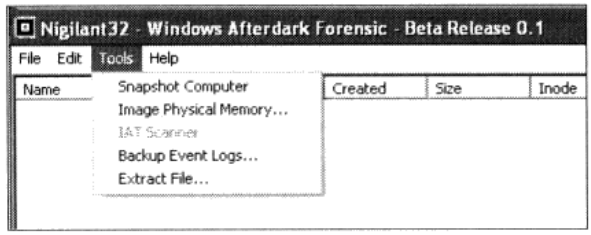


图 1.14 Nigilant 图形界面工具

是，这些工具包有很多重复部分。批处理风格的工具包使用与 Nigilant32 等工具相同的 Windows API。

远程取证方法

远程取证方法通常包括一些列的命令，通过网络获取远程系统的信息。这种方法对很多系统都比较实用，因为登录和运行命令的过程都可以自动执行。在安全圈内部，我们称之为可扩展性。一些工具可以很好地和 SysInternals 的 psexec. exe 配合工作，很多信息也可以通过 Windows 管理机制（WMI）收集。不管使用何种方法，记住（a）调查员需要远程登录权限，（b）每次登录、运行命令并输出信息，安全事件日志中会添加记录（假设相应级别的审计已激活）。还记得提过的易变数据的变化顺序吗，所以远程取证的第一件事就是收集安全事件日志的数据内容。

Windows 批处理文件可以作为构建远程取证方法的基础。通过在命令行中带 3 个参数（远程系统的名称或 IP、登录的用户名和口令），构建一个包括一些列命令的脚本来收集必须的信息。一些命令需要通过 psexec. exe 来执行，这将会复制执行程序到远程系统中，程序将结果输出到标准输出（STDOUT）或者将输出重定向到文件中，就像运行本地命令一样。其他命令需要使用 UNC 路径（系统名称前加 \ \ ）和登录口令做参数，这样就不需要使用 psexec. exe。最后，可以利用 WMI 机制构建 VBScript 或 Perl 脚本来收集数据。Microsoft 提供了使用 WMI 代码的不同例子脚本库[63]，包括 Perl 脚本的例子[64]，使得构建定制工具包的工作异常简单。

实现远程取证方法的批处理文件相当简单：

```
psexec. exe \\ %1 -u %2 -p %3 -c tlist. exe -c > tlist-c. log
psexec. exe \\ %1 -u %2 -p %3 -c tlist. exe -t > tlist-t. log
psexec. exe \\ %1 -u %2 -p %3 -c tlist. exe -s > tlist-s. log
psexec. exe \\ %1 -u %2 -p %3 -c openports. exe -fport > openports-fport. log
psexec. exe \\ %1 -u %2 -p %3 c:\windows\system32\netstat. exe -ano > %1\netstat-ano. log
```

这个批处理文件（remote. bat）可以这样启动：

```
C:\forensics\case007>remote. bat 192. 168. 0. 7 Administrator password
```

批处理执行完成后，调查员在本地系统中获得 5 个命令输出文件，可用来分析。

如果读者对使用 WMI 机制收集远程系统信息感兴趣，而对 VBScript 又不是很熟悉，可以看一下 WMI 命令行调用程序 wmic. exe[65]。Ed Skoudis 在 SANS 因特网安全

风暴中心的 Blog 上写过一篇关于 wmic. exe 的很好的启蒙教程[66]，其中包括收集远程系统安装的补丁信息的例子。通过 wmic. exe 可以使用很多 Win32 类[67]获取相当多的信息。例如，显示本地系统的进程信息，命令如下：

```
C:\>wmic PROCESS GET ProcessId,Name,ExecutablePath
```

这是一个相当简单而直观的命令，命令执行完的结果显示在控制台上。可以将输出重定向到文件中，甚至可以选择输出为其他不同的格式，例如，CSVs（可使用电子表格程序打开或 Perl 脚本分析）和 HTML 表格。使用/Node:，/User:和/Password:参数，可以将 wmic. exe 命令集成到批处理文件中，收集远程系统的一系列信息。更进一步，管理员可以使用这些命令来构建硬件和软件资产列表，确定哪些系统需要更新补丁等。WMI 是管理 Windows 系统的一个很强大的界面，wmic. exe 提供了方便地执行自动化命令的可能性。

代码中加入错误处理、恢复机制和活动日志功能，可以构成高效、可扩展的信息收集方法，从一个中心位置来管理，并且将结果保存在一起。Tchnology Pathways 公司的 ProDiscover（IR）[68]应急响应工具就是采用这种方法实现的商业工具。取证人员可以从中心位置安装代理（Agent），通过代理获取需要的信息，也可以删除代理程序。同时要感谢 ProDiscover 实现了 ProScript API 调用功能，响应人员可以使用 Perl 脚本来实现整个收集过程。这一方法使登录的次数降低到最少，减小了对安全事件日志的影响，同时也减少了需要安装到远程系统软件的数量。ProDiscover IR 具有开机获取的功能，也可以获取整个内存的数据。

远程取证方法的局限在于响应人员必须可以通过网络登录到远程系统。如果 Windows风格的登录被（通过 NetBIOS）禁用（NetBIOS 未安装、防火墙/路由器禁用了相关协议等），这种方法就不可用了。

混合方法

前面提到过有两种基本的响应方法。但还有第三种方法，就是将本地和远程方法进行混合使用，为了简单化，姑且称之为混合方法（事实是，我想不出一个好的名称）。这种方法常用于响应者不能登录到远程系统，但是又需要收集一些系统的信息到中心位置。响应人员（或助理）带着光盘或闪存（理想情况下是带有写保护开关）到远程系统控制台运行工具收集信息。利用工具执行时，每个工具会将输出结果通过网络传输到中心“取证服务器”。这种方式中，不需要进行远程登录，工具从不可修改的可行介质运行，在“被调查”系统的硬盘中很少写入数据[69]。通过良好的计划和流程约定，响应人员可以将其和系统的交互降低到最小，减小使用过程中选择和输入命令的数量，从而降低出错的机会。

也许实现混合方法最简单的方式就是批处理文件。我们已经看到通过各种工具收集信息的方法。其中最多的就是本地取证方法所采用的方式，使用命令行工具并将输出结果重定向到文件。那么怎样将系统中收集到的信息发送到中心位置？一种方式就是使用 netcat[70]，它被称为“TCP/IP 瑞士军刀”，因为可以在各种情况下使用这一工具。在我们的例子中，不需要对 netcat 做功能详尽地描述，只需要将信息从一个系统传送到

另外一个系统。首先，需要在取证服务器上构建一个“侦听者”，使用如下的命令行：

```
D:\forensics>nc -L -p 80 > case007.txt
```

这个命令行告诉 netcat（nc.exe）在端口 80 侦听（很难，需要在连接关闭时继续侦听），任何从该端口传入的信息将被发送（重定向）到 case007.txt 文件中。设置完成以后，就可以修改批处理文件，使得所有的输出通过 netcat 传送到取证服务器的“侦听者”：

```
tlist.exe -c | nc %1 %2
tlist.exe -t | nc %1 %2
tlist.exe -s | nc %1 %2
openports.exe -fport | nc %1 %2
netstat.exe -ano | nc %1 %2
```

将这个文件保存为 hybrid.bat，然后如这样从命令行启动（这里 D:\代表光驱驱动器）：

```
D:\>remote.bat 192.168.1.10 80
```

命令行文件执行完成，所有的数据都会安全地从被调查的系统发送到取证服务器来用于分析。

有几个免费工具实现了混合方法。一个是取证服务器项目（Forensic Server Project，FSP），伴随我的第一本书于 2004 年 7 月出版，这几年又有所改进。FSP 是开源的，使用 Perl 编写，任何人都可以使用。FSP 实现的思路来源于 netcat，响应人员需要从被调查机器的光盘上运行工具，并且将输出结果通过 netcat 发送出去。Netcat 负责将信息发送到服务器的侦听程序，而不是将结果显示在屏幕上。这种方式在一些情况下很有效，但是随着命令数量的增长，命令有一系列的参数需要输入，这种方式变得越来越笨重。当更多的命令需要输入时，错误的概率也逐渐升高，有时连批处理文件都不能满足要求。所以我决定启动取证服务器项目，一个用来自动收集、存储、管理开机取证数据的应用框架。

取证服务器（FSP）包括两个组件：服务器和客户端。服务器组件就是 FSP（抱歉我找不到一个好名字）。将 FSP 文件复制到取证工作站（开机取证时我使用笔记本），启动运行，FSP 会侦听网络连接。FSP 还可以处理简单的案件管理任务：日志、存储等。

当收到来自客户端组件的连接信息，FSP 将会按照接收到的不同“动作”或指令进行响应。

当前的客户端组件称为第一响应人员实用工具（First Respoinder Utility，FRU）。FRU 和客户端紧密相关，因为要在被调查的系统上运行，可以使用光盘或 USB 闪存来运行。FRU 是一个简单的框架，里面会用到很多第三方工具，例如，本章前面讨论到的工具。当其中的命令执行时，FRU 会捕捉命令的输出（也就是通常在命令行窗口看到的输出）并发送到 FSP，FSP 会记录输出和执行的命令。FRU 还可以收集特定的注册表键值或者特定注册表键下的所有数据。在所有命令执行完成，数据也收集完成之后，FRU 告诉 FSP 关闭日志文件，FSP 将执行关闭操作。

FRU 通过初始化文件（也就是.ini）文件来控制，类似于 Windows 3.1 的 INI 文件，包括 4 个部分。第一部分是 [Configuration]，有一些默认设置用于 FRU 连接到

FSP——特别需要提到的是服务器的地址和端口。这在小型环境中很有用，或者可用于大型机构的区域办公室，这些设置可以通过命令行覆盖。

第二部分是［Commands］，列出了用于执行、收集信息的外部第三方工具。这些工具可以是任何可以输出到控制台（STDOUT）的 Widnows 可执行程序。我自己也用 Perl 写了一些小工具，然后编译成独立的可执行程序，用于没有安装 Perl 的系统。这些小工具对收集系统信息很有效，同时可以由 FRU 的 INI 来定义。这部分的格式很重要，并且和其他部分有所不同，每一行的定义如下：

```
<index> = <command line>::<filename>
```

Index 是命令运行的次序。例如，可能想要某指令第一个执行，index 就可以用来定义这个顺序。命令行是要执行工具的名称加上需要包括的命令行参数，就像我们在命令行窗口中执行这个程序一样。这两部分使用等号（=）隔开，随后是两个冒号（::）。大部分情况下，最后一个部分使用一个冒号分割就够了，但是一些工具（psloglist.exe）有可能使用冒号作为选项参数，所以这里必须使用两个冒号。最后一个元素是生成的文件名，大部分情况下使用工具名称作为文件名，使用.dat 作为扩展名。当输出发送到 FSP 服务器时，输出会写入指定目录的文件中，文件名的前面还会使用被调查的系统名称作为前缀。这样就可以从多个系统同时采集数据了。

FRU 工具的一个重要评论：基于系统的事实情况，作为调查者，需要与正在运行的系统交互，必须修改第三方工具的名称。一个好的方法是使用特殊的文件名，例如，*f*_或 fru_开头的文件名。这是由于和系统的交互总是会以某种方式记录下来（参见第 4 章），也是因为 Windows XP 的程序预处理机制（参见第 5 章）。还记得诺卡德交换原理吗？将调查中记录的信息与系统中原始信息区分开来是很重要的。

FRU INI 文件中的一行如下所示：

```
6 = openports.exe -fport::openports.dat
```

如果调查人员想要复制被调查系统的文件，可以使用另外一个客户端工具——FCLI，图 1.15 是这个 FCLI 的图形用户界面。

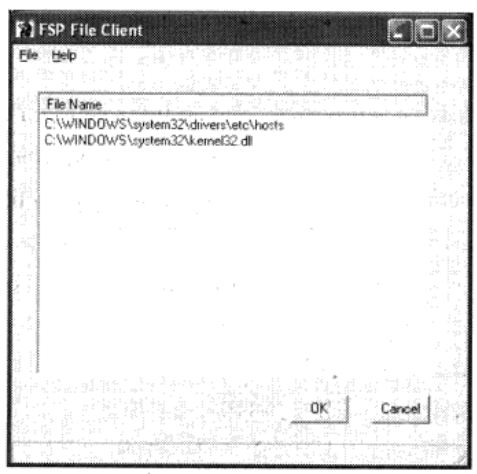


图 1.15　文件复制工具图形用户界面

要使用这个复制工具（FCLI），调查员只要在工具界面上选择 File | Config 来输入 FSP 服务器的 IP 地址和端口。然后选择 File | Open 选择要复制的文件。选择好文件之后，单击 OK 按钮。FCLI 会首先收集文件的 MAC 时间戳信息和其他元信息，计算文件的 MD-5 和 SHA-1 哈希值，发送到 FSP 服务器，随后 FCLI 将文件的二进制数据内容复制到服务器上。完成文件复制操作后，FSP 服务器会计算被复制文件的哈希值，与 FCLI 客户端计算的哈希值做比较。所有操作自动完成，不需要调查人员的介入，并且 FSP 会记录操作日志。

小结

本章讨论开机取证的内容，尤其是针对系统易变信息的获取。其中有很多信息可以加深我们对事件的理解，这些信息在关机获取硬盘镜像之前得到。我们还讨论了计算机中呈现内容可能出现的变化，以及只能采用开机取证方式的各种情形。

所有本章提到的 Perl 脚本都可以在随书附赠的光盘中找到，同时也有一个使用 Perl2Exe 编译的独立可执行版本。用于 Technology Pathways 公司的 Prodiscover 工具的 ProScripts 脚本也在光盘中，但是只有脚本。

光盘中还有几个视频文件，介绍取证服务器（Forensic Server Project，FSP）的安装和使用，包括下载播放器的指南。

参考资料

1. www.vulnwatch.org/netcat/
2. www.ntsecurity.nu/toolbox/pmdump/
3. www.microsoft.com/technet/sysinternals/Miscellaneous/Strings.mspx
4. http://info.sen.ca.gov/pub/01-02/bill/sen/sb_13511400/sb_1386_bill_20020926_chaptered.html
5. http://msdn.microsoft.com/library/default.asp? url =/library/enus/sysinfo/base/file_times.asp
6. www.microsoft.com/technet/sysin-ternals/SystemInformation/PsLoggedOn.mspx
7. www.microsoft.com/technet/sysinternals/Security/LogonSessions.mspx
8. www.symantec.com/avcenter/venc/data/backdoor.subseven.html
9. www.microsoft.com/technet/sysinternals/utilities/psfile.mspx
10. www.microsoft.com/resources/documentation/windows/xp/all/proddocs/enus/openfiles.mspx
11. http://en.wikipedia.org/wiki/NetBIOS
12. http://support.microsoft.com/kb/q163409/
13. http://support.microsoft.com/kb/119495/EN-US/
14. http://support.microsoft.com/? id=837243.

15. http://support.microsoft.com/kb/137984/
16. http://vil.nai.com/vil/content/v_100559.htm
17. http://support.microsoft.com/default.aspx? scid=kb;en-us;236995
18. http://support.microsoft.com/kb/222193/EN-US/
19. www.microsoft.com/whdc/devtools/debugging/default.mspx
20. www.diamondcs.com.au/index.php? page=consolecmdline
21. www.microsoft.com/resources/documentation/windows/xp/all/proddocs/en-us/tasklist.mspx
22. www.sysinternals.com/Utilities/PsList.html
23. www.sysinternals.com/Utilities/ListDlls.html
24. www.sysinternals.com/Utilities/Handle.html
25. http://msdn.microsoft.com/library/default.asp? url=/library/enus/wmisdk/wmi/win32_process.asp
26. http://support.microsoft.com/default.aspx? scid=kb;en-us;314056
27. http://support.microsoft.com/default.aspx? scid=kb;en-us;250320
28. http://securityresponse.symantec.com/avcenter/venc/data/backdoor.xts.html
29. http://securityresponse.symantec.com/avcenter/venc/data/backdoor.litmus.203.b.html
30. http://support.microsoft.com/kb/927229
31. www.microsoft.com/technet/sysinternals/Networking/TcpView.mspx
32. www.foundstone.com/
33. www.diamondcs.com.au/openports/
34. www.insecure.org/nmap/index.html
35. http://support.microsoft.com/kb/314850
36. www.ntsecurity.nu/toolbox/promiscdetect/
37. http://support.microsoft.com/? kbid=892853
38. www.wireshark.org
39. http://support.microsoft.com/default.aspx? scid=KB;EN-US;Q224993&
40. http://unxutils.sourceforge.net/
41. http://msdn.microsoft.com/library/default.asp? url=/library/enus/wmisdk/wmi/win32_service.asp
42. http://support.microsoft.com/kb/182086/EN-US/
43. http://support.microsoft.com/kb/314834/EN-US/
44. http://support.microsoft.com/default.aspx? scid=kb;en-us;555041
45. www.download.microsoft.com/download/2/8/0/2800a518-7ac6-4aac-bd85-74d2c52e1ec6/tuning.doc
46. www.sysinternals.com/Utilities/Autoruns.html
47. www.ntsecurity.nu/toolbox/pstoreview/
48. www.codeproject.com/tools/PSExplorer.asp

49. www.washingtonpost.com/wpdyn/content/article/2006/02/14/AR2006021401342.html
50. www.microsoft.com/windows/ie/using/howto/customizing/autocomplete.mspx
51. www.nirsoft.net/utils/pspv.html
52. www.forensicideas.com/tools.html
53. www.ntsecurity.nu/toolbox/pstoreview/
54. www.sysinternals.com/Utilities/PsLogList.html
55. www.somarsoft.com/
56. http://support.microsoft.com/? kbid=311272
57. www.microsoft.com/technet/sysinternals/SystemInformation/Regmon.mspx
58. www.microsoft.com/technet/sysinternals/SystemInformation/Filemon.mspx
59. www.wireshark.org/
60. http://tools.phantombyte.com/
61. www.foolmoon.net/security/wft/index.html
62. www.agilerm.net/publications_4.html
63. www.microsoft.com/technet/scriptcenter/default.mspx
64. http://isc.sans.org/diary.php? storyid=1622
65. http://msdn.microsoft.com/library/default.asp? url=/library/enus/wmisdk/wmi/wmic.asp
66. http://isc.sans.org/diary.php? storyid=1622
67. http://msdn.microsoft.com/library/default.asp? url=/library/enus/wmisdk/wmi/win32_classes.asp
68. www.techpathways.com/ProDiscoverIR.htm
69. http://en.wikipedia.org/wiki/Netcat

快速解决方案

开机取证

- 诺卡德交换原理表明，两个对象接触时物质会在其间交换。这个原理也适用于数字领域。
- 调查员在运行系统中所做的，甚至不做任何事，都会对系统有影响并留下痕迹。系统在没有用户交互的情况下运行也会留下痕迹。
- 痕迹在该存在的时候不存在，也是一种痕迹。
- 易变信息的顺序告诉我们，一些数据的“生命周期”比其他数据更加短暂。
- 在应急响应时，大部分易变数据都是必须获取的。
- 开机取证的理由必须彻底理解并记录在文档中。
- 公司的安全策略可以规定调查的第一步为开机取证。

收集什么数据

- 为调查员提供案件线索的很多数据存在于正在运行的系统中，一些数据只在有限

的时间范围内存在。

- √ 很多时候，需要从系统中收集的易变数据依赖于被调查事件的类型。
- √ 获取易变数据时，遵从 RFC 3227 对易变数据次序的规定和诺卡德原理的理解。
- √ 收集易变数据及在调查中使用这些数据的关键是详尽的文档。

非易变信息

- √ 非易变信息（例如，系统设置）有可能会影响调查的结果，所以也必须在开机取证的过程中收集。
- √ 一些非易变数据会影响随后开机取证的决定，同时也会影响随后调查和后期分析的决定。
- √ 开机取证过程中要收集的非易变信息取决于网络结构、安全和事件响应策略，以及系统配置。

开机取证方法

- √ 有 3 种基本的开机取证方法：本地、远程和两者的综合。了解现场情况和实现这些方法的不同会增大收集信息的弹性。
- √ 现场采用的方法依赖于网络结构、部署选项，甚至组织的架构设置，不过可以根据现场情况使用不同的方法。
- √ 在选择取证方法时，注意操作会在系统中留下痕迹，并且是对系统的直接影响，如导致注册表键增加、文件增加或修改、可执行文件加载到内存中。不过这些改变是有限度的、可以区分的，过程中必须明确记录采用的方法和操作。

常见问题

下面的这些问答用来测试读者是否已经完全理解本章的内容，并可以帮助读者在实际案件调查过程中应用本章知识。如果还有其他问题，浏览 http://www.syngress.com/solutions 并单击 Ask the Author 一栏。

问：何时进行开机取证？

答：这个问题没有硬性标准。不过，越来越多的规范（例如，SEC，HIPAA，FISMA，Visa PCI 等）都对安全方法、安全机制提出了明确规定，并且随着安全事件问题（用户敏感信息问题）解决规范的出台，开机取证显得越来越重要。

问：我参与了一个案件的办理，对方使用了“木马辩护”。开机取证怎样帮助解决这一问题？

答：通过收集系统运行的进程、网络连接信息，以及其他发现关于木马和后门的痕迹，可以确定系统开机时是否有这类工具在运行。后期的调查包括对文件系统进行分析、调度任务等类似分析，可以确定木马被安装的可能性。但是收集开机系统的易变信息可以确定当时是否有木马正在运行。

问：我现在没有做开机取证。为什么要做呢？

答：通常组织会倾向于选择“删除-重装”方式，此时管理员会擦除可疑系统的硬盘，从干净的介质重新安装操作系统，重装应用，然后从备份系统加载数据。这可能被认为是最简便的方法，不过这种方法不能在第一时间确定事件发生的原因。也许有人会说：“我重装了操作系统，也打了补丁”。很好，不过事件不仅仅是由于系统没有打补丁。系统、应用程序的一个弱口令也会导致系统被入侵，没有补丁可以解决这类问题。如果不确定事件发生的原因和解决办法，那么事件还会再发生，就在干净的新系统被接入网络之后，而且本章内容已经揭示：大量有价值的信息存在于开机运行的系统中——物理内存、运行进程、网络连接和剪贴板内容等——这些都可能会对调查产生巨大的影响。

第 2 章
开机取证：数据分析

本章主要内容：

- ■ 数据分析

- √ 小结
- √ 快速解决方案
- √ 常见问题

引言

现在已经从系统中提取出一些易失证据，但问题是怎么才能“听懂”这些易失证据“说出”了什么呢？在拿到系统进程列表之后，怎么才能确定哪个进程是恶意程序呢？又该怎么判断系统是否已经被入侵，攻击者是否还连在被入侵计算机上呢？这些问题最终归结为怎样使用易失证据重现犯罪活动发生的过程，特别是在对获取的硬盘镜像进行后期分析的时候。

本章正是为了解决上述这些问题。要从搜集到的易失证据中找什么、挖掘怎样的线索，依赖于所调查案件的实际情况。怎样在数据的海洋中找到想要的信息？在本章里，并不奢求能在这么短的篇幅中回答所有的问题。希望能够提供足够的理论和实例，建立分析易失数据的思路。这样在遇到本章没有涉及的情况时，也能根据这一思路自行制定行之有效的分析方案。也许在本章之后，读者会对搜集和分析易失证据具有更深刻地理解。

数据分析

很多文章提到，为了排错和解决问题应该在计算机关闭之前收集哪些数据。例如，e-Evidence Info 网站[1]，这是一个每月更新的站点，其中包括最新的取证领域相关的会议报告、论文和文章链接，也包括易失信息搜集方面的内容。尽管有很多文章谈到了易失信息的收集，却不能解决数据信息关联和分析方面的问题。本章着重解决这些问题。

首先需要查看工具的输出，看得到了哪些数据。在使用第 1 章中讨论的工具时，得到的是某一时间点系统状态的快照。许多情况下，可以直接从某个工具的输出结果中判断问题出在哪里。例如，可能会从 Windows 任务管理器或 tlist. exe 的输出结果中发现一些“异样”的进程（包括执行文件路径和命令行参数）。对于一个熟悉 Windows 操作系统，特别是熟悉 Windows 中各个“正常”进程的调查人员来说，这些信息足以让他立即找出问题所在。

提 示

Microsoft 知识库序列号为 263 201[2]的文章给出了 Windows 2000 系统中默认进程的相关信息。

可惜，许多调查人员甚至系统管理员对系统还不够熟悉，不能一眼看出哪些是系统默认/正常进程。这也有 Windows 操作系统各个版本之间（Windows 2000，XP，2003 及 Vista 等）区别甚大的原因。例如，Windwos 2000 中的默认进程就与 Windows XP 中的不同，这还是指默认安装的干净操作系统，没有考虑应用程序的情况。另外，不同的硬件配置还会安装不同的驱动程序和应用。还能举出一些变化的因素，关键是要明

白：判别一个进程是不是合法进程，需要综合考虑很多因素。所以最重要的是建立分析数据的流程，以解决调查中出现的问题，这点很重要。流程规定应该先做什么、后做什么的基本框架，然后才能根据案件的实际情况决定，是不是要略过其中的一些步骤或者增加一些检查项目。如果没有流程，就很难发现分析过程中出现的问题，也很难改进和提高。

最好的方法可能就是马上着手分析。在搜集和分析易失证据时，明确查找目标会很有帮助。计算机应急事件发生时，IT 管理人员和应急响应人员面临的最大问题之一就是：如何根据现有的信息去追溯问题的根源。一个例子就是网络入侵检测系统报警或者防火墙日志中出现异常记录，很多时候这是恶意软件（例如，蠕虫）感染的症状。通常，日志或报警信息中包含源和目标的 IP 地址，以及端口信息。通过源 IP 地址可以找到发起网络数据流的系统，而使用有关的端口信息可以确定发出流量的进程，从而找出恶意软件。

警　告

虽然网络上所有的信息流总是由计算机中的进程产生的，但是有些进程的运行时间并不是很长，根据防火墙 4 小时前的日志定位系统中的进程可能不太现实。如果网络数据周期性出现，确保检查所有的可能性。

另一个需要特别注意的问题是，恶意软件作者经常会采用常见名称或管理员熟悉的合法名称来命名他们的程序，调查员使用这些名称在互联网上搜索时，搜索结果会显示这是一个正常的文件，于是线索就这样从指缝中溜走了。

警　告

在调查某公司网络中的蠕虫爆发事件时，我发现部分感染源自于 Windows 系统安装的服务，服务的可执行文件镜像文件名是 alg. exe。使用这个文件名进行搜索，管理员认为这是一个合法应用，名称为应用层网关服务（Application Layer Gateway Service）[3]，该服务注册在 CurrentControlSet \Services 下的 ALG 子键，执行文件指向%SystemRoot%\system32 \alg. exe。但是，我发现的服务是在 Application Layer Gateway Service 子键中（这是第一个问题：键名不对），执行文件镜像的路径指向%SystemRoot%\alg. exe。这个案例说明：仅仅根据可疑文件的文件名去判断，很容易落入恶意软件作者设下的陷阱。甚至一些经验丰富的恶意软件分析人员也不能幸免于此。

为了让读者更好地理解上述这些理论知识，现在来看几个实际的例子。

案例一

这个案例场景出现过无数次，就是管理员得到报告说系统中发现可疑的或者反常的活动。这种反常的活动可能是用户/服务器管理员发现系统中存在一些奇怪的文件，而

且这些文件无法删除。

这种情况下，调查人员可能无法使用传统的后期分析方法（时间或业务不允许），而需要快速响应并解决问题。通常也就是进行开机取证，这种取证要求获取系统当前状态信息，快速分析，找出问题的根源。从开机系统中获取信息的过程是可以重复的，但是信息本身是不会重现的，因为运行系统一直处于变化之中。

系统中发生的任何事情，都是系统中某些进程运行的结果。虽然这句话“对大部分人来说，显而易见”（我读研究生时，教授在讲解六阶微分方程时经常这么说），但在应急响应过程中，有些调查员会迫于压力把它忘在脑后。在调查时必须要记住：系统中发生的任何事，总是进程或线程执行的结果。

提 示

在“使用 Windows 内存分析方法揭开 Rootkit 迷雾”一文中，Jesse Kornblum[4]指出：Rootkit 和其他恶意软件一样，需要运行或者被执行。理解这点是开机取证的关键。

那么调查人员应该如何找出系统中的可疑进程呢？答案是通过开机取证的数据获取和分析。信不信由你，不只一次有客户拿着系统中硬盘（或者硬盘镜像）找到我，要我分析系统中运行了哪些进程。事实上，要想知道开机系统中到底发生了什么问题，必须在系统关机之前提取其中的信息。使用第 1 章中讨论的工具，可以及时地获取系统某一时间的状态信息。因为要提取的信息只存在于易失内存中，一旦系统关机，这些信息就无可挽回地丢失了。

在这个案例场景中，出问题的是一台 Windows 2000 系统。该系统是一个运行 IIS 5.0 的内部网服务器，用户在访问服务器上的网页时发现不能得到任何信息，浏览器上看到的总是空白网页。这很奇怪，一般来说总会有一些报错信息吧。于是我在取证工作站（IP 地址是 192.168.1.6）上启动了一个取证服务器（FSP）进程（fspc.exe，见第 1 章的讨论），命令如下：

```
C:\fsp>fspc -c cases -n testcase1 -i "H. Carvey" -v
```

然后我拿出取证光盘，这张光盘上包含第 1 章中讨论的工具和第一时间响应工具（FRU，fruc.exe），开始在出问题的系统中收集相关的信息。在这样的事件中，我的第一原则是最小化对系统的影响（还记得诺卡德交换原理吗），并且尽可能地提高工作效率和缩短取证时间。为做到这点，我找出了这种情况下所需状态信息的最小集合，并且选出了提取信息所需的工具（FRU 使用的 fruc.ini 文件保存在附赠光盘的 ch2\samples 目录中）。fruc.ini 文件的 [Commands] 部分内容如下：

```
1 = psloggedon.exe::psloggedon.dat
2 = netusers.exe -l -h::netusers-lh.dat
3 = tlist.exe -c::tlist-c.dat
4 = tlist.exe -s::tlist-s.dat
5 = tlist.exe -t::tlist-t.dat
```

```
6 = handle.exe -a -u::handle-au.dat
7 = listdlls.exe::listdlls.dat
8 = openports.exe -fport::openports-fport.dat
9 = openports.exe -netstat::openports-netstat.dat
10 = autorunsc.exe -l -d -s -t -w::autorunsc-ldstw.dat
11 = svc.exe::svc.dat 12 = auditpol.exe::auditpol.dat
```

列表中命令将会被顺序执行，这些命令会收集系统中登录的用户（本地和远程登录）、登录历史记录、自动启动项目、进程、网络连接、开放的端口、服务及审计策略等信息。这些命令不仅提供了当时系统状态的完整视图，还包括后续调查活动所需的信息。

根据第 1 章中介绍的内容，FRUC 程序可以有多个配置（ini）文件。在调查员工违反公司使用策略案例时，可以通过新增的配置文件收集剪贴板和系统保护存储中的信息。

将这张光盘放到出问题的 Windows 2000 系统的光驱中，启动命令行程序并输入如下命令：

```
E:\>fruc -s 192.168.1.6 -p 7070 -f fruc.ini -v
```

几秒钟的时间，系统中要收集的易失信息就全部被提取出来，并且通过网络传送到取证工作站上。

本场景提取的数据可以在附赠光盘 ch2\samples 目录中找到，文件名为 testcase1.zip。

回到取证工作站，testcase1 目录中包含 16 个需要的数据文件。使用 FSP 的一个好处是该工具能够自动记录取证过程，fruc.ini 文件中包含提取信息使用的工具和命令行参数。这个文件和使用的工具存在于光盘中，不能被修改。只要光盘被妥善保存，就可以随时检查当时运行的工具（包括版本）及使用的选项参数。testcase1 目录中有一个名为 case.log 的文件，该文件包含 fruc.exe 传送过来的数据信息列表及每个数据文件的 MD-5 和 SHA-1 哈希值。还有一个 case.hash 文件，其中保存了 case.log 文件的 MD-5 和 SHA-1 哈希值。

真正感兴趣的信息包含在剩下的 14 个文件中。在开始分析时，我最关心的一件事是检查系统中有无可疑进程。要发现这点，可以查看 *tlist -c* 命令的输出结果，这条命令会显示系统中每个活动进程的命令行参数。例如，其中可以很快找到 FRUC 进程的信息：

```
1000 FRUC.exe Command Line: fruc -s 192.168.1.6 -p 7070 -f fruc.ini -v
```

该文件中可以看到很多正常的进程，接下来要找的是 IIS 服务器的进程：

```
736 inetinfo.exe Command Line: C:\WINNT\system32\inetsrv\inetinfo.exe
```

这是正常的 IIS 的进程。再往下看几行，另一个进程立刻跳了出来，这个进程太奇怪了，也太可疑了：

```
816 inetinfo.exe Command Line: inetinfo.exe -L -d -p 80 -e c:\winnt\system32\cmd.exe
```

大多数情况下，IIS Web 服务器中只有一个 inetinfo.exe 进程，而这台机器中却有两个。这还不算什么，正常版本的 inetinfo.exe 位于 system32 \ inetsrv 目录，就像那个进程 ID 为 736 的 inetinfo.exe 那样。而进程 ID 为 816 的 inetinfo.exe 是从 System32 目录运行的，并且该进程的参数像是在启动一个 netcat!

要确定 ID 为 816 的进程确实绑定于端口 80（这是用户报告的异常所在），还需要进一步的信息。打开 fruc.ini 中第九个命令（openports.exe -fport）的输出文件：

```
736 inetinfo        -> 443      TCP    C:\WINNT\system32\inetsrv\inetinfo.exe
736 inetinfo        -> 21       TCP    C:\WINNT\system32\inetsrv\inetinfo.exe
736 inetinfo        -> 25       TCP    C:\WINNT\system32\inetsrv\inetinfo.exe
736 inetinfo        -> 1026     TCP    C:\WINNT\system32\inetsrv\inetinfo.exe
816 inetinfo        -> 80       TCP    C:\WINNT\system32\inetinfo.exe
```

通常情况下，应该是进程 ID 为 736 的 IIS 进程绑定端口 80，实际情况却是 ID 为 816 的进程，而且 openports.exe -netstat 命令的输出也证实了这一点：

```
TCP     0.0.0.0:80          0.0.0.0:0          LISTENING     816
```

根据上面的分析，已经确定了 ID 为 816 的进程为可疑进程，该进程导致了用户报告的异常情况。通过查看其他命令的输出，未发现可疑服务，也未发现其在自动启动位置出现。handle.exe 工具的输出显示该进程是以管理员身份运行的，但是它没有打开文件，而且 openports.exe -netstat 命令的输出结果显示系统中没有使用端口 80 的当前连接。现在问题搞清楚了，但还需要查出这个程序是如何进入系统并运行起来的。

案例二

网络环境中，另一种常见的情况是 IDS 报警或防火墙异常日志。大多数情况下，管理员看到的是网络中发出的非正常、可疑数据包。这可能是类似 IRC 僵尸和蠕虫感染的结果，通常在用户浏览 Web 页面时，包含恶意代码的网页会利用系统漏洞感染系统。最初进入系统的是小型下载器，这些下载器会自动从网上下载并安装 IRC 僵尸程序。然后这些 IRC 僵尸程序会和 IRC 服务器建立联系，等待来自主控程序的命令。

警告

IRC僵尸程序会引发严重的安全问题，而且该问题由来已久。由僵尸组成的大军，或称“僵尸网络”，被用于一系列的网络犯罪活动。2006年2月19日发布的华盛顿邮报杂志中[5]，Brian Krebs讲述了一个名为“0x80”的IRC僵尸管理程序的故事。故事明确地告诉世人构造僵尸网络是多么容易及僵尸网络的危害。几个月后，Robert Lemos在安全焦点网站上撰文指出[6]，IRC僵尸正在从客户机-服务器模式向P2P框架迁移，这使得僵尸网络更加难以清除。

对于网络蠕虫来说，一旦蠕虫成功感染系统，就会试图感染网络上的其他系统。蠕虫通常会扫描IP地址段，寻找与当前被感染系统具有相同漏洞（现在的蠕虫会利用多个漏洞来感染）的主机。一些蠕虫在扫描时相当过分，2003年1月出现的“SQL Slammer”蠕虫[7]狂扫互联网，产生的巨大数据流导致整个网络被堵塞，联网的服务器甚至ATM取款机全部瘫痪。

既然说到了拒绝服务攻击，就不得不谈到另一个与此有关的话题。有时系统管理员会收到其他同行的通知说：“嗨！老兄，你管的服务器好像有问题”。这种情况一般是另一台计算机的管理员发现自己的计算机被蠕虫扫描或被拒绝服务攻击，通过分析捕获的网络数据，找到了攻击来源IP地址使用者的相关信息（通常分配的是一个IP地址段，而不是单独的IP地址），接着会试图联系该段网络的管理员。这事很正常。时至2007年，如果系统管理员收到这样的邮件，也不用大惊小怪。

不管系统管理员是怎么知道自己的机器被黑了，响应方式都一样。该问题的难点是只有IP地址和端口信息，管理员需要明确事件的真相。一般来讲，处理这类案件首先要确定系统的物理位置，然后获取和分析系统中的信息。

与上一个案例场景类似，首先在取证工作站上运行FSP，然后将FRUC光盘插入被检查的计算机中，收集系统中的易失信息。

提示

这次搜集到的信息存放于附赠光盘ch2\samples目录下的testcase2.zip压缩包中。

回到取证工作站，查看*tlist -t*命令（该命令以树形结构显示进程树，子进程以缩进的方式列在父进程之下）的输出结果。我们发现980号进程很奇怪：

```
System Process (0)
System (8)
  SMSS.exe (140)
    CSRSS.exe (164)
    WINLOGON.exe (160) NetDDE Agent
      SERVICES.exe (212)
```

```
        svchost.exe (404)
        spoolsv.exe (428)
        svchost.exe (480)
        regsvc.exe (532)
        mstask.exe (556) SYSTEM AGENT COM WINDOW
        snmp.exe (628)
        VMwareService.e (684)
        WinMgmt.exe (600)
        svchost.exe (720)
          wuauclt.exe (1080)
        inetinfo.exe (736)
        svchost.exe (1192)
      LSASS.exe (224)
  explorer.exe (520) Program Manager
    VMwareTray.exe (1232)
    VMwareUser.exe (1256)
    WZQKPICK.exe (1268) About WinZip Quick Pick
    CMD.exe (812) Command Prompt - svchost 192.168.1.28 80
      svchost.exe (980)
```

有必要解释为什么“980”这个进程看上去很奇怪，在默认安装的 Windows 2000 系统中一般应该只有两个 svchost.exe 进程。

提 示

Microsoft 知识库文章 Q250 320[8]提供了 Windows 2000 系统中 svchost.exe 进程的描述信息（Q314 056[9]号文章给出了 Windows XP 下 svchost.exe 进程的相关信息）。文章中 tlist -s 示例输出不仅显示了运行的两个 svchost.exe，同时还指出了相关的注册表键。Q263 201[10]给出了 Windows 2000 下默认进程的列表。

tlist -t 命令的输出结果显示了这个多出来的 svchost.exe 进程，而且与其他进程不同的是，该进程是从命令行窗口运行的，而不是由 services.exe 运行。

再来看看 *tlist -c* 命令的输出，980 号进程启动的命令行参数如下：

```
980 svchost.exe
    Command Line: svchost 192.168.1.28 80
```

openports.exe -fport 命令的输出显示 980 号进程正在使用一个客户端口：

```
980     svchost          ->     1103     TCP     C:\WINNT\system\svchost.exe
```

openports.exe -netstat 命令显示 980 号进程与远端系统的端口 80 建立了活动连接：

```
TCP     192.168.1.22:1103          192.168.1.28:80          ESTABLISHED     980
```

根据上面收集到的信息，此时应该安装一个网络嗅探器（如 WireShark[11]），分析这两台计算机之间到底在传些什么数据。另外根据收集到的其他易失信息，980 进程并没有打开什么文件（通过 handle. exe 的输出），同时也没有发现其他可疑进程。

既然讨论的是系统中的进程，那么了解一些有关进程创建和进程之间关系的知识总是有益的。以刚才查看的 *tlist -t* 命令输出为例，大部分系统进程源自于 System 进程（Windows 2000 中进程 ID 是 8，Windows XP 中 ID 为 4），而大多数用户进程源自 Explorer. exe，也就是系统命令解释器（Shell），或者叫程序管理器。一般来说（不排除特定情况），System 进程是 Service. exe 的父进程，而 Service. exe 又是 svchost. exe 等很多服务的父进程。在用户进程方面，命令行程序（cmd. exe）是 Explorer. exe 的子进程，而命令行中运行的命令（如 *tlist -t*），又是 cmd. exe 的子进程。

这对开机取证有何意义呢？回头再看 *tlist -t* 命令的输出，可以看到进程标识为 980 的 svchost. exe 是 cmd. exe 的子进程，而 cmd. exe 是 Explorer. exe 子进程…… 这里是不应该出现 svchost. exe 系统服务的。

好吧，再深入一步……要是 980 号进程被注册成系统服务会怎么样呢？虽然此时 *tlist -t* 的输出不会发现异常，但还是可以通过 *tlist -c* 命令的输出看出端倪，该命令可以列出各进程启动的命令行参数。由于 Windows 文件保护机制（WFP），这个恶意的 svchost. exe 进程肯定不能从 system32 目录启动。Windows 文件保护机制是从 Windows 2000 开始使用的、对关键的系统文件及其他重要文件进行保护的一套机制。当受到 WFP 保护的文件遭到修改时，WFP 就会被唤醒，并且立即用缓存中该文件的备份把被修改过的文件替换回来（同时会在系统日志中留下记录）。Windows 2000 中的 WFP 曾经可以被绕过，不过好在这些漏洞已经被修补。在 WFP 有效的情况下，被修改的 svchost. exe 文件是不可能存在于 system 32 目录中的。它可能会从Windows \ system 目录或 temp 目录运行，这很容易暴露其恶意性。

敏捷分析

不进行任何开机取证的一个最大借口是没有办法从获取的大量数据中找出问题的根源。开机取证中获取易失（非易失）数据的大部分工具会产生大量的数据，调查员会对这一大堆数据感到无从下手。从上面两个例子来看，在查找产生问题的原因时并不需要收集很多的数据。例子中使用工具的选择主要考虑了两个简单要素：①恶意软件首先要运行才能对系统产生影响；②恶意软件必须常驻于系统才能对系统产生持续的影响（例如，恶意软件都想在系统重启和用户重新登录后保持运行）。这两点也是筛选数据和定位问题的基本规则。为了能够快速、敏捷地分析，必须进行自动化的数据精简。

虽然上面的两个例子都很简单，但是它们仍然指出了开机取证的要点。两个案例中定位可疑进程的方法和 2002 年调查 russiantopz 僵尸程序[12]的方法基本一样。实际上，这个方法和差异分析方法也是同源的（查找两个状态之间的差异）。但是对于司法取证或安全顾问来说，有一点要牢记：在绝大多数案件中，事件之前

的系统原始基准状态并不存在，调查员必须依靠对操作系统和应用的理解来作为基准。例如，在第一个案例中，如果系统中只有一个 inetinfo. exe 进程，而调查员也不知道是否被感染的系统运行了 Web 服务，他就要根据所掌握的知识（例如，inetinfo. exe 进程运行信息）与 svc. exe 的输出结果做比较，这个例子中的信息如下：

```
736,W3SVC,World Wide Web Publishing
Service,C:\WINNT\system32\inetsrv\inetinfo.exe,Running,Auto,Share Process,#
```

这种相关分析可以通过脚本自动化完成，如果合法服务（如前所示）对应一个合法的进程（736 号 inetinfo. exe 进程），就可以做数据精简。

注 释

样例中收集系统服务信息的 svc. exe 工具会以逗号分割的文本格式（. csv）显示信息，所以结果很容易使用电子表格程序打开和分析。输出信息的各个列分别为进程 ID、服务名、服务显示名称、可执行文件路径、服务状态、服务启动模式、服务类型及是否有一个描述该服务的字符串（有用“#”表示，无用“*”表示）。大多数情况下，恶意软件没有服务描述字符串，这就需要进一步调查了。

对有经验的调查人员来说，inetinfo. exe 进程没有关联的 W3SVC 描述字符串是一个明显的恶意软件标志，或者至少需要对该进程进行仔细地检查。

不过调查员也应该知道，inetinfo. exe 还能提供 MS FTP 服务和 SMTP 邮件服务，如下所示：

```
736 inetinfo.exe Svcs: IISADMIN,MSFTPSVC,SMTPSVC,W3SVC
```

再强调一遍，要是发现 inetinfo. exe 进程没有提供相应的服务，则该进程非常可疑。另外，这个检查可以自动完成。例如，如果 FRUC 工具的输出结果经过分析以后导入到数据库中，就可以使用 SQL 语句进行数据提取和相关分析。

提 示

在 2007 年 BlackHat 大会上，Kevin Mandia 在演讲中指出：根据他所在的公司近几年进行的大量计算机安全事件响应的经验，现在越来越多的恶意软件作者倾向于把编写的恶意软件注册成系统服务，以便常驻于系统。我自己的经验也显示了这一点。

上面的例子表明，在对系统有了足够的了解之后，通过使用自动分析和数据精简的方法，可以快速而准确地找出问题产生的原因。自动化的过程是很重要的，因为对于计算机应急响应来说，其特点之一就是“时间紧，任务急”，这种状况下是最容易犯错误的。自动化则可以将过程用编码的方式固定下来，并且可以重复使用。只要我们对描述系统状态的易失信息足够了解，就可以快速收集信息，对信息

进行相关分析，进而确定事件的性质及其影响的范围。这构成了敏捷响应的基础，也就是使用文档的过程、彻底的方法，快速响应。如果需要，还可以获取更多易失数据。使用这种最小化方法，预先精简需要分析和关联的数据，能够获得更好的效果。

至于应该如何分析和自动化处理，这点依赖于调查人员的经验及对系统和应用程序内部工作原理的认识和了解。

提示

几年前，我的一个朋友总会把他在事件中获取的易失信息发送给我。他使用批处理程序和一些列工具来获取数据，并且是在案件完成之后很长时间才会将原始数据文件发给我，请我找出其中可能存在的问题。在对原始系统状态和特性一无所知的情况下，我必须从他给我的数据中寻找线索。这花了我不少时间和精力，不过由此我也磨炼了分析技巧甚至开发出一些相关分析的工具。

有些精简数据和分析的规则可以加入到固定的操作流程中，甚至可以写成脚本以提高效率。其中一个例子就是 svchost. exe 进程。一些恶意软件作者利用系统中总是存在多个 svchost. exe 进程（根据我的经验，Windows 2000 中有两个，Windows XP SP2 里有 5 个，Windows 2003 里有 7 个）的情况，将恶意软件的名字设成一样（如赛门铁克公司定义的 Ranky 后门程序[13]）。但是我们知道正常的 svchost. exe 有一些基本规律，其中一个就是执行程序位于 system32 目录。因而，就可以编写一个 Perl 的脚本检查 *tlist -c* 命令的输出，自动标记那些可执行文件不是位于 systm32 目录中的 svchost. exe 进程。

这种方法是“人工忽略”（该术语来自 Marcus Ranum[14]）分析方法的一个变体。人工忽略方法的基本原理就是剔除所有已知正常内容，剩下的就是要找的东西。关于该方法的详细介绍请参见第 5 章中“事件日志”部分。

敏捷分析和快速响应的关键就在于精简所要调查的数据量。这可以是将获取的数据转换成一种更易于分析的形式，或者是去掉已知是正常的信息，从而减少实际调查需要的数据量。

扩大范围

要是案件实际情况比刚才看到的场景更为复杂该怎么办呢？我们经常可以在传媒上看到安全专家说网络犯罪正变得越来越复杂（事实也确实如此）。那么应该怎么处理更为复杂的情况呢？毕竟，事件中并非所有的恶意进程都会一直运行在系统中的。例如，利用浏览器漏洞进入系统的下载器程序，在下载完需要的目标软件之后就会退出。该进程的信息，包括进程使用的网络连接，也就不再存在。

不久之前，我负责调查的一个事件中包括一个加密的可执行文件，尝试了 20 多个杀毒引擎，都不能识别它，而且还有另外的麻烦，在所有被该神秘文件感染的系统中找不到与其同名的执行进程。对该恶意软件的动态分析（详见第 6 章“动态分析”相关部

分）显示，恶意软件在注入 IE 进程之后就退出了。这也是我们找不到与神秘文件同名运行进程的原因，于是所有的精力都放在对 IE 的调查上。我们对所有系统中提取到的易失数据进行了梳理，证实了这点。这些系统在未运行 IE 窗口程序的情况下，都有一个 iexplore. exe 进程在运行，并且在向因特网发送数据包。

这个特殊案例的有趣之处不在于恶意软件所使用的代码注入技术，也不在于那么多的杀毒引擎都不能识别这个恶意软件。最有意思的是该恶意软件与几位 SensePost[15]研究人员在 2002 年拉斯维加斯黑客大会上提出的 Setiri 概念蠕虫非常相似。Setiri 会把 IE 当成是一个 COM 服务组件，并且通过 IE 去访问互联网。Dave Roth 编写过一个名为 IEEvents. pl[16]的 Perl 脚本，只要对这个脚本略做修改，就可以采用不可见方式打开 IE（桌面上没有可见的窗口），并且可以访问网页。

上面这个案例说明了什么？我只想指出有些事件会变得如何复杂。利用浏览器漏洞进入系统，通过下载器安装后门，与进程注入相比并不特别复杂。恶意软件作者使用的另一项技术是利用浏览器帮助对象（BHO 的更多信息参见第 4 章），这常见于间谍软件。系统中常见的两个 BHO 分别是 Adobe PDF Reader Link Helper 和 DriveLetterAccess。使用 listdlls. exe 工具可以列出 IE 进程使用的 BHO：

```
C:\Program Files\Adobe\Acrobat 7.0\Activ eX\AcroIEHelper.dll
C:\WINDOWS\System32\DLA\DLASHX_W.DLL
```

要是互联网上的某台 Windows 系统被黑客入侵了，调查时应该能够发现登录的痕迹（在安全事件日志或用户最后登录时间中），以及打开文件或运行进程的痕迹等。如果黑客没有使用 Microsoft 登录机制（远程桌面、net use 等），而是通过后门访问系统，那就应该能够看到运行的进程、打开的句柄和网络连接等信息。

如果对事件性质有所了解，可以针对问题情况，更快的进行开机取证，不仅在数据获取的时候提高效率，也可以在数据相关分析时加快进度。

应对

很多时候，在确认事件之后立即面临的问题就是："现在该怎么办?"遗憾的是并没有确定的答案，这取决于网络和信息系统结构。例如，在案例场景中，我们发现恶意进程以系统管理员身份运行。虽然这是设置的场景，但实际事件中发现进程以管理员身份甚至是 System 权限运行并不稀奇。此时，流行的看法是已经不能再信任系统中的任何信息（例如，不能再相信刚才工具搜集到的易失信息），惟一可接受的做法就是擦除硬盘，从干净的介质重装系统，重新加载备份的数据。

对我来说，这一切太麻烦了，尤其是可能在不久之后又得重来一遍。你可能会想，凭什么啊？好吧，现在的情况是用 pslist. exe 工具找到了恶意进程，而且发现相对于系统开机运行时间来说，恶意进程的运行时间并不算很长。这说明进程是在系统启动之后运行的。例如，我现在正在写本章内容，我的系统已经运行了 8 个多小时。这一点可以通过 pslist. exe 输出的最右一列"经过的时间"信息轻易看出：

```
smss   1024   11      3   21   168      0:00:00.062      8:28:38.109
csrss  1072   13     13   555  1776     0:00:26.203      8:28:36.546
```

但是那些在系统启动之后，用户打开的进程如下：

```
uedit32    940    8    1    88    4888    0:00:03.703    4:07:25.296
cmd        3232   8    1    32    2008    0:00:00.046    3:26:46.640
```

虽然黑客可以修改硬盘上文件的MAC时间戳误导调查人员，但是进程运行的时间信息是很难伪造的。根据这个信息，调查员可以建立事件发生的时间线，评估攻击所造成的影响（类似于前面案例使用的方法）。最终目标是确定事件的根源，避免以后再发生类似的情况，随后还要检查其他相关系统。否则，如果只是简单的重装系统，把存在同样问题的计算机放在网上，过不了多久系统又会被入侵。如果是由于补丁问题，打上补丁就好了。但如果是系统管理员使用了弱口令或者是应用配置问题，打再多的补丁也没有用。

再来考虑另一种情况，恶意软件被注册成系统服务，pslist.exe显示进程运行时间和系统启动时间一样长。因为没有Windows API可以更改注册表键的最后写入（Last-Write）时间，所以可以提取有关服务注册表键的最后写入时间来判定服务安装的时间。另外，有经验的调查人员都知道安装服务需要管理员权限，所以检查该时间前后的用户登录和用户活动情况，可能会发现事件根源所在。

再强调一遍：找出事件的根本原因才能修正问题，不仅是对被入侵的系统，对其他系统也是如此。

警 告

Microsoft知识库文章Q328 691[17]“MIRC木马相关的攻击检测和修复”在“攻击方向”一节是这么说的：“当前分析结果表明，攻击者是利用系统管理员账号的弱口令进入系统的。本次攻击中，Microsoft没有找到任何使用未知安全漏洞的迹象。”要是系统配置保持不变，重装系统、应用和数据，只会导致再度入侵。公司环境中，公用的管理员账户常常使用容易记住的口令，事件后重装系统还会使用同样的用户名和口令。

确定事件发生的根本原因似乎是不可能完成的任务，但是调查人员如果有扎实的系统知识，能够选择正确的检查技术，并且愿意仔细研读本书，这个任务会变得相当容易。

防范

IT部门的员工在事件响应过程中（IT人员通常是第一时间响应人员）可以按照系统加固指南和配置管理流程来处理系统，而不应该简单地重装操作系统和应用程序。例如，仅仅保留系统运行必须的服务和进程，这样可以有效减少攻击者的目标。对必须运行的服务，加强其安全配置。如果系统是Web服务器，并不需要FTP服务，就应该禁用或卸载相关组件，甚至开始就不应该安装。接着，应该配置Web服务器，仅保留必要的脚本映射[18]（2001年红色代码[19]蠕虫事件中，禁用了.ida脚本映射的服务器是不会受感染的）。另外，还可以考虑安装URLScan[20]防护工具。

最小化权限原则也适用于系统用户设置。只有必要的用户才应当被授予适当的访问系统的权限。如果某个用户不必访问系统，不必从控制台登录也不必远程访问，系统中就不应该有他的账号。在我处理过的几个网络入侵案件中，就是由于采用弱口令的老的用户账号为黑客打开了入侵之门。另一个案例中，被入侵计算机上的日志记录显示，当有人使用某个账号登录系统时，账号的真正主人还正在 33 000 英尺高空的飞机上呢，而且该用户很少访问出问题的系统，他的账号实际上处于一种无人监管的状态。

通过减少系统上的攻击目标，也就加大了黑客获得系统访问权限的难度（可以提高到很难），黑客也就不能破坏系统中的数据，或者通过系统来攻击别人。攻击者的试探行为可能会在加固的系统中留下很多痕迹，这些痕迹也是对管理员的提醒，久攻不下攻击者只能放弃。相比之下，我个人更喜欢分析未成功的攻击尝试留下的大量日志（在 Nimda[21] 蠕虫流行时），而不是由于缺少加固和监控措施不断被入侵的系统。就算是一台使用上述手段配置的计算机被黑掉了，上面也会有更多的数据可供分析。

小结

在开机取证获取数据之后，接下来就是分析数据并提供及时有效的反应。许多时候，调查人员会感觉被淹没在易失数据的海洋中，如果不知道要找些什么则事情会更糟。根据案件的性质，调查人员可以做出一些初步的判断，进而可以把正常的数据（如已知是正常的进程、网络连接、用户活动等）从要分析的数据中排除掉，接下来调查人员还可以进行自动的数据关联，进一步减少需要分析的数据，从而减少可能产生的失误。

所有这一切都是为了应急响应工作更加及时、准确和有效。

参考资料

1. www. e-evidence. info
2. http://support. microsoft. com/? kbid=263201
3. www. microsoft. com/technet/prodtechnol/windowsserver2003/technologies/management/svrxpser_7. mspx#E5C
4. http://en. wikipedia. org/wiki/Jesse_Kornblum
5. www. washingtonpost. com/wpdyn/content/article/2006/02/14/AR2006021401342. html
6. www. securityfocus. com/news/11390
7. www. cert. org/advisories/CA-2003-04. html
8. support. microsoft. com/? kbid=250320
9. http://support. microsoft. com/kb/314056/EN-US/
10. http://support. microsoft. com/? kbid=263201
11. http://www. wireshark. org/
12. www. securityfocus. com/infocus/1618
13. www. symantec. com/security_response/writeup. jsp? docid = 2006-081415-2212-

99&tabid=2
14. www.ranum.com/
15. www.sensepost.com/research_conferences.html
16. www.roth.net/perl/scripts/
17. http://support.microsoft.com/default.aspx?scid=kb;en-us;328691
18. www.microsoft.com/technet/prodtechnol/acs/proddocs/accrsc_iisscr.mspx?mfr=true
19. www.cert.org/advisories/CA-2001-19.html
20. www.microsoft.com/technet/security/tools/urlscan.mspx
21. www.cert.org/advisories/CA-2001-26.html

快速解决方案

数据分析

- √ 开机取证的特点之一就是“时间紧、任务重”。调查人员使用数据精简和自动化技术才能有效地完成响应工作。
- √ 在数据收集和分析工作完成之后，对事件的最终响应可能取决于非技术因素，如商业环境、政治环境等。
- √ 找出事件发生的根本原因是节省时间和金钱的最好方法。
- √ 使用“最小化权限”原则配置系统可以有效防止事件的发生。至少，会使得系统更加难以入侵并可以得到攻击尝试的报警信息。

常见问题

下面的这些问答用来测试读者是否已经完全理解本章内容，并可以帮助读者在实际案件调查过程中应用本章知识。如果还有其他问题，浏览 http://www.syngress.com/solutions 并单击 Ask the Author 一栏。

问：进程和系统服务之间有何区别？

答：从开机取证的角度来看，这两者之间并没有太大的差别，只是两者的启动方式及所处的用户上下文环境不同而已。Windows 系统的服务实际上也是进程，并且可以随系统启动自动运行。进程作为服务启动时，经常具有 System 级别的权限，而对于通过注册表设置自动启动的进程来说，它是运行于用户上下文环境的。

问：我在防火墙日志中看到一些时断时续的非正常流量。流量似乎源自于网络中的一个系统，发送到一个不明系统。看见不明流量之后，我获取了这台计算机中的易失信息，但是没能发现任何相关的活动网络连接，也没能找到使用相应端口的活动进程。6小时之后，防火墙日志中又出现了类似的异常记录。现在该怎么办？

答：在本章案例使用的 fruc.ini 文件中，我使用了 Microsoft/SysInternals 的 autorunsc.exe 工具来收集系统自动运行信息。检查计划任务，有些恶意进程可能会创建子

进程，通过子进程来进行网络通信。

问：我正在调查一起事件，但是在系统中找不到任何有用的线索。

答：有很多时候，Windows 系统中一些“奇怪的”或者“反常的”现象实际上是由于我们不熟悉系统造成的，而不是真有什么入侵事件发生。一些响应人员仅仅由于对系统不熟悉而质疑特定文件和目录的存在。一个真实的案例是，管理员删除了C:\Windows\Prefetch目录中的所有.pf 文件。几天之后，这些文件又神秘地回来了，他感到系统一定有木马或后门。

Chapter 3

第 3 章
Windows 内存分析

本章主要内容：

- 获取物理内存镜像
- 分析物理内存镜像
- 获取进程内存

√ 小结

√ 快速解决方案

√ 常见问题

引言

第 1 章讨论了如何搜集一个正在运行中的 Windows 系统里包含的易失信息。看一下“RFC 3227”给出的易失信息变化的次序，我们发现列表中第一个需要获取的易失信息就是系统物理内存（一般就是指 RAM）。尽管系统中打开的端口、运行的进程等易失信息的提取方法已经广为人知了，但是如何提取、分析物理内存还是比较新的领域。根据公开报道，这一问题的研究大约起步于 2005 年夏季。

现在必须回答的最重要的问题是：“为什么?”。为什么要提取系统物理内存？这样做很重要吗？有什么用？要是没有提取会错过哪些信息？到目前为止，调查人员获取物理内存的目的是想发现更多信息——那些通过传统的硬盘镜像分析手段不能获取的信息，特别是口令。很多程序会提示用户输入口令，口令对话框消失后，则只能在内存中寻找口令了。恶意软件分析人员在对付一些加了很强的壳的恶意软件时也会对系统内存进行分析。恶意软件运行时必须要把自己脱壳加载到内存中。所以对于静态分析时很难脱壳的恶意软件，如果能在系统中运行恶意软件，然后把恶意软件在内存中提取出来，就等于让恶意软件自己脱壳，接下来的分析就会容易许多。最后，rootkit 会隐藏进程、文件、注册表键及网络连接端口。大多数工具都发现不了用 rootkit 隐藏的信息，但是通过分析系统的物理内存可以找到这些痕迹。

当越来越多的调查人员开始把内存视为扩大侦查视野和获取更多证据的一个新的信息源时，内存分析及其与后期取证分析结果的相关分析就会变得更加容易。

内存分析简史

在过去，“分析”内存镜像主要是指运行 strings 或者 grep 程序，搜索内存镜像文件，试图从中发现如登录口令、IP 地址、E-mail 地址之类有助于调查的字符串。这种“分析”方法的缺点在于：很难把找到的信息和某个具体的进程联系起来。例如，找到的某个 IP 地址究竟是与案件有关，还是来自其他进程？那个看起来如登录口令的字符串是怎么留在内存中的？这是黑客用来连接系统中特洛伊木马的口令么，还是用户的聊天数据？

怎么样才能更好地对内存镜像进行分析在一些司法取证相关的论坛上早就成了热门话题。包括我在内的一些研究人员也意识到应该研究系统物理内存的获取和分析框架，并且开发与之相应的工具。

2005 年夏季，数字取证研究工作组（Digital Forensic Research Workshop，DFRWS）[1]发起了一次“内存分析挑战赛”，其目的在于鼓励内存取证技术的公开、研究和工具开发。DFRWS 网站上提供了两个物理内存镜像文件（两个镜像都是使用 Helix[2] 1.6 中改进版的 dd.exe 程序获取的），参加挑战赛的选手要根据 DFRWS 提供的案件背景资料，通过分析这两个内存镜像文件来回答一些问题。最后，Chris Betz、George M. Garner 爵士和 Robert-Jan Mora 组合共同赢得了比赛。提供的分析报告反映出他们对内存取证技术的研究取得了令人满意的成果，并且他们还开发了与之相适应的有效工

具。可惜的是这些工具没有公开发布。

这次挑战赛之后，其他一些研究人员沿着他们的思路，继续研究和工具开发工作。Andreas Schuster[3]在他的英文Blog中公开了部分研究成果及实现工具，同时公布的还有不同Windows版本（主要是Windows 2000和Windows XP）中EPROCESS和ETHREAD结构的格式。Joe Stewart则在TRUMAN Project[4]里提供了一个名为pmodump.pl的Perl脚本工具，该工具可以从物理内存镜像中提取指定进程所使用的内存，这对恶意软件分析很有用。2006年秋，Mariusz Burdach在BlackHat大会[5]上提供了一些关于内存取证方面的信息（一开始是关于Linux系统的，但是后来也支持Windows系统）。此外，Jesse Kornblum也提供了一些关于内存取证方面的新方法，包括如何通过物理内存镜像确定原先的操作系统。2006年夏，内布拉斯加州立大学的研究员Tim Vidas[6]，发布了一个名为pro-cloc.pl的Perl脚本，该脚本可以从内存镜像或者崩溃转储文件中提取运行的进程列表。

获取物理内存镜像

那么，怎样获取系统物理内存镜像呢？有多个方案，这些方案各有优缺点。本节的目的是使读者能真正理解各个解决方案及其技术细节。这样，在现场进行取证时就能结合案情，采用最合适的手段，进行有效的取证。

基于硬件的方案

2004年2月，《数字调查杂志》上刊登了一篇Grand Idea Studio公司[7]研究人员Brian Carrier和Joe Grand的文章——“数字调查中基于硬件的内存获取流程”。文章中，Brian和Joe提出用一种名为Tribble（这个名字可能是来自《星际旅行》中的一种可爱又让人头疼的小动物）的硬件扩展卡来获取系统物理内存的方法。Tribble可以用来把系统的物理内存复制到外接存储设备中。这样调查人员就可以在既不引入任何新代码（不会更改系统内存）也不使用系统中的代码（系统中的代码中可能存在rootkit，不应该被认为是可靠的代码）的情况下完成系统物理内存的获取工作。作者构建了一个Tribble概念设备，设计为可以插入系统总线的PCI扩展卡。还有一些能获取系统物理内存的设备，主要是用于系统硬件调试，也可以把将它们用于取证调查。

如DFRWS 2005内存取证挑战赛中遇到的问题那样，基于软件的内存获取方法的一个缺点是用于获取内存镜像的程序必须装入内存。尤其是在Windows系统中，程序可能（视程序的设计、编译和链接方法而定）使用攻击者替换的系统API或动态链接库。下面讨论硬件的设备获取的优缺点。

优点：Trbbile这类硬件设备的优点在于操作方便、易于理解。使用这种方法获取物理内存镜像不需要引入额外的代码，从而减小获取过程被外界影响的可能性。

缺点：使用硬件镜像系统物理内存方法的最大缺点在于硬件必须事先安装到系统上，所以Tribble设备还不能被广泛的应用。其他可用的设备主要用于硬件调试，但也需要在事前安装。

利用火线接口

利用火线设备/协议的特殊性，使用相应的软件，调查人员可以获取系统的物理内存镜像。火线设备使用直接内存访问（DMA），也就是说，可以不通过 CPU 直接访问系统内存。DMA 设备能更快的读写内存中的数据。调查人员只需要一个包含适当软件并能向火线设备内存的特定区域写指令的控制器，就可以不经过操作系统镜像物理内存，而且速度快、延迟小。

Adam Boileau[8]提供了一个使用 Linux 和 Python 来提取系统物理内存的方法[9]。实现这一方法的软件是运行在 Linux 下的，并且需要/dev/raw1394 设备、Adam 的 pythonraw 1394 库、libraw 1394 库和 Swig（Swig 通过生成封装代码，使得其他语言可以访问 C/C++头文件）的支持。Adam 演示了如何使用工具获取屏幕锁定的 Windows 系统的内存镜像，并且从中分析用户口令，最后成功登录系统。

英国格温特郡的 Jon Evans 警官安装了 Adam 的工具，成功地从 Windows 系统及一些 Linux 系统中获取了系统物理内存镜像。作为他硕士论文的一部分，Jon 发表了一篇介绍文章，大致总结了如何在 Knoppixv 5.01，Gentoo linux 2.6.17 和 BackTrack 等 Linux 发行版上安装和使用 Adam 的工具。在所有必要的（包括 Adam 的工具）工具包都被下载并安装完毕之后，Jon 就在目标系统识别火线接口设备上做手脚，通过使用 Linux 的 romtool 命令加载一个包含 iPod 状态控制信息（Control Status Register，CSR）的数据文件（这个 CSR 文件包含在 Adam 的工具包中），让目标 Windows 系统将火线接口上的 Linux 系统误认为 iPod 设备。这个方法的优缺点分述如下。

优点：现在许多计算机系统的主板上中都集成了火线/IEEE 1394 接口。同时，现有的代码可以在 Linux 和 Mac OS 系统上直接访问目标系统的物理内存。

缺点：Arne Vidstrom 指出了这个方法存在的一些问题[10]，通过火线接口制作物理内存镜像可能会导致系统挂机或者丢失部分内存中的信息。2006 年 10 月，George M. Garner 爵士通过电子邮件列表指出，在一些测试中，使用火线接口方法获取的镜像与使用 George 编写的的软件获取的内存镜像在一些重要的偏移地址具有明显的差异，这个差异只能归结于镜像方法产生的错误，而且使用火线接口方法可能导致目标系统蓝屏死机（Blue Screens of Death，BSoD，下面会进一步讨论），这可能都是由火线接口本身的特性造成的。

崩溃转储

我们都看到过蓝屏死机[11]（BSoD）导致的崩溃转储，事实上，Microsoft 在这方面已经是声名狼藉了。除了提示用户系统出现了严重故障之外，“蓝屏死机”给用户带来的更多是烦恼。但是，如果想要得到 Windows 系统漂亮、完美的内存镜像，那么一次完整的崩溃转储可能是惟一的方法。为什么这么说呢？因为在生成崩溃转储时，系统会被冻结，物理内存中的数据（加上大约 4Kb 的头部信息）会被写入磁盘。这样就完整地保存了系统状态，并且保证从开始进行崩溃转储之时起，系统状态不会被人为地修改。

崩溃转储中的信息对于调查人员来说是非常有用的。首先，崩溃转储实际上就是系

统被即时冻结后的一个快照。我曾经在几个案件中通过分析系统中生成的崩溃转储文件，找到感染或者入侵的大量信息。其次，Microsoft公司提供了一系列可以分析崩溃转储的工具——不光是调试工具[12]，还有基于调试工具的内核内存空间分析工具[13]。

听起来这个主意不错，不是吗？可是崩溃转储会在硬盘写一个约1GB大小的文件，这很可能就会破坏硬盘证据（不能真正减小调查对系统的影响），你现在还认为这是个好主意吗？某种情况下，可能会更有用……也许在某些案件中内存的崩溃转储极其重要，为了得到它，即使往硬盘上写上一堆数据也在所不惜。不过，还有两个问题需要解决：首先，不是所有系统在默认情况下都会生成完整的崩溃转储；其次，默认情况下，Windows不能仅靠输入命令生成崩溃转储。

第一个问题解决起来相对比较简单，参照Microsoft知识库中编号为Q254 649的文章[14]。这篇文章列举了崩溃转储的3种类型：小内存转储（64KB）、内核内存转储和完全崩溃转储。我们需要的是完全崩溃转储，因为它包含完整的物理内存数据。这篇文章中也指出，Windows 2000 Pro和Windows XP会生成小内存转储，所有版本的Windows 2003生成完全崩溃转储。根据我的经验，Windows Vista RC1默认情况下也生成小内存转储。

此外，根据Microsoft知识库编号为Q274 598[15]文章描述，物理内存大于2GB的系统不会生成完全崩溃转储。这是因为，要生成一个如此庞大的内存转储所需要的时间和磁盘空间都太多了（完全崩溃转储大小是物理内存大小+1MB）。

Microsoft知识库编号为Q307 973[16]的文章描述了如何在Windows中配置系统故障和恢复选项。这些选项主要为系统管理员和IT经理提供，用于事件发生之前安装和配置系统，但是有关注册表的设置会为调查分析提供线索。例如，如果系统被配置为（默认或其他原因）生成完全崩溃转储，如果系统管理员报告说看到过系统蓝屏，调查人员就应该想到：可以在系统中找到一个完全崩溃转储文件。

注 释

调查人员在处理崩溃转储文件时要特别小心，特别是对于处理敏感数据但不保存的系统。一些情况下，处理信用卡号码、个人社会安全号码等敏感信息的系统发生了崩溃转储，转储文件中可能就会包含这些信息。虽然编写程序的程序员会特别注意，不让这些敏感信息被写入硬盘，崩溃转储却会将内存中的敏感信息写入硬盘。

那么，假设事先已经把系统配置成在系统出现问题时生成完全内存转储了（这一配置是整个系统配置策略的一部分），现在的问题是，如何在必要时“强迫”系统当机生成一个完全内存转储呢？答案是要设置注册表中的一个键（详见Microsoft知识库编号为Q244 139[17]的文章），在设置完这个键并重启系统（以便使设置生效）之后，按住Ctrl键，同时按Scroll Lock键两次，即可生成一个内存转储。下面谈谈这个方法的优缺点。

优点：技术上来说，使用崩溃转储来获取系统内存镜像也许是惟一正确的方法。这

是因为一旦 KeBugCheck API 函数被调用，整个系统就会被冻结，物理内存中数据就会被写入页交换文件，继而被写入系统分区上的崩溃转储文件。同时 Microsoft 还提供了内核内存空间分析工具[18]（该工具包括引擎、插件和用户界面）等一系列调试工具，用于分析崩溃转储文件。

缺点：某些版本的 Windows 操作系统在默认情况下不会生成完全内存转储（如 Vista RC1。我在 Vista RC1 中安装一个驱动时总有问题，想通过 BSoD 找出问题所在，结果在每次关闭机器之后生成都是小内存转储）。另外，为了通过按键强制 Windows 生成崩溃转储，必须在事件发生之前修改注册表键并重启计算机。即使已经配置好了，崩溃转储还是会在计算机的硬盘上写入一个和物理内存同样大小的文件。如 Microsoft 知识库 Q274 598 号的文章所述的那样，页交换文件至少必须设置为系统物理内存大小+1MB。为了正确获取物理内存，这点必须事先配置好，遗憾的是人们常常会忽略。

提 示

Citrix 网站上的一篇技术支持文章[19]里提供了一种使用 LiveKD. exe[20]和 Microsoft的调试工具包获取完全内核转储的方法。运行 LiveKD. exe 后，输入指令“.dump /f <文件路径>”就能获取转储文件。该文章也指出使用这种方法获取的内存镜像可能会不一致，因为执行内存转储的时候，系统仍在运行。

利用虚拟机

VMware 是一款流行的虚拟机软件（本书大量使用了 VMware WorkStation 5. 5. 2），使用它可以在一台计算机硬件上创建多台虚拟计算机，甚至组建虚拟网络。虚拟机好处很多，例如，可以安装一个客户操作系统并给它拍一个“快照”。在虚拟机中完成各种测试工作，包括安装和监视/分析恶意软件。一旦出现问题，可以返回到快照状态。我曾看到运行在虚拟机中的生产系统。

运行 VMware 会话时，可以“挂起”（suspend）这个会话，也就是暂时“冻结”系统，如图 3. 1 所示。

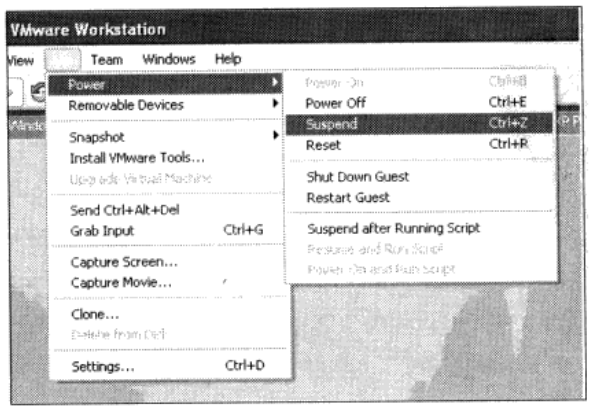


图 3. 1　VMware Workstation 5. 5. 2 中挂起一个会话

当一个VMware会话被挂起时，VMware会把该系统的“物理内存”镜像以DD格式（这种格式下面马上会讲到）存放到一个扩展名为.vmem的文件中。

VMware并不是惟一的虚拟机软件，Microsoft的VirtualPC或者开源的Bochs[21]都是不错的虚拟机软件。这些虚拟机软件在第6章中也会提到。不过我没有测试这些虚拟机生成物理内存镜像的功能。下面谈谈优缺点。

优点：如果可以，挂起一个VMware会话的方式是最简单快捷的，而且对系统内存的影响最小。

缺点：Vmware和其他虚拟技术还没有被广泛使用。

休眠文件

当一个Windows操作系统（Windows 2000及其后续版本）“休眠”的时候，系统的电源管理组件会将系统物理内存压缩后，存放到系统分区根目录下的Hiberfil.sys文件中。该文件的大小足以容纳未经压缩的系统物理内存，但压缩存放可以有效减少磁盘I/O并提高恢复的性能。在系统的引导过程中，如果系统发现有效的Hiberfil.sys文件，NTLDR会将该文件的内容加载到物理内存，并且将系统的控制权交给内核中处理休眠恢复的代码（加载驱动程序等）。该功能常用于笔记本电脑。下面是优缺点。

优点：分析休眠文件的内容，可以得到休眠之前系统中发生的情况。

缺点：休眠文件中的内容是经过压缩的，而且在大多数情况下，休眠文件中不包括当前内存中的数据。

DD

DD是UNIX系统中“数据转储”（data dumper）工具的缩写。DD用途广泛，可以复制文件，甚至可以制作整个硬盘的镜像。DD长期以来一直是标准取证镜像格式，大多数的取证获取/分析软件都支持DD格式。GMG Systems公司发布了一个可以免费下载的取证获取工具包[22]，其中包含可用于Windows系统（支持Windows 2000和XP）的改进版DD工具。在用户模式下访问\Device\PhysicalMemory对象，该工具可以获取Windows系统的物理内存。下面这行命令会将本地系统的物理内存镜像到ram.img文件：

```
D:\tools>dd if=\\.\PhysicalMemory of=ram.img bs=4096 conv=noerror
```

当然，在实际取证工作中，可以将物理内存镜像到移动存储介质或是网络共享目录。这个DD工具还可以生成压缩镜像并生成镜像的加密哈希。由于内存的易变特性，只有在镜像写入磁盘后计算哈希才是可行的。如果用户做了两次内存镜像，即使时间间隔很短，镜像内容和哈希也是不一样的。这种情况下，哈希仅用来保证镜像获取后的数据完整性。

其他工具使用与DD.exe类似的进程获取内存镜像。ProDiscover IR（本书使用4.8版）通过移动介质（CD、闪存盘）或网络发布远程服务器插件，使得调查人员可以利用这些插件远程获取物理内存镜像（包括系统BIOS）。该功能用户界面如图3.2所示。

该方法的一个问题是，Windows 2003 SP1只允许内核态的驱动程序访问\Device\

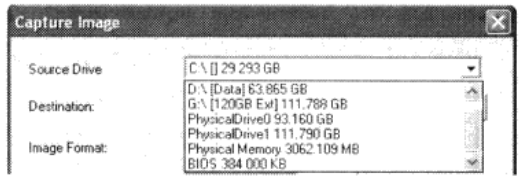


图 3.2 ProDiscover IR 获取内存镜像对话框

PhysicalMemory 对象，用户模式下禁止访问。为解决此问题，GMG System 公司又开发了一个称为 KntDD 的新工具，KntDD 包含在 KnTTools 工具包中。KnTTools 仅授权司法工作人员和安全专业人士使用，KntDD 能够做到：

（1）使用多种方法获取物理内存，包括使用\Device\PhysicalMemory 对象。

（2）可以运行于 Windows 2000 到 Vista 的所有 Microsoft 操作系统，包括 AMD 64 版本的操作系统。

（3）可以将原始内存“镜像”转换成 Microsoft 崩溃转储格式，从而使用其调试工具分析。

（4）可以保存到移动存储介质（USB、火线接口），也可以保存到 TCP/IP 网络。

（5）专为取证用途设计，包括审计日志和完整性校验功能。

KntTools 企业版还包括如下功能：

（1）支持加密镜像的功能，包括使用 X.509 证书、AES-256 加密算法（默认）、Windows 2000 中降为 3DES 算法。

（2）使用 KnTDDSvc 服务程序获取内存镜像

（3）一个可以远程部署 KnTDDSvc 服务的模块，可以通过远程管理共享“推送”或者基于 SSL 的 Web 网站“拉”到远程系统上，在服务执行之前采用加密方式验证程序。

要使用 dd.exe 或者类似的工具获取内存镜像，必须牢记诺卡德交换原理。dd.exe 必须在内存中加载并运行，这会导致其他程序使用的内存空间会被保存到页面交换文件。

另外还要记住，使用 dd.exe 时系统是活跃的，并没有像崩溃转储那样冻结系统。也就是说，在使用工具读取物理内存的过程中内存会改变。例如，正在读取第 30 个内存页时，运行的进程会改变第 11 个内存页中的内容。镜像整个系统物理内存所需的时间是由 CPU 的速度、系统总线的速度及磁盘 I/O 等因素共同决定的。现在面临的问题是：在生成内存镜像这段时间内，会发生足以影响分析结果的改变吗？

下面来看这个方案的优缺点。

优点：绝大多数情况下，dd.exe 可能都是最好的获取物理内存的方法。它不会关闭系统，也不会对存放内存镜像的介质有任何的要求（例如，使用 netcat，通过网络保存到另外一个系统中，而不是保存到本地硬盘）。另外，能够分析内存镜像中进程、网络连接等信息的工具可以免费使用。最后，KnTTools 可以支持 Windows 2003 SP1 等后续版本。

缺点：使用 dd 或者 KnTTools 获取系统物理内存的主要问题在于，在获取内存镜

像时，系统仍然在运行！这不仅意味着工具本身会占用内存，而且会在获取的过程中改变内存，已经读取的内存页面也会发生变化。问题就在于系统没有被及时冻结，这也是它和传统取证中的硬盘复制的差别所在。

尽管如此，使用DD获取物理内存镜像仍然是当前最常用的方法。

分析物理内存镜像

在拿到系统的物理内存镜像之后，该怎么处理呢？2005年夏季前，调查人员在尽力获取内存镜像（通常使用dd.exe工具）之后，标准处理流程是使用strings.exe提取字符串或grep工具搜索（目标是得到E-mail地址、IP地址等），或者两种方法同时使用。虽然使用这种方法确实能发现一些信息（例如，用户名“附近”保存的类似口令的字符串），但是这种方式不能提供信息的上下文环境。例如，字符串是来自Word文档还是文本文件？或者是复制到系统剪贴板中的一段文字？哪个进程在使用这个字符串？

2005年的DFRWS内存分析挑战赛就像一剂催化剂，带动了对内存信息的进一步研究和分析。通过定位特定进程（或内存中的其他对象）及进程使用的内存，调查人员可以更深入地理解其中的信息，并且在分析时忽略正常进程，集中分析“不常见的”进程和数据。一些团队/个人还编写了一些可以从内存镜像中提取进程和其他数据结构详细信息的工具。

本部分接下来的内容都是以分析DFRWS 2005内存分析挑战赛中提供的内存镜像为主线展开的，以此来演示物理内存镜像分析的工具和技巧。大家可能会问：怎么还分析这么老掉牙的东西？Windows 2000不就是个新版的MS-DOS吗？我承认这是事实，但还是认为这是个很好的样本，因为各路高手已经做了很多详细分析示例，同时镜像可以免费下载。

进程基础

这部分主要讨论如何从物理内存镜像中分析与进程相关的信息。这是由于大多数公开的研究和工具都是围绕进程及其相关信息开展的。并不是说内存中其他信息被排除在外，只是大多数研究者的重点是进程。本章的稍后部分也将讨论如何从内存镜像中提取其他信息，现在还是先把注意力集中在进程上。

EProcess结构

Windows操作系统中，“进程”从宏观上讲就是指一个正在执行的程序，而从操作系统的角度讲，就是指一个EProcess数据块。EProcess数据块是一个数据结构，其中维护进程的多个属性，以及指向进程相关的其他一些属性和数据结构的指针（线程、进程环境块）。由于数据结构就是字节序列，序列中有特殊的含义和目的，所以需要调查人员的分析。在对EProcess进行分析时有一点要格外注意：对于不同版本的Windows操作系统来说，EProcess的大小和结构可能不尽相同。甚至对于同一个版本的Windows操作系统来说，如果SP补丁包版本不同，EProcess大小和结构也有可能不同。

Andreas Schuster 在他的 Blog[23] 中公开了对 EProcess 结构的研究结果，从而可以很方便地查看 EProcess 结构信息（以及 Windows 操作系统中其他一些结构信息）。首先，下载 Microsoft 调试工具[24]，同时下载对应于当前操作系统及 SP 补丁包的符号文件库。再到 sysinternals.com 网站上（在浏览器的地址栏输入 sysinternals.com 域名之后，浏览器会自动重定向到 Microsoft 网站，因为 Mark Russinovich 已经去 Microsoft 工作了）下载 liveKD.exe 工具。方便起见，可以把 LiveKD 复制到 Microsoft 调试工具的安装目录下。下载完成之后，打开一个命令行窗口，切换到 Microsoft 调试工具所在的目录，输入下面的命令：

```
D:\debug>livekd -w
```

上面的指令会打开 WinDbg 图形用户界面。要看 EProcess 及所有的子结构体“长什么样”，输入 dt -a -b -v _EPROCESS 这条指令。其中，-a 参数显示数组中每一个成员的当前值，每行一个成员，并且注明该成员在数组中的索引；-b 参数把各个子结构体的各个成员也展开显示；-v 参数显示包括每个子结构体大小在内的更详细信息。有些情况下，配合-r 参数显示递归结构效果会更好。下面给出 Windows 2000 系统上执行上述命令的部分显示结果：

```
kd> dt -a -b -v _EPROCESS
struct _EPROCESS, 94 elements, 0x290 bytes
      + 0x000 Pcb          : struct _KPROCESS, 26 elements, 0x6c bytes
      + 0x000 Header              : struct _DISPATCHER_HEADER, 6 elements, 0x10 bytes
      + 0x000 Type         : UChar
      + 0x001 Absolute     : UChar
      + 0x002 Size         : UChar
      + 0x003 Inserted     : UChar
      + 0x004 SignalState        : Int4B
      + 0x008 WaitListHead       : struct _LIST_ENTRY, 2 elements, 0x8 bytes
      + 0x000 Flink              : Ptr32 to
      + 0x004 Blink              : Ptr32to
      + 0x010 ProfileListHead : struct_LIST_ENTRY,2elements,0x8bytes
      + 0x000 Flink              : Ptr32 to
      + 0x004 Blink : Ptr32to
      + 0x018 DirectoryTableBase : (2 elements) Uint4B
```

完整的调试器输出要长得多（加上头部，整个结构有 0x290 个字节），不过别紧张，本部分只介绍重要的（从取证的角度看）结构元素。

EPROCESS 中最重要的一个成员是指向进程环境块（PEB）的指针。进程环境块中包含大量的信息，对取证比较重要的信息如下：

指向 PPEB_LDR_DATA 结构（其中存放的是进程的加载器使用的数据）的指针，PPEB_LDR_DATA 结构中包含进程中使用的动态链接库的指针。

指向可执行文件镜像加载基地址（ImageBaseAddress，在 PEB 偏移 0x008 处）的指针，通过这个指针可以找到内存中可执行文件的起始位置。

指向包含进程参数结构（ProcessParameters，在PEB偏移0x010处）的指针，该结构包含进程中加载的DLL的路径，可执行文件镜像的原始路径，以及创建进程时传递进来的参数等信息。

如果能从物理内存镜像提取这些信息，对于案件的侦查是很有帮助的。接下来的部分介绍这方面的技巧。

进程创建机制

现在已经有了一些关于进程所使用的结构的初步知识了，这些知识有助于理解操作系统是如何对进程进行管理的，特别是在进程被创建的时候。

一个进程被创建时，系统要执行很多操作，大致可以分为6个阶段：

（1）打开可执行文件镜像。在这一阶段，需要确定该可执行文件需要在哪个子系统中运行（Posix，MS-DOS还是Win 16等）。另外，操作系统还要检查镜像文件执行选项the Image File Execution Options注册表键（详见第4章）是否存在名为Debugger的值，要是有，则可执行文件会被改成该键下的字符串，然后重新开始这一阶段。

（2）创建EProcess对象，在这一阶段会创建内核进程块（KProcess）、进程环境块并初始化进程的地址空间。

（3）创建初始线程。

（4）通知Windows子系统一个新的进程及其主线程已经创建了，同时会传递进程及其主线程的句柄、父进程的ID及该进程是否Windows进程的标识。

（5）启动初始线程的执行，这时会确定进程的环境，线程将要使用的资源也会被分配好。

（6）进程的地址空间将在这时初始化完毕，新进程及其主线程的上下文环境也都被准备好了。

现在，进程的内存地址空间中EProcess块结构（包括KProcess结构）和进程环境块结构已经分配完成。此时进程至少有一个主线程，随着进程的执行会占用更多内存资源。如果进程挂起或崩溃，至少可以得到上述结构数据进行分析。

分析内存镜像

DFRWS 2005内存挑战中描述的工具可以分析内存数据并列出活动进程列表。该工具使用特定的值/偏移（由系统文件导出）来识别活动进程列表的开始，然后遍历该双向链表，从而得到所有的活动进程信息。活动进程列表的偏移地址来自于重要的系统文件ntoskrnl.exe。

Andreas Schuster给出了一个截然不同的解决方案。应该说，这个解决方案相当的暴力——在进程的地址空间中会有一些特殊的字符串，找到这些特殊的字符串之后，就能把各个进程的EProcess块都找出来了。Andreas这个方法里首先要找的是*DISPATCHER_HEADER*这个结构体。这个结构体位于EProcess结构体偏移0x00的位置上（实际上*DISPATCHER_HEADER*包含于Kprocess结构）。使用LiveKD，可以看到该结构体中包含的各个成员。

```
+ 0x000 Header              : struct _DISPATCHER_HEADER, 6 elements, 0x10 bytes
   + 0x000 Type             : UChar
   + 0x001 Absolute         : UChar
   + 0x002 Size             : UChar
   + 0x003 Inserted         : UChar
   + 0x004 SignalState      : Int4B
```

简单来说，Andreas 发现系统中所有进程的 DISPATCHER _ HEADER 结构体中的几个成员值都是一样的。他检查了从 Windows 2000 一直到早期的 Vista 系统中所有进程中 DISPATCHER _ HEADER 结构体中的成员，结果发现在各个版本的 Windows 操作系统中 Type 的值都是固定不变的。另外，Size 值在多个系统中也是不变的（例如，Windows 2000 或者 XP 中各个进程中，Size 的值都是一样的），但是在某些系统之间可能会有些变化（例如，在 Windows 2000 中 Size 的值都是 0x1b，但是在早期 Vista 中 Size 的值改成 0x20）。

使用这些信息，加上该结构体的大小和结构特征，Andreas 写了一个名为 ptfinder. pl 的 Perl 脚本，这个脚本列出内存镜像中进程和线程信息。在 2006 年的 DFRWS 会议上，Andreas 发表了一篇文章，名为“在 Windows 内存镜像中搜索进程和线程”[26]。这篇文章里详细介绍了这一内存分析方法的原理，以及如何判别找出的 EProcess（或者其他内核数据块）是不是一个真正的 Eprocess 数据块（或者其他内核数据块）等问题的解决方法。

注 释

2006 年秋季，ForensicZone. com 的 Richard McQuown 为 Andreas Schuster 的 PTFinder 编写了一个图形界面的前端应用程序（PTFinderFE）。PTFinder 是一个 Perl 脚本程序，运行时需要在系统中安装 Perl 解释器（Linux 的发行版大多自带 Perl 解释器，Windows 用户可以在 ActiveState. com 免费下载和安装 Perl 解释器）。

PTFinderFE 工具不仅可以自动检测操作系统的内存镜像（用户不需要手工输入），还提供了图形化的输出。这是个很不错的工具，特别是在图形界面方面。

2006 年春天，我写了几个用于辅助分析 Windows 物理内存镜像的工具。由于当时可用的镜像是 DFRWS 2005 内存分析挑战赛提供的 Widnows 2000 系统，同时不愿意过多地纠缠于各个不同版本 Windows 操作系统内核数据结构之间的细微差别，所以我的这几个工具只是针对 Windows 2000 系统。4 个工具都是 Perl 命令行脚本，可以在本书附带的光盘中找到。下面就来具体介绍这 4 个脚本。

Lsproc. pl

Lsproc，名称为列举进程（List Processes）的缩写，功能类似 PTFinder. pl，不过

lsproc. pl 只能列举进程而不能列出线程。这个脚本的作用就是列出镜像中所有的进程。lsproc. pl 运行需要一个参数——物理内存镜像的路径：

```
c:\perl\memory>lsproc.pl d:\dumps\drfws1-mem.dmp
```

lsproc. pl 输出到控制台（如 STDOUT），一共分成 6 列，分别是类型栏（这一列全是 Proc，希望以后能加入线程）、父进程 ID（PPID）、进程 ID（PID）、进程名称、进程数据结构在内存镜像文件中的偏移位置和进程创建时间。下面是输出的一部分[①]：

```
Proc    820     324     helix.exe           0x00306020    Sun Jun 5 14:09:27 2005
Proc    0       0       Idle                0x0046d160
Proc    600     668     UMGR32.EXE          0x0095f020    Sun Jun 5 00:55:08 2005
Proc    324     1112    cmd2k.exe           0x00dcc020    Sun Jun 5 14:14:25 2005
Proc    668     784     dfrws2005.exe(x)    0x00e1fb60    Sun Jun 5 01:00:53 2005
Proc    156     176     winlogon.exe        0x01045d60    Sun Jun 5 00:32:44 2005
Proc    156     176     winlogon.exe        0x01048140    Sat Jun 4 23:36:31 2005
Proc    144     164     winlogon.exe        0x0104ca00    Fri Jun 3 01:25:54 2005
Proc    156     180     csrss.exe           0x01286480    Sun Jun 5 00:32:43 2005
Proc    144     168     csrss.exe           0x01297b40    Fri Jun 3 01:25:53 2005
Proc    8       156     smss.exe            0x012b62c0    Sun Jun 5 00:32:40 2005
Proc    0       8       System              0x0141dc60
Proc    668     784     dfrws2005.exe(x)    0x016a9b60    Sun Jun 5 01:00:53 2005
Proc    1112    1152    dd.exe(x)           0x019d1980    Sun Jun 5 14:14:38 2005
Proc    228     592     dfrws2005.exe       0x02138640    Sun Jun 5 01:00:53 2005
Proc    820     1076    cmd.exe             0x02138c40    Sun Jun 5 00:35:18 2005
Proc    240     788     metasploit.exe(x)   0x02686cc0    Sun Jun 5 00:38:37 2005
Proc    820     964     Apoint.exe          0x02b84400    Sun Jun 5 00:33:57 2005
Proc    820     972     HKserv.exe          0x02bf86e0    Sun Jun 5 00:33:57 2005
Proc    820     988     DragDrop.exe        0x02c46020    Sun Jun 5 00:33:57 2005
Proc    820     1008    alogserv.exe        0x02e7ea20    Sun Jun 5 00:33:57 2005
Proc    820     972     HKserv.exe          0x02f806e0    Sun Jun 5 00:33:57 2005
Proc    820     1012    tgcmd.exe           0x030826a0    Sun Jun 5 00:33:58 2005
Proc    176     800     userinit.exe(x)     0x03e35020    Sun Jun 5 00:33:52 2005
Proc    800     820     Explorer.exe        0x03e35ae0    Sun Jun 5 00:33:53 2005
Proc    820     1048    PcfMgr.exe          0x040b4660    Sun Jun 5 00:34:01 2005
```

lsproc. pl 输出的第一行进程名是 helix. exe。根据 DFRWS 2005 挑战赛网站上提供的信息，获取内存镜像使用了 Helix Live CD[27]，工具名称是 helix. exe。

上面只给出了 lsproc. pl 的部分输出结果，镜像中共有 45 个进程。另外要注意的是，部分进程名称后面带有“(x)”，这表示该进程已经退出。这种情况下，物理内存（例如，内存页）已经被释放，但还没有被覆盖，所以看起来这些信息还在内存中。

---

① 这些是对 DFRWS 2005 内存挑战赛的 dfrws2005-physical-memory1. dmp 镜像分析的部分结果，这个内存镜像需要到 DFRWS 网站下载——译注。

注 释

仔细观察 lsproc. pl 的输出结果会发现一些问题。一个是 csrss. exe 进程（进程 ID=168）的创建时间显然比其他进程的创建时间早了一两天。另外，似乎有两个 winlogon. exe 进程（进程 ID 分别是 164 和 176），时间也很怪。Andreas Schuster 也注意到了这个问题，根据他 Blog 上的文章[28]，创建内存镜像的系统启动时间是 2005 年 6 月 5 日（星期天）大约 00：32：27。问题是，这些进程是从哪儿来的？

正如 Andreas 在自己的 Blog 中指出的，如果没有被检查的系统的背景信息，要解答这个问题是很困难的。不过关于被检查系统的一些信息已经被明确记录，其中说明在数据获取的时候系统发生了崩溃。

完全可能是因为系统重启以后一些数据仍然保存在内存中。并没有规范说明 Windows 系统关机或者崩溃的时候，内存中的数据会全部清空。可能的情况是，内存中的数据保持了以前的状态，如果在重启的时候没有被覆盖，数据就可能被分析得到。很多 BIOS 版本会在系统启动的内存检测过程中覆盖内存，但是这个功能经常会被禁用，以加快系统启动的速度。

这个问题引出了一个需要研究的全新领域，正如 Andreas 所说[29]，该领域“有着光明的前景”。

Lspd. pl

Lspd. pl Perl 脚本可以列出进程更详细的信息。和下面将要讨论其他工具一样，lspd. pl 依赖 lsproc. pl 的输出以获取信息。Lspd. pl 有两个输入参数：镜像文件的路径和 lsproc. pl 输出的需要分析的进程偏移量。虽然 lsproc. pl 分析的时间比较长，lspd. pl 则要快得多，因为它已经知道该到哪里去找信息。

下面是对具体进程的分析，要分析的是 dd. exe，进程 ID 为 28。使用 lspd. pl 完成这一功能的命令如下：

```
c:\perl\memory>lspd.pl d:\dumps\dfrws1-mem.dmp 0x0414dd60
```

注意，pspd. pl 有两个输入参数：镜像文件路径及使用 lsproc. pl 得到的要查询的进程在镜像中的偏移位置。可以分几部分来看 lspd. pl 的输出结果，首先是从 EProcess 块中读出的一些有用信息：

```
ProcessName             :dd.exe
PID                     :284
ParentPID               :1112
TFLINK                  :0xff2401c4
TBLINK                  :0xff2401c4
FLINK                   :0x8046b980
BLINK                   :0xff1190c0
SubSystem               :4.0
```

```
ExitStatus              :259
CreateTime              :SunJun  514:53:422005
ExitCalled              :0
DTB                     :0x01d9e000
ObjTable                :0xff158708(0x00eb6708)
PEB                     :0x7ffdf000(0x02c2d000)
        InheritedAddressSpace          :0
        ReadImageFileExecutionOptions  :0
        BeingDebugged                  :0
        CSDVersion                     :ServicePack1
        Mutant              = 0xffffffff
        ImgBaseAddr         = 0x00400000(0x00fee000)
        PEB_LDR_DATA        = 0x00131e90(0x03a1ee90)
        Params              = 0x00020000(0x03a11000)
```

lspd. pl 也会从 Eprocess 结构指向的其他数据结构中读取信息。例如，可以看到在 lspd. pl 的输出信息中还包括进程可执行文件的路径及创建进程时的参数（强调一下，这很重要）：

```
Current Directory Path = E:\Shells\
DllPath                =
E:\Acquisition\FAU;.;C:\WINNT\System32;C:\WINNT\system;
C:\WINNT;E:\Acquisition\FAU\;E:\Acquisition\GNU\;E:\Acquisition\CYGWIN\;E:\I
R\bin\;E:\IR\WFT;E:\IR\windbg\;E:\IR\Foundstone\;E:\IR\Cygwin;E:\IR\somarsof
t\;E:\IR\sysinternals\;E:\IR\ntsecurity\;E:\IR\perl\;E:\Static-
Binaries\gnu_utils_win32\;C:\WINNT\system32;C:\WINNT;C:\WINNT\System32\Wbem
ImagePathName          = E:\Acquisition\FAU\dd.exe
Command Line           = ..\Acquisition\FAU\dd.exe if=\\.\PhysicalMemory
of=F:\intrusion2005\physicalmemory.dd conv=noerror --md5sum --verifymd5  --
md5out=F:\intrusion2005\physicalmemory.dd.md5 --
log=F:\intrusion2005\audit.log
Environment Offset     = 0x00000000 (0x00000000)
Window Title           = ..\Acquisition\FAU\dd.exe if=\\.\PhysicalMemory
of=F:\intrusion2005\physicalmemory.dd conv=noerror --md5sum --verifymd5  --
md5out=F:\intrusion2005\physicalmemory.dd.md5 --
log=F:\intrusion2005\audit.log
Desktop Name           = WinSta0\Default
```

另外，lspd. pl 也会列出进程中使用各个不同模块（DLLs）的名字及内存中的句柄（文件句柄等）信息。例如，lspd. pl 显示 dd. exe 进程中打开了如下的文件句柄：

```
Type : File
      Name = \intrusion2005\audit. log
```

从前面的命令行信息可以看出，dd. exe 生成的日志记录在 F:\intrusion 2005\audit. log 文件中，这说明了为什么进程打开了该文件的句柄。使用从其他进程中导出的这类信息，就可以确定调查中需要关注哪些文件。本例中，可以假设在 ImagePath-

Name 中提到的 E:\是指 CD-ROM，因为 Helix 可能从光盘运行。可以从硬盘镜像分析注册表来验证这点（不过，在 DFRWS 2005 内存挑战赛中没有给出相关计算机的硬盘镜像）。还可以使用类似的信息找出有关 F:\的更多内容。这部分详见第 4 章。

最后，lspd.pl 还会根据进程可执行文件镜像基地址（通过将虚拟内存地址转换为物理内存镜像中的偏移位置）来校验该地址上装载的是不是一个有效的可执行文件镜像。这个校验很简单，也就是该地址上开始的两个字节是不是 MZ。这两个字节并非绝对正确的校验，但是正常的 PE 文件（Windwos 下可执行文件的标准格式，如以.exe,.dll,.ocs,.sys 为扩展名的文件）都是以 Mark Zbikowski 的缩写开头，这位仁兄是早期的 MS-DOS 和 Windows NT 的系统架构师之一。PE 文件及其文件头格式会在第 6 章详细介绍。

分析进程内存

本章开头曾经提到过需要得到证据的上下文，这可以通过提取进程所使用的内存部分得到。过去，调查人员使用 string.exe 或者 grep 工具对内存镜像进行分析，查找字符串（可能是口令）、IP 地址、电子邮件地址或者 URL 之类的信息。不过在分析时，并不能得到有关信息的上下文。有时，调查人员使用十六进制编辑器打开物理内存镜像，在一个类似用户名的字符串边上发现另一个字符串。这种情况下，他可能会猜测说“哦，这个可能是这家伙使用的口令”，仅此而已。这并不是分析物理内存镜像的最好办法。还记得在第 1 章里介绍的诺卡德交换原理吗？凭什么说在内存中找到的 IP 地址（或者是一个目录之类的数据）就一定与某个事件或者进程有关联呢？不过，如果通过内存中的进程结构定位并提取内存镜像中进程使用的内存数据，然后再进行搜索，就可以确定进程上下文了。

lspm.pl 工具可以自动提取进程内存数据。lspm.pl 使用的参数和 lspd.pl 一样（都是镜像文件的路径加上进程在镜像文件中的偏移量），然后就能把该进程所有使用的内存提取出来，写到当前目录的一个文件中。例如，要提取所有 dd.exe 进程所使用的内存，输入如下命令：

```
c:\perl\memory>lspm.pl d:\dumps\dfrws1-mem.dmp 0x0414dd60
```

程序的输出如下所示：

```
Name : dd.exe -> 0x01d9e000
There are 372 pages (1523712 bytes) to process.
Dumping process memory to dd.dmp...
Done.
```

现在就得到了一个名为 dd.dmp 的文件，这个文件共有 1 523 712B，其中包含在这个镜像文件创建时物理内存中所有被 dd.exe 进程所使用的内存页（共 372 个）。可以使用 string.exe 或者 Foundstone.com 的 BinText（如图 3.3 所示）工具分析文件，查找其中的 ASCII 和 unicode 字符串，也可以用 *grep* 工具搜索 IP、E-mail 地址、信用卡和社会安全号等信息。

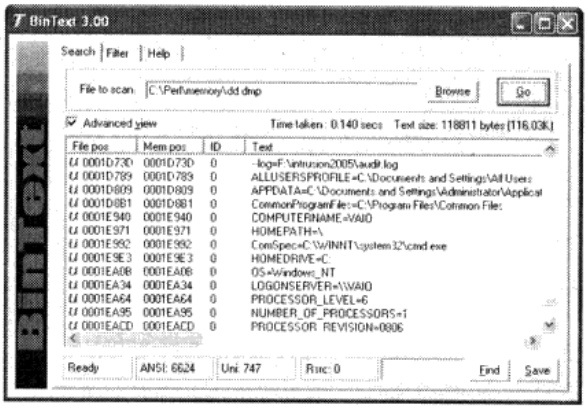


图 3.3　BinText 中的进程内存数据

从图 3.3 中可以看到，dd.exe 进程中使用的一些 unicode 字符串，其中包括会话中的系统名称和 *LogonServer* 名字。所有这些信息都有助于调查人员对案情的进一步理解。其中最重要的一点是，现在可以将内存中找到的数据与某个进程关联起来。

提取进程可执行文件镜像

之前已经讲过，当进程开始启动的时候，对应的可执行文件（的一部分）会被读入内存。使用 lspd.pl 获取的进程信息之一就是可执行镜像的加载基地址（内存镜像里的偏移量），而且 lspd.pl 会检查相应位置上是不是一个有效的 PE 头部。接下来可以进一步分析这个 PE 文件头（详见第 6 章），看是否可以从内存镜像中把这个进程对应的可执行文件完整提取出来。lspi.pl 工具可以自动完成这一工作。

Lspi.pl 是一个 Perl 脚本，和之前两个 Perl 脚本——lspd.pl 和 lspm.pl 接受的参数是一样的，可以定位进程可执行文件的起始地址。要是该地址指向一个有效的可执行文件，lspi.pl 就会根据 PE 头部数据结构中的内容定位内存中的数据并重组可执行文件内容。

好了，现在就用这个工具提取镜像中 dd.exe（PID 为 284）进程的文件内容，输入下面的命令：

```
c:\perl\memory>lspi.pl d:\dumps\dfrws1-mem.dmp 0x0414dd60
```

Ldpi.pl 命令输出如下：

```
ProcessName       : dd.exe
PID               : 284
DTB               : 0x01d9e000
PEB               : 0x7ffdf000(0x02c2d000)
ImgBaseAddr       : 0x00400000(0x00fee000)
e_lfanew = 0xe8
NTHeader = 0x4550
```

```
Reading the Image File Header
Sections = 4
Opt Header Size = 0x000000e0(224bytes)
Characteristics:
        IMAGE_FILE_EXECUTABLE_IMAGE
        IMAGE_FILE_LOCAL_SYMS_STRIPPED
        IMAGE_FILE_RELOCS_STRIPPED
        IMAGE_FILE_LINE_NUMS_STRIPPED
        IMAGE_FILE_32BIT_MACHINE
Machine = IMAGE_FILE_MACHINE_I860
Reading the Image Optional Header
Opt Header Magic = 0x10b
Subsystem          : IMAGE_SUBSYSTEM_WINDOWS_CUI
Entry Pt Addr      : 0x00006bda
Image Base     : 0x00400000
File Align     : 0x00001000
Reading the Image Data Directory information
Data Directory      RVA           Size
--------------      ---           ----
ResourceTable       0x0000d000    0x00000430
DebugTable          0x00000000    0x00000000
BaseRelocTable      0x00000000    0x00000000
DelayImportDesc     0x0000af7c    0x000000a0
TLSTable            0x00000000    0x00000000
GlobalPtrReg        0x00000000    0x00000000
ArchSpecific        0x00000000    0x00000000
CLIHeader           0x00000000    0x00000000
LoadConfigTable     0x00000000    0x00000000
ExceptionTable      0x00000000    0x00000000
ImportTable         0x0000b25c    0x000000a0
unused              0x00000000    0x00000000
BoundImportTable    0x00000000    0x00000000
ExportTable         0x00000000    0x00000000
CertificateTable    0x00000000    0x00000000
IAT                 0x00007000    0x00000210
Reading Image Section Header Information
Name      Virt Sz      Virt Addr    rData Ofs    rData Sz     Char
----      -------      ---------    ---------    --------     ----
.text     0x00005ee0   0x00001000   0x00001000   0x00006000   0x60000020
.data     0x000002fc   0x0000c000   0x0000c000   0x00001000   0xc0000040
.rsrc     0x00000430   0x0000d000   0x0000d000   0x00001000   0x40000040
.rdata    0x00004cfa   0x00007000   0x00007000   0x00005000   0x40000040
Reassembling image file into dd.exe.img
```

```
Bytes written = 57344
New file size = 57344
```

如上所示，lspi. pl 输出的信息相当详细，其中很多信息对于调查人员来说甚至难以理解，除非这个调查人员正好对恶意软件分析很熟悉。在第 6 章详细讨论这些信息。现在重要的是“读取镜像分区头信息”（Reading Image Section Header Information）下面列出的几行及提取出来的数据被放到哪个文件中了。分区头信息提供了可执行文件在内存中的大致组成情况，Lspi. pl 正是根据这些信息尝试重构可执行文件的。如果 lspi. pl 发现有关的各个内存页都在物理内存中，就把各个内存页读出来，按可执行文件镜像中的顺序把它们重组成一个可执行文件。重新组成的可执行文件的文件名就是进程名，后面再加上. img 后缀，以防止该可执行文件被误运行（万一重构的可执行文件是一个恶意软件呢）。只要分区头信息中列出的各个内存页中有一个不在内存中（可能由于长期没有执行，被交换到硬盘上去了；或者尚未被执行到，未被读入内存），可执行文件就不能被重组。这时，lspi. pl 就会返回一个信息，告诉调查人员，由于某个内存页不在物理内存中，所以无法重组可执行文件。

现在必须提醒：使用 lspi. pl 从内存中提取的文件可能会与原始可执行文件的内容有所出入。这是因为可执行文件的有些段在内存中是可写的，随着程序的运行，这些段将会发生改变。程序运行时，根据其运行的环境，以及程序当前的执行状态，程序中的一些变量、地址等组成元素会发生变化，甚至代码本身也不能幸免（有些代码是可以自修改的，如加壳）。不过还是有一些办法来确定程序自身信息的。首先是可以读取编译器在生成可执行文件时加入的一些信息，这一招对于正规的软件公司开发的程序特别有效。一般会在可执行文件中发现一个. rsrc 段，这个段是 PE 文件中常见的一个段，在这个段里都存放了大量的资源，如对话框、版本字符串等。图 3. 4 就是用 BinText 工具分析 dd. exe. img 文件的一个截图，看到其中的 unicode 字符串 *VS _VERSION _INFO*[30] 及其他一些字符串。

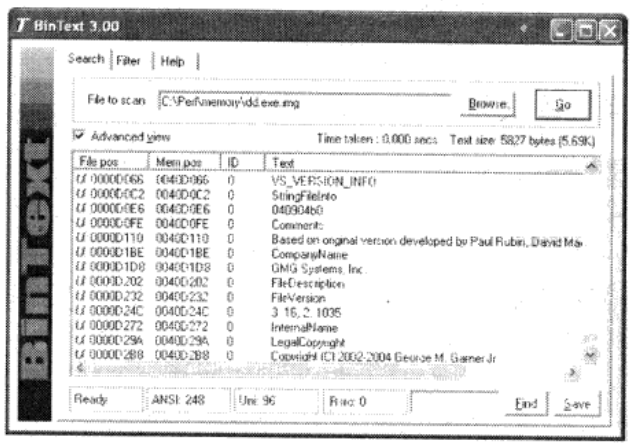


图 3. 4　利用 BinText 工具分析的截图

另一个分析方法是计算文件哈希。读者可能会想“嗨，不对吧？刚才不是在说

lspi. pl重构出来的文件可能会与原始文件不一样吗，哈希会有什么用?”是的，这个想法是对的……至少在某一点上是对的。不能使用 MD-5 哈希来做比较，因为文件只要有一点点改变，甚至只被改动了一个位，计算出来的哈希也会完全不一样。现在该怎么办呢?

2006 年夏季，Jesse Kornblum 发布了 ssdeep[31]工具，这个工具实现了名为上下文触发的分段哈希算法，也叫模糊哈希。如果要了解这一算法细节，建议读者进一步阅读 Jesse 在 DFRWS 2006 发表的关于该主题的论文[32]。简单来说，这一算法就是根据计算分段哈希从而得出两个文件相同数据序列在文件中所占的比重。因为已经知道在这个案例中制作物理内存镜像的是 George Garner 写的 dd. exe，所以用 ssdeep. exe 来比较原始的 dd. exe 和提取的 dd. exe. 镜像，看会有什么结果。

首先用 ssdeep. exe 计算提取的镜像文件的哈希：

```
D:\tools>ssdeep c:\perl\memory\dd. exe. img > dd. sdp
```

把计算结果存放在 dd. sdp 文件中，这样就能用 sdeep. exe 工具的其他选项来比较原始可执行文件和提取出来的镜像文件的相似度：

```
D:\tools>ssdeep -v -m dd. sdp d:\tools\dd\old\dd. exe
d:\tools\dd\old\dd. exe matches c:\perl\memory\dd. exe. img (97)
```

使用-d 或者-p 参数，可以在一条指令中完成比较：

```
D:\tools\>ssdeep -d c:\perl\memory\dd. exe. img d:\tools\dd\old\dd. exe
C:\perl\memory\dd. exe. img matches d:\tools\dd\old\dd. exe (97)
```

不管用上述哪种方法，都可以知道 lspi. pl 提取出来的文件镜像与原始文件的相似度是 97%。

记住，要完成哈希比较，必须知道 lspi. pl 提取出来的文件和哪个文件进行比较。ssdeep. exe 是一个很新、功能很强大的工具，就是在生成哈希和比较的时候要花点时间。

内存镜像分析和页交换文件

到目前为止，所有分析工作都是只针对物理内存镜像自身的——没有借助其他信息。也就是说，如果 lspm. pl 等同类工具只对内存镜像进行分析，提取的数据可能会不完整，因为被交换到页交换文件（Windows 中的 pagefile. sys）的内存页不会被内存镜像工具所获取。为了解决这个问题，2006 年春季 Nicholas Paul Maclean 发表了题为“获取和分析 Windows 内存”（Acquisition and Analysis of Windows Memory）的学位论文（很抱歉，网上没有这篇论文的有效链接），论文中详细描述了 Windows 系统中内存管理系统的内部工作机制，并且提供了一个开源的工具 *vtop*（用 Python 语言实现）用以重构进程的虚拟内存空间。

Jesse Kornblum 在他的个人网站上发表了一篇全名为“*Using Every Part of the Buffalo in Windows Memory Analysis*”的文章，也发表在《数字调查月刊》、杂志上。在这篇文章里，Jesse 演示了虚拟内存地址是如何转换成一个物理内存地址的，同

时也讨论了如何把页交换文件纳入内存分析体系，从而构建出一套更加复杂、完整和准确的内存分析方法。

根据内存镜像判断操作系统类型

当你正在处理一个系统的物理内存镜像时，要是有人问你："老兄，你现在分析的是个什么系统啊?"，你该怎么说？是不是只能说"Windows"。朋友，这个问题可能比想象的要重要，毕竟名气和实力二者缺一不可。在对系统镜像进行分析时，确定Windows的版本是首先要解决的问题，因为版本的不同会导致取证的手法也有所不同。在内存分析时这个问题会更加重要，因为不同版本的操作系统内存中定义线程和进程的数据结构会不尽相同，甚至对一些版本相同SP补丁包不同的Windows操作系统也是如此。

所以要分析Windows内存镜像，必须要知道Windows具体的版本和甚至补丁信息。如果不知道，就要使用所有的工具挨个对镜像分析，直到有一个工具能从镜像中正确地分解出各个进程为止，那样就太费时间了。在我和Andreas Schuster的电子邮件往来中，Andreas建议检查系统内核的加载位置。因为同一个版本（同一个SP补丁包）的Windows操作系统在启动时，总是把系统内核加载到同一个物理内存地址上的[①]。但是不同版本的Windows操作系统，这个内核的加载位置却是有可能不同的。怎么找到内核的加载地址呢？最简单的方法就是通过LivdKD工具，LiveKD启动时会显示如下所示的信息（Windows XP SP2）：

```
Windows XP Kernel Version 2600 (Service Pack 2) MP (2 procs) Free x86compatible
Product: WinNt, suite: TerminalServer SingleUserTS
Built by: 2600.xpsp.050329-1536
Kernel base = 0x804d7000 PsLoadedModuleList = 0x8055c700
```

注意上面黑体标识的信息——那是加载系统内核的基地址。将这个值减去0x80000000，得到0x4d7000，跳转到物理内存镜像0x4d7000偏移地址。如果该地址前两个字节是MZ，可以认为这个位置上是一个标准的Windows PE文件，也就是系统内核。此时，可以用lspi.pl类似的代码分析PE头信息并定位段数据结构。由于系统内核就是标准的Windows PE程序，其中带一个包含*VS_VERSION_INFO*结构的资源段。根据Microsoft提供的数据结构信息，就可以知道这个镜像来自哪个版本的操作系统。

在本书附送的光盘中，有一个名为osid.pl脚本可以用来分析内存镜像的操作系统版本。开始这个脚本的名字是kern.pl，当时我打算把这个工具加到Rick McQuown开发的PTFinderFE工具包中。后来Rick给我发送了一封电子邮件，要求我精简这个脚本的输出结果。于是我对该脚本做了一些修改（改变了输出结果，添加了一些启动参数

---

① 这个内核是指NTOSKrnl.exe，在系统启动时是由NTLDR模块将其加载到内存中的。NTOSkrnl.exe是一个标准的Windows PE文件，虽然这个文件带了一个.exe的扩展名，但实际上NTOSKrnl.exe却是一个DLL文件，它带有许多导出函数供其他程序使用，另外如其他DLL一样，NTOSKrnl.exe也有一个入口点函数，NTLDR在加载完NTOSKrnl.exe之后就会把系统控制权交给这个入口点函数——译注。

等），并且改名为 osid. pl。

使用这个工具最简单的方法是从命令行运行，加上镜像文件的路径作为参数：

```
C:\Perl\memory>osid.pl d:\hacking\xp-laptop1.img
```

也可以使用-f 参数来指定镜像文件：

```
C:\Perl\memory>osid.pl -f d:\hacking\xp-laptop1.img
```

上面两条命令的执行结果是一样的。本例中，内存镜像来自一个 Windows XP SP2 系统，所以脚本的返回结果是 *XPSP2*。如果想要看到更多信息，可以使用-v 参数，输出结果如下：

```
OS : XPSP2
Product : Microsoft«Windows» Operating System ver 5.1.2600.2622
```

如上所示，详细输出中包括了 VS_VERSION_INFO 结构中的产品名称和版本信息。如果使用 osid. pl 分析 DFRWS 2005 内存镜像挑战赛提供的内存镜像，可得到如下结果：

```
OS          : 2000
Product : Microsoft(R) Windows(R) 2000 Operating System ver 5.00.2195.1620
```

这个脚本还可以对 VMWare 的. vmem 内存文件进行分析。下面是对一个 Windows 2000 虚拟机的. vmem 文件的分析结果：

```
OS       :  2000
Product  :  Microsoft(R) Windows (R) 2000 Operating System ver 5.00.2195.7071
```

分析内存池

由于采用了分页的内存管理技术，所以 Windows 内存分配的单位是页（4096B）。如果每一个细微的操作（如拷贝字符串到剪贴板）都要申请 4KB 的内存页，就太浪费了。所以内存管理器会提前分配几页内存（构成内存池），然后在需要的时候从中分配给应用程序①。Andreas Schuster 在这一领域里进行了深入而细致地研究。虽然 Microsoft 公司公开了常用的内存池的头部标记的含义，但还没有对这些内存池进行有益分析的文档。Microsoft 调试工具中的 pooltag. txt[33] 文档中列出了常用内存池头信息，另外，Microsoft 知识库中的一篇文章描述了如何定位第三方工具使用的内存池头部/标记[34]。Andreas 使用类似的方法确定 Windows 2000 的物理内存镜像用于存放网络连接数据的内存池的格式[35]，找到了 Windows 2000 系统中 tcpip. sys 驱动使用的内存池，并且从中得到网络连接有关的信息。

内存池分析技术的不利点在于，不同版本的 Windows 操作系统中内存池头部标记似乎是固定不变的，内存池中存储的数据格式却是经常变化的，而且目前还找不到关于这方面信息的文档。

---

① 这种方法对于用户态代码来说就是堆，而对于内核态代码来说就是内存池——译注。

获取进程内存

调查过程中，调查员更关心的可能是某个特殊的进程而不是系统中的所有进程，而且对内存中该进程所使用的内存信息更感兴趣。现在有办法提取进程使用的完整内存数据——不仅包括物理内存中的内容，还包括虚拟内存和页交换文件中的数据。

有若干工具可以完成这一工作。其中之一是Arne Vidstrom编写的pmdump.exe[36]工具，可以从NTSecurity.nu网站下载。按照NTSecurity.nu网站上的介绍，pmdump.exe工具可以在不停止进程的情况下获取进程的完整内存镜像。这与前面的讨论一样，在获取的过程中，进程的内存数据会发生改变，所以产生的是一个“模糊的”进程内存镜像。pmdump.exe输出的文件不能使用调试工具来分析。

Tobias Klein给出了另外一个获取进程内存镜像的免费（但不是开源的）工具Process Dumper[37]。Process Dumper（pd.exe）会导出整个进程地址空间、进程元数据和进程的运行环境，由于结果是输出到系统控制台（STDOUT）的，所以很容易就能将输出结果重定向到文件或者网络上（通过第1章中讨论的*netcat*或者其他类似工具）。仔细阅读Tobias为pd.exe编写的文档，没有发现在导出的过程中会调试、终止或冻结目标进程的提示。Tobias还提供了一个图形界面的工具，用以分析内存镜像中的元数据和内存信息。这些工具是对内存获取工具的扩展，可以从进程内存镜像中提取RSA私钥和证书[38]。

有些地方推荐使用Microsoft公司提供的userdump.exe工具来制作进程的内存镜像。这个工具可以快速生成进程的内存镜像，既不用附加调试器，也不会终止进程，而且userdump.exe生成的内存镜像可以使用Microsoft提供的调试器进行调试。userdump.exe的缺点在于需要先在系统中安装一个驱动，一些情况下这是不可行的。

基于和Robert Hensing的讨论（Robert Hensing是前Microsoft PSS安全小组的成员），导出进程的比较好的方法是使用Microsoft调试工具中的adplus.vbs脚本。Adplus的字面意思是自动导出增强版，原始作者正是Robert（根据该脚本自带的文档，版本1～5是由Robert写的，版本6由Israel Burman接手编写）。在调试工具包的帮助文件中（debugger.chm）中包含了大量关于这个脚本和调试器（cdb.exe）的大量信息。整个调试工具不需安装驱动并可从光盘上运行。也就是说，可以把有关的工具（adplus.vbs、cdb.exe及相关的动态链接库）刻录到光盘中（adplus.vbs运行需要Windows脚本环境——也就是cscript.exe 5.6，大多数Windows系统中安装有这个环境），运行工具并将进程内存导出到网络共享或USB移动存储设备。镜像制作完毕后可以用免费的调试工具来分析，也可以用BinText之类的工具从镜像中提取ASCII，unicode和资源字符串。此外，还可以使用其他一些工具收集进程有关的更多信息，Handle.exe[39]（该工具需要管理员权限运行）可以列出进程打开的所有句柄（文件、目录等），listdlls.exe[40]可以列出进程加载的所有模块的镜像文件路径和版本。

调试工具包帮助文件和Microsoft知识库文章286 350[41]提供了大量有关adplus.vbs的帮助信息。adplus.vbs可以在获取进程内存镜像时挂起进程（也就是挂起、导出，然后再恢复进程）或者杀掉进程（挂起、导出、终止进程）。要使用挂起方式获取进程内

存，输入如下命令：

```
D:\debug>cscript adplus.vbs -quiet -hang -p <PID>
```

这条命令会在调试目录下生成一个子目录，该目录名称以 *Hang_mode* _开头，后面跟着生成镜像的日期和时间（注意，使用*-o* 参数可以指定输出目录）。目录中包括 adplus.vbs 报告文件、内存镜像文件（要制作多个进程的内存镜像可以使用多个*-p* 参数）、进程列表文件（由 tlist.exe 工具生成，使用*-noTList* 参数可以关闭这个输出）及一个包含目标进程加载的所有模块（DLL）的文本文件。

adplus.vbs 收集的所有信息对调查来说非常有用，但是这个工具只能导出系统 API 能“看得见”的进程。对于连系统都找不到的进程（例如，被 rootkit 隐藏的进程），adplus.vbs 就没有办法了。

小结

本章讨论了几种应急响应过程中获取目标系统物理内存和进程内存的方法。在第 1 章中讨论的是开机取证方法，涉及内存中易失信息（进程、网络连接等）提取的各种工具。在获取这些信息的时候必须牢记：这些工具使用的系统 API 很可能被黑客破坏了。任何开机取证方式都不能避免这点，因此在搜集有关信息时应该使用多个工作原理不同的工具。rootkit 可以让 tlist.exe，pslist.exe 等基于系统 API 的工具找不到恶意软件的进程，但是获取系统物理内存并分析，就可以把系统中所有的进程列举出来，其中包括被内核模式 rootkit 隐藏的进程（rootkit 的进一步信息参见第 7 章）。

参考资料

1. www.dfrws.org
2. www.e-fense.com/helix/
3. http://computer.forensikblog.de/en/topics/windows/memory_analysis/
4. www.lurhq.com/truman/
5. www.blackhat.com/html/bh-federal-06/bh-fed-06-speakers.html#Burdach
6. http://nucia.unomaha.edu/tvidas/
7. www.grandideastudio.com/
8. www.storm.net.nz/projects/16
9. www.python.org
10. http://ntsecurity.nu/onmymind/2006/2006-0902.html
11. http://en.wikipedia.org/wiki/Blue_Screen_of_Death.
12. www.microsoft.com/whdc/devtools/debugging/default.mspx.
13. www.microsoft.com/downloads/details.aspx?FamilyID=E84D3B35-63C3-445B-810D-9FED3FDEB13F&displaylang=en.
14. http://support.microsoft.com/kb/254649/

15. http://support.microsoft.com/kb/274598/
16. http://support.microsoft.com/kb/307973/
17. http://support.microsoft.com/kb/ 244139/
18. www. microsoft. com/downloads/details. aspx? FamilyID = E84D3B35-63C3-445B-810D-9FED3FDEB13F&displaylang=en
19. http://support.citrix.com/article/CTX107717&parentCategoryID=617
20. www.microsoft.com/technet/sysinternals/SystemInformation/LiveKd.mspx
21. http://bochs.sourceforge.net/
22. http://users.erols.com/gmgarner/forensics/develop
23. http://computer.forensikblog.de/en/topics/win-dows/memory_analysis/
24. www.microsoft.com/whdc/devtools/debugging/default.mspx
25. Mark E. Russinovich and David A. Solomon, Windows Internals, fourth edition, Chapter 6 (Redmond, WA: Microsoft Press, 2005)
26. www.dfrws.org/2006/proceedings/2-Schuster.pdf
27. www.e-fense.com/helix/
28. http://computer. forensikblog. de/en/2006/04/per-sistance _through _the _boot _process.html
29. http://computer.forensikblog.de/en/2006/04/data_lifetime.html
30. http://msdn. microsoft. com/library/default. asp? url =/library/en-us/mobilesdk5/html/wce50lrfVSVERSIONINFO.asp
31. http://ssdeep.sourceforge.net/
32. http://dfrws.org/2006/proceedings/12-Kornblum.pdf
33. www.microsoft.com/whdc/driver/tips/PoolMem.mspx
34. http://support.microsoft.com/default.aspx? scid=kb;en-us;298102
35. http://computer.forensikblog.de/en/2006/07/finding_network_socket_activity_in_p ools.html
36. www.ntsecurity.nu/toolbox/pmdump/
37. www.trapkit.de/research/forensic/pd/index.html
38. www.trapkit.de/research/sslkeyfinder/index.html
39. www.microsoft.com/technet/sysinternals/ProcessesAndThreads/Handle.mspx
40. www.microsoft.com/technet/sysinternals/ProcessesAndThreads/ListDlls.mspx
41. http://support.microsoft.com/kb/286350

快速解决方案

获取物理内存镜像

√ 有几种方法可以用来制作系统的物理内存镜像。调查人员应该熟悉每一种镜像物理内存方法的优缺点，以便选择最合适的方法。

- 从开机系统中获取系统内存镜像时，不能保持数据的一致性，这是因为获取的时候系统处于运行状态，从而带来数据的改变。
- 在获取系统物理内存镜像时，调查人员必须牢记诺卡德交换原理。

分析物理内存镜像

- 依赖于获取物理内存镜像不同方式，可以使用不同的工具对之进行分析。以前使用 string. exe，BinText 和正则表达式搜索等分析方法比较流行，2005 年春季以来的研究更加注重于对特定进程信息的提取。
- 物理内存镜像中包含各种有用的信息和对象，例如，进程、剪贴板中的数据和网络连接等。
- 进一步的研究表明，将页交换文件与内存镜像一起分析可以得到更多信息。

获取进程内存

- 一些情况下，调查员不必获取整个物理内存镜像，而只需获取单个进程的镜像。
- 获取单个进程内存镜像的前提是系统和调查工具可以在活动进程列表中找到这个进程。使用某种方式（参见第 7 章）隐藏的进程可能看不到，所以没有办法通过工具来获取。
- 获取进程的内存镜像不仅可以获取进程使用的物理内存，还包括进程在页交换文件的内存。
- 获取进程的内存镜像之后，还可以获取该进程打开的句柄和加载的模块等信息。

常见问题

下面的这些问答用来测试读者是否已经完全理解本章的内容，并可以帮助读者在实际案件调查过程中应用本章知识。如果还有其他问题，浏览 http://www. syngress. com/solutions 并单击 Ask the Author 一栏。

问：为什么要获取开机系统的内存镜像？这有什么用，它能告诉我一些什么有用信息呢？

答：正如第 1 章所讨论的，系统在开机状态下具有大量与案件调查紧密相关的重要信息。这些信息存在于系统的内存中，一旦关机将永远消失。可以使用不同的第三方工具（详见第 1 章）来收集这些信息，但是获取系统的物理内存镜像也是非常重要的。物理内存中不仅包括其他工具能够找到的信息，而且还能找到一些传统方法找不到的信息（如被 rootkit 隐藏的信息）。另外还有可能找到一些已经退出的进程的信息，甚至重启后“残留”的上一次开机时运行的进程信息。

问：在获取系统物理内存镜像之后，应该怎么样分析它呢？

答：调查员习惯于使用基于文件的搜索工具来分析内存镜像。String. exe 和 grep 工具搜索镜像时可用来定位内存镜像中类似口令、电子邮件地址和 URL 等信息。现在已经有一些工具可以用来分析内存镜像中的进程、进程详细信息（命令行参数、句柄

等）、线程及其他一些对象（如进程的可执行文件镜像）。这对恶意软件分析（详见第 6 章）和传统的计算机取证分析都特别有用。

问：我参与了一起失踪案的调查。在分析失踪者家里的一台电脑时，发现系统桌面上有一个打开的即时聊天（IM）应用窗口。我翻动这个聊天工具的屏幕，想看失踪者和别人聊了些什么，但是什么也没看到。这个进程中可能有重要的信息，我该怎么办呢？

答：要是关心的只是这个 IM 进程，获取整个系统的内存镜像就没必要了。此时可以获取这个进程所使用的内存，再使用其他的工具提取进程的相关信息，如加载的模块、打开的句柄及启动进程时使用的命令行参数等。信息获取完毕之后，下一步就是保存聊天会话的内容。然后应该对进程的内存镜像进行搜索，看其中有没有残存的上次聊天的会话记录或者其他有用的线索。

Chapter 4

第 4 章
注册表分析

本章主要内容：

- ■ 注册表内部结构
- ■ 注册表分析

- √ 小结
- √ 快速解决方案
- √ 常见问题

引言

对于绝大多系统管理员和取证调查人员来说，如果把 Windows 操作系统看作一幅风景画，那么注册表就是这幅风景画中的黑暗洞穴。实际上，注册表是一座真正的信息金矿。很多情况下，攻击者使用的软件会在注册表中留下痕迹，会给调查者留下相关事件的线索。知道在注册表的什么地方获取信息及如何解释这些信息，对于了解操作系统中进行的各种活动是大有帮助的。

本章的主要目的是加深读者对注册表的了解，同时向读者提供注册表所包含的大量信息。除了系统配置信息，Windows 注册表还保存了有关最近访问文件和用户活动的大量信息。针对各种不同类型的案件，所有这些信息都是非常有价值的。本章所讨论的绝大多数注册表分析方法都是基于静态分析/后期分析（post mortem）的，也就是说，分析对象是获取的系统镜像。在一些特定情况下，讨论对开机系统（Live System）的分析方法，并且提供针对开机系统的注册表键值和数据。需要指明的是，当以开机系统分析代替静态分析时，会在讨论中予以说明。

本章讨论的重点内容是取证调查过程中所关心的各种注册表键和数值，但是本章并不是所有感兴趣注册表值的全集。操作系统安装完成后，与操作系统相关的注册表键基本上会保持不变，但是在安装了系统修补包和补丁后，注册表会发生相应地变化。各种应用程序的情况和操作系统基本一致，不同版本的应用程序其注册表项也会发生变化。本章是一本有关注册表分析的指导和参考手册，有一点要记住，没有真正完全的指导手册。本章的目的是描述注册表文件的格式、如何调查注册表、注册表的重要键值及如何分析它们。通读本章，就会对注册表所保存的信息有一个较为深入的理解，能够熟练地获取和分析这些信息。

注册表内部结构

注册表是什么？回顾 DOS 和 Windows 的早期时代（3.1 和 3.11 等），这些系统的配置信息（驱动、设置）主要由下述几个文件管理：autoexec.bat，config.sys，win.ini（Windows）和 system.ini（Windows）。这些文件中的各种设置决定加载什么程序，系统如何对用户输入作出反应。

后来的 Windows 系统用注册表替代了这些配置文件，注册表是一个中央分层[1]的数据库，负责管理应用程序、硬件设备和用户的各种配置设定。这个数据库（广义上的）代替了 Microsoft 以前版本操作系统中所使用的基于文本的配置文件。

绝大多数情况下，管理员可以通过某些中间应用程序能够直接和注册表交互。其中最常用的就是 Microsoft 自带的图形界面软件——注册表编辑器（Regedit.exe 或者 Regedit32.exe）。但是，如果要访问不允许用户直接访问的注册表键或者是注册表项，就必须安装特殊的软件。Windows XP 和 Windows 2003 发布包中有一个 CLI 工具——reg.exe，能够在命令提示符或者是脚本（Scripts）中使用。有关更多 GUI 注册表编辑工具信息参考“RegEdit 和 RegEdt32”介绍。

regedit. exe 程序运行后，regedit. exe 浏览区域中显示的是具有 5 个根目录（根目录也称为“Hives”，配置单元）的、类似树状的结构，如图 4. 1 所示。这种目录结构能够使管理员很容易地浏览整个注册表，如浏览文件系统一样。

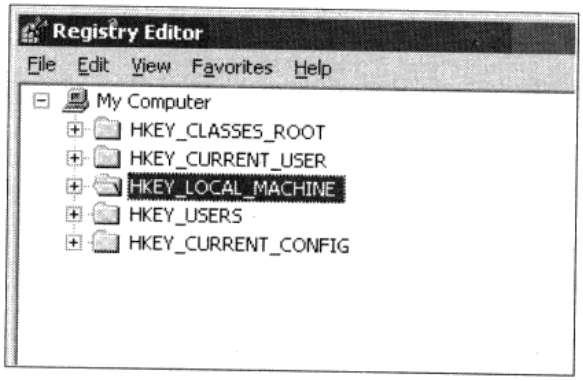


图 4. 1　regedit. exe 显示的 5 个根配置单元

工具和陷阱

RegEdit 和 RegEdt32

有意思的是，这两个操作系统自带的程序在不同的操作系统上是不一样的[2]。例如，在 Windows 2000 和 Windows NT 系统上，RegEdit. exe 具有很好的搜索能力，但是不能够修改注册表键值的访问控制列表（Access Control Lists，ACL），也不能正确解释 *REG_EXPAND_SZ*（可变长度字符串）或者 *REG_MULTI_SZ*（多字符串）数据类型。如果使用 RegEdit. exe 编辑上述两种数据类型的变量，那么变量的数据类型就会变为 *REG_SZ*（固定长度字符串），这就意味着该注册表键值失去了它的功能。RegEdt32. exe 的另一个不足是，它不能输入或者输出配置单元文件（. reg）。

在 Windows XP 和 Windows 2003 系统中，RegEdit. exe 程序可以完成所有功能。Windows XP 和 Windows 2003 系统中的 RegEdt32. exe 的功能与 RegEdit. exe 一致，实际上 RegEdt32. exe 只是一个调用 RegEdit. exe 的小程序而已。

注册表中的每个配置单元在操作系统中都扮演着重要的角色，起着非常重要的作用。HKEY_USERS 管理系统中所有曾登录的用户的配置信息。HKEY_CURRENT_USER 管理当前登录用户的配置信息；HKEY_LOCAL_MACHINE 管理系统配置信息，包括硬件设置信息和软件设置信息；HKEY_CURRENT_CONFIG 管理系统启动时所使用的硬件信息；HKEY_CLASSES_ROOT 管理系统中打开各种类型文件所需软件的相关配置信息，这个配置单元同时也是 HKEY_CURRENT_USER\Software\Classes（用户设定）和 HKEY_LOCAL_MACHINE\Software\Classes（系统设定）的子类。

上述注册表结构是比较清晰的，但是并没有说明这些配置单元来自哪里，存储在文件系统的什么位置。注册表编辑器中可见的注册表项大部分存储在几个文件中，表 4.1 详细说明了每个配置单元的存储位置。

表 4.1 注册表路径及其对应文件。

注册表路径	文件路径
HKEY_LOCAL_MACHINE\System	%WINDIR%\system32\config\System
HKEY_LOCAL_MACHINE\SAM	%WINDIR%\system32\config\Sam
HKEY_LOCAL_MACHINE\Security	%WINDIR%\system32\config\Security
HKEY_LOCAL_MACHINE\Software	%WINDIR%\system32\config\Software
HKEY_LOCAL_MACHINE\Hardware	可变配置单元
HKEY_LOCAL_MACHINE\System\Clone	可变配置单元
HKEY_USERS\User *SID*	Documents and Settings/*User*/NTUSER. dat； Vista 下目录变为“User/*user*”
HKEY_USERS\. Default	%WINDIR%\system32\config\default

从表 4.1 可以看出，有几个注册表路径在硬盘上并不存在相对应的文件，这些配置单元是在系统启动过程中创建的，系统关机后就不复存在。在静态分析和对开机系统进行调查时，记住这点是非常重要的。

除包含不同配置单元外，注册表还支持不同数据类型的变量。表 4.2 列出了各种数据类型及相应的描述。

表 4.2 注册表数据类型及描述

数据类型	描述
REG_BINARY	二进制
REG_DWORD	32 位整数(4 字节)
REG_SZ	固定长度字符串
REG_EXPAND_SZ	可变长度数据串
REG_MULTI_SZ	多字符串，以空格、逗号或者其他分隔符分隔
REG_NONE	无定义数据类型
REG_QWORD	64 位整数(8 字节)
REG_LINK	unicode 字符串，命名符号链接
REG_RESOURCE_LIST	一系列嵌套的数组，用于存储资源列表
REG_RESOURCE_REQUIREMENTS_LIST	一系列嵌套的数组，用于存储硬件资源的驱动列表
REG_FULL_RESOURCE_DESCRIPTOR	一系列嵌套的数组，用于存储物理硬件设备使用的资源列表

如表 4.2 所示，注册表支持多种多样的数据类型。注册表键的数据类型设置毫无规律可循，用于相同目的注册表键可能有不同的数据类型，这样在对注册表进行查找时，就很可能会遇到问题。一个程序可能把它最近访问的文件列表存储为 ASCII 字符串类型，而另一个程序就可能存储为 unicode 字符串类型，这就意味着如果搜索 ASCII 字符，就会漏掉 unicode 字符。事实上，Microsoft 知识库的一篇文章[3]特别提到，注册表编辑器的查找工具只能用于查找 ASCII 字符，无法查找双字或者是二进制数据。

配置单元文件内的注册表结构

知道注册表文件位置只是了解注册表的开始，本节要在配置单元文件二进制数据层面分析注册表结构。对于这点，很多人可能持怀疑态度，认为没有必要对底层结构进行分析，但是通过对注册表底层结构的了解，就可以在其他位置和资源中（例如，页面文

件、物理内存、未分配空间等）利用关键词搜索获取许多额外的注册表信息。对注册表了解的越多越透彻，就越有可能在操作系统中发现有用的注册表信息。

Mark Russinovich 为 *Windows NT Magazine* 杂志写过一篇很好的文章——“深入了解注册表”（Inside the Registry）[4]，这篇文章详细描述了注册表的不同组成成分（单元）。Mark 和 David Solomon 合著的“*Windows Internals*”一书中，也有这部分内容。

注册表的每种类型单元都有其特定的结构，包含特定类型信息。注册表的各种类型单元及其描述如下所示：

- **键（Key）单元**　包含注册表键信息、到其他单元的偏移量及该键的 LastWrite 时间，特征符是 kn；
- **值（Value）单元**　保存注册表值及其数据，特征符是 kv；
- **子键列表（Subkey List）单元**　由一系列指向其他键单元的索引（偏移量）构成，是父键（Parent Key）单元的所有子键；
- **值列表（Value List）单元**　由一系列指向其他值单元的索引（偏移量）构成，是键单元的所有值；
- **安全描述符（Security Descriptor）单元**　包含一个键单元的安全描述符信息，特征符是 *ks*。

图 4.2 显示的是除安全描述符外，注册表编辑器显示注册表各种单元类型。

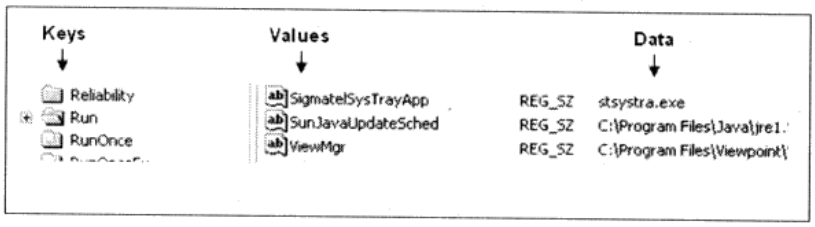


图 4.2　除安全描述符外，注册表的键、值和数据

图 4.3 说明了用十六进制编辑器打开注册表文件的部分内容。图中显示了键单元的特征符和值单元的特征符。需要注意的是，由于是小尾码表示键单元的特征符显示为 *nk*，值单元特征符显示为 *vk*。

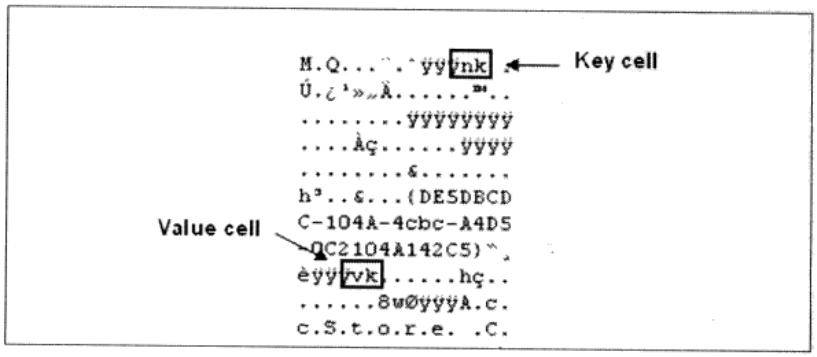


图 4.3　注册表原始文件中的键单元、值单元特征符

前文曾提及，调查过程中这些特征符可以提供非常有用的信息。利用这些特征符，就可以在获取镜像的未分配空间甚至是内存转储里面，找到注册表键和值信息（有关内存转储的更多信息见第 3 章）。

提示

理解注册表键和值的二进制结构对于调查分析是非常有用的。涉及一定程度注册表分析的绝大多数调查工作都会将注意力集中在注册表文件自身。但是，在未分配空间和内存转储中，同样可以发现图 4.3 所示的特征符。对注册表键和值结构的深刻理解将会为读者提供一种从未分配空间和内存转储中，获取相关数据并将其正确解析的方法。例如，可以在内存中找到注册表键，从而获取 *LastWrite* 时间。

关于注册表配置文件各种单元二进制结构的最好参考资料并不是由 Microsoft 提供的。令人惊奇的是，这些有用的资料是一个 Linux 工具开发者（Peter Nordahl-Hagen）提供的。Peter 开发了 Offline Registry and Password Editor 工具[5]，利用这个工具将 Windows 系统分区加载到 Linux 系统后，就可以编辑注册表，甚至可以修改 Windows 用户口令。Peter 提供了这个软件的源代码，其中最值得感兴趣的文件是 ntreg.h，这是一个定义构成注册表各种单元结构的 C 语言头文件。

在 Peter 提供的源代码中，键单元是由长度为 76B 的固定参数结构和可变长度的键名两部分构成的。因此，如果从内存转储或者是未分配簇中定位了键单元的特征符，就可能获取键单元内容。获取从特征符开始的 74B（特征符占用 2B，以小尾码表示 0x6B6E），就会获得键名和 *LastWrite* 时间。

例如，假设已经在一个系统的未分配簇或者是内存转储中定位了键单元。定义一个变量$*offset*，表示到已定位键单元的偏移量，解析这个键单元并从中提取数据的 Perl 代码如下所示：

```
seek(FH,$offset,0);
read(FH,$data,76);
my %nk;
my (@recs) = unpack("vvV17vv",$record);
$nk{id}             = $recs[0];
$nk{type}           = $recs[1];
$nk{time1}          = $recs[2];
$nk{time2}          = $recs[3];
$nk{time3}          = $recs[4];
$nk{no_subkeys}     = $recs[6];
$nk{ofs_lf}         = $recs[8];
$nk{no_values}      = $recs[10];
$nk{ofs_vallist}    = $recs[11];
$nk{ofs_sk}         = $recs[12];
$nk{ofs_classname}  = $recs[13];
$nk{len_name}       = $recs[19];
```

```
$nk{len_classname} = $recs[20];
# Get the name
seek(FH,$offset + 76,0);
read(FH,$data,$nk{len_name});
```

在这段代码中，FH是Perl文件句柄，表示包含所处理数据的文件，*$data*是一个可变变量，表示从文件读取的内容。这段代码提供了一个从键单元中提取基本信息的简单、直观的方法。同样可以提取键单元的LastWrite时间、子键数目、键单元所关联的值及另外数据结构等。例如，其他键单元列表（子键）和值列表（键的值）的偏移量。

值单元长度由18B固定参数结构可变长度名称和可变长度数据3个部分构成。值单元中的一个4B（双字）值定义了其数据类型。使用与键单元同样的解析技术，就可以获取任意来源值单元信息，分析出其实际表示的值。解析值单元的Perl代码如下所示：

```
seek(FH,$offset,0);
my $bytes = read(FH,$data,20);
my (@recs)     = unpack("vvVVVvv",$data);
$vk{id}        = $recs[0];
$vk{len_name}  = $recs[1];
$vk{len_data}  = $recs[2];
$vk{ofs_data}  = $recs[3];
$vk{val_type}  = $recs[4];
$vk{flag}      = $recs[5];
if ($vk{len_name} == 0) {
$vk{valname} = "Default";
}
else {
seek(FH,$offset + 20,0);
       read(FH,$data,$vk{len_name});
       $vk{valname}  = $data;
}
```

对于调查来说，即便是可以解析各种来源的数据，但是获取的信息可能还是不足的，如很难获取有关某个值属于哪个注册表键、创建时间及用途等信息。从未分配空间和内存转储中提取的有关值和键单元的信息只能是作为注册表调查的有益补充，不能作为注册表分析的全部。

基于Peter所提供的源代码，本书作者编写了名为Offline Registry parser的Perl脚本，简称regp.pl，包括了上述的Perl代码。该脚本解析原始注册表文件，提取键单元和值单元信息，并且以ASCII格式在控制台显示这些信息。控制台输出可以重定向到文件，用于归档、搜索、或者和其他文件比较。使用这个脚本及类似代码有几个好处，首先脚本提供了如何解析原始注册表文件的开源实例，具备Perl编程能力的读者

能够很容易扩展这个脚本的能力，例如，将结果输出到电子表格或者是数据库中，以便更加容易地处理。第二，这个脚本仅仅依赖于基本 Perl 功能，没有使用特殊的与平台相关的模块，也不依赖 Microsoft 的 API，这就意味着，该脚本能够运行在任何支持 Perl 的平台上，包括 Linux。如果在一个由 Linux 系统构成的取证平台上运行了 TSK[6] 或者 PyFlag[7] 工具，对注册表文件的解析、查看、分析也不会有问题。最后，与其他取证工具相比，虽然这个脚本没有提供新功能，但是该工具可以更快速、更有效地完成数据处理过程。

Regp. pl 脚本（包括利用 Perl2Exe 编译的 Windows 执行版本）收集于随本书发行的光盘中。

Peter 提供的源代码中，另一个应注意的文件是 sam. h，该文件提供了有关注册表 SAM 配置单元内部结构的有用信息。在“查找用户”一节将会讨论这一结构。

注册表作为日志文件

注册表的键单元构成了注册表编辑器中显示的键或者是目录。键单元是注册表中惟一包含时间值的结构，称之为 *LastWrite* 时间。*LastWrite* 时间是一个 64B 的 FILETIME 对象，类似于文件的最后修改时间（更多信息参看“注册表键 *LastWrite* 时间”）。这一信息不但提供了系统中特定用户行为的时间框架参考，而且在某些情况下，还会提供某个特定值加入键单元的时间，或者是这个特定值被修改的时间。

背景注释

注册表键 *LastWrite* 时间

注册表键有几个和自身相关联的特性，其中之一就是 *LastWrite* 时间，类似于文件或者是目录的最后修改时间。*LastWrite* 时间以 FILETIME[8] 对象（结构）方式存储，FILETIME 对象表示的是以 100ns 时间间隔为单位的数值，起始时间为 1601 年 1 月 1 日（不要问为什么选取这个时间作为起始点）。为了把这个 64b 值转化为易读的形式，需要把它转变为 SYSTEMTIME 结构[9]。把 FILETIME 结构直接转变为 SYSTEMTIME 结构，需要调用 *FileTimeToSystemTime()* API，结果显示为 UTC 格式，这是一种宽松定义的格林尼治时间（GMT）。如果要把 FILETIME 对象转换为准确的当前系统时间，需要调用 *FileTimeToLocalFileTime()* API，这个 API 考虑本地系统的时区信息和夏时制时间。

在对被入侵系统进行调查取证时，*LastWrite* 时间是非常有用的。在 ProDiscover 软件中打开注册表视图，定位到 Windows 服务所属的注册表键区域，对第一层次的键单元按照 *LastWrite* 时间排序，这样就能够很容易确认在系统被入侵期间安装的服务程序，如远程访问后门甚至是 Rootkits。

提示

Rootkits 将在第 7 章中详细讨论。一个由 Network Assocites 命名为 W32/Opanki. worm! MS06-040 的蠕虫病毒就是个很好的例子[10]。该蠕虫病毒在操作系统中安装了一个 Rootkit 组件，创建了两个 Windows 服务（还有一些其他行为）。由于绝大多数危害行为发生在系统安装之后，因此按照 *LastWrite* 时间对服务键进行排序经常可以很快暴露出存在的问题。

除了 *LastWrite* 时间之外，注册表中还有其他一些特殊键值同样保留了时间戳信息。本章稍后将会讨论这些特殊键值，但是就目前而言，知道注册表键的时间信息是由 8B 组成的 FILETIME 对象就足够了。

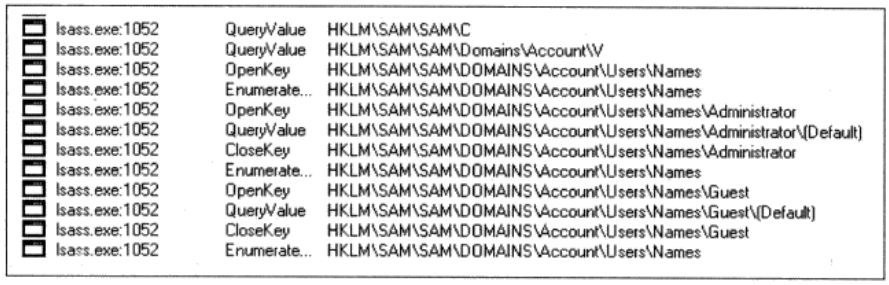
之所以把注册表看成是日志文件，是因为通常而言日志文件是由与时间相联系的行为或者事件构成，注册表恰好满足这一条件。注册表内的成千上万个条目中，并不是所有的条目都是取证调查所关心的。不同类型的调查过程中，所关心的是一些特定的键单元。如果了解这些键单元和值单元是如何建立、修改及系统如何使用，就会知道注册表实际上就是由一系列时间戳相关联的事件构成。

监视注册表变化

对于任何调查过程都适用的完整注册表资源是不存在的。本书配套光盘第 4 章目录下的数据表格文件包含许多在各种调查过程有用的注册表资源，但这些并不是注册表资源的全部。很多情况下，同一软件的不同版本在安装过程中所创建的注册表键以及使用过程中所更改的注册表键是有变化的。即便是在本书印刷、准备发售之后的很短时间内，就会出现新的软件，并且在注册表内增加配置和设定信息。

因此，对我们来说，确定哪些注册表键和注册表值有用是非常重要的。一个常用方法是先对注册表进行快照，执行一个细小动作后（也就是说，只做一个动作），再对注册表进行快照，比较两次快照之间的不同点，从而发现变化的注册表键值。适用于上述快照分析的一个较好工具是 InControl5[11]，InControl5 也是一个用于各种分析工作的主要工具。使用 InControl5 工具进行分析一般包括两个阶段，首先运行 InControl5 并对系统进行快照；在完成必要的工作后（例如，安装 P2P 文件共享程序或者是恶意软件），再次运行 InControl5 对系统进行快照并进行分析。绝大多数情况下输出的 HTML 格式报告已经能够满足分析需要，报告中相关注册表键的增加、修改或者是删除都描述得非常清楚。

另外一个发现有用或者重要注册表键的方法是利用 Microsoft 的 Regmon[12] 工具来实时监视注册表。例如，如果想知道用户信息在注册表中的保存位置时，先启动 RegMon 监控工具，然后在命令提示符中执行 net user 命令，就能够确定在获取用户信息的过程中，系统访问了哪些注册表键和值。图 4.4 显示的是在监视 net user 命令执行过程中，Regmon 的部分输出内容。从图可以看出，虽然执行的是 Net. exe 程序去获取用户信息，但 Net. exe 把这个请求传递给了 Lsass. exe，实际上是 Lsass. exe 访问注册表并获取系统中的用户名信息。



lsass.exe:1052	QueryValue	HKLM\SAM\SAM\C
lsass.exe:1052	QueryValue	HKLM\SAM\SAM\Domains\Account\V
lsass.exe:1052	OpenKey	HKLM\SAM\SAM\DOMAINS\Account\Users\Names
lsass.exe:1052	Enumerate...	HKLM\SAM\SAM\DOMAINS\Account\Users\Names
lsass.exe:1052	OpenKey	HKLM\SAM\SAM\DOMAINS\Account\Users\Names\Administrator
lsass.exe:1052	QueryValue	HKLM\SAM\SAM\DOMAINS\Account\Users\Names\Administrator\(Default)
lsass.exe:1052	CloseKey	HKLM\SAM\SAM\DOMAINS\Account\Users\Names\Administrator
lsass.exe:1052	Enumerate...	HKLM\SAM\SAM\DOMAINS\Account\Users\Names
lsass.exe:1052	OpenKey	HKLM\SAM\SAM\DOMAINS\Account\Users\Names\Guest
lsass.exe:1052	QueryValue	HKLM\SAM\SAM\DOMAINS\Account\Users\Names\Guest\(Default)
lsass.exe:1052	CloseKey	HKLM\SAM\SAM\DOMAINS\Account\Users\Names\Guest
lsass.exe:1052	Enumerate...	HKLM\SAM\SAM\DOMAINS\Account\Users\Names

图 4.4　Regmon 的部分输出

动手实验

监视注册表

下载 RegMon 工具并复制到系统中，执行 Regmon，单击停止捕获（单击放大镜图标，在该图标上会出现一个红叉子）。打开命令提示符（CMD）输入 net accounts，不要按 Enter 键。转到 Regmon 程序启动捕获，回到命令提示符界面，按 Enter 键。当命令提示符界面中出现账户策略时，再回到 Regmon 程序停止捕获。为了简化分析，可以使用 lsass. exe 作为条件进行过滤，查看哪些注册表键被访问并提供了相关信息。这样在静态调查过程中，要获取账户策略信息，就知道去访问哪些键了。

本章后面将会多次提及 RegMon 和 InControl5 的使用，对读者来说，拥有这些工具是非常必要的（由于版权问题，这些工具不在本书配套光盘中发布）。

注册表分析

现在，我们已经知道了注册表的结构，因此如何获取并分析注册表内部信息将是关注的重点。这些注册表信息中的大部分对于开机调查是非常有用的（不包括由于权限问题无法访问的键），除了注册表的动态部分外，所有这些注册表信息，对于静态调查都是有效的。

ProDiscover 提供了一个简单的接口（实际上是 API）用于在静态调查过程中访问注册表。当一个案件加载到 ProDiscover 中时，调查者所要做的仅仅是右击内容视图中 Windows 目录，选择“增加到注册表视图 ProDiscover”选项将会定位所须的文件并进行分析，形成注册表视图，如图 4.5 所示。

ProDiscover 所使用的脚本语言为 ProScript，它基于 Perl 语言，能够以自动方式执行，这就为从注册表提取信息提供了极大的便利。ProScript. pm 模块提供了 API 接口，用于脚本和 ProDiscover 之间的交互。通过 ProDiscover 用户界面能够完成的事情也可以用 Proscript 完成。对于 Windows 注册表和静态取证分析，ProScript 能够完全自动化的收集和格式化大量冗长乏味的数据。本章给出了大量用于执行各种任务的 ProScript（例如，在静态分析过程中从注册表中提取和分析信息的 ProScript）。所有 Pro-

Script 代码位于随本书附赠光盘的 ch4 \ code 目录下。

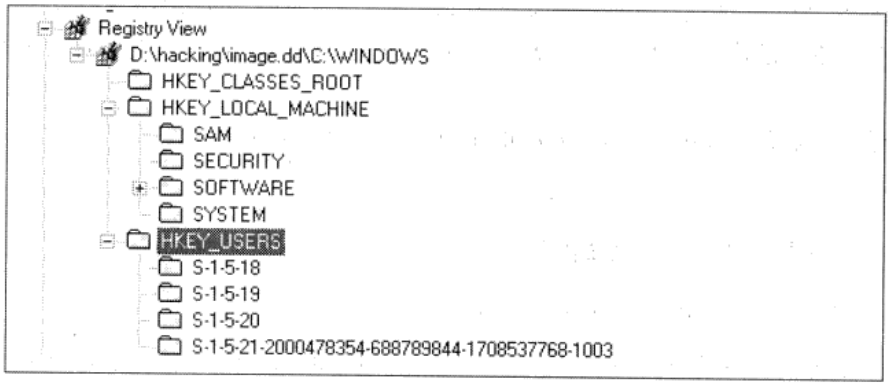


图 4.5　ProDiscover 中的注册表视图

系统信息

静态分析过程中，如果分析对象是已经获取的 Windows 系统镜像，调查者可能会对该系统的很多基本信息感兴趣。在开机调查过程中，这些基本信息中的绝大部分是比较容易获得的。例如，在大多数情况下，简单地查看“资源管理器”，就可以确定操作系统的版本（Windows 2000，XP，2003，Vista 等）；或者可以通过右击“我的计算机”，选择“属性”选项也能查看大量信息，包括操作系统版本、修补包版本及计算机名称等。这些信息实际上存在注册表内，在静态分析过程中也能访问这些信息。

上文曾提及，*CurrentControlSet* 是注册表的动态部分，在获取的镜像内是找不到这个注册表键的。“找到 *CurrentControlSet*”信息栏阐明了在活动系统中确定哪个 *ControlSet* 是 *CurrentControlSet* 的几种方式。一旦确定了哪个 *ControlSet* 是当前使用的，就可以把调查工作集中到这个特定 *ControlSet* 的注册表键。

取证技艺

找到 *CurrentControlSet*

大多数时候，在对获取镜像进行分析时，调查者对注册表视图（ProDiscover）的两个 *ControlSet* 中哪个是操作系统使用的 *CurrentControlSet* 是非常感兴趣的。要确定 *CurrentControlSet*，首先要找到注册表中的 HKEY_LOCAL_MACHINE\System\Select 键，其中显示的是如图 4.6 所示的几个值。

(Default)	REG_SZ	(value not set)
Current	REG_DWORD	0x00000001 (1)
Default	REG_DWORD	0x00000001 (1)
Failed	REG_DWORD	0x00000000 (0)
LastKnownGood	REG_DWORD	0x00000002 (2)

图 4.6　在镜像中定位 *CurrentControlSet*

如图 4.6 所示，操作系统作为 *CurrentControlSet* 使用的 *ControlSet* 序号是 1。在注册表视图中可以看到两个 *ControlSets*：*ControlSets01* 和 *ControlSet02*（某些情况下，可能会存在不同的序号，如 *ControlSets03* 和 *ControlSet04*，但是通常只有两个 *ControlSets*）。

下述注册表键的 *ComputerName* 值中可以发现计算机名称：

```
SYSTEM\CurrentControlSet\Control\ComputerName\ActiveComputerName
```

下述注册表键可以发现系统的最后关机时间：

```
SYSTEM\ControlSet00x\Control\Windows
```

这个注册表键下的 *ShutdownTime* 值是一个 FILETIME 对象，能够与系统中的其他时间相关联（例如，事件日志项，将在第 6 章讨论），并且可能形成系统使用和活动的时间线。

下述注册表键也是调查过程所需要的：

```
SOFTWARE\Microsoft\Windows NT\CurrentVersion
```

这个注册表键中的几个值提供了有关操作系统的基本信息。*ProductName*，*CurrentBuildNumber* 和 *CSDVersion* 提供了关于操作系统名称、版本（包括修补包）的信息。*RegisteredOrganization* 和 *RegisteredOwner* 值，虽然不一定填写，但也能提供有关操作系统的进一步信息。*ProductId* 和 *InstallDate* 也是同样有用的。

时区信息

下述注册表键可以发现有关时区设置的信息：

```
SYSTEM\CurrentControlSet\Control\TimeZoneInformation
```

该信息对于建立操作系统活动时间线是极为重要的。本章余下部分讨论从注册表中获取信息的各种脚本，其他章节讨论文件、事件日志项等。其中大部分有效工具都会获取有关时间和日期信息，利用来自 *TimeZoneInformation* 键的 *ActiveTimeBias* 值（以分钟列出）对系统其他事件源的时间信息进行转换或者规范化，如日志文件的条目。

共享

默认情况下，Windows2000，XP，2003 和 Vista 系统会在系统中创建隐藏管理共享，包括 IPC$ 共享、ADMIN$ 共享、指向系统根硬盘的 C$ 共享和 D$ 共享等。如果用户创建了一个另外的共享（例如，通过 net share 命令创建），这个共享会出现在下述键（除非特别指出，本节所有的注册表键都位于 HKEY_LOCAL_MACHINE 配置单元）：

```
SYSTEM\CurrentControlSet\Services\lanmanserver\Shares
```

利用 Offline Registry Parser 工具提取样本镜像中的 SYSTEM 配置单元文件信息时，得到下述条目信息：

```
\ $$$ PROTO. HIV\ControlSet001\Services\lanmanserver\Shares
LastWrite time: Fri Aug 20 15:19:35 2004 (UTC)
        --> Temp;REG_MULTI_SZ;CSCFlags = 0 MaxUses = 4294967295 Path = C:\Temp
Permissions = 0 Remark =  Type = 0
```

使用 LiveView[13]工具将这个特殊镜像转换为 VMware 虚拟机格式，在 Vmware 中运行该虚拟机，在虚拟机中增加共享（C:\Temp 目录共享为 Temp）。上述注册表键的 *LastWrite* 时间就是该共享加入系统的时间。如果存在多个共享（一般是在一段时间内不断增加的），该注册表键的 *LastWrite* 时间对调查者来说可能就没有什么用处了，这是因为该注册表键没有保留最近使用列表（MRU，本章后面讨论 MRU 列表）。

此外，默认情况下共享注册表键下也可能没有任何共享列表，这只是意味着用户没有创建任何新共享。如果要了解用户或者管理员是否采取了相应的措施禁用隐藏的共享，则需要查看下述所列注册表键下的值：

```
SYSTEM\CurrentControlSet\Services\lanmanserver\parameters
```

如果此键下有名为 AutoShareServer[14]的值且其数据为 0，这就意味着该系统已被特别处理过，无法创建隐藏的管理共享。

审计策略

系统的审计策略[15]是由安全配置单元维护的，位于 Policy\PolAdtEv 注册表键下，该注册表键下默认值的数据类型为 REG_NONE，该值包含的二进制信息代表编码后的审计策略。利用 Offline Registry Parser 工具在一个样本镜像中提取的审计策略如下所示：

```
\SECURITY\Policy\PolAdtEv
LastWrite time: Fri Sep 9 01:11:43 2005
    --> Default;REG_NONE;01 17 f5 77 03 00 00 00 03 00 00 00 00 00 00 00 00 00 00 00
00 00 00 03 00 00 00 03 00 00 00 00 00 00 00 03 00 00 00 09 00 00 00
```

二进制数据的第一个双字（首 4B）表示审计策略是否启用，在这个例子中，值为 01，意味着启用了审计策略（00 表示没有启用审计策略）。Windows 2000 和 XP 系统有 9 种可被审计的事件类型，每种由字节序列中一个双字表示，最后一个双字未被使用。

为了解密这些信息，需要了解一些格式。把下面的模板映射到从 PolAdtEv 注册表键下的二进制数据上：

```
0Z XX XX XX AA 00 00 00 BB 00 00 00 CC 00 00 00 DD 00 00 00 EE 00 00 00 FF 00 00 00 GG 00 00
00 HH 00 00 00 II 00 00 00 XX 00 00 00
```

值 Z 决定审计策略是否启用（1 代表启用，0 代表未启用）。其余的值对应如下（忽略 X 代表的值）：

AA	Audit System Events(审计系统事件)
BB	Audit Logon Events(审计登录事件)
CC	Audit Object Access(审计对象访问)
DD	Audit Privilege Use(审计特权使用)

EE	Audit Process Tracking(审计过程跟踪)
FF	Audit Policy Change(审计策略更改)
GG	Audit Account Management(审计账户管理)
HH	Audit Directory Service Access(审计目录服务访问)
II	Audit Account Logon Events(审计帐户登录事件)

对每个字母对，00 表示无审计，01 表示审计成功的事件，02 表示审计失败的事件，03 表示审计成功和失败的事件。上文从样本镜像获得的数据显示，要对系统事件、登录事件、策略更改事件、账户管理事件和账户登录事件的成功及失败进行审计。

调查过程中，这些信息对调查者来说是非常有用的，因为它告诉调查者可以在事件日志中找到哪几类事件的审计记录。

无线 SSIDs

在活动系统上（绝大部分是笔记本），Windows XP 管理一个曾经接入过该系统的 SSIDs（Service set identifiers）列表，这一列表由下述注册表键维护：

```
SOFTWARE\Microsoft\WZCSVC\Parameters\Interfaces\{GUID}
```

键中的 *GUID* 是无线网卡的 GUID。在这个注册表键下有一个名为 *ActiveSettings* 的值和几个名为 *Static*#000*x* 的值（此处，x 是从 0 开始的整数）。这些值都是二进制形式的，其二进制数据包含任何曾经访问过的无线接入点 SSIDs。从二进制数据偏移量为 0x10 位置开始的双字类型数值包含 SSID 的长度。SSID 的名称紧接着这个双字，长度由双字确定。图 4.7 显示的是 *ActiveSettings* 值的内容，图中 SSID 已被高亮显示。

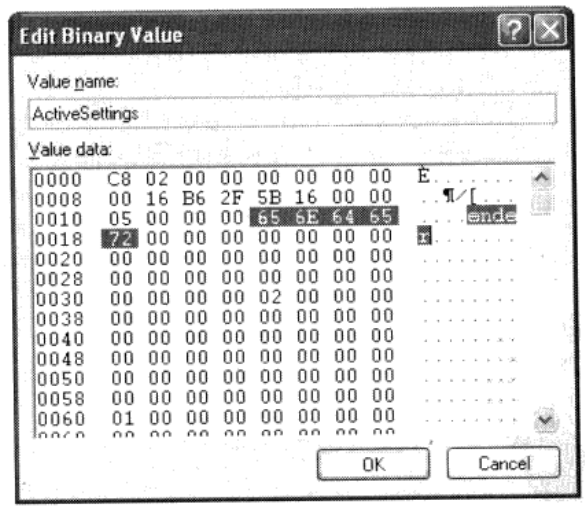


图 4.7 *ActiveSettings* 值的内容，数据来自活动样本系统

自动启动位置

注册表中的自动启动位置指无需和用户交互就可以调用程序的位置，以及用户通常不了解，但同样可以运行程序的位置。最初提供这些位置主要是为了方便用户，例如，自动

启动笔记本上的触摸板驱动程序、反病毒和防火墙软件等，能够减少很多用户操作。另外，用户为了方便会让一些程序（例如，即时聊天客户端）在登录系统时自动启动。

2004年，Microsoft研究中心网络安全和系统管理研究组的成员在Usenix大会上发表了一篇文章，文章题目是“*Gatekeeper: Monitoring Auto-Start Extensibility Points for System Management*”。在这篇文章里，作者把自动启动位置称为自动启动扩展点（auto-start extensibility points，ASEPs）。这篇文章采用了与本书作者不同的方式对ASEP进行分类，并且提供一幅图表用以说明间谍软件最常用的ASEP是浏览器帮助对象（BHO）。BHO实际上是IE浏览器启动时自动调用的并能够监视用户行为的动态链接库（DLL）。BHOs案例研究将说明如何使用自动启动/ASEP。

你知道吗？

案例研究：BHOs

对BHO感兴趣的时候，我还是一个财务服务公司的安全管理员。我的老板向顾客提供信用监控和盗窃保护服务，通过对信用卡的多层次监控，能够为用户提供从季度财务报告到实时查询E-mail通知等服务。如绝大多数生意一样，我的老板也有竞争对手。有一次，我接到来自公司商业发展部门的电话，报告说他们很可能遇到了一个网络安全问题。公司的一些用户在浏览公司网站时，发现所有包含我们公司名称的地方都被高亮显示，单击这些高亮区域就会转到竞争对手的网站。

这意味着客户的计算机上感染了BHO（已经感染了几个客户，同时我们公司商业发展部门的同事已经确认如何被感染），并在不停地监视用户的浏览行为。各个公司可以向广告软件/BHO提供者订购这样的服务，无论何时，只要竞争对手的名字出现在网页上，BHO就会自动把该公司名称转换为指向BHO订购者网站的超级链接。因此，假如消费者电子公司A和广告软件提供者签订协议，当这个广告软件感染了用户的系统时，每次当用户加载的网页上有A的竞争对手名字时，这些名字就会被指向A公司的超级链接代替。

幸运的是，Microsoft知识库文章KB298 931[16]向用户提供了如何禁止BHO的方法。这篇知识库文章中的链接提供了更多有关BHO的细节信息，BHOs位于下述注册表键下：

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\Browser Helper Objects
```

注意这个HKEY_LOCAL_MACHINE下的注册表键，位于该键下的BHO只要浏览器启动就会加载，与用户无关。

本书附赠光盘上的Perl脚本Bho.pl，可以查看安装在活动系统上的所有BHOs。

一个天真的想法认为绝大多数用户和管理员并不需要熟悉注册表内的自动启动位置，而且还认为只有少数几个注册表键（特别是到处存在的 Run 和 RunOnce 键）可用于上述目的。操作系统提供的文档和程序部分支持了这种想法。例如，在一个活动的 Windows XP 系统上运行 MsConfig 命令就可以调用系统配置工具[17]。运行这个命令的操作如下：单击“开始”菜单，选择“运行”选项，输入 MsConfig，按 Enter 键。图 4.8 显示的是系统配置工具打开后 Startup 选项卡的内容。

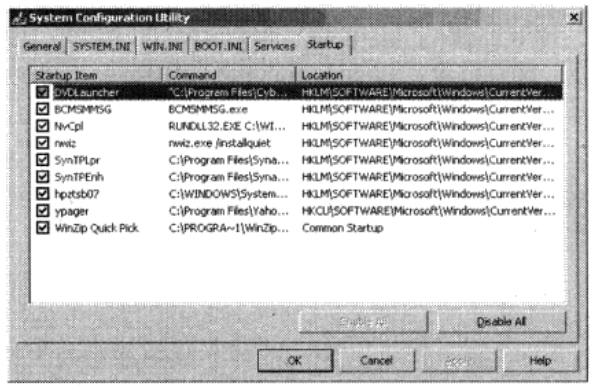


图 4.8　系统配置工具 Startup 选项卡

每位读者都应在自己的 Windows XP 系统上运行这个命令，并且仔细查看位置（Location）那列的内容，就会发现出现的注册表键包括 HKEY_CURRENT_USER 和 HKEY_LOCAL_MACHINE 中的 Run 键，不幸的是，这也是惟一出现的注册表键。程序开发者可以利用名为 Win32_StartupCommand 的 WMI 类自动获取这些注册表键的内容[18]，和系统配置工具一样，Win32_StartupCommand 仅仅检查注册表内很有限的几个启动位置。

另一个极端的想法是注册表内启动位置的数量是如此之多，因此对这些位置的检查最好留给专业人士或者专业人员开发的软件去完成，例如，使用商业反病毒和反间谍软件工具进行检查。

实际上，真实情况刚好介于这两种想法之间。注册表内确实有大量的自动启动位置，但是其中很多是不需要考虑的。接下来的几节中，我们把这些自动启动位置分为系统启动时访问的注册表键、用户登录时访问的注册表键和用户在系统上执行动作访问的注册表键 3 个部分分别讨论，并且列举查看这些自动启动位置的方法。需要说明的是，这 3 个部分讨论的自动启动位置并不是注册表内全部的自动启动位置。本书附赠光盘中的电子表格文件 regref.xls 包含几个工作表，每个工作表中都包含属于工作表目录下的多个注册表键，每个注册表键都有一个简要地描述，包括适用范围、描述该键功能的可信参考等。可以将这个电子表格文件作为进行注册表分析的起始点，由于该表格并不能列出所有的程序，以及不断的版本变化等，因此不能认为这个表格包含了全部的内容。

系统启动

系统启动时访问的注册表自动启动位置也是恶意软件作者最喜欢的地方，这些位置可以不经用户允许，甚至在不用登录系统的情况下调用恶意软件。其中一个位置是Windows服务，更明确的是：

```
HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services
```

系统启动时，由Current *ControlSet*值决定使用哪个*ControlSet*设定，接下来扫描该*ControlSet*内的服务列表，调用标示为自动运行（Start值为0x02）的服务。

通常情况下，当执行入侵分析时主要依靠的是系统镜像和事件报告（基本上，报告中除了“我们在什么时间看到了什么”以外，不会有别的内容）。面对这样的情况，初步调查的简单做法是用ProDiscover打开镜像文件，加载注册表视图，定位标记为*Current*的*ControlSet*，基于*LastWrite*时间对服务键下的子键进行排序。这些时间通常与操作系统安装日期（此时安装了绝大部分服务和驱动）及管理员已知的合法软件安装日期排列一致。一些案件中，本书作者发现了由攻击者安装的服务和驱动，包括Rootkit和一个名为rdriv.sys驱动文件，该文件是一个用于Trojan木马和IPC病毒的Rootkit驱动文件。另外一些案件中，本书作者定位到的一些服务，不管是名字还是*LastWrite*时间都无法与事件报告相关联。

利用这种分析技术，还可以发现一些有趣的事情。例如，WinPcap工具安装的npf.sys驱动，这个驱动也会被与WinPcap工具类似功能的软件安装，当然也可能会被攻击者安装。

用户登录

依据Microsoft文档的描述，直到用户登录后，才算完成了一个完整的系统启动过程。用户登录系统后，系统会访问和解析特定的注册表键，运行这些注册表键下列出的程序。这些键包括[19]：

（1）HKLM\Software\Microsoft\Windows\CurrentVersion\Runonce

（2）HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run

（3）HKLM\Software\Microsoft\Windows\CurrentVersion\Run

（4）HKCU\Software\Microsoft\WindowsNT\CurrentVersion\Windows\Run

（5）HKCU\Software\Microsoft\Windows\CurrentVersion\Run

（6）HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce

上述注册表键部分采用了简写，*HKLM*代表HKEY_LOCAL_MACHINE，*HKCU*代表HKEY_CURRENT_USER。

新用户登录到系统时，系统就会解析（1）、（3）、（5）、（6）注册表键，运行这些键下所列的程序[20]。如果系统运行于安全模式，默认忽略Run注册表键下的程序列表。但是在Windows XP和2003系统中，即便该系统是以安全模式启动[21]，只要RunOnce值［键（1）和（6）］是以星号（*）开始，就会强制该值所关联的程序运行。此外，在Windows XP系统上，键（1）、（3）、（5）和（6）是为向下兼容而准备的，这样为较早版本Windows（早于Windows XP）开发的软件仍可以运行[22]。

用户活动

此类下自动启动注册表位置是与用户动作相关联的，如打开 IE 或者 Outlook 程序。如果在系统上启动 RegMon 后，仅仅移动鼠标或者是打开一个程序（或者什么也不做），就会发现有相当数量的注册表访问记录。与其他的自动启动位置一样，恶意软件（病毒、木马、蠕虫等）作者同样会利用这些注册表键，以确保他们的产品能够生存的更久、运行的更好。其中一个应特别注意的位置如下：

```
HKEY_LOCAL_MACHINE\Software\Classes\Exefile\Shell\Open\command
```

这个注册表键和其他类型文件（Bat 文件、Com 文件等）的注册表键控制该类文件打开时的动作。例如，在 Windows 资源管理器中，右击任何文件都会出现一个快捷菜单，在菜单的顶部有一个粗体的单词“打开”。用粗体表示的动作就是在大多数情况下，双击文件会执行的操作。当双击一个文件时，Windows 就会扫描注册表，查找此类文件的设置并根据该类文件的设置来决定需要采取的操作。如 SirCam[23]和 PrettyPark[24]这类恶意软件就是利用了此类注册表键来维持其在被感染系统上的生存。

例如，假设要在系统上玩纸牌游戏，进入命令提示符，输入下述命令：

```
C:\>dir /s sol.exe
```

此命令输出的是纸牌游戏执行程序在文件系统的位置（在 Windows XP 家庭版中，sol.exe 位于 C:\Windows\system32目录）。如果想知道双击纸牌游戏的图标时会发生什么事情，可以输入下述命令：

```
C:\>ftype exefile
```

此命令的输出是 *exefile*= “%1”%*。这意味着系统会用第一个参数（文件名）和后续参数来调用这个文件。可以在 Shell \ Open \ command 中增加注册表项，这样在这个特定类型文件打开时，就会调用其他文件。这个注册表键下的默认注册表项只应包含简单的“%1”%*，不能有其他数据，该注册表项中的任何其他数据都是可疑的，要立刻调查。

另一个需要分析的具有相似信息的注册表项如下：

```
HKEY_CLASSES_ROOT\Exefile\Shell\Open\Command
```

并不是只有 HKEY _ CLASSES _ ROOT 键下的 ExeFile 项具有运行软件的功能，有些恶意软件会用同样的方法修改其他的注册表项以确保其生存。例如，有一些后门软件修改了下述注册表键的项目：

```
HKEY_CLASSES_ROOT\Word.Document.x\shell\open\command
```

在这个例子中，X 是 Word 软件的版本号（8，9 等）。该注册表项告诉系统，只要通过 Windows 资源管理器打开 Word 文档，如双击文档，恶意软件就会执行。

另外一个可能会以相似方式使用的注册表键如下：

HKEY_LOCAL_MACHINE\Software\Microsoft\Command Processor\AutoRun[25]

只要命令处理器（cmd.exe）运行，这个值所列的软件就会运行，也就是说，只要运行命令提示符，就会运行该软件。该注册表键下的默认值是空的。HKEY _ CURRENT _ USER 下也有同样的值，如果此处存在这样一个项目，它的优先级是高于 HKEY _ LOCAL _ MACHINE 下项目的。

实现上面所述的操作是非常容易的。打开 RegEdit 定位到命令处理器注册表键，在 Windows XP PRO SP2 系统中，这个值是可见的，并且没有关联任何数据。右击这个值选择“修改”选项，按 Enter 键。图 4.9 表示能够将其他数据加入该注册表值。

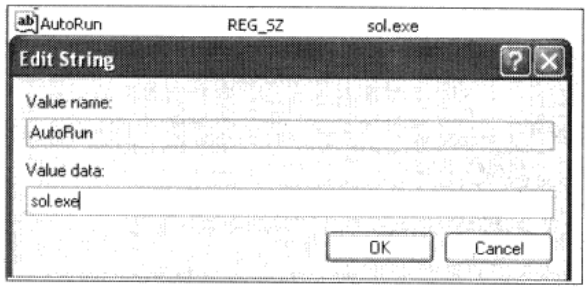


图 4.9 在命令处理器注册表键中增加数据

一旦修改完成，单击 OK 按钮。单击 Start | Run，输入 Cmd，按 Enter 键。命令提示符打开，但刚才选择的程序（sol.exe）也打开了。图 4.9 中选择的程序具有很好的 GUI 界面，因此在运行命令提示符时，能够清楚显示运行了其他程序。同样的道理，在这种机制下其他操作也能够被执行，例如，安静的增加用户、增加服务或者是启动没有 GUI 界面的木马后门等。

谈及 GUI，存在一个很少了解的注册表键，只要 GUI 程序启动，这个键就可将 DLL 加载到内存。这个键是

HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows\AppInit_DLLs[26]

2000 年，J. D. Glaser（以前服务于 FoundStone 公司，现就职于 NTObjectives 公司）在 BlackHat 和 Usenix 大会上做了有关追踪服务器泄密和在 AppInit _ DLLs 注册表键中发现可自动运行注册表项的报告。

AppInit _ DLLs 注册表键作为恶意软件的掩藏位置是相当有效的，一个已知的利用此位置的间谍软件是 CoolWebSearch。为什么这个注册表键如此有效？在作者讲授“应急响应实战”课程时，最初安排的训练是一个简单的“感染”练习，这个练习的主要目的是观察参加者的调查过程，而不是看哪个参加者能够首先发现这个感染。参加这个课程的学员包括新手、有经验的 Windows 管理员、负有管理 Windows 系统责任的 UNIX 和 Linux 资深管理员。每个学员所选择的第一步肯定是在桌面上运行 GUI 工具，当然他们使用的 GUI 工具是不同的，有事件查看器、任务管理器甚至是资源管理器等。显然，使用 GUI 程序进行调查是参加者的本能反应。

Windows 操作系统对特定事件的发生提供了外部报警功能，如用户的登录、退出、屏幕保护启动等。这些通知事件是由下述注册表键管理的：

HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon\Notify \[27]

此注册表键下的项目指向接受特定事件通知的 DLL。如果在 Google 上搜索 Winlogon\Notify，则会列出一个很长的列表，都是有关利用此注册表键功能恶意软件的。在对系统进行取证分析时，按照 LastWrite 时间对 Winlogon \ Notify 下的子键进行排序是个很好的做法。对与入侵事件发生时间相近的注册表项要特别注意，同样需要注意的注册表项还有版本信息可疑的或者根本没有版本信息的 DLLName 项（从可执行文件中获取版本信息见第 6 章）。

位于 Winlogon 键（上文已列出）下 TaskMan 注册表项也是调查人员非常感兴趣的。该注册表项允许用户使用选择的程序代替任务管理器。这一注册表项在默认情况下是不存在的，但可以增加。实际上，在安装来自 Sysinternals 的 Process Explorer 时，就允许用户选择是否用该软件代替任务管理器。绝大多出情况下，如果在调查过程中发现 Winlogon 键下有 TaskMan 注册表项，就需要对该注册表项所列的程序进行仔细调查。

另一个应引起注意的注册表项如下：

HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options

该注册表项允许用户（通常是软件开发人员或者是程序调试人员）指定一个软件运行过程中所需调用的特定调试器。Microsoft 提供了几篇有关利用该注册表项调试 CGI 程序[28]及在 Windows XP 下关闭系统自动更新[29]的知识库文章。例如，在 Image File Execution Options 键下创建一个新的子键，子键值为需要阻止运行的应用程序名（不需要程序路径），在该子键下增加名为 Debugger 的注册表项，该项的取值为 ntsd，这样在运行该程序时就会运行调试器从而导致该程序立即退出。然而，Dan Epps[30]提出了一种利用该键值作为“攻击向量”的方法，实际上就是一种恶意软件生存的方法。具体做法如下：首先在 Image File Execution Options 键下增加一个子键，子键键值为需要替代的可执行程序名（例如，Notepad. exe，不需要程序路径，只需可执行文件名）；接着在该子键下增加名为 Debugger 的字符串类型注册表项，取值指向命令提示符如图 4. 10 所示。

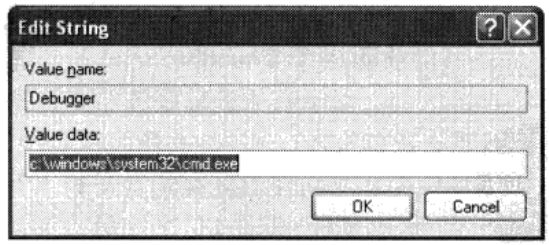


图 4. 10　在 Image File Execution Options 键下增加 Debugger 项

单击 OK 按钮，在 Start | Run 中输入 Notepad 后，运行的是命令提示符，而不是 Notepad。这一过程不需要重启。

对上述注册表键还有多种利用方法。假设 Debugger 注册表项指向已经被木马感染的 Notepad 程序，那么执行 Notepad 时，不但 Notepad 会执行，后门、蠕虫等恶意软件也会执行。现在已经掌握的大量恶意软件样本都会在注册表中建立据点，因此对这些注册表键的检查是非常重要的。

枚举注册表自动启动位置

目前，从活动系统中获取自动启动信息的最有效工具是来自于 SysInternals. com 的 Autorun 工具[31]，该工具包括 GUI 和 CLI 两个版本，图 4. 11 显示的是 AutoRun 工具的 GUI 版本。

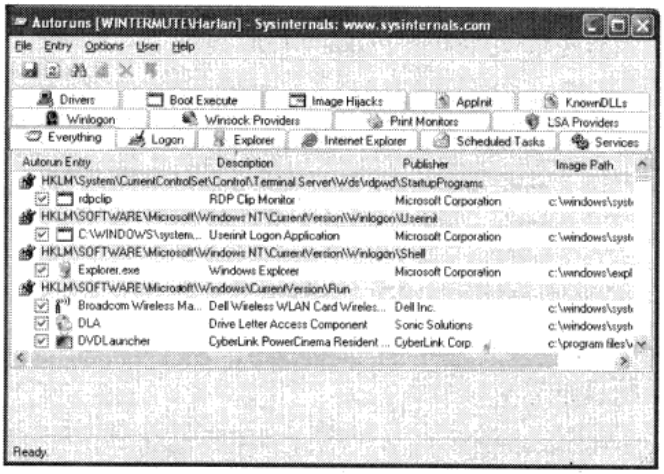


图 4. 11　AutoRun 工具 GUI 版本界面

如图 4. 11 所示，AutoRun 检查大量的注册表键值并显示自动启动项的位置。AutoRun 同样检索注册表项指向的可执行程序的发布者和描述信息，并且在 Image Path 列中显示出来。这些信息为调查人员提供了可执行程序是否可疑及是否需要进一步调查的快速指示。

除 AutoRun 外，还有其他一些获取自动启动位置的工具，其中一个基于 VB 脚本（Visual Basic Scripts）的工具是 Slient Runners，该工具来源于 www. silentrunners. org 网站，此网站上包含 VB 脚本（该脚本 2004 年 5 月 12 日首先发布于 www. ntbugtraq. com 网站）枚举调用点的完整列表。这个脚本基本上适用于绝大部分 Windows 操作系统（包括 Windows 98）。使用这个脚本时应注意的是，枚举的自启动位置是与操作系统版本相关的，这些与操作系统相关的位置在列表中已经标明。

在静态调查过程中，调查人员需要的工具不仅能够查看利用系统镜像文件重构的注册表，而且还要能够枚举注册表。ProDiscover 的 ProScripts 功能允许调查人员使用为活动系统开发的类 Perl 脚本去搜索静态镜像中的注册表。类似 AutoRun 的工具应保持更新，这样就能提供具备自动启动功能的注册表键的完整列表。

USB 移动存储设备

当一个 USB 移动存储设备，如闪存盘插入 Windows 系统时，就会在注册表中留下痕迹（有些痕迹同时还保留在 setupapi. log 文件中）。当移动存储设备插入计算机时，即插即用管理器（Plug and Play，PnP）接收这一事件，并且在设备的固件（Firewre information，此信息并不存储在设备的存储区内）中查询有关该设备的设备描述符信

息，如厂商、型号、序列号、数据传输模式等。接着，PnP 管理器利用这些信息查找适用于该设备的驱动程序（基于.inf 文件的内容）。如有必要，装载这些驱动（这一过程信息记录在 setupapi.log 文件中）。当该设备被识别后，在下述键值下创建一个新的注册表键。

```
HKEY_LOCAL_MACHINE\System\CurrentControlSet\Enum\USBSTOR
```

在这个键值下，会看到与下面结构类似的子键：

```
Disk&Ven_###&Prod_###&Rev_###
```

这个子键代表设备类标识符（device class identifier），标识设备的一个特定类。上述子键中“###”代表的区域由 PnP 管理器依据在设备描述符中获取的数据填写。例如，一个 1GB 的 Geek Squad U 盘的设备类 ID 如下所示：

```
Disk&Ven_Best_Buy&Prod_Geek_Squad_U3&Rev_6.15
```

利用 UVCView[32] 工具可以查看设备描述符的内容。上述的 Geek Squad 内存 U 盘的部分设备描述符信息如图 4.12 所示。

```
iManufacturer:                    0x01
     English (United States)  "Best Buy"
iProduct:                         0x02
     English (United States)  "Geek Squad U3"
iSerialNumber:                    0x03
     English (United States)  "0C90195032E36889"
```

图 4.12 UVCView 显示的部分设备描述符

从图 4.12 可见，设备描述符中的生产商、设备型号信息和设备类 ID 的内容是互相对应的。

设备类 ID 一旦建立，就需要为特定设备建立一个惟一的实例标记（unique instance identifier，UID）。注意图 4.12 中的 *iSerialNumber* 的值，这实际上就是移动存储设备的 UID，相当于网卡的 MAC 地址。使用这个值作为设备惟一的 UID，就可以把具有同一设备类标识的多个存储设备在操作系统中区分开来。下述说明来自 USB FAQ[33]。

如果设备有序列号，Microsoft 使用该序列号标记同类设备的不同实例。例如，如果两个设备的描述符具有相同的 *idVendor*，*idProduct* 和 *bcdDevice* 值，那么 *iSerialNumber* 值就会将两个设备分开。

图 4.13 表示的是注册表中设备类 ID 及其下的 UID。

虽然大部分设备厂商会在设备中提供序列号以便通过 Microsoft 认证程序[34]，但是并不是所有的移动存储设备都有序列号。对于没有序列号的设备，PnP 管理器会为这些设备创建一个类似下面格式的 UID：

```
6&26c97b61&0
```

需要注意的是，这个 UID 的第二个字符是 &，如果一个 USBStor 键下 UID 的第二个字符是 &，就意味着该设备的设备描述符中没有序列号，也就是说，无法在相同类

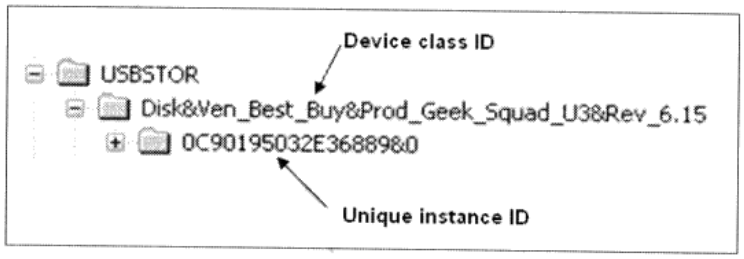


图 4.13　Regedit 中显示的部分设备类 ID 和 UID

型的设备中确定是哪个接入系统了。因此，如果 UID 中的第二个字符中不是 &，就意味着能够惟一标记接入系统的设备。在调查范围包括多个系统和多个存储设备的情况下，调查人员应使用包括 UVCView、注册表痕迹、MountedDevices 键和文件快捷方式等多种办法确定接入系统的每个设备。

UID 建立后，该键值由包括 *FriendlyName* 在内的多个注册表项构成。其中，调查人员最感兴趣的是 *ParentIdPrefix* 注册表项。对于该注册表项，Microsoft 没有提供任何有关该注册表项如何创建及该注册表项在系统内是否惟一的信息。但是 *ParentIdPrefix* 可以用来关联注册表内的其他信息。

例如，利用 UID 和 *ParentIdPrefix* 可以确定 USB 设备最后接入操作系统的时间。在一个活动系统上，HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\DeviceClasses 键值下有一系列的子键，其中应注意的是两个设备类：{53f56307-b6bf-11d0-94f2-00a0c91efb8b} 和 {53f5630d-b6bf-11d0-94f2-00a0c91efb8b}。这两个设备类是在 ntddstor.h 头文件中定义的磁盘和卷的全局惟一标识符（GUIDs）。定位到第一个 GUID，可以看到一些具有相当长名称的子键，如下所示：

```
USBSTOR#Disk&Ven_Best_Buy&Prod_Geek_Squad_U3&Rev_6.15 #0C90195032E36889&0 #
{53f56307-b6bf-11d0-94f2-00a0c91efb8b}
```

在这个键值中，黑体字的部分就是 UID。这个键值的最后写入时间就是设备最后接入系统的时间。同样，利用 *ParentIdPrefix* 可以实现 GUID 卷设备的关联。下面的键值中黑体部分就是 *ParentIdPrefix*：

```
##?# STORAGE # RemovableMedia # 7&326659cd&0&RM # {53f5630d-b6bf-11d0-94f2-
00a0c91efb8b}.
```

这个键值的最后写入时间同样也是该卷设备最后接入系统的时间。在 MountedDevices 节中会介绍更多有关从注册表中获取 *ParentIdPrefix* 相关信息的资料。

USB 设备问题

长久以来，人们（特别是计算机安全专业人员）就意识到 USB 移动存储设备对安全，特别是对于公司内部安全所构成的威胁。自从软盘出现起，随着存储容量的不断增加，存储设备的尺寸却在不断变小。在撰写此书时，2GB 和 4GB 的内存 U 盘已经有售，

而且价格合理。如果想从公司的计算机内盗窃文件甚至是完整的数据库，最好的工具毫无疑问就是俗称“闪存”的 USB 设备了。拆下闪存的塑料壳，里面就是一个拇指指甲盖大小的电路板，这就是闪存的字面解释。

更为糟糕的是，USB 存储设备不仅体积小、容量大，而且到处都是。以前拥有 64MB 闪存的绝大部分是系统管理员，而现在几乎每个人都有这些设备，用来存储图像、演示等文件。以前在体育馆、办公室、公共汽车随处可见的 iPods 和 MP3 播放器，现在经常出现在办公桌的笔记本上。现在，一个 8GB 的 iPod Nano 播放器价格大概为 200 美元。如果在公司内网中笔记本计算机上接入这些移动存储设备，如何能够判断使用者是在听音乐还是下载公司的财务预测报告、补偿计划、合同标书及其他的秘密信息呢？

USB 存储设备的另一个问题与自动播放（AutoPlay）功能有关，这是 Windows 提供的增强用户体验（Enhanced User Experience）的一部分。在 Windows 操作系统上，放入 CD 或者是 DVD 光盘后，系统会检测到新放入的介质，如果发现根目录中存在 Autorun. inf 文件，就会解析并执行该文件中的 *Run*＝行和 *Load*＝行程序。默认情况下，对于闪存这类移动存储设备的自动运行功能是不启用的。自动播放功能是由 NoDriveTypeAutoRun[35] 注册表键控制的（需要注意的是，该键值在 HKEY_USERS 和 HKEY_LOCAL_MACHINE 下都有）。

U3 公司为用户提供了一个增强用户程序便携性的工具，该工具在闪存的起始部分建立一个标记为 CDFS（CD 文件系统）的小分区，因此 Windows 系统把这个分区识别为 CD 而不是移动存储设备。这样 CDFS 分区上的程序就可以运行，而移动存储设备剩余的空间就可以用来存储数据。这就意味着，虽然默认情况下移动存储介质的自动播放功能是关闭的，但是由于默认情况下 CDFS 分区的自动播放功能是打开的，所以同样可以自动运行程序。

当把一个 U3 类型的闪存接入 Windows 操作系统时，在注册表中会产生两个独立的设备类 ID 项，如下所示：

```
CdRom&Ven_Best_Buy&Prod_Geek_Squad_U3&Rev_6.15
Disk&Ven_Best_Buy&Prod_Geek_Squad_U3&Rev_6.15
```

上述两个设备类 ID 下的子键具有相同 UID。当实施静态取证调查时，需要特别注意这些相同 UID 的设备，因为这些设备能够自动运行程序，可能会造成泄密或者是感染恶意软件。实现这一感染过程并不困难，首先攻击者会创建一个定制的 ISO 镜像并安装到 CDFS 分区上，接着删除所有的 U3 程序或者设备上的标记。如果这个定制的设备插入系统中，CDFS 分区的自动运行功能就会启用，这样攻击者想进行的操作（安装木马后门、收集被保护的存储信息及口令等）都会自动执行。

USBDumper 工具说明提供了更多有关移动存储设备的威胁信息，虽然没有与注册表分析关联，但是还应引起安全专业人员的注意。

工具和陷阱

USBDumper

一个名为 USBDumper 的工具以不同的方式揭示了 USB 移动存储设备的另一个安全威胁[36]。USBDumper 安装在 Windows 操作系统上，无论何时，只要 USB 移动存储设备接入系统，该工具就会以安静方式将设备上文件数据复制到系统中。同样应被提及的另一个工具则能够安静获取接入系统中 USB 移动存储设备的镜像，这样，不只设备中当前存在的文件会被获取，而且设备中已经删除的文件也会被获取。Bruce Schneier 于 2006 年 8 月 25 日在他的 Blog（Schneier and Security）中详细探讨了这两类问题。

Mounted Devices

MountedDevices 注册表键存储着挂载到 NTFS 文件系统中的各种设备和卷的相关信息，该键值的完整路径是 HKEY_LOCAL_MACHINE\System\MountedDevices。例如，当一个 USB 设备接入 Windows 系统时会被分配一个盘符，这个盘符会在 MountedDevices 注册表键中显示出来。如果一个设备分配的盘符是 F:\，那么 MountedDevices 注册表键中显示的值就是*DosDevices*\F:。利用设备 UID 键值中 *ParentIdPrefix* 就可以实现 MountedDevices 注册表键和 USBStor 键值之间的映射。上节中讨论过的 USB 设备的 *ParentIdPrefix* 值是 *7&326659cd&0*。注意这个值不是设备的 UID，因此这个值不是本书前面所讨论的设备序列号。

一旦获取了 USB 设备的 *ParentIdPrefix* 值，通过对 MountedDevices 注册表键下 *DosDevices* 注册表项所包含 *ParentIdPrefix* 数据的分析，就可以确定分配给这个 USB 设备的盘符。在活动系统上，对 MountedDevices 注册表键下的每个 *DosDevices* 注册表项右击，选择 Modify 选项，出现 Edit Binary Value 对话框后，可以查看是否包含 *ParentIdPrefix* 值。在注册表中 *ParentIdPrefix* 值的数据类型是串（String），但是 *DosDevices* 项的数据类型是二进制（binary data），所以需要做一些必要的格式转换工作。图 4.14 图解说明了 *DosDevices*\F:项的 Edit Binary Value 对话框。

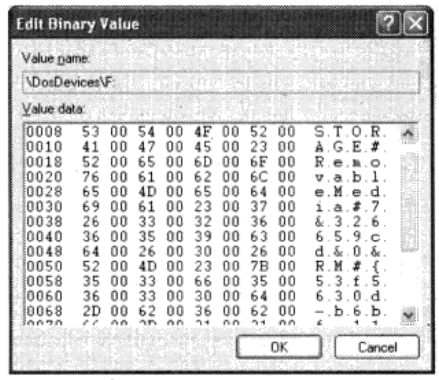


图 4.14　*DosDevices*\F:项数据

图 4.14 清楚显示了此设备的 *ParentIdPrefix* 值是 *7&326659cd&0*，利用 *ParentIdPrefix* 值在 MountedDevices 注册表键和 USBStor 键值之间进行映射，可以确认分配给该 USB 存储设备的驱动器盘符。在静态取证调查过程中，如果发现注册表的其他部分或者是快捷文件指向 F:\，那么可以将 UID 键值、MountedDevices 注册表键的最后写入时间（*LastWrite*）、文件的 MAC 时间进行关联，形成时间链（TimeLine）。

警告

当利用 *ParentIdPrefix* 值在 MountedDevices 注册表键和 USBStor 键值之间进行关联时，一定要记住，接入系统的不同设备可能会分配同样的盘符。

在 MountedDevices 注册表键下，有些 *DosDevices* 表项（例如，*DosDevices*\C:）的数据只有 12B 长（也就是 3 个双字 DW(ORDs)，一个典型的例子如图 4.15 所示。

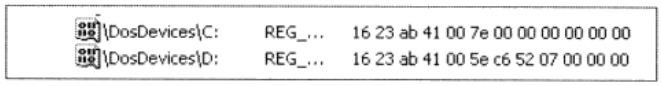


图 4.15 *DosDevices*\C: 项和 *DosDevices*\D: 项数据

如图 4.15 所示的二进制数据的第一个双字（DW(ORD）是磁盘特征符（也就是卷 ID，注意卷 ID 和卷序列号是不同的，译者注），磁盘特征符位于磁盘 MBR 扇区的 0x1b8（十进制为 440）位置。第二个双字和第三个双字是分区偏移量。

就图 4.15 所示的 *DosDevices*\C: 表项而言，第二个和第三个双字转换成十六进制是 0x7e00，十进制是 32 256。磁盘的每一个扇区为 512B，因此 32 256/512＝63，因此 C 分区起始位置为 63 扇区。图 4.15 所示的 *DosDevices*\D: 表项，其二进制数据的第一个双字和 *DosDevices*\C: 项是一致的，也就是说，具有相同磁盘特征符，但后面的分区偏移数据不一致。这是因为 D 分区和 C 分区是同一个物理硬盘上的不同分区。利用分区偏移数据，可以计算出 C 分区至少 29GB：

```
0x0752c65e00 = 31 453 502 976/1024/1024/1024 = 29. 29GB
```

下面说明在取证过程中如何获取这些信息。首先利用 Prodiscover 软件创建一个新案件。在注册表视图中定位到 MountedDevices 注册表键下的\DosDevices\C: 项，在二进制数据中可以看到磁盘特征符为 5D EC 5D EC，在案件中打开簇视图（起始位置为 0），图 4.16 引用自 Prodiscover，显示的是 MBR 扇区一部分。

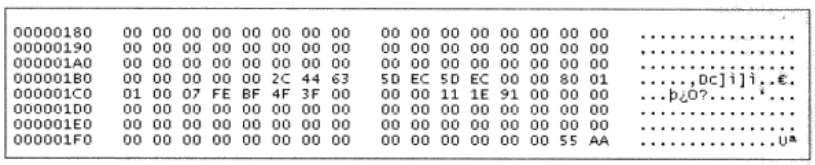


图 4.16 Prodiscover 簇视图显示的磁盘特征符

从图 4.16 知，在 MountedDevices 注册表键中发现的磁盘特征符在偏移量为 0x1b8 的位置清楚可见。

这点的重要性不言而喻，如果已经获得了硬盘的镜像，就可以从 MBR 和 MountedDevices 注册表键中得到磁盘特征符并对其进行验证。如果有其他外置磁盘和案件联系在一起，这些信息就变得非常有用了。在本书作者作为应急反应人员和取证工程师的日常工作中，经常使用外置硬盘存储类似客户数据、硬盘镜像、日志文件等数据。之所以这么做，是因为一旦完成案件调查工作，提交了最终调查报告，就可以很容易地擦除外置硬盘上的数据。但如果是在操作系统硬盘上管理这些数据，就不得不先擦除数据，然后重新安装操作系统、应用程序和数据，包括 PGP 密钥等。另外，撰写本书时，所有与本书有关的文件都保存在一个 120GB 的西部数据（Western Digital）USB 硬盘上。这个设备在 USBSTOR 键值下的设备类 ID 如下所示：

```
Disk&Ven_WDC_WD12&Prod_00UE-00KVT0&Rev_0000
```

正如期望的那样，这个设备具有序列号（可以通过 UVCView 软件验证），注册表中的这个 UID 是可以作为证据使用的。但是，在注册表 UID 键值下，没有名为 *ParentIdPrefix* 的键值。当这个设备接入计算机时，显示为 *G*:\盘，该设备的很多信息包括磁盘特征符，都存储在 MountedDevices 注册表键中，这样即便是没有 *ParentIdPrefix* 值，一样可以确定接入系统的外置硬盘。

MountedDevices 注册表键下的设备卷，同用户配置单元（一般而言是 NTUSER.dat 文件）中的 MountedDevices2 键值下的子键是相互关联的。MountedDevices2 位置如下：

```
Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2
```

列在这个键值下的子键包含各种形式。首先，子键中具有 GUIDs 的可以与 MountedDevices 键下的\??\Volume 注册表项相互映射，这样就为确定那一个用户（如果存在多用户）使用这个卷提供了可能，同时基于这个键值的最后写入时间（*LastWrite*），就可以知道该卷的最早接入系统的时间。

查找用户

注册表中有关用户的信息存储在 SAM 配置单元中。如果不采用特殊步骤手工编辑该配置单元存取权限，通常情况下，即便是管理员也无法读取该配置单元文件。这样做的优点显而易见：注册表的大部分内容即便有些混乱也不会影响系统的正常运作，但是包括 SAM 配置单元在内的一小部分注册表内容即使是发生微小的改变也会造成系统不稳定。

SAM 配置单元中包含很多有用信息，但这些信息是按照二进制格式编码的。幸运的是，Peter Nordahl-Hagen 提供的 sam.h 文件中给出了解密 SAM 文件结构的方法，这就为我们将编码数据解释为可读格式提供了相当大的帮助。

随本书赠送光盘中的 UserDump.pl 的 ProScript 脚本文件（v.0.31，20060522）安装后，就可用于从 Prodiscover 的注册表视图中获取用户和组信息。要运行这个 ProScript 脚本，单击菜单条中的 Run ProScript 按钮，从 Run ProScript 对话框中选择 Pro-

Script 脚本位置，选择 UserDump. pl 脚本，一旦脚本执行完成，从 SAM 配置单元中获取的信息会在结果窗口中显示出来，就可以进行选择、复制、粘贴、报告等操作。

下面举例说明 UserDump. pl 脚本所返回的信息。这个 ProScript 脚本获取的是系统中用户账户的信息，包括用户名、描述、账户建立日期、登录次数和用户标志等。如果时间信息存在，该 ProScript 脚本同样会显示最后登录时间。

```
Username : Administrator
Comment : Built-in account for administering the computer/domain
Acct Creation Date : Thu Aug 19 17:17:29 2004
RID : 500
Logins : 0
Flags :
        Password does not expire
        Normal user account
Username : Mr. Evil
Acct Creation Date : Thu Aug 19 23:03:54 2004
RID : 1003
Logins : 15
Flags :
        Password does not expire
        Normal user account
```

这些用户信息位于下述路径的 *F* 值中：

```
SAM\SAM\Domains\Account\Users\{RID}
```

{*RID*} 或者有关的标识符是安全标识符（SID）的一部分，SID 是标识用户、组和计算机账户的惟一号码。Microsoft 不但提供了有关 SID 如何创建的大量信息，还提供了一个 RIDs 列表[37]，包括众所周知的用户、组和 Aliaes 的列表（SAM \ SAM \ Domins \ Builtin \ Aliases）。

该注册表键内的 *F* 值为二进制数据类型，必须对其进行正确解析（见 sam. h 文件）才能获取全部信息。*F* 值的二进制数据内容中包含一些重要的日期信息，特别是几个以 64 位 FILETIME 格式表示的时间戳信息，这些重要的时间值及其位置如下：

- Bytes 8～15　表示账户的最后登录日期
- Bytes 24～31　表示最后重设口令的日期（如果口令未被重设或改变，表示账户创建时间）
- Bytes 32～39　表示账户有效日期
- Bytes 40～47　表示最后一次登录失败日期（如果登录时账户名正确，则表示最后一次输入错误口令日期）

AccessData 公司的 Registry Viewer 工具能够自动解密上述这些信息，如图 4. 17 所示。

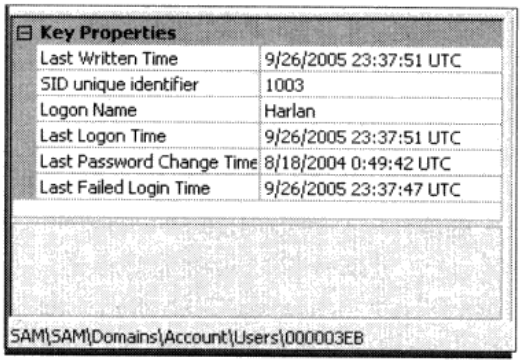


图 4.17　Registry Viewer 工具显示的用户 F 值的部分内容

Registry Viewer 工具可从 AccessData 公司网站下载（www. accessdata. com）[38]，在没有软件狗的情况下能以 Demo 方式运行。要利用 Registry Viewer 解码 *F* 值，首先必须从镜像文件中获得原始注册表文件（SAM 文件）并复制到其他位置。

Parse:: Win32Registry[39] Perl 模块提供了一种免费的、跨平台方法，能够解析从镜像文件中提取的原始注册表文件。要在 Windows 系统中安装这个模块，只需要下载这个模块文件并把它复制到 Perl 安装目录中的“\site\lib”子目录即可。随书赠送光盘中的 Perl 脚本文件 sam _ parse. pl 利用这个模块不仅可以获取和显示 Registry Viewer 可以获得的信息，还包括一些其他信息：

```
Name : Harlan
LastWrite = Mon Sep 26 23:37:51 2005 (UTC)
Last Login = Mon Sep 26 23:37:51 2005 (UTC)
Pwd Reset = Wed Aug 18 00:49:42 2004 (UTC)
Pwd Fail = Mon Sep 26 23:37:47 2005 (UTC)
--> Password does not expire
--> Normal user account
Number of logins = 35
```

如上所示，sam _ parse. pl 脚本不仅显示了从 SAM 文件用户信息中获取的时间信息，还解析了用户标志，并且给出了用户登录的次数。

由于 Win32Registry 模块并不依赖 Windows APIs，因此利用这一模块的 Perl 脚本是与操作系统平台无关的。这就意味着，当调查人员想解析原始注册表文件内容时，并不仅限于 Windows 平台。只要原始注册表文件有效，利用这一模块的代码可以运行在 Linux，Windows，甚至 MacOS X 等支持 Perl 的平台上。

SAM\SAM\Domains\Account\Users\{RID} 键下还有一个 *V* 值，同样是二进制数据类型，从中可以解析出用户账户设定信息，例如，用户全名、描述、启动脚本路径和加密后口令散列值等。

UserDump. pl 还可以获取系统中的组信息，包括组名、描述及组内用户：

```
Group : Administrators
```

```
Comment : Administrators have complete and unrestricted access to the
computer/domain
--> 500 (Administrator)
--> 1003 (Mr. Evil)
```

有关组成员信息是由 SAM \ SAM \ Domins \ Builtin \ Aliases 注册表键维护的。Aliases 键下的每个 RID 子键都有一个二进制格式的 *C* 值，对 *C* 值的解析可以确定某个用户所隶属的组。解析 *C* 值二进制数据结构的最好办法是由 Andreas Schuster 的 Blog[40] 提供的，这种方法已经写进 UserDump. pl ProScript 脚本。

用户和组成员信息对于了解案件背景是相当重要的，特别是在了解哪些用户访问过系统及他们的访问等级方面。在开机调查过程中利用合适的工具可以获取许多有关用户和组的信息。通过对注册表所维护的各种结构的深入了解，在静态分析过程中也可以获取相似信息。

追踪用户活动

许多注册表键能用于追踪用户活动。这类注册表键与自动启动/用户活动注册表键是不同的，这些是与用户执行特定动作相关的注册表键。这些注册表键位于用户目录下的 NTUSER. dat 文件中，当用户执行特定动作时，这些键值，包括键值的 *LastWrite* 时间就会被更新。这就是本章前面所提到的可以把注册表看成是日志文件的概念。另外，还有一些注册表键也可用于追踪用户活动，这些注册表键时间戳信息的增加或者修改是与注册表项数据相关联的，这些时间戳信息是由注册表项数据维护的。

注册表中记录用户活动的大部分位置位于 HKEY _CURRENT _USER 配置单元内，这些位置中使用最多的是最近使用列表 MRU 注册表键，在这些注册表键下的每个注册表项都有一个标识符，如一个小写字母。MRU 列表是按照其被访问的时间进行排序的。

UserAssist 注册表键

有很多文章提及 UserAssist 注册表键，但是其中大部分只是提出了有关 UserAssist 注册表键的问题。我们感兴趣的这个特定注册表键位于 NTUSER. dat 文件中的下述注册表键之下：

```
Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{GUID}\Count
```

上述键值中的 *GUID* 是全局惟一标识符，一般情况下，UserAssist 键下有两个 *GUID* 键：5E6AB780-7743-11CF-A12B-00AA004AE837 和 75048700-EF1F-11D0-9888-006097DEACF9。在 HKEY _CLASSSES _ROOT 配置单元内，5E6AB780-7743-11CF-A12B-00AA004AE837 *GUID* 指向 Internet 工具条（% SystemRoot% \ system32 \ browseui. dll），75048700-EF1F-11D0-9888-006097DEACF9 *GUID* 指向活动桌面（% SystemRoot% \ system32 \ SHELL32. dll）。这些键的重要性不言而喻，因此需要对这些键进行分析、获取其中的内容。

在每个 Count 键下都有几个（实际上是很多）注册表值。例如，一台操作系统是

Windows 2000，已经运行了5年之久的计算机上，Internet Toolbar键下有超过18 000个站点记录，活动桌面键下大约有400个记录。这些键有什么特别之处？简言之，它们记录了用户活动，特别是以类似日志文件的方式记录用户活动。

然而，在REGEDIT中定位到这些键后，只有类似HRZR_HVGBBYONE的记录，看不到可理解的记录。这是因为对这两个键下的注册表项名称进行了Rot-13加密，Rot-13加密也称为凯撒置换（Caesarian Cipher），是一种利用字母表中13个位置之后字母替代当前字母的算法。利用简单的转换算法（在Perl中，tr/N-ZA-Mn-za-m/A-Za-z/），就可以把HRZR_HVGBBYONE转换为其原始数据：UEME_UITOOLBAR。这样的转换虽然提高了可读性，但是离完全解析的要求还是有些差距。

这两个键下的注册表项虽然采用了Rot-13加密，但是能够很容易地解密。实际上，uAssist.pl的Perl脚本（随书赠送的光盘中）说明了在活动系统上完成这一转换过程是非常简单的。真正的宝藏是该键下与注册表项相关联的数据。大部分情况下，解密后的注册表项名指向应用程序或者可执行程序。在此情况下，项值数据一般是16B（4个双字），不但包含程序运行计数（应用程序曾经运行的次数），还包含程序的最后运行时间（8B的FILETIME对象）。运行计数存储在第二个双字中，从5开始计数，也就是说，如果运行计数是6，程序的实际运行次数是一次。FILETIME对象存储在第三个和第四个双字中。

解密后的注册表项名称大部分以UEME_开始，接着是*RUNPATH*，*RUNPIDL*，*RUNCPL*等。对这些标志的说明如下：

- **RUNPATH** 指向文件系统的绝对路径，当在Windows浏览器中双击可执行程序图标或者是在开始-运行中输入应用程序名产生。
- **RUNCPL** 指向控制面板程序。
- **RUNPIDL** PIDL是ID列表指针（Pointer to ID List），IE命名空间的一部分，指向一个对象。在UserAssist键下，通常是开始-文档中选择的快捷方式或者是LNK文件。

下面举例说明，分析对象是一台2006年8月份购买的DELL Latiltude D820计算机。购买这台计算机后，用户格式化了硬盘并重新安装了操作系统，对DELL计算机来说，这意味着需要下载并安装几个驱动程序。在该系统上运行uAssist.pl脚本后，出现下述项目：

```
UEME_RUNPATH:F:\D820\D820_A02_bios.exe
Mon Aug 7 16:35:39 2006 – (1)
```

这个脚本以本地时间返回FILETIME对象，这样就清楚表明D820_A02_bios.exe程序执行过一次，执行时间是2006年8月7日。uAssist.pl脚本执行后的其他项目还包括

```
UEME_RUNCPL:"C:\WINDOWS\system32\desk.cpl",Display
Thu Aug 24 21:27:45 2006 – (1)
UEME_RUNCPL:"C:\WINDOWS\system32\powercfg.cpl",Power Options
Thu Aug 24 21:27:07 2006 – (1)
```

从上述项目可以看到，控制面板的显示程序和电源管理程序在8月24日执行过，且只执行了一次。为说明上述项目中时间为最后执行时间，在10月4日晚上10:55左右再次执行控制面板中的显示程序后，重新运行uAssist.pl脚本，发现DisPlay项的执行时间已经变为Oct 4 22:55:59 2006。

由此可见，这些键记录了特定应用程序执行的次数和最后运行时间。这些信息对于案件取证是非常重要的。例如，如果发现UEME_RUNCPL：timedate.cpl项目，就意味着控制面板的时间和日期程序执行过，系统的时间就有可能被认为调整过。

调查过程中，Pnu.pl脚本（在随书赠送光盘内）能够帮助收集NTUSER.dat文件内的信息。该脚本使用Perl::Win32Registry模块访问原始NTUSER.dat文件，查找指向活动桌面GUID的UserAssist键。这个脚本获取该键的*LastTime*时间，解密并获取键下注册表项名称。该脚本的输出格式如下所示，按照每个注册表项数据中的时间进行排列：

```
LastWrite time = Mon Sep 26 23:33:06 2005 (UTC)
Mon Sep 26 23:33:06 2005 (UTC)
        UEME_RUNPATH
        UEME_RUNPATH:C:\WINDOWS\system32\notepad.exe
Mon Sep 26 23:26:43 2005 (UTC)
        UEME_RUNPATH:Z:\WINNT\system32\sol.exe
Mon Sep 26 23:22:30 2005 (UTC)
        UEME_UISCUT
        UEME_RUNPATH:Downloads.lnk
Mon Sep 26 23:16:26 2005 (UTC)
        UEME_RUNPATH:C:\Program Files\Morpheus\Morpheus.exe
Mon Sep 26 23:16:25 2005 (UTC)
        UEME_RUNPATH:Morpheus.lnk
Mon Sep 26 23:15:04 2005 (UTC)
        UEME_RUNPATH:C:\Program Files\Internet Explorer\iexplore.exe
Mon Sep 26 23:04:08 2005 (UTC)
        UEME_RUNPATH:d:\bintext.exe
```

UAssist.pl脚本（版本号为0.11 20060522，在随书赠送光盘内）是一个在Prodiscover中可用的ProScript脚本。只要在Prodiscover中完成注册表视图，即可执行该脚本。该脚本解析系统所有用户UserAssist键项、提取相关信息、解密注册表项名称、获取运行次数和最后运行时间。当上述任务完成后，该脚本按照时间戳顺序对所有的值进行排序，因而获取的信息能用于时间线分析。这一版本的ProScript脚本将获取结果发送到Prodiscover的命令窗口中，这样调查人员就可以选择、复制、粘贴这些结果到其他文件或者是报告内。

你知道吗?

案件研究：磁盘整理?

某公司有一个调查案例，该公司的一个雇员涉嫌删除了自己系统内的一些文件，在删除文件中发现了可疑的文件名，但是无法获取文件内容，而且还有线索表明该雇员不但删除了文件，还进行了磁盘整理操作。Prefetch 目录中的信息表明磁盘整理工具曾运行过，但是 UserAssist 键中的信息表明这个雇员只运行了控制面板中的添加或删除程序，没有运行过控制面板中的磁盘整理工具。最后发现这个结果是由 Windows XP 每三天自动进行的磁盘整理操作造成的。

提 示

如果需要调查的操作系统安装了 IE 7.0，在 UserAssist 注册表键下存在第三个子键。

MRU 列表

很多程序都维护一个最近访问文件列表，也就是 MRU 列表。在这些程序运行时，这些文件名一般出现在文件下拉菜单的最下面。

注册表中最著名的 MRU 列表是 RecentDocs 键：

```
\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs
```

这个键下的所有注册表项都是二进制数据类型。在所有项目中，需要注意的是名称为数字和 MRU 列表 Ex 的注册表项。以数字命名的注册表项包含访问的文件名（unicode 编码格式），以 MRU 列表 Ex 命名的注册表项包含文件访问顺序（双字）。例如，在某个计算机上，MRU 列表 Ex 项数据第一个双字的值是 0x26（十进制 38），而名为 38 的注册表项指向的是一个曾经打开的目录。假如在 RecentDocs 键下增加一个注册表项及其相对应的数据，MRU 列表 Ex 项的数值也会随之变化，同时 RecentDocs 键的 *LastTime* 时间也会发生改变，这样就能够确定最后访问文件的时间。

RecentDocs 键也包含很多子键，每个子键的名称是打开过的文件扩展名（.doc,.txt,.html 等）。这些子键下注册表项的维护方法同 RecentDocs 键一样：注册表项以数字命名，其数据为二进制数据类型，数据内容为访问的文件名；MRU 列表 Ex 注册表项同样也是二进制数据类型，内容为访问文件的顺序，最后访问的文件排在最前面。

另外一个常见的 MRU 列表位于 RunMRU 键下：

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\RunMRU
```

这个键记录了在 Start | Run 对话框中输入的命令列表。图 4.18 表示的是该键下的内容。

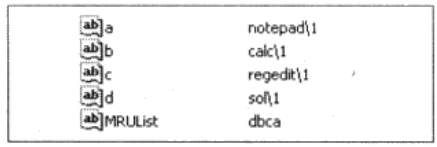


图 4.18 RunMRU 键的部分内容

RunMRU MRU 列表为文本格式，这比 RecentDocs 键下的内容要更加易读。同 RecentDocs 键一样，最近输入的项目排在 MRU 列表的最前面。在 RunMRU 键下增加项目的方法是单击“开始”菜单，选择“运行”选项，输入命令或者是文件名。对远程系统访问的项目及对移动存储介质上程序和文件访问的项目也会出现在 RunMRU 键下。

与 RunMRU 键类似的一个注册表键是 TypedURLs 键：

```
HKCU\Software\Microsoft\Internet Explorer\TypedURLs
```

与 RunMRU 键相似，TypedURLs 键记录了在 IE 地址栏中输入的 URLs 列表。这一信息与临时网络文件结合，就可以知道哪些是用户手工输入地址访问的站点，哪些是用户点击链接访问的站点。

在下述注册表键下可以找到另外一个 MRU 列表：

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\ComDlg32\OpenSaveMRU[41]
```

这个注册表键记录通过 Windows shell 打开和另存为文件的 MRU 列表。与 RecentDocs 键类似，OpenSaveMRU 键同样包含子键，子键名为打开或者保存文件的扩展名。这些子键的数据类型与 RunMRU 键类似，是字符串类型，非常易读。OpenSaveMRU 下子键的内容在很多方面是非常有用的。首先，在正常使用系统的过程中，有一些文件扩展名并不是经常出现的，这样在 OpenSaveMRU 键下的包含这些扩展名的子键就有可能只有一个名为 *a* 注册表项。这样，MRU 列表就只有一个惟一的 *a* 列表项，如图 4.19 所示。因此，这个子键的 *LastWrite* 时间就是该文件的打开或者保存时间。

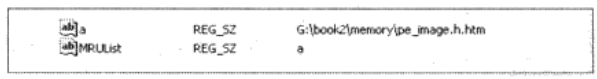


图 4.19 OpenSaveMRU 键下子键的内容

在 Prodiscover 中打开一个 Windows XP 系统的镜像，定位到用户的 OpenSaveMRU 键下的 exe 子键，exe 子键的 *LastWrite* 时间是 2004 年 8 月 27 日上午 11：18，exe 子键的内容如图 4.20 所示。

由图 4.20 可见，最近访问的文件是 Ethereal 安装工具（Ethereal 是网络抓包分析工具集，现在称作 Wireshark），用于安装 Ethereal 程序。这一信息可与 UserAssist 键（使用 uAssist.pl 脚本，版本 0.11，20060522）内容一起进行关联分析。运行该脚本后，得到如下输出内容：

a	REG_SZ	C:\Documents and Settings\Mr. Evil\Desktop\lalsetup250.exe
MRUList	REG_SZ	cdba
b	REG_SZ	C:\Documents and Settings\Mr. Evil\Desktop\netstumblerinstaller_0_4_0.exe
c	REG_SZ	C:\Documents and Settings\Mr. Evil\Desktop\ethereal-setup-0.10.6.exe
d	REG_SZ	C:\Documents and Settings\Mr. Evil\Desktop\WinPcap_3_01_a.exe

图 4.20　OpenSaveMRU 键下 exe 子键内容

```
--> Fri Aug 27 15:34:54 2004
        --> UEME_RUNPATH:C:\Program Files\Ethereal\ethereal.exe
--> Fri Aug 27 15:33:02 2004
        --> UEME_RUNPATH:C:\Program Files\Cain\Cain.exe
--> Fri Aug 27 15:28:36 2004
        --> UEME_RUNPATH:C:\Documents and Settings\Mr.
        Evil\Desktop\ethereal-setup-0.10.6.exe
--> Fri Aug 27 15:15:08 2004
        --> UEME_RUNPATH:C:\Documents and Settings\Mr.
        Evil\Desktop\WinPcap_3_01_a.exe
--> Fri Aug 27 15:14:44 2004
          --> UEME_RUNCPL
        --> UEME_RUNCPL:"C:\WINDOWS\System32\appwiz.cpl",Add or Remove Programs
--> Fri Aug 27 15:12:35 2004
        --> UEME_RUNPATH:C:\Program Files\Network Stumbler\NetStumbler.exe
--> Fri Aug 27 15:12:11 2004
        --> UEME_RUNPATH:C:\Documents and Settings\Mr.
        Evil\Desktop\netstumblerinstaller_0_4_0.exe
```

上述数据清楚表明，该用户在 2004 年 8 月 27 日那天是相当忙碌的（注意，该 ProScript 脚本提取的是原始 FILETIME 数据并将其转化为 UTC 时间。UTC 时间大约等于 GMT 时间。ProDiscover 程序显示的所有时间都是相对于调查者计算机系统的时间并按照时区设置进行了调整）。时区设置（TimeZoneInformation）注册表键表示的是在系统运行过程中所设定的中心时区，并且可按照夏时制自动调整。在通常的系统上，ActiveTimeBias 注册表项的数据是 300min（5h），而在调查上述案件的计算机上，ActiveTimeBias 的值为 240min（4h）。由上述信息就可做出如下判断：在 Ethereal 安装程序保存后大约 10min，“嫌犯”就安装了这个程序（（8 月 27 日 11:18+4 小时）-8 月 27 日 15:28=10 分钟）。

OpenSaveMRU 键所包含的信息对于显示外置存储设备也是非常有用的。子键不仅提供了外置存储设备接入系统的 *LastWrite* 时间信息，而且还可以结合 MountedDevices 键的内容，以提供有关外置存储设备更多的信息。

下面的注册表键包含另一个 MRU 列表：

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\FileExts
```

该键下的子键与系统曾打开文件的扩展名相对应。在文件扩展名子键下，有两个名为 OpenWithProgIDs 和 OpenWithList 的子键。当用户双击与子键名相同的扩展名文件

时，这两个子键下的注册表项数据告诉系统调用哪个程序去打开该扩展名类型文件。

搜索辅助（Search Assistant）

Windows XP 系统中，当用户单击“开始”菜单，选择“搜索 | 文件或文件夹”后，在对话框中输入的搜索条件保存在下述注册表键内：

```
HKCU\Software\Microsoft\Search Assistant\ACMru
```

ACMru 键通常由 5001，5603，5604 和 5647 4 个子键组合而成。5001 子键包含网络搜索助手（Internet Search Assistant）的 MRU 列表；5603 子键包含 Windows XP 中文件和文件夹搜索的 MRU 列表；5604 子键包含搜索“文件中的一个字或词组”的 MRU 列表；5647 包含搜索“计算机或人”的 MRU 列表。这些子键下的注册表项名称为 3 位数字，最小的数字（如 000）代表最近的搜索，子键的 *LastWrite* 时间就是最近搜索的执行时间。

了解这些子键的功能和它们的组合方式，更有利于调查人员洞察用户的活动规律。特别是在调查过程中想要了解用户在什么时间做了什么，这是非常有用的。

对于 Windows 2000 这种较早的操作系统，其搜索信息存储在不同的注册表位置中。如果操作系统是由 Windows 2000 升级到 XP 的，其搜索信息注册表键如下所示：

```
Software\Microsoft\Internet Explorer\Explorer Bars\
{C4EE31F3-4768-11D2-BE5C-00A0C9A83DA1}
```

基于 HKEY _ CLASSES _ ROOT \ CLSID 键的内容，上述 GUID 指向 shell. dll 中的文件搜索浏览区。该键下有 FilesNamedMRU 和 ContainingTextMRU 两个子键，分别与 Windows XP 系统下的 5603 和 5604 子键相对应。

连接其他系统（Connecting to Other System）

当用户利用网络驱动器映射向导（右击“我的电脑”图标，选择“映射网络驱动器”选项）连接远程系统时，就会在下述注册表键下生成 MRU 列表：

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\Map Network Drive MRU
```

该键下的注册表项以字母命名，MRU 列表项表示用户连接或者共享磁盘的顺序。

不管用户使用的是网络驱动器映射向导还是 *net use* 命令，用户增加到系统中的卷都会出现在下述注册表键下：

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2
```

前文曾提及，MountPoints2 子键下名为 GUIDs 的注册表项能够与 MountDevices 键下的\ \?? \Volume 注册表项互相映射。GUIDs 项也能与 MountPoints2 \ CPC \ Volume 子键下的注册表项相映射。

例如，在一个测试系统上，使用 *net use* 命令连接另一台计算机上的 C $ 共享，在测试机的注册表中生成了 # #192. 168. 1. 22#c$和# #192. 168. 1. 71# c$两个子键。这两个 IP 地址（测试环境使用的简单网络，没有设置域，因此计算机名称基本上以 IP 地址代

替）还出现在下述注册表键下：

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\ComputerDescriptions
```

这个注册表键记录了网络浏览器所能看到的计算机描述。如果计算机系统接入网络，该键下一般会有几个计算机名称；如果是独立的计算机系统，如家庭计算机或者是未接入网络的计算机，该键下就不存在注册表项了。

在本书作者的家庭计算机上，只有作者使用 *Net use* 命令连接过的系统才会出现在该键下；在本书作者的工作计算机上（通过 VPN 连接到公司内网），ComputerDescriptions 键下出现的都是作者曾经连接过的计算机，其中一台计算机的描述为 Samba2.2.7a，这就意味着这台计算机运行的是 Samba Linux 系统。由于该注册表键位于 NTUSER.dat 文件内，因此同一计算机上不同用户之间，这些注册表项是不同的。

IM 和 P2P

现在，即时通信（IM）和点对点（P2P）文件共享程序是非常流行的，几乎每个联网的人都在使用。在 IM 中，用户输入的信息很快会以 E-mail 方式发送给接收者，并且等待接收者的应答。即便是远在千里之外，也可以立即收到朋友登录 IM 程序的通知。利用 P2P 文件共享程序，可以找到大量有用的（或者没有那么有用）文件：音乐、视频和图像等。即时通信和点对点文件共享的广泛流行也催生出了各种不同的服务框架和客户端程序。雅虎、美国在线和 Microsoft 都有自己的 IM 客户端程序，这几个程序的功能各有不同，而且在系统中留下的“足迹”也是独一无二的。除此之外，还有各种第三方程序可用于代替上述这些客户端程序，甚至可将这些客户端程序集成到同一个界面中。例如，Trillian（Trillian.cc）允许用户将其他 IM 集成到一个程序中，这样用户只需要登录这个单独的界面就可以访问多个 IM 网络。MeeBo.com 网站提供了一个与 Trillian 功能类似的基于 Web 界面的应用服务。

P2P 网络的应用也在不断激增。对于取证分析而言，每一个 IM 和 P2P 程序都是空前的挑战。例如，如果 IM 程序默认情况下并不保存对话信息，调查人员如何能够确定嫌疑人曾和谁聊天？或者调查人员如何能够确定一份存储的对话记录是用户特意保存的还是第三方插件自动记录的？对 P2P 来说，调查者如何确定嫌疑人所使用的搜索条件，从共享网络上下载了哪个文件，以及这些文件来自哪里？

由于 IM 和 P2P 客户端程序之间差异是如此之大，以至于对每一个程序的完全取证分析都需要一本书来说明。如其他程序一样，IM 和 P2P 客户端程序的不同版本之间同样存在变化，包括增加新的功能、创建新的注册表项和创建新的文件等。对这些程序进行取证分析所遇到的困难将是前所未有的。例如，美国在线较早版本的 IM 客户端程序在用户配置文件中（User 配置单元）保存一系列特定的注册表键。在这些注册表键内可以发现加密后的用户口令，这是用户选择自动登录 AIM 网络而不是手工输入口令的结果。如果在某些情况下，必须获取用户在 AIM 上的活动情况，就可以利用这个加密后的口令在其他系统上生成类似的配置文件，以用户身份登录。美国在线 IM 客户端程序的新版本是 AIM Triton，该程序的主要变化是不再为每个对话打开单独的窗口，而是将所有的对话以标签的方式集成到一个窗口中显示。但是从取证的角度来看，其最主

要的变化体现在注册表中，在 HKEY_CURRENT_USER\Software 键下已经不存在 AOL 或者 AIM 表项了。

更糟糕的是，虽然数年来，取证调查人员和法律机构遇到很多需要对 IM 和 P2P 进行分析的情况，但是目前仍没有可用的数据库或者是 Wiki 条目。这仍旧是一个需要不断研究的领域。

提 示

还有一些可用于跟踪用户活动的注册表键。例如，ShellNoRoam\BagMRU 键[42]保存用户的查看设定和定制目录信息，这就意味着用户使用了命令解释程序（Shell，在 Windows 中，一般就是 Windows Explorer）进行了定制。Explorer\Streams 键[43]和 StreamMRU 键保存窗口关闭时的位置以及尺寸信息。这些信息说明用户特意改变过桌面上程序窗口的位置和尺寸，这也就表明用户和 Windows Explorer 解释器进行过交互。

Windows XP 系统还原点

Windows XP 的系统还原功能保存一系列的系统还原点。在系统变得不稳定或者是表现异常时，可以将系统还原到以前某个工作正常的配置。能够将系统恢复到某一个工作正常的时刻是一项非常重要的功能，这点已经得到很多用户的认可。

对于用户和取证调查人员来说，系统还原是一个非常有用的工具。毕竟，系统还原功能是在后台执行的，不需要用户介入，能够安静地备份系统的关键配置信息。系统还原点的创建需要特定的触发条件，例如，安装新的程序或是安装未签名的驱动系统的自动升级安装等。系统还原点也可手工创建，默认情况下系统还原服务每天创建一个系统还原点。

为更好地理解系统还原点对于取证分析的用处，需要对系统还原点有一些深入了解：系统还原点备份什么内容，不备份什么内容，注册表键如何控制系统还原行为。

系统还原恢复下述项目：

- 注册表
- 本地配置文件
- COM+数据库
- Windows 文件保护 DLL 缓存
- WMI 数据库
- IIS MetaBase
- 与 Platform SDK 系统还原中监控的文件扩展名列表<*include*>部分扩展名相同的文件

系统还原不会恢复的项目如下：

- DRM 设置
- SAM 配置单元

- WPA 设置（Windows 认证信息不恢复）
- Platform SDK 中系统还原监控文件扩展列表中所列的特定文件和目录
- 与 Platform SDK 中系统还原监控文件扩展名列表中<*include*>部分扩展名不匹配的文件
- 保存在用户配置中的用户创建数据
- 重定向文件夹内容

有一点是需要特别注意的，系统还原服务并不恢复 SAM 配置单元文件，但至少对该文件的一部分进行了备份。检查恢复点的内容，会发现 SAM 配置单元的备份文件是同其他注册表文件的备份在一起的。

因为本章的目的是注册表分析，因此在系统还原点中最感兴趣的是注册表文件的特定备份，如 NTUSER. dat，SYSTEM，SOFTWARE 和 SAM。图 4. 21 列出了 Prodiscover 中显示的还原点下快照（Snapshot）目录中的内容。

Select	File Name	File Exten...	Size
☐	Repository		
☐	ComDb	Dat	22512 bytes
☐	domain	txt	36 bytes
☐	_REGISTRY_MACHINE_SAM		24576 bytes
☐	_REGISTRY_MACHINE_SECURITY		45056 bytes
☐	_REGISTRY_MACHINE_SOFTWARE		9134080 bytes
☐	_REGISTRY_MACHINE_SYSTEM		4558848 bytes
☐	_REGISTRY_USER_	DEFAULT	241664 bytes
☐	_REGISTRY_USER_NTUSER_S-1-5-18		241664 bytes
☐	_REGISTRY_USER_NTUSER_S-1-5-19		229376 bytes
☐	_REGISTRY_USER_NTUSER_S-1-5-20		229376 bytes
☐	_REGISTRY_USER_NTUSER_S-1-5-21...		679936 bytes
☐	_REGISTRY_USER_USRCLASS_S-1-5-...		8192 bytes
☐	_REGISTRY_USER_USRCLASS_S-1-5-...		8192 bytes
☐	_REGISTRY_USER_USRCLASS_S-1-5-...		16384 bytes

图 4. 21　Prodiscover 中显示的还原点中快照目录的部分内容

如图 4. 21 所示，从取证的角度来看，系统还原点备份了相当多有用的信息。虽然备份到还原点中的注册表文件仅是 System32\config 目录中注册表文件的一部分，但是仍就能够为调查人员提供很多系统以前配置的有用信息。

在第 5 章还会对系统还原服务中的文件进行分析。本章主要将注意力放在注册表文件上。

现有的分析技术，特别是使用如 Offline Registry Parser 等工具进行分析的方法，不但对 System32\config 目录中的原始注册表文件是有效的，对系统还原点中的注册表文件同样也是有效的。实际上，本章所讨论的很多注册表键和注册表项同样可以在还原点的备份注册表文件中找到。这就使调查人员可以查看系统过去的配置信息及那时系统上所安装的软件。

关于系统还原点还有一些需要注意的问题。默认情况下，系统还原至少需要 200MB 磁盘空间，如果磁盘空间需求得不到满足，系统还原服务就会暂停，等待控件的释放才能再次恢复。这点是很重要的，特别是在有些调查过程中，目标系统中没有系统还原点，此时，有一些调查人员可能怀疑是不是某些用户访问了 System Volume Information 目录并删除了还原点，但是实际的情况可能是系统根本没有创建还原点。

分析系统还原点要注意的另一个问题是 SYSTEM\ControlSet00x\Control\Backu-

pRestore 注册表键对系统还原功能所起的重要作用，该注册表键可以确定备份哪些内容、不备份哪些内容；还原哪些内容、不还原哪些内容。BackupRestore 注册表键有 3 个子键（AsrKeysNotToRestore，FilesNotToBackup 和 KeysNotToRestore），每个子键下都存在注册表键或者是文件列表（如果是文件列表，每个项目以通配符加扩展名格式显示，表示所有具备该扩展名的文件）。这些注册表项及其数据同样能够影响调查者在静态调查过程中所看到的或者所访问的内容。

系统还原配置信息[44]由下述注册表键维护：

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\SystemRestore
```

该键下有几个重要的注册表项：*RPGlobalInterval* 项说明隔多久创建新的还原点，默认值是 86 400，也就是说系统每天创建一个还原点（60 秒×60 分/小时×24 小时/天=86 400）；*DisableSR* 项值如设为 1，将停用系统还原功能，默认情况下，该值设为 0；*RPLifeInterval* 项表示系统还原点的最长保存时间默认值是 7 776 000，也就是 90 天（7 776 000/86 400=90 天）。

在活动的 Windows XP 系统上，访问系统还原配置信息的一个简单办法是使用 SystemRestore[45]和 SystemRestoreConfig[46]WMI 类。随书附赠光盘中的 sr. pl 的 Perl 脚本提供了一个如何使用这些类的例子。该 Perl 脚本通过 SystemRestoreConfig WMI 类获取系统还原配置设置信息（基本上都是注册表项）并显示每个还原点的信息（序列号、创建日期、为什么创建还原点的字符串描述等）。通过 SystemRestore 类从还原点文件中获取的信息将在第 5 章中说明。

从调查的观点来看，还原点内的注册表文件的作用是不可低估的。还原点内保存的注册表文件包含了活动系统的大部分注册表信息。如果在没有可用的系统镜像文件情况下查看还原点内注册表文件内容，需要从 SysInternals. com 下载 psexec. exe 程序到 Windows XP 系统，在命令行中输入 psexec -s cmd。该命令打开一个以 SYSTEM 权限运行的命令提示符。输入下述命令，进入 System Volume Information 目录：

```
cd \sys *
```

从此目录进入包含还原点的子目录：

```
cd _restore *
```

在这个目录下，如果将所有的目录列出，会看到一些还原点。每个还原点目录名都以 *RP* 开始。如果进入任何一个还原点目录接着进入快照子目录，就会看到备份的部分注册表文件。从这里可以将这些注册表文件复制到其他目录以进一步分析。一个常用的分析方法是使用 Offline Registry Parser，只需输入下述命令即可：

```
C:\tools\repl.pl <path>
```

如果系统上没有安装 Perl，可以使用 Perl2Exe 编译的独立可执行程序。上述命令的输出是 ASCII 格式。另一个查看注册表文件的方法是使用 RegEdit。打开 RegEdit，选择 HKEY _ USERS 配置单元，单击“文件 | 加载”选项配置单元，然后定位到从还原点中复制出来的文件。当询问时，对配置单元进行命名，名称无限制，可以是 Test 或者是 Test System 等。图 4. 22 表明的是以上述这种方式加载的系统配置单元。

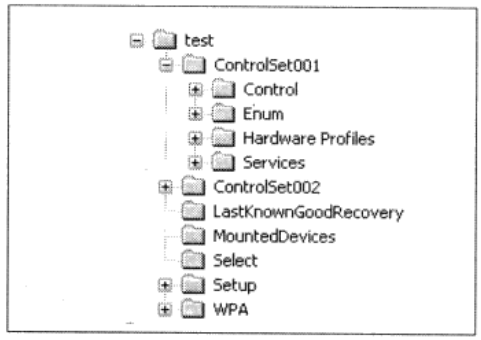


图 4.22　RegEdit 中加载的还原点系统配置单元

至此，如在活动系统上一样，可以查看注册表键的内容，也可以利用工具获取注册表项及其数据，还可以导出配置单元内的注册表项。

下面举例说明如何使用这些分析技术。假设在案件调查过程中，怀疑某个软件从系统中被删除了。现在已经在 RegEdit 中加载了用户配置单元，利用相关工具从配置单元信息中获取了有关的用户活动。已经解析了 UserAssist 键内容，并且发现控制面板中的添加或删除程序在某个特定日期运行过。同样注意到，在这个日期之前有几个还原点。从还原点中将软件配置单元文件复制出来，调查该配置单元其中的 Software\Microsoft\Windows\CurrentVersion\Uninstall 子键，就可以确定这个有问题的软件是否是在这个日期之前安装的。事实上，依赖于软件的安装方式，在软件卸载时还可能会创建一个系统还原点，从中甚至可以发现这个软件。

小结

对于管理员、顾问和取证分析人员而言，了解如何浏览注册表及从中获取特定信息是一项相当有用的技能。对知道从哪里发现信息并能正确解释所获取信息的调查人员来说，注册表会打开所有的秘密。对注册表的深入研究不类似印第安纳·琼斯那样，转眼之间就能发现古老的秘密。

在随书附赠光盘中的 regref. xls 文件中，包含注册表内用户 MRU 列表、自动启动位置等的完整列表（不敢说“完全”）。虽然这是一个完整列表，但是应该说这个列表只是一个开始。regref. xls 文件中所列的注册表列表是多种渠道获得的，如网络、AutoRuns 程序、反毒网站的恶意软件报告及个人经验等。

本章所列的注册表键对于某个特定案件可能是非常重要的，但是不能认为是注册表键的一个完全参考。具备同样功能的注册表键的名称和位置可能不同，例如，不同的程序之间，甚至同一软件的不同版本之间可能都会有所区别。没有一本书能作为注册表键的完整的、权威的参考资源。即便是有，也会相当大且昂贵，而且极有可能是出版后马上过期。本章的目的是说明什么样的信息是有用的及如何着手发现这些相关信息。

光盘内容

随本书附赠的光盘中包含一个与本章配套的目录。这个目录包含几个子目录，Code 子目录包含本章中所介绍的代码，包括 Perl 脚本和 ProScripts 脚本（为 Prodiscover 特别编写的 Perl 脚本）。spreadsheet 子目录包含一个注册表参考电子表格文件，这个电子表格文件列出了调查人员所感兴趣的注册表键、注册表项及其简单描述。这个电子表格文件包括不同的工作表，每个工作表代表的是不同的功能区域。

samples 子目录包含几个来自真实案例的注册表文件，不仅包括 system32\config 目录下的文件，还包括来自系统还原点的文件。我们鼓励读者采用各种方法分析这些文件，例如，在十六进制编辑器中打开这些文件，使用任何工具，包括 Code 目录中提供的工具解析这些文件。

参考资料

1. http://support.microsoft.com/kb/256986
2. http://support.microsoft.com/default.aspx?scid=kb;en-us;141377
3. http://support.microsoft.com/default.aspx?scid=kb;en-us;161678
4. http://www.microsoft.com/technet/archive/winntas/tips/winntmag/inreg.mspx?mfr=true
5. http://home.eunet.no/pnordahl/ntpasswd/
6. http://www.sleuthkit.org/
7. http://pyflag.sourceforge.net/
8. http://support.microsoft.com/default.aspx?scid=kb;en-us;188768
9. http://msdn.microsoft.com/library/default.asp?url=/library/en-us/sysinfo/base/filetimetosystemtime.asp
10. http://vil.nai.com/vil/Content/v_140546.htm
11. http://home.planet.nl/~pa0joz/pc_util.html
12. http://www.microsoft.com/technet/sysinternals/SystemInformation/Regmon.mspx
13. http://liveview.sourceforge.net/
14. http://support.microsoft.com/kb/288164/en-us
15. http://support.microsoft.com/default.aspx?scid=kb;EN-US;q246120
16. http://support.microsoft.com/kb/298931
17. http://support.microsoft.com/kb/310560
18. msdn.microsoft.com/library/default.asp?url=/library/en-us/wmisdk/wmi/win32_startupcommand.asp
19. http://www.microsoft.com/technet/prodtechnol/winxppro/reskit/c29621675.mspx
20. http://support.microsoft.com/default.aspx?scid=kb;EN-US;137367
21. http://support.microsoft.com/kb/314866/EN-US/

22. http://liveview. sourceforge. net/
23. http://support. microsoft. com/? kbid=311446
24. http://support. microsoft. com/? kbid=310585
25. http://www. microsoft. com/resources/documentation/Windows/2000/server/reskit/enus/Default. asp? url=/resources/documentation/Windows/2000/server/reskit/enus/regentry/942. asp
26. http://support. microsoft. com/kb/197571
27. http://msdn. microsoft. com/library/default. asp? url=/library/en-us/secauthn/security/registry_entries. asp
28. http://support. microsoft. com/default. aspx? kbid=238788
29. http://support. microsoft. com/default. aspx? scid=kb;en-us;892894
30. http://silverstr. ufies. org/blog/archives/000809. html
31. http://www. sysinternals. com/ntw2k/freeware/autoruns. shtml
32. http://www. microsoft. com/whdc/device/stream/vidcap/UVCView. mspx
33. http://www. microsoft. com/whdc/system/bus/usb/USBFAQ_intermed. mspx
34. http://www. microsoft. com/whdc/device/storage/usbfaq. mspx#ERCAC
35. http://www. microsoft. com/technet/prodtechnol/windows2000serv/reskit/regentry/93502. mspx? mfr=true
36. http://www. hak5. org/wiki/USB_Hacksaw
37. http://support. microsoft. com/kb/157234/
38. http://www. accessdata. com/support/downloads/
39. http://search. cpan. org/~ jmacfarla/Parse-Win32Registry-0. 24/lib/Parse/Win32Registry. pm
40. http://computer. forensikblog. de/en/2006/02/list _ members _ of _ a _ windows _ group. html
41. http://support. microsoft. com/default. aspx? scid=kb;en-us;322948
42. http://support. microsoft. com/kb/813711
43. http://support. microsoft. com/kb/235994
44. http://support. microsoft. com/kb/295659
45. http://msdn. microsoft. com/library/default. asp? url=/library/en-us/sr/sr/systemrestore. asp
46. http://msdn. microsoft. com/library/default. asp? url=/library/en-us/sr/sr/systemrestoreconfig. asp

快速解决方案

注册表内部结构

√ Windows 注册表是一个二进制的配置信息数据库，不但包含操作系统和应用程

序的各种设定信息，还包括用户和系统交互过程等各个方面的信息。

- √ 通过对各种注册表结构（键、项）的了解，可以解析和查看存储在内存和未分配空间中的注册表数据。
- √ 部分的注册表数据是易失的，在操作系统启动时建立，但是在系统的静态镜像中不存在。
- √ 注册表键（或者某些值）具有与其相联系的时间戳，能够用于时间线分析。基于这一原因，注册表可以看作是日志文件。
- √ 注册表所维护的信息没有明确的统一标准。例如，部分 MRU 键以二进制方式保存数据，而其他 MRU 键以 ASCII 方式保存数据。还有一些注册表键的项名称是以 ROT-13 算法加密的。理解特定键和项的数据结构对于解析和分析注册表数据是非常重要的。
- √ 有很多有效工具可用于在活动系统上监控注册表的访问和修改，获取的信息可用于定位特定注册表键，发现应用程序和用户活动所遗留的痕迹。

注册表分析

- √ 注册表中许多位置（键、项）包含的信息可用于绝大部分常规调查过程，而其他一些位置包含的信息可用于入侵、欺骗、违反用户策略等特殊调查过程。
- √ 一些特定的注册表键和项可能对某一调查过程是重要的，但是通常情况下，对多个注册表键和项的相互关联分析才能提供最完整的信息。
- √ 调查过程中，Windows XP 系统还原点包含的部分注册表信息是非常有用的。例如，通过对保留的 SAM 文件内容的分析，调查者能够判断出用户组成员是否发生改变，最近是否安装了新的应用程序。

常见问题

下面的这些问答用来测试读者是否已经完全理解本章的内容，并可以帮助读者在实际案件调查中应用本章知识，如果还有其他问题，浏览 www. syngress. com/solutions 并单击 Ask the Author 一栏。

问：如何确定是否安装了 BHOs，并找出其存在的位置?

答：BHOs 是由可以影响系统全部用户的 HKEY _LOCAL _MACHINE 配置单元维护的。BHOs 列表位于 Software \Microsoft \Windows \CurrentVersion \Explorer \Browser Helper Objects 键下。在该键下，每个 BHO 都是一个子键，子键名为 BHO 的 GUID。记住这些 GUID，转到 HKEY _LOCAL _MACHINE 配置单元内的 Softwre \Classes \CLSID 键，搜索每个 GUID，一旦发现与 BHO 相同的 GUID，该键下 *default* 项就是 BHO 的名称，InProcServer 子键下 *default* 项就是 BHO 所调用的 DLL 文件。Bho. pl 脚本（在随书附赠的光盘中）可用于从系统中获取 BHOs 信息。

问：搜索系统的过程中，在用户配置单元文件中（Software \Microsoft \Windows

\CurrentVersion\InternetSettings\ZoneMap\Domains）发现一些包含 Web 站点的子键，这些键是什么？表示的内容是什么？

答：这些项目是由系统管理员加入 IE 的。在 IE 菜单中选择“工具 | Internet 选项 … | 受限站点”，单击“站点”按钮，就可以添加这些站点。然而，仔细分析这些站点列表会发现，某些恶意软件也会在此处增加站点，这样用户就无法访问这些站点。该键下的项目不但可作为用户活动或者是组策略变更的表现，同样可作为恶意软件入侵的证据。

问：调查过程中，基于从 Windows XP 系统所获取的信息，可以确认某个用户具备本地管理员权限；而通过和 IT 主管的讨论发现，所有用户应该只有用户级别的访问权限。如何能够追踪这个用户加入管理员组的时间？

答：系统用户信息是由 SAM 文件维护的。组成员注册表键的 *LastTime* 时间能够提供一些线索。另外，在 Windows XP 系统还原点内有足够的历史数据帮助确定用户加入用户组的最后时间，这样就可以确定其加入管理员组的最早时间。

Chapter 5

第 5 章
文件分析

本章主要内容：

- 事件日志
- 文件元数据
- 其他分析方法

- √ 小结
- √ 快速解决方案
- √ 常见问题

引言

Windows 系统维护的一些文件在取证方面是很有用的。事实上，许多调查人员可能还没意识到 Windows 系统用于跟踪各种活动和功能的文件中所产生的大量数据。了解各种用于系统内部维护信息文件的位置可以使调查人员将其与其他方面找到的信息进行关联，从而减小分析的不确定性。

本章讨论一些可以在 Windows 系统中找到的日志文件及其包含的信息，同时涉及一些取证调查中有价值的特殊文件。本章还会讨论一些实际存在于文件和文件系统中的与其紧密关联的信息。

事件日志

事件日志（Event Log）可能是 Windows 系统中最为人所知的日志了，相当于 Linux 系统中的 syslog。事件日志记录 Windows 系统中每天发生的各种各样的事件，经过配置还可以记录一些附加事件（在第 4 章中讨论过）。事件日志可以分成几类，并且记录在不同的日志文件中，如安全、系统及应用程序日志。事件日志提供了大量的信息，可以用来对系统进行排错（troubleshooting）分析，同时也可以帮助调查人员了解系统中所发生的事件。

提 示

大多数 Windows 系统中，可以使用资源工具包中的 auditpol. exe 工具查询和设置审计策略。在 Windows XP SP 2 和 2003 SP 1 中，auditusr. exe 可以为每个用户设置审计策略。例如，为所有用户设定登录审计，对特定用户设定更详细地审计。Auditeuser. exe 所做的策略更改会修改 HKEY_LOCAL_MACHINE \ SYSTEM \CurrentControlSet \Control \Lsa \Audit \PerUserAuditing \System 注册表键。通过这个工具，调查员可以知道事件日志中可能会记录哪些事件、用户/管理员的技术水平如何。

理解事件

Windows NT 家族操作系统中，从 Windows 2000，XP 到 2003，事件日志保持为同样的二进制结构，文件包括一个头部和一系列事件记录。根据操作系统的设计思路，当某些事件发生时，如用户登录或注销，就会产生相应的事件记录。一些事件会默认记录，其他一些是根据注册表键值（RolAdEvt）中保存的审计配置记录的，如第 4 章讨论的。事件日志的其他配置，包括文件大小、保留时间等被保存在下列注册表键中：

```
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Eventlog\<Event Log>
```

默认情况下，Windows 2000，XP 和 2003 都拥有应用程序、安全和系统事件日志。作为域控制器的系统还会有文件复制（File Replication）和目录服务（Directory Service）事件日志，域名服务器（DNS）系统有 DNS 事件日志。

管理员最熟悉的就是使用事件查看器（Event Viewer）与事件日志进行交互，事件查看器是事件日志进行管理的图形界面工具。当管理员查看 Windows XP 的事件记录时，会看到类似如图 5.1 所示的界面。

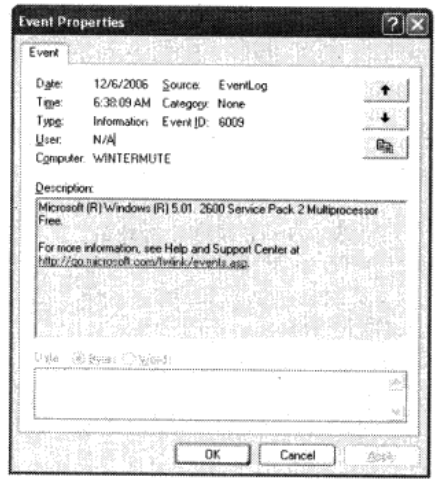


图 5.1 在用事件查看器中查看 Windows XP 事件记录

当事件查看器打开一条事件记录时，会显示事件描述信息：读取事件记录中的字符串值，然后定位系统中相应的消息文件（DLL），两者组合得到描述信息。这些消息文件包含消息字符串，主要用来支持 Windows 操作系统的国际化，同时事件记录的字符串值会被插入消息串的合适位置。这些消息文件包含不同语言（英语、汉语等）的消息字符串，在使用时填补必要的信息（例如，系统名称、时间戳等，这些信息来自日志记录）。

在 Windows 2003 之前，登录事件只记录登录系统的 NetBIOS 名称。从 2003 开始，安全事件日志同时记录工作站的名称和系统 IP 地址，如图 5.2 所示。

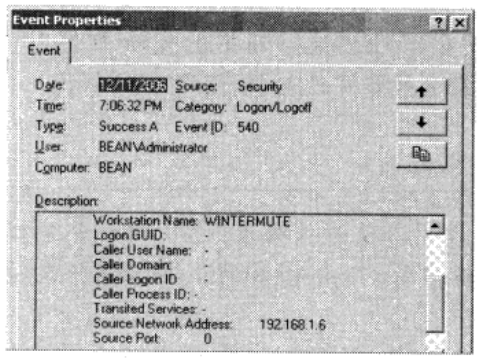


图 5.2 Windows 2003 事件记录显示了 IP 地址

图 5.2 显示的信息（例如，源网络地址）在调查中可能非常有用，特别是因为事件记录中明确记录了远程系统的 IP 地址。这个信息可以被用于确定登录的来源及登录的企图。

即使没有 DLL 消息文件，区分不同事件日志的含义也不困难，因为日志记录中还有其他特征信息。在图 5.2 中，记录中可以看到 Event ID、事件来源和其他可以参考的特征信息。还有一个时间戳信息，可用于时间线分析，实际上一条事件记录中有两个时间戳（这个在后面讨论）。Microsoft 为可能会被关注的事件提供了很多的信息。例如，如果登录/注销事件的审计和日志被激活时（参见第 4 章相关内容），调查人员应该可以在安全事件日志中看到编号为 528（成功登录）和 538（注销）的记录。如果发现几个事件记录的编号都是 528 时，就需要检查登录类型，因为有 9 种不同的登录类型代码。表 5.1 列出了各种成功登录的安全登录类型代码[1]及其含义。

表 5.1 登录事件的类型

登录类型	标题	描述
2	交互式	用户从控制台登录
3	网络	用户或者计算机从网络登录，例如，通过 net use、访问网络共享或者从网络中使用 net view 查看共享（该类型已经被事件编号 540 代替了）
4	批处理	为批处理程序保留
5	服务	服务登录
6	代理	不支持
7	解锁	用户解除工作站的锁定
8	网络明文	用户登录到网络，并且用户认证信息是以未加密的方式传递的
9	新认证	进程或线程克隆了当前令牌，但是对外连接使用了新的认证信息
10	远程交互	使用终端服务或者远程桌面登录
11	缓存的交互	用户使用本地存储的认证信息登录（域控制器不能提供认证服务）
12	缓存的远程	与远程交互是相同的，用于内部审计目的
13	缓存的解锁	工作站解锁登录尝试

对于一般的情形，Microsoft 提供了两篇知识库文章（299 475[2]和 301 677[3]），列出了 Windows 2000 的安全事件描述。列举的安全事件有一个简略描述及占位符（%1，%2 等），这些占位符会被事件日志中的字符串所替代。

提示

Windows 2000 中引入了事件编号 540（网络登录），用来代替事件编号 528 第三类（成功从网络登录）。在 Vista 中，登录事件又被合并到一个事件编号中（4624）。

对于其他事件记录，许多站点提供了关于事件记录的详细信息，如事件为何产生等。应用程序事件日志中特定记录的详细信息可能需要到厂商那里去查找。Even-

tID. net 可能是查找事件记录帮助信息最好的网站之一。这个网站是收费的，但是如果要花费很多时间来搜索、调查不同类型的事件日志，这个花费就会很大。在很多实例中，只要简单的提供事件编号就能得到事件的信息，信息来自不同的渠道并有相关的参考链接。例如，如果搜索事件编号 6009，会得到 4 个不同的事件来源。可以单击其中一个详细信息链接（6009 是 EventLog 产生的事件），从中还可以得到 2 个评论和 3 个指向 Microsoft 站点的链接，Microsoft 的站点提供了事件 6009 的详细信息。在这个例子中，在相当短的时间内就可以发现 Windows 系统启动就产生了该事件（因此，该事件记录产生的时间相当于系统启动的时间），并且操作系统的版本信息也被写到事件的描述字段中。

提 示

6009 号事件记录可以用来确定和校验主机系统的操作系统和系统名称。其中，计算机（Computer）一栏包含主机名，事件记录的描述字段包含用于识别操作系统版本的字符串。

EventID. net 以外，一个非常好的 Windows 事件日志信息来源是 Eric Fitzgerald 的“Windows 安全日志和其他秘密”个人网志[4]。Eric 的网志包含相当多与事件日志相关的有用信息，还包括如何满足 Visa PCI 兼容标准、审计提示及窍门等文章。Microsoft 站点上也有事件和错误消息中心：通过高级搜索功能[5]，可以获得各种事件日志项目的信息。

提 示

还记得第 4 章中有关 USB 移动存储设备痕迹的内容吗？在 Windows 2000 中，当一个 USB 移动存储设备连接到系统时，移动存储服务就会产生一条编号为 134 的事件记录，当设备被移除时，产生编号为 135 的事件。这些事件在 Windows XP 中已经没有了，KB329 463[6]号文章道出了其中原委：

在安装了补丁以后，Netshell 不再监听即插即用设备到达通知，因而就不会收到新设备的通知。

因此，不能期望日志中能够看到 USB 移动存储设备插入或移除的通知了。

同样，Randy Franklin 的终级 Windows 安全（UltimateWindowsSecurity. com）站点也不错。其中有很多关于 Windows 安全事件日志的页面，包括事件编号参考表和百科。就安全事件日志方面的内容来说，这个网站很值得访问和收藏。

事件日志文件格式

调查期间，可能需要分析 . evt 事件日志文件，但其文件格式是未知的（本章讨论的事件日志格式适用于 Vista 之外的所有 Windows 系统）。因此在从镜像中取得 . evt 文件之后会考虑使用事件查看器打开。不过，这样做可能会得到一个错误提示：事件日志

文件“损坏”。有时，调查员还会想到在未分配簇中查找对案件有用的事件日志信息。无论哪种情况，了解事件日志文件结构的细节都是很有用的。

Windows 事件日志（Windows2000，XP 和 2003 版本）是具有清晰、可识别特性的二进制文件格式，调查员可以识别并解析事件日志文件，也可以识别和解析文件、未分配空间中的事件记录信息。每个事件日志由一个头部和一系列事件记录组成，这两部分将会详细讨论。事件日志是以循环缓存方式维护的，因此当新的事件记录被添加到文件中，旧的事件记录会被循环覆盖。

事件日志头部

事件日志头部包含在有效事件日志文件的起始 48B 中。如果 .evt 文件没有被破坏，其头部会和图 5.3 中的样例类似。

```
00000000h: 30 00 00 00 4C 66 4C 65 01 00 00 00 01 00 00 00 ; 0...LfLe........
00000010h: 30 00 00 00 F0 A9 00 00 AD 00 00 00 01 00 00 00 ; 0...ð©..-.......
00000020h: 00 00 01 00 09 00 00 00 80 3A 09 00 30 00 00 00 ; ........€:..0...
```

图 5.3　事件日志头部

事件日志头部由 12 个独立的 DWORD 值组成。表 5.2 列出了其中的 9 个并提供了简单地描述。

表 5.2　事件日志头部结构

偏移	大小	描述
0	4B	记录大小；.evt 文件头部大小为 0x30（48）字节，事件记录头部大小是 56B
4	4B	幻数（LfLe）
16	4B	.evt 文件中最老事件记录的偏移
20	4B	.evt 文件中下一条事件记录写入的位置
24	4B	下一条事件记录的 ID
28	4B	最老事件记录的 ID
32	4B	.evt 文件的最大长度（取自注册表）
40	4B	事件记录的保留时间（取自注册表）
44	4B	记录的大小（与偏移 0 处的 DWORD 值重复）

幻数是头部中的重要属性，从头部的 0x04 位置开始（第二个 DWORD），其值为 *LfLe*。这个值是 Windows 事件日志特有的，并且与事件记录相关联。Microsoft 称此值为 *ELF _ LOG _ SIGNATURE*（Microsoft 网站上关于事件记录结构的描述说明“这个 *DWORD* 值总是设置成 *ELF _ LOG _ SIGNATURE*”）。注意，记录的大小（头部大小为 0x30，也就是 48 字节）在头部结构的头尾部分都会出现。这样，调查员可以使用自动程序或手工方法（使用十六进制编辑器）定位头部或者一条事件记录，该方法适用于事件日志文件、未分配空间或者未知类型文件。调查员可以通过下一条被写入记录的 ID 号及最老的事件记录 ID 号信息来确定事件记录的总数。

注 释

处理文件时，我们使用术语幻数（Magic Number）特指文件内一系列特殊的字节，幻数可用来惟一标识文件及类型。在文件特征分析时会用到幻数，通过检测文件中幻数来判断其与该类型的扩展名是否匹配。事件日志文件中，幻数是0x654c664c，或者如图5.3所示为4C 66 4C 65。在不考虑字节序时，它们可以转换成字符串 *LfLe*，但仍然是幻数。

事件日志文件的最大长度值和事件记录保留时间是从所属的系统注册表中获取的。

事件记录结构

事件记录有一些值和事件日志头一样，但是事件记录中包含了更多的内容，如图5.4所示。不过，事件记录的基础头部会稍微比文件头大些，为56B。事件记录整个长度为244B（0xF4），前56B为记录头。

```
00000030h: F4 00 00 00 4C 66 4C 65 01 00 00 00 3D E1 20 43 ; ô...LfLe....=á C
00000040h: 3D E1 20 43 64 02 00 00 08 00 15 00 06 00 00 00 ; =á Cd...........
00000050h: 00 00 00 00 72 00 00 00 1C 00 00 00 56 00 00 00 ; ....r.......V...
00000060h: 00 00 00 00 EE 00 00 00 53 00 65 00 63 00 75 00 ; ....î...S.e.c.u.
```

图5.4 事件记录结构示例

如图5.4所示，事件日志的幻数出现在事件记录的第二个 *DWORD* 值中，与日志头部一样。表5.3是事件记录前56B数据结构信息。

表5.3 事件记录前56B数据结构信息

偏移	大小	描述
0	4B	事件记录长度，即记录大小
4	4B	保留；幻数
8	4B	记录号
12	4B	产生的时间；使用UNIX时间格式存储，即距离1970年1月1日00：00：00的秒数，世界标准时间（UTC）
16	4B	写入时间，格式同上
20	4B	事件编号，是事件源中惟一的事件标识；事件编号连同源名称一起使用可以定位所属事件源消息文件中的事件描述字符串
24	2B	事件类型（0x01=错误；0x10=失败；0x08=成功；0x04=信息；0x02=警告）
26	2B	字符串数量
28	2B	事件类别
30	2B	保留的标识
32	4B	结尾记录编号
36	4B	字符串偏移；事件记录中描述字符串的偏移
40	4B	用户SID长度；用户SID的字节大小（如果是0，表示没有提供用户SID）
44	4B	记录中用户SID的偏移
48	4B	数据长度；该记录相关的二进制数据的长度
52	4B	数据的偏移

表 5.3 列出了一条事件记录的头 56B。记住，记录本身的实际长度在记录的第一个和最后一个 DWORD 中（记录大小将实际记录框在当中，就如文件头结构一样）。有了这些信息，对事件日志文件的内容解析就很简单了，可以很容易地获取并显示事件记录。

有了事件记录结构的定义，查找和分析未分配空间中的事件记录成为可能。使用事件日志幻数作为路标，分析员可以在未分配空间中搜索，一旦找到幻数，读取前面的 DWORD 就可以得到事件记录的长度，然后获取完整的事件记录。即使不能得到整条事件记录，头 56B 也能提供重组事件记录的部分信息。

工具和陷阱

读取事件日志

在一次案件协查中，案件中的一位分析员非常熟悉 Linux，并且使用 PyFlag[7] 作为取证分析工具。他决定让我来打开事件日志以提取可用的记录。他已对此做过尝试，但是当复制 .evt 文件到 Windows 桌面系统并尝试用事件查看器打开时，发现信息提示文件已经损坏。

我对事件日志和事件记录的格式做过研究，所以对已有的 Perl 脚本进行了一些调整，分析得到这些事件日志文件中的所有事件记录。不过，我发现其中一个日志文件数据记录与其日志头的定义不一致。无论采用何种方法，得到的日志记录总是比头信息中定义的多一条。经过一段时间的调查，我发现对于 API 来说，在事件日志被清除后，第一条事件记录之前有一个区域会作为缓存区域保留下来。这个缓存区域不会被 API 读取，如果系统已经继续正常运行，新的事件记录会写入这个区域。忽略其具体细节，这个缓存确实包含了一条完整的事件记录。由于使用的工具没有调用系统 API，而是用读取文件二进制的模式来分析事件记录信息，所以工具并不认为它是缓存区域。

尽管丢失的事件记录对案件没有重要的影响，但的确反映了理解特定文件格式对 Windows 系统分析的重要性，可能情况下，开发不依赖 Windows API 的工具用于日志文件分析处理。这不仅可以发现“隐藏的”信息，还可以在 Windows 以外的平台上进行分析（特别是 Linux）。调查人员将不再局限于在 Windows 平台上分析 Windows 镜像。

附送光盘的 ReadEvt 目录中有 3 个 Perl 脚本，可以用来获取 Windows 2000，XP 和 2003 系统中事件日志文件的信息。第一个脚本 evtstats.pl 显示 .evt 文件的简单统计信息，如下所示：

```
G:\>evtstats.pl d:\cases\appevent.evt
Max Size of the Event Log file                  = 65536 bytes
Actual Size of the Event Log file               = 65536 bytes
Total number of event records (header info)     = 200
Total number of event records (actual count)    = 206
```

```
Total number of event records (rec_nums)        = 206
Total number of event records (sources)         = 206
Total number of event records (types)           = 206
Total number of event records (IDs)             = 206
```

脚本分析事件日志文件的头信息，确定文件中应该存在的记录数，然后根据每条事件记录的不同标记，扫描分析整个事件日志文件，统计实际找到的记录数。

第二个脚本，lsevt.pl 使用 File::ReadEvt Perl 模块分析事件日志文件，并且使用简单列表格式显示事件记录，如下所示：

```
Record Number:198
Source:SecurityCenter
Computer Name:PETER
Event ID:1800
Event Type:EVENTLOG_INFORMATION_TYPE
Time Generated:Mon Sep 19 21:07:32 2005
Time Written:Mon Sep 19 21:07:32 2005
```

第三个脚本，lsevt2.pl 提供了比 lsevt.pl 更多的灵活性，例如，选择逗号分隔的输出格式。这样，调查人员可以使用下面的命令行来处理 Windows 事件日志文件：

```
C:\Perl>perl lsevt2.pl-f d:\cases\appevent.evt-c>testevt.csv
```

这样就可以使用 Excel 打开结果文件 testect.csv，进行排序、查找和分析，而且 lsevt2.pl 不需要使用 File::ReadEvt 模块。

所有的 Perl 脚本都是以二进制方式分析事件日志文件，没有使用 Windows API。这样，不仅可以在 Windows 以外的平台（Mac OS X，Linux 等）上分析事件日志文件，而且即使事件日志查看器提示文件数据损坏时，仍然可以分析这个文件。脚本所需要的 File::ReadEvt 模块也在光盘中。模块安装很简单，复制 ReadEvt.pm 文件到 Perl 安装路径下的 site\lib\File 目录中就可以了。复制完成后，就能够在 Perl 脚本中使用这个模块了。

另一种处理那些事件查看器报错事件日志文件的方法是使用十六进制编辑器修改其文件头。Stephen Bunting 是特拉华大学警局的队长，他有一个详细的步骤列表[8]，用于修复损坏的事件日志文件头，使修好的文件可以在事件日志查看器和其他使用 Windows API 访问 .evt 文件的程序中打开和查看。

警 告

2007 年 2 月，Andreas Schuster 在 Blog 中记录了有关事件日志记录的一种特殊情况[9]，该情况下，.evt 文件尾部的一条记录会绕到文件的开始，该记录的部分数据跟在文件头后面。如果根据幻数来分析事件日志，将不会正确得到这条被分割的记录（本章的脚本就不可以），因为只有部分记录数据能被识别。Andreas 提供了一个范例 .evt 文件，可以用来测试这种情况。

Vista 事件日志

Vista 在许多方面改变了 Windows 操作系统，包括系统使用的事件日志结构。例如，这个服务的名称从事件日志改成 Windows 事件日志，并且使用了全新的格式。Vista 使用 XML 格式来存储事件，支持事件记录中心收集方式。

Vista 仍然维护 3 个主要事件日志类别（应用程序、安全和系统），但现在有更多的类别，记录各种不同的事件记录，如图 5.5 所示。

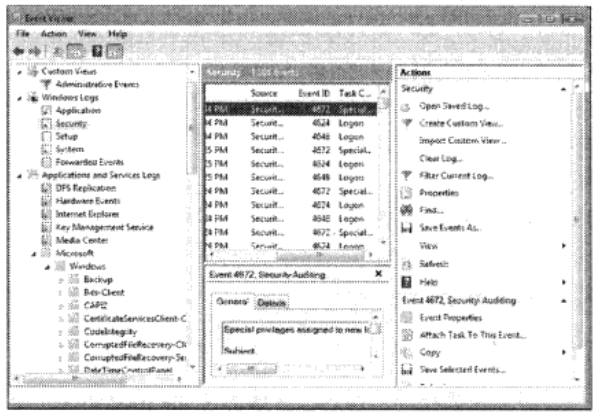


图 5.5 Vista 事件查看器

如图 5.5 所示，Vista 有更多的事件日志类型，包括 Internet 浏览器和硬件事件（安装 IE 7.0 也会在 Windows XP 和 2003 中增加 IE 浏览器事件日志）。同时注意图 5.5 的右下方的动作（Actions），其中有一个项目是“附加任务到这个事件……（Attach Task To This Event……）”。这个项目很有趣，利用它可以开发用于分析 Vista 事件日志的工具，在应急响应和取证分析过程中使用。

Andreas Schuster 和 Eric Fitzgerald 在 Blog 中写过一些有关 Vista 记录存储结构的文章。

在运行的 Vista 系统中，wevtutl.exe 命令可用来获得在事件查看器图形界面中没有显示的 Windows 事件日志相关信息。下面的命令将显示系统中存在的事件日志列表：

```
C:\>wevtutil el
```

利用本命令的输出可以进一步列出特定事件日志的配置信息，包括日志文件的名称和路径：

```
C:\>wevtutil gl log name
```

本命令显示的大部分信息也可在下述注册表键中找到：

```
HKEY_LOCAL_MACHINE\System\ControlSet00x\Services\EventLog\log name
```

这些都是应急响应和取证分析中有价值的信息，必须开发出相应的工具和技术以提取 Vista 系统中这部分信息。

IIS日志

Microsoft的因特网信息服务器（IIS）是一个网站服务器平台，有很多用户使用，同时也是攻击者的首选目标之一。IIS安装很容易——有时管理员根本没有意识到系统中IIS服务器在运行。攻击者喜欢它也是有原因的。由于编码和配置问题导致的网站服务器漏洞，不仅会使网站服务受影响，也会导致整个服务器平台被入侵。分析网站服务器生成的日志是揭示攻击尝试和入侵细节的最好方法之一。

IIS网站服务器日志通常位于%WinDir%\System32\LogFiles目录。每个虚拟服务器的日志保存在相应的子目录中，使用服务名称命名。多数情况下，只有一个网站服务器实例运行，因此日志的子目录为W3SVC1。调查过程中，可能发现多个名为W3SVCn的子目录，其中n是虚拟服务器的号码。不过，日志的存储位置是由管理员配置的，可以被修改指向到任何位置，甚至保存到共享的驱动器。默认情况下，日志文件是ASCII文本格式（这个也是由管理员配置的），这意味着它们很容易打开和搜索。有些情况下，日志文件会相当大，尤其是一些活跃的网站，因此手工打开和搜索是不切实际的。可以使用Perl脚本或grep工具来定义搜索，如果要找的东西很确定，也可以使用编辑器的“查找/搜索”功能。

搜索最大的问题在于如何在大量的网站服务器日志中筛选出证据信息，这犹如大海捞针。在一个访问频繁的服务器上，日志文件可能相当大，要在其中搜索相关的数据也是相当费劲的。有时一个受害者的事故报告能帮助调查者确定攻击发生的时间段，通过时间信息可以精简日志数据。不过，这也不是总有效。调查者可能会发现在事件报告之前几个星期或数月之前服务器已经被入侵。这时该怎么办？

回溯到因特网发展很久以前的1997年[10]，Marcus Ranum提出了一个概念，称为“人工忽略”（artificial ignorance，AI）。其基本思路是如果移除网站服务器日志中的所有合法活动，那么剩下的将是“不寻常的”的记录。

工具和陷阱

实现AI

我曾经使用“人工忽略”的方法过滤各种项目，并且它很有用。我写过一个Perl脚本，该脚本会遍历整个企业网络（当时我在一家300～400人的小公司工作），收集所有登录到域中的系统的特定注册表键。我在午饭期间运行这个脚本，午餐过后得到一个漂亮的日志文件，日志可以用Excel中打开和分析。不过，日志文件相当大，而我只想看到关注的内容。因此我开始检测发现的条目，经过验证发现一些条目是正常的，这些正常条目会被添加到“已知良好”的文件中。然后，修改脚本，让它只收集“已知良好”文件中不存在的注册表键。经过一段时间的调整，日志记录的项目从数页减小到不足半页。

对于网站服务器日志，实现这种类型的AI是相当容易处理的。例如，假设正在调

查的案件中，有一个网站服务器可能被入侵，在服务器上的文件数量不多——只包括主页文件 Index. html 和不到 10 个其他 HTML 文件（about. html，contact. html，links. html 等）。

IIS 网站服务器日志是以 ASCII 字符保存的（默认方式），格式相当简单，因此很容易使用调查员喜欢的脚本语言打开文件，每次读取一行日志记录进行处理。IIS 日志通常在文件的顶部会有数据列定义，如果网站服务器重启，则在文件其他地方也会有列定义。使用列定义作为处理的依据，可以分析得到每条记录的相关信息，例如，请求动作类型（GET，HEAD，POST），被请求的页面、状态和返回的相应代码[11]。如果发现日志记录中被请求的页面不是已知的页面，可以记下文件名、请求的时间和日期、请求的源 IP 地址并存入单独的文件进行分析。

警 告

本书没有提供上述处理技术的代码，因为不是所有的 IIS 网站日志都具有相同的格式。日志记录的信息是由网站服务器管理员配置的，因此不能提供一劳永逸的解决方案。而且，不同案件之间的搜索需求也会不同。例如，一个案件中调查员可能对所有被请求的不存在页面感兴趣，另一个案件中感兴趣的则是来自某个 IP 地址或地址段的请求，在其他案件中感兴趣的可能是产生特定响应代码的请求。

“人工忽略”是搜索网站服务器日志的一种方法，该技术很有弹性，可以应用于各种日志和文件的处理。另外一种技术是查找特定攻击留下的痕迹。当系统结构信息很详细并知道黑客获得的访问级别和其他信息时，采用这种技术会比较有效。同样，如果攻击时间前后公布了某一特定漏洞，或者某种特定入侵尝试正在流行，搜索特定的痕迹也会很有效。

例如，如果 IIS 网站服务器使用 MS SQL 数据库服务器作为后端，SQL 注入就是需要调查的一种攻击方式。攻击者可能通过提交需要后端数据库处理的查询到网站服务器，获取网站的更多信息并上传文件等。SQL 注入攻击的明显特征是日志文件条目中存在 xpcmdshell。xpcmdshell 是 MS SQL 服务器的扩展存储过程，攻击者利用它可以使用与数据库服务相同的权限（通常是 System 级别的权限）执行外部命令。

基于不同关键字或词组的搜索还可以解决其他一些问题。例如，网站服务器日志中存在 vti _auth \author. dll 可能意味着 FrontPage 扩展权限问题[12]，这可能导致网站页面被破坏。以前用于查找 Nimdar 蠕虫[13]（参见 CERT 安全公告“系统痕迹”部分）时使用的其他特征包括企图执行 cmd. exe 和 tftp. exe 的 URL。

IIS（和其他网站服务器）日志的分析是个很大的主题，比较适合用整章内容来介绍。不过，对大多数日志文件来说，数据精简原理是相同的：去除所有应该用于记录合法活动的条目。或者，如果知道或者想查找什么，可以使用特征串来查找指定活动的迹象。

背景注释

FTP 日志

我曾经协助过一个调查，有人通过远程管理工具，例如，WinVNC 或者 PCAnywhere 访问 Windows 系统，并且使用安装的 Microsoft FTP 服务器来传输文件。与 IIS 网站服务器相似，FTP 服务器的日志也在 LogFiles 目录下，位于 MSFTPSVCx 子目录中。没有迹象表明该人企图隐藏或者扰乱他在系统中存在的痕迹，这样我们就可以使用 FTP 日志作为活动时间线分析的初始参考。FTP 日志默认格式很不错，其中不仅有访问的时间戳和使用的用户名，还有 FTP 客户端链接来源的 IP 地址。我们将这些信息与注册表中活动时间戳（例如，UserAssist 键等）及事件日志（几个事件 ID，有 10 个条目显示 FTP 连接因为处于不活动状态超时）进行关联分析，得到该用户在系统中活动的清晰描述。

因特网浏览器历史

与网站服务器日志相对的是因特网浏览器（IE）的网站浏览历史。IE 默认安装于 Windows 系统中，并且是许多用户的默认浏览器。在一些案例中，如企业用户，一些企业网站点（用于提交时间卡信息或者旅行费用等）是专门为 IE 而设计的，不支持其他浏览器（如火狐或者 Opera）。使用 IE 浏览网页时，IE 会保留活动的历史记录，调查人员可以开发相应的工具分析用户上网行为并获取证据。IE 浏览历史记录文件保存在用户数据目录下，位于 Local Settings \Temporary Internet Files \Content. IE5 子目录中。在这个目录下，调查人员会发现多个子目录，这些目录的名称由 8 个随机字符组成。

这些目录的结构和内容，包括每个目录中的 index. dat 文件的结构，已经有很多地方提供了详细描述和分析，在这里就不重复这些信息了。对于正在运行的系统，调查员可以使用 WebHistorian 工具（本书写作时，Mandiant. com 网站有 1. 3 版本下载）来分析浏览器历史。在分析镜像时，调查人员可以使用类似 ProDiscover 工具中因特网历史查看器合并和导出上网历史记录。每个子目录中的 Index. dat 文件（无论是来自开机系统还是从镜像中获取），可以使用 Index Dat Spy[14] 和 Index. dat Analyzer[15] 等工具来查看。

提示

通常，在执行调查时，有些地方应该能够找到我们所了解的信息。例如，如果系统审计功能设置为记录成功登录时，就可以从注册表中看到不同用户的最后登录时间，同时在安全事件日志中也能看到成功登录的事件记录。对浏览器历史来说，IE 可以设置保留多少天的访问 URL 历史。该设置在用户注册表配置项中（NTUSER. DAT 文件，开机系统的 HCU），具体位于\Software \Microsoft \Windows \CurrentVersion \Internet Settings \URL History 键。这个值是 DaystoKeep，默认数据是 0x014 或者十进制的 20。如果这个值对应的数据不是默认设置，可以

假设它被修改了，最可能是通过 IE“工具”菜单下的“Internet 选项”设置修改的，该值的修改在“常规”选项卡的“历史”选项。注册表键的最后写入时间将显示这个值是什么时候被修改的。

许多调查人员已经可以熟练使用浏览器历史来描述用户的行为活动。例如，其中可以找到用来下载恶意软件工具的参考站点，如 Myspace. com 或其他用户不该访问的站点。与系统中大部分取证要素一样，要找寻的证据依赖于案件的特性。不过，不要忽略任何细节，一些细小的信息也能为整个案件提供线索或背景信息。

其他日志文件

Windows 系统维护一些其他鲜为人知的日志文件，在操作系统初始安装及每天的操作时产生。其中一些日志文件用来记录安装过程中的动作和错误，另外一些日志文件只有当特定事件发生时才会被创建或添加新的记录。了解这些日志文件的存在、掌握日志文件和条目创建的原因，并且可以对其中的信息进行分析和理解，将会对调查非常有用。这一节介绍几个这种类型的日志文件。

Setuplog. txt

该文件位于 Windows 目录中，用于记录 Windows 系统安装过程中的信息。对调查员来说，这个文件中最重要的就是包括所记录活动的时间戳信息，显示了系统安装的日期和时间。该信息可以协助建立系统活动的时间线。

下面显示的是来自一个 Windows XP SP 2 系统的 setuplog. txt 文件的部分内容：

```
08/07/2006
16:14:22.921,d:\xpsprtm\base\ntsetup\syssetup\syssetup.c,6434,BEGIN_SECTION,
Installing Windows NT
08/07/2006
16:14:24.921,d:\xpsprtm\base\ntsetup\syssetup\wizard.c,1568,,SETUP:
Calculating registery size
08/07/2006
16:14:24.921,d:\xpsprtm\base\ntsetup\syssetup\wizard.c,1599,,SETUP:
Calculated time for Win9x migration = 120 seconds
16:14:24.937,d:\xpsprtm\base\ntsetup\syssetup\syssetup.c,6465,BEGIN_SECTION,
Initialization
08/07/2006
16:14:24.984,d:\xpsprtm\base\ntsetup\syssetup\syssetup.c,6585,BEGIN_SECTION,
Common Initialiazation
08/07/2006
16:14:25.000,d:\xpsprtm\base\ntsetup\syssetup\syssetup.c,1674,BEGIN_SECTION,
Initializing action log
08/07/2006 16:14:25.046,d:\xpsprtm\base\ntsetup\syssetup\log.c,133,,GUI mode
Setup has started.
08/07/2006
```

```
16:14:25.078,d:\xpsprtm\base\ntsetup\syssetup\syssetup.c,1679,END_SECTION,In
itializing action log
08/07/2006
16:14:25.093,d:\xpsprtm\base\ntsetup\syssetup\syssetup.c,1764,BEGIN_SECTION,
Creating setup background window
```

警 告

写本书的时候，我对各个版本 Windows 系统中的 setuplog.txt 进行了调查和比较。在 Windows 2000 中，这个文件有时间戳，但是没有包括日期。在 Windows XP 和 2003 中，文件的内容相似，日志的每个条目中都有带日期的时间戳。在 Vista 中没有发现 setuplog.txt 文件。

从上面 Windows XP 系统 setuplog.txt 的部分输出可以看到，文件中的每个条目都包含日期和时间戳。即使文件的创建、最后修改时间被改变或者文件删除，这个文件的内容依然能为调查提供有价值的信息。

警 告

如果正在分析一个系统镜像，并且发现 setuplog.txt 的时间戳信息不太合理（例如，与其他信息的时间线对应不上），这个系统可能是通过克隆（Ghost）镜像安装的或者从备份中恢复的。记住，setuplog.txt 文件记录了安装时的活动，因此操作系统必须是“安装”到系统上，该文件才能提供有用的信息。

Setupact.log

该文件位于 Windows 目录中，记录了安装过程中图形部分所发生的一系列动作信息。在 Windows 2000，XP 和 2003 中，文件中没有与所记录的各种活动的时间戳，但是该文件的创建和最后修改时间信息为调查员提供了操作系统何时安装的线索。在 Vista 操作系统中，这个文件的很多活动记录项都包含了日期和时间戳信息。

SetupAPI.log

位于 Windows 目录中的 setupapi.log 文件维护着 Windows 系统中设备、服务包和补丁安装记录。Windows XP 及其后续版本在这方面的日志记录更加广泛，尽管 Microsoft 这个文件的主要目的在于排除故障，此文件中的信息对于调查员来说却非常有用。

Microsoft 有一个名为“用 SetupApi 日志文件对设备安装进行排错”的文档，它提供了大量关于 setupapi.log 文件的有用信息。例如，setupapi.log 文件包含 Windows 安装的头信息区域，列出了操作系统版本及其他信息。如果 setupapi.log 文件由于某种原因被删除了，操作系统会创建一个新的文件并插入新的 Windows 安装头。

驱动程序的安装同样也会记录到这个文件中，调查人员可以根据时间戳跟踪系统中的这类活动痕迹。第 4 章已经介绍当 USB 移动存储设备（闪存、iPod 和其他类似的设

备）连接到 Windows 系统时，注册表中留下了记录。当设备第一次被连接到系统时，系统会查找相应的驱动并加载该驱动。现在看一段 setupapi. log 文件的内容：

```
[2006/10/18 14:11:53 1040.8 Driver Install]
#-019 Searching for hardware ID(s):
usbstor\disksony_____sony_dsc_______5.00,usbstor\disksony____sony_dsc_____
__,usbstor\disksony____,usbstor\sony____sony_dsc_________5,sony____sony_dsc
________5,usbstor\gendisk,gendisk
#-018 Searching for compatible ID(s):usbstor\disk,usbstor\raw
#-198 Command line processed:C:\WINDOWS\system32\services.exe
#I022 Found "GenDisk" in C:\WINDOWS\inf\disk.inf; Device:"Disk drive"; Driver:"Disk
drive"; Provider:"Microsoft"; Mfg:"(Standard disk drives)"; Section name:"disk_in-
stall".
#I023 Actual install section:[disk_install.NT].Rank:0x00000006.Effective driver
date:07/01/2001.

#-166 Device install function:DIF_SELECTBESTCOMPATDRV.
#I063 Selected driver installs from section [disk_install] in "c:\windows\inf\
disk.inf".
#I320 Class GUID of device remains:{4D36E967-E325-11CE-BFC1-08002BE10318}.
#I060 Set selected driver.
#I058 Selected best compatible driver.
#-166 Device install function:DIF_INSTALLDEVICEFILES.
#I124 Doing copy-only install of "USBSTOR\DISK&VEN_SONY&PROD_SONY_DSC&REV_5.00\
6&1655167&0".
```

从上面的日志片段可以看到一个索尼生产的 USB 移动存储设备在 2006 年 10 月 18 日第一次连接到系统。基于第 4 章的内容，从日志的最后一条可以看出该设备没有序列号。不过，Driver Install 节中的日期和时间戳显示了设备第一次插入系统的日期，配合相应注册表键的“最后写入时间”可以确定该设备在系统使用的时间线。

Netsetup. log

Netsetup. log 文件在系统安装时创建。Windows XP 中，该文件位于 Windows \ Debug 目录。文件记录了系统工作组和域成员信息，记录的所有信息都有时间戳。Netsetup. log 文件中的时间戳与 setuplog. txt 中记录的时间处于同样的时间片。如果系统的“工作组”或者“域”属性被修改，修改的信息会被添加到文件中。例如，笔者的个人笔记本在 2006 年 8 月 7 日安装了 Windows XP 操作系统，netsetup. log 和 setuplog. txt 文件会记录该时间。2006 年 9 月 19 日，为了授权文件共享，修改系统的工作组信息（从 WorkGroup 改为 Home）。这个信息伴随着时间戳被记录到 netsetup. log 文件。如果系统被添加或移出一个域，日志条目也将添加到这个文件。

计划任务（Task Scheduler）日志

很多人不熟悉 Widnows 系统的计划任务服务，通过 at. exe 或控制面板中的计划任

务向导可以对其进行管理。这个服务允许具有管理员权限的用户去计划一个任务，使其在将来的某点运行或者每天、每周或每月的特定时间重复运行。这对系统或整个网络的管理很有帮助。幸运的是，该服务在 SchedLgU. txt 文件中记录了很多调度任务的运行日志。这个日志文件就是下列注册表键中 *LogFile* 值定义的默认名称：

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\SchedulingAgent
```

在 WindowsXP 中，schedlgu. txt 文件默认保存在 Windows 目录。但是在 Vista 中，schedlgu. txt 的默认保存目录是 Tasks（C:\Windows\Tasks）。

如果管理员没有使用计划任务，日志中将仅包括计划任务服务开始和退出的时间信息。由于计划任务服务通常被设置为与系统一起启动，这个信息也可以用来分析系统的开关机时间。

如果一个任务被计划执行，Windows XP 中 schedlgu. txt 日志文件中记录的信息大致如下：

```
"At1. job" (regedit. exe)
        Started 9/26/2006 4:35:00 PM
"At1. job" (regedit. exe)
        Finished 9/26/2006 4:35:04 PM
        Result:The task completed with an exit code of (0).
"Pinball. job" (PINBALL. EXE)
        Started 9/26/2006 4:36:00 PM
"Pinball. job" (PINBALL. EXE)
        Finished 9/26/2006 4:36:07 PM
        Result:The task completed with an exit code of (0).
```

第一个任务通过 at. exe 设置，第二个任务（Pinball. job）通过计划任务向导设置。任务文件保存在 Windows\Tasks 目录中。

背景注释

隐藏计划任务

有一个有效的隐藏计划任务的方法。通过 at. exe 或计划任务向导创建一个计划任务，打开控制面板中的计划任务管理程序，可以看到新建的任务。现在关闭程序，打开一个命令行窗口，转到 Windows\Tasks 目录，并且使用 attrib. exe 程序设置任务文件为“隐藏”属性。完成之后，计划任务管理程序就不再可以看到设置为“隐藏”属性的任务。当然，命令行和资源管理器在默认情况下，也看不到这个隐藏的文件。不过，该任务还是会按计划执行。

写第一本书时，我也被这个特性吓了一跳。当时我是利用假期写作，并且使用蜘蛛牌游戏程序进行上述设置。不过，当时没有删除文件，期间利用周末出去休息了。当我回到办公室时，看到蜘蛛牌程序桌面上打开了，第一个想法是有人到过我的办公室！然后发现事情的真正原因后，删除了这个 .job 文件。

不幸的是，日志文件中没有记录任务执行文件的全路径，但提供了程序执行的时间信息。

XP 防火墙日志

可能是由于新闻媒体和 Windows XP SP 2 对安全问题宣传力度比较大的缘故，大多数人对 Windows XP 中的防火墙组件还是相对熟悉的。多数用户甚至没有看过或与 XP 防火墙交互过，但它是默认开启的。这个防火墙可以被禁用[16]，并且可以通过公司配置策略来进行管理。Windows XP 防火墙也可手动配置[17]，以允许特定应用程序访问网络。

Window XP 防火墙有一个日志文件，其中记录了多种出现的活动信息，但是默认情况下，不会记录日志。图 5.6 是防火墙日志设置对话框的默认设置。

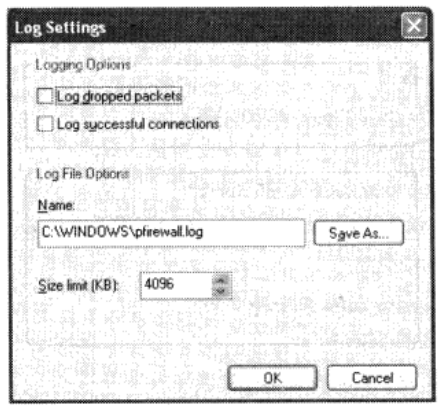


图 5.6 Windows XP 防火墙日志设置

可以看出，日志选项相当有限。默认情况下日志是不启用的，因此大多数系统中找不到防火墙日志文件 pfirewall. log。没有日志文件并不意味着防火墙没有启用。不过，如果系统中有这个防火墙日志文件，读者就会发现该文件的格式相当直观易懂。如下是防火墙日志的部分内容：

```
#Version:1.0 #Software:Microsoft Internet Connection Firewall
#Time Format:Local
#Fields:date time action protocol src-ip dst-ip src-port dst-port size tcpflags tcpsyn
tcpack tcpwin icmptype icmpcode info
2003-10-10 10:21:11 DROP ICMP 131.107.0.2 131.107.0.1--60----80-
2003-10-10 10:21:16 DROP ICMP 131.107.0.2 131.107.0.1--60----8 0-
2003-10-10 10:21:21 DROP ICMP 131.107.0.2 131.107.0.1 --6 ----80 -
2003-10-10 10:21:26 DROP ICMP 131.107.0.2 131.107.0.1 --60 ----80 -
2003-10-10 10:21:34 DROP TCP 131.107.0.2 131.107.0.1 1045 21 48 S 1226886480 0 16384 ---
2003-10-10 10:21:37 DROP TCP 131.107.0.2 131.107.0.1 1045 21 48 S 1226886480 0 16384 ---
2003-10-10 10:21:43 DROP TCP 131.107.0.2 131.107.0.1 1045 21 48 S 1226886480 0 16384 ---
```

防火墙日志头部的 Fields 标志显示了日志记录中各个部分含义和对应关系，通过

它可以解析日志文件中的内容。从上面的日志片段信息可以看出有几个 ICMP 包被丢掉（可能来自 ping. exe 程序），还有几个到端口 21 的连接尝试，端口 21 是 FTP 服务的默认端口。

警 告

如果不能详细了解系统及其运行环境，就很难分析 pfirewall. log 文件中记录的行为活动。例如，在基于公司防火墙和 IDS 日志分析网络行为时，管理员经常会问这些记录的活动信息有什么含义。

对单机系统来说，端口 80（网页服务器）或者端口 21（FTP 服务器）等已知端口的访问尝试并不表明系统中有相应的服务，只是有人在尝试确定端口上是否有服务在运行。这是类似端口扫描的前期试探活动，如果日志表明同时有几个系统都有类似的尝试行为，这可能预示着大范围的端口扫描。

这里要说明的重点是，一条日志记录表明对特定端口的访问行为并不定意味着系统中那个端口是打开的（系统中有服务在该端口监听）。这是一种常见的误解，尤其是发现木马后门程序在进行大范围扫描的时候。

为了方便查看，可以使用一些免费工具来分析该文件，甚至可以使用彩色标记某些条目。通过在 Google 上查找 XP，firewall 和 viewer 的等多种组合，能够找到满足需求的分析工具。

提 示

附送光盘第 5 章目录中包含了一个 samples 子目录。该目录中包含一个名为 nmap _scan _XP. txt 的文件，文件中包括使用 nmap 工具扫描 Windows XP SP2 系统的指令和标准输出结果。另外一个 pfirewall _nmap _scan. txt 文件中包含目标系统记录的部分数据包。说明，nmap 扫描位置在 192. 168. 1. 18，目标系统是 192. 168. 1. 6。

Dr. Watson 日志

Dr. Watson 工具[18]很早以前就随 Windows 一起发布了，但直到今天也很少有人讨论它。当一个程序在系统中发生错误时，Dr. Watson 工具会收集系统和程序有关错误信息并保存到一个文本日志文件中，该文件可以发给支持人员用于系统排错和改进程序。在对系统进行调查时，这个信息也很有用。

由 Dr. Watson 产生的文本日志文件名为 drwtsn32. log，保存在下面的目录中：

```
C:\Documents and Settings\All Users\Application Data\Microsoft\Dr Watson
```

Dr. Watson 工具的配置信息保存在下面的注册表键中：

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\DrWatson
```

这个注册表键下包含一些列在 Dr. Watson 图形界面中可见的值，也就是通过“开始”菜单运行 drwtsn32 时会看到的。默认情况下，日志文件会保存 10 个程序异常信息。这些值将指示调查者，如果系统发生异常日志文件中会有什么内容。

错误发生时，Dr. Watson 工具保存的信息会添加到 drwtsn32. log 文件中。Dr. Watson 首先会记录描述信息，开始文字为“Application exception occurred:”，描述信息包含程序名称和错误发生的日期和时间：

```
App:C:\Perl\bin\perl. exe (pid = 4040) When:8/21/2006 @ 10:17:35. 859
```

日志文件包括错误程序的全路径文件名和时间戳信息。这些信息对于调查很有帮助，如果发生错误的程序就是恶意程序、入侵者复制的工具或非法使用的工具，则会更有价值。在概述信息之后，Dr. Watson 会记录系统信息、运行进程的列表、程序加载的模块（DLLs）列表和堆栈转储信息，保存到日志文件中以用于对程序的异常分析。调查员通过日志中的信息能够知道：用户登录到系统的确切日期、什么进程在运行（这可以显示哪些程序曾被安装）、异常程序加载了哪些动态链接库（可能会显示 IE 安装的 BHO 对象，以及任何注入到进程中的动态链接库等）。

注 释

Dr Watson 日志在证明或确认系统中活动的时间线时非常有用。一个案子中，某人访问系统并上传了一些工具，在试图运行工具的时候发生了应用异常。我们找到了他访问系统的日志，日志显示是何时上传工具的（包括了发起连接的 IP 地址），事件日志记录显示了应用异常弹出的消息，Dr. Wastson 日志记录了应用程序的崩溃信息。除了应用程序名称之外，还有应用程序的用户上下文及当时运行的进程列表。所有这些信息使得现场情况更加清晰：在访问系统之前运行的程序有哪些、哪些是后来添加到系统的、何时使用的。

Dr. Watson 还会产生一个崩溃转储文件（user. dmp），该文件与文本日志文件位于同一目录。这个转储文件中包括进程异常发生时使用的私有页面，但不包含可执行程序的代码页（EXE，DLL 等）。这个 user. dmp 文件能够使用 WinDbg 工具打开，该工具是 Microsoft 调试工具的一部分。不过，user. dmp 文件会不断被随后的异常覆盖，因此只能看到最近一次异常产生的文件。

崩溃转储文件

崩溃转储文件在第 3 章讨论过。为了讨论的完整性，在这里再简单提一下。

第 3 章讨论了怎样配置和产生崩溃转储文件，但是大部分情况下，我发现系统本身并不会被修改。如果在事件和调查现场找到一个崩溃转储文件，最好是看其中包含什么内容。可以使用 dumpchk. exe（针对 Windows 2000/2003[19] 和 XP[20]）类似的工具来验证转储文件是否合法。然后可以使用调试工具（如 WinDbg）加载转储文件，在调试器中使用! process 0 0 命令显示崩溃时运行的进程列表，使用 lm kv 命令查看加载的核心模式驱动信息，而且可以使用 strings. exe，BinText. exe 和正则表达式等来查找特定的信息。

回收站

大部分的司法调查人员明白一个古老的名言：当一个文件被删除时，它没有真的消失。Windows 桌面上的回收站更好地说明了这点。回收站如同丢弃文件的一个隐喻，好比将纸张揉成一团然后扔进垃圾桶。回收站同样允许回收和恢复“不小心”丢掉的文件。可以打开回收站，选择先前丢弃的文件，然后将其恢复到之前的位置。

因此，当一些文件通过系统命令解释程序（shell）删除时——也就是用户在桌面或资源管理器中选中并删除文件时——它不是真正地消失。文件仅仅是被移到“回收站”，默认情况下，回收站就是每个驱动器根目录下的 Recycler 目录。在很多案件中，这个目录能够提供重要的信息。

为了更好地理解这个目录中的信息如何被用来作为证据，现在看通过系统命令解释器（Shell，这里是指资源管理器）删除文件时会发生了什么。用户通过 Shell 删除文件时（不是使用命令行中 del 或者 erase 命令），在回收站目录（Recycler）中会为用户创建一个子目录，目录名称是用户的安全标识符，或者叫 SID。例如，从命令行来看，这个子目录可能如下所示：

```
C:\RECYCLER\S-1-5-21-1454471165-630328440-725345543-1003>
```

从桌面打开回收站时，用户子目录会自动打开以便于查看。因此如果使用用户的笔记本，用该用户的账户登录并打开回收站查看内容，将看到这个用户“删除”的文件。如果切换成另一个账户并重复这个过程，将自动看到当前活动用户账户删除的文件。

当查看镜像中的回收站目录时，会看到系统中每个活动用户子目录，子目录中是通过系统 Shell 删除的文件，如图 5.7 所示。

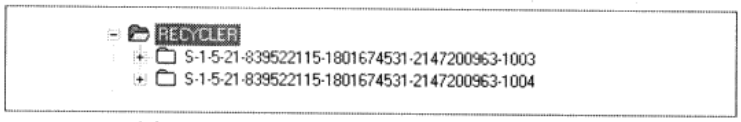


图 5.7 通过 ProDiscover 查看回收站的示例

在每个子目录中，可能会看到一系列文件，这取决于用户活动及用户每隔多久清空回收站。发送到回收站的文件会使用特殊的命名约定保存[21]，一旦理解其命名约定，就可以很容易区分不同类型的文件，找到感兴趣的文件。文件被移入回收站时，使用以下约定对其重命名：

```
D<文件原始驱动器符><#>.<原始扩展名>
```

文件名以字母 D 开头，接着是被删文件的原驱动器字母，后跟从 0 开始的文件序号（换句话说，第五个删除文件的号码是 4）。这个文件保留了原来的扩展名。同时，一条记录被添加到该目录的 INFO 2 文件中，这是当前回收站中所有文件的日志文件。删除文件的索引号用来指向 INFO 2 文件中保存文件原文件名和路径的记录。

幸运的是，Keith Jones（原先任职于 Foundstone，现在 Madiant）详细记录了文档 INFO 2 文件格式，因此这个信息对取证分析更加实用。INFO 2 文件中的记录对应于回收站中的每个删除文件，每条记录中包含记录号、驱动器符、文件移入回收站的时间戳、文件大小、文件原始名称和全路径，同时使用了 ASCII 和 unicode 编码。

INFO 2 文件以 16B 头部开始，头部最后的 *DWORD* 值是每条记录的大小。这个值是 0x320（低位在前），转换成十进制就是 800B。第一条记录紧跟着头部开始，长度是 800B。

记录的第一个 *DWORD*（4B）可以忽略。文件的原始全路径和名字是以 NULL 结束的 ASCII 格式的字符串，从第一个 DWORD 后面开始，占用了记录的第一个 260B。打开 INFO 2 文件，将看到 ASCII 格式的文件名的大部分空间是 0。这些 0 可以忽略，直接得到文件名。记录中余下的项如下：

- 记录号，*DWORD* 类型，位于记录偏移 264B 处
- 驱动器标记，*DWORD* 类型，位于记录偏移 268B 处。驱动器标记用来确定文件是从哪个驱动器删除的；其中，2=C:\、3=D:\，以此类推。
- 文件移入回收站的时间戳，是 64b 的 FILETIME 类型，位于记录偏移 272B 处。
- 被删除文件的大小（以簇大小为单位），*DWORD* 类型，位于记录偏移 280B。

剩余的记录空间包含 unicode 格式的文件名，从 284B 开始到记录结束（516B）。简单地去除 NULL 字节就可以得到英文 ASCII 格式的文件路径和名称（注释：unicode 格式是 2B 宽，移除 unicode 格式中第二个字节将得到英文 ASCII 格式）。

附送光盘中的 recbin. pl Perl 脚本能够找回每条记录的各类信息，显示记录号、文件删除到回收站的时间戳（UTC 格式，没有考虑系统的时区设置）、原始的文件路径和名称。该脚本使用 INFO 2 文件的全路径作为参数，对输出进行调整可以得到调查员需要的任何结构和格式。

Keith Jones 还提供了一个名为 *rifiuti* 的工具（意大利语“垃圾”的意思），用于处理 INFO 2 文件的内容。rifiuti. exe 可以从 Foundstone. com 网站免费下载，分析结果可以使用电子表格程序打开和查看。

背景注释

近距离看回收站

调查者应该同样注意在回收站中没有被保存在用户 SID 子目录中的文件，以及不满足命名约定的文件。这可能预示着用户或恶意程序的恶意行为——用于隐藏文件。调查者也应该注意类似诺顿的反病毒程序会使用回收站，诺顿的回收站保护程序会在目录中保存一个名为 nprotect. log 的文件。取证工具厂商 Datlifter 有一个 Nprotect 查看器[22]工具，可以解析 nprotect. log 文件的内容。NProtect 查看器是 Datalifter . Net 工具套装的一部分。

我在镜像中挖掘和分析时，会检查 INFO 2 文件的最后修改时间。这可以显示 INFO 2 文件最后一条记录是何时添加的，这基本等同于最后一个文件的删除时间。如果回收站中的用户子目录仅包含 desktop. ini 和 INFO 2 文件，并且 INFO 2 文件比较小，那么它的最后修改时间就是用户清空回收站的时间（用户右击“回收站”，选择快捷菜单的“清空回收站”选项）。

系统还原点

系统还原点中维护的注册表文件在第4章讨论过了。这一部分主要讨论还原点中保存的日志文件。

Rp.log 文件

Rp.log 是还原点日志文件，位于还原点（RPxx）目录中。还原点日志包含一个标识还原点类型的值、还原点创建事件的描述性名称（例如：程序或设备驱动安装，程序卸载等）和还原点创建的 64bFILETIME 时间信息。还原点类型使用 4B（*DWORD* 类型），开始于文件的第 4 字节。还原点描述是 NULL 结尾的 unicode 字符串，从文件的第 16（0×10）字节处开始。创建时间是 8 字节（*QWORD* 类型），位于文件的 528（0×210）字节偏移处。

附赠光盘中的 sr.pl 的 Perl 脚本（与第 4 章讨论的 sr.pl 脚本相同）能够在开机系统中运行并收集还原点信息。该脚本使用 SystemRestore WMI 类来访问每个还原点的 *RestorePointType*，*Description* 和 *CreationTime* 值，并且显示给用户。

光盘中的 SysRestore.pl Perl 脚本是一个 ProScript 脚本，可以在 ProDiscover 中使用，用于获取 Windows XP 系统镜像还原点目录中的 rp.log 文件的信息。该脚本打开每个目录中的 rp.log 文件，获取还原点描述和还原点创建日期信息。

还原点描述对于调查者很有帮助，尤其是在需要查找应用程序安装和卸载信息的时候。应用程序及无签名的驱动安装时、自动更新安装时和系统恢复执行时，系统会创建一个还原点。

还原点也能够手工创建。还原点创建时，引发还原点创建的事件描述会记录到 rp.log 文件中。很多时候会看到 System Checkpoint 描述，这是由 Windows XP 每 24 小时创建的还原点（默认设置）。Software Distribution Service 指的是 Windows 更新被安装。此外，还可能看到 Installed QuickTime，Removed ProDiscover 4.8a 和 Installed Windows Media Player 11 等还原点描述信息。还原点描述告诉调查员特定程序安装或卸载的日期信息。

还原点的创建日期对于调查者也是很有用的。不仅是由于系统活动时间线添加了新的信息，而且调查者也能够使用创建时间来检测系统时间是否改动。如果连续的还原点没有连续的创建日期，这预示着某人修改了系统时间（基于一系列还原点的连续性，如 RP80，RP81，RP82 等）。

Change.log.x 文件

还原点创建以后，关键的系统和应用文件会被一直监控，以确定系统能够被恢复到特定的状态。文件的改变被记录，若必要，会为系统还原保留整个文件。这些修改记录于 change.log 文件，它位于还原点目录中。还原点文件系统驱动检测到被监控文件改变时，change.log 文件会记录源文件名、一个序号及其他必要的的信息，如发生修改的类型（文件删除、文件属性修改或内容修改）。如果被监控的文件需要保留（在文件被删除时），文件会被复制到还原点目录中，并且重命名为 Axxxxxxx.ext 格式，其中 x 表示序列号，.ext 是文件原来的扩展名。

系统重启时，第一个 change. log 文件会被添加一个序列号（change. log 文件被改名为 change. log. 1），并且创建一个新的 change. log 文件。要注意还原点目录中是找不到 change. log 文件的，能够找到的是名为 change. log. x 的文件，x 是 change. log 文件的序号。

每个 change. log. x 文件包括一系列变更日志记录。在网站上[23]可以找到描述这些记录二进制格式的详细信息（用 0xABCDEF12 作为“幻数”可以在未分配空间中找到变更日志记录）。使用站点中的信息，我创建了一个用来处理和分析 change. log. x 文件内容的 Perl 脚本。附送的光盘上有这个 Lscl. pl 脚本。

位于根目录中的 Fifo. log 是另一个由系统还原维护的文件。当系统还原达到 90%容量时，将采用先进先出方式（FIFO）删除还原点，将容量减少至最大值的 75%（可能是默认也可能是用户定义的值）。Fifo. log 文件保存着被监控分区上采用这一原则删除的还原点列表，同时也包括删除的日期和时间。创建的还原点超过 90 天时，也会被以先进先出方式删除[24]。

Prefetch 文件

从 Windows XP 开始，Microsoft 操作系统开始使用“预读”（prefetching）技术来改进系统性能。Windows XP，2003 和 Vista 默认执行启动预读，XP 和 Vista 还会默认启用应用程序预读。

在启动预读方面，缓存管理器会监视硬页故障（需要从磁盘读取数据）和软页故障（需要将内存中的数据添加到进程工作集），时间是在下面事件的第一个结束之前——启动过程的头两分钟、所有 Windows 服务启动的第一分钟或者用户命令解释器启动的前 30s。故障数据会和涉及的文件与目录一起处理，从而使得所有数据从同一个文件访问，而不是从散布于硬盘各处的不同文件和目录中获取。因此，该技术减少了启动系统需要的时间。

在应用程序预读时，缓存管理器监视进程启动的前 10s。处理的数据被写入到位于 Windows \Prefetch 目录的 . pf 文件中。文件名称为应用程序名称加一个破折号再加上程序路径的十六进制哈希。例如，Windows XP 系统中，写字板程序从 C:\ Windows 目录和 C:\Windows \system32目录下分别运行时，会创建两个不同的 . pf 文件（因为一些原因，Windows XP 在两个目录中都有写字板程序）。预读是由下面的注册表键控制的：

```
HKEY_LOCAL_MACHINE\SYSTEM\ControlSet00x\Control\Session
Manager\Memory Management\PrefetchParameters
```

在这个键中有一个名为 EnablePrefetcher 的值。该值中的数据定义了系统使用何种形式的预读：

- 0：预读被禁止
- 1：程序预读启用
- 2：启动预读启用

■ 3：程序和启动预读都启用

在 Windows XP 和 Vista 中，EnablePrefetcher 的默认值是 3；Windows 2003 中是 2。需要注意的是，Windows XP 限制程序预读为 128 个 .pf 文件。

Prefetch 目录中 .pf 文件的一些信息对调查是极其有用的。文件偏移 144B 处的 *DWORD*（4B）值是该程序被运行的次数。文件偏移 120B 处的 64b FILETIME 值对应程序最后一次运行的时间。这个值是以 UTC 格式存储的，等同于 GMT 时间。附送光盘中的 prefetch. pl 是一个 ProScript 脚本，用于分析 Prefetch 目录中的 .pf 文件，可以得到程序的运行次数统计和最后一次运行时间。pref. pl 的 Prel 脚本可以在开机系统中运行，分析 Prefetch 目录以提取 .pf 文件的 MAC 时间戳（下一节介绍 MAC 时间戳）和最后运行时间信息，分析结果以逗号分隔的文本格式发送到控制台（可以使用 Excel 电子表格程序打开）。

尽管 .pf 文件不能提供运行程序的用户信息，但是文件名会揭示哪个程序运行过（如果运行过不同目录中的多个同名程序，需要分析哪个 .pf 对应哪个程序），此外 .pf 文件提供程序运行次数和最后一次运行时间信息。

运行程序的路径信息以 unicode 字符串格式保存在 .pf 文件中（还有其他一些字符串），如图 5.8 所示。

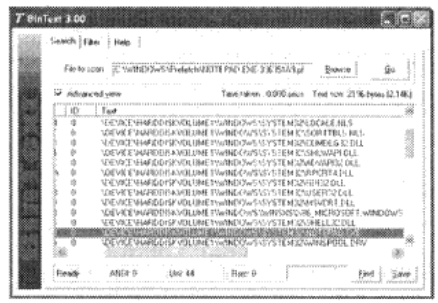


图 5.8 .pf 文件中的文件路径实例

.pf 文件中的很多信息可以与注册表中（参见第 4 章）或者是事件日志中的信息关联起来，从而确定当时的登录用户是谁，程序都是由谁运行的等。这个关联分析的一个好处就是如果用户安装、运行然后删除了程序，程序的痕迹会留在 Prefetch 目录中。当我和司法官员说起网络犯罪中使用的信息隐藏程序问题时，他们都说除非有迹象表明该类程序被使用过，否则不会查找信息隐藏内容。特定程序名称的 .pf 文件就是那种迹象。

快捷方式文件

实践证明，快捷方式文件对调查来说也很实用。通常快捷方式（扩展名为 .lnk 的文件）每天都会创建和使用。用户访问硬盘、移动存储设备或者网络共享的文件，系统就会在 Recent 目录（是用户目录下的一个隐藏目录）中创建一个快捷方式。

快捷方式能够提供用户访问过的文件和网络共享信息，以及用户一度连接和使用的移动存储设备信息。AccessData 公司的 FTK 和 Guidance Software 公司的 EnCase 等商业取证分析工具都提供了分析 .lnk 文件内容的功能。MiTec 提供的免费工具 WFA

(Windows File Analyzer)[25]也可以分析 .lnk 文件中的信息

不久之前，Jesse Hager 发布了一篇名为《Windows 快捷方式文件格式》的文章(当时可以在 I2S-Lab[26]找到)，文中详细描述了快捷方式文件中各字节偏移处包含的内容。Nathan Weilbacher[27]参考 Jesse 的论文为 ForensicFocus.com 网站写了一篇文章，细述了 Windows 快捷方式文件中的证据的价值。

附送光盘中的 lslnk.pl 的 Perl 脚本实现了 Jesse 论文中对快捷方式文件的解析，使调查者可以清晰地查看 Windows 快捷方式文件内部结构，显示目标文件的 MAC 时间戳、各种标识、属性设置和本地卷信息等。下面是一个实例：

```
Shortcut file is on a local volume.
Volume Name = C-DISK
Volume Type = Fixed Volume
SN = 0x303d30de
```

如果目标文件来自网路共享，lslnk.pl 将获取共享路径，如下所示：

```
File is on a network share.
Network Share name = \\192.168.1.22\c$ Z:
```

Lslnk.pl Perl 脚本以二进制模式打开快捷方式文件，分析内容时未使用 Windows API。该脚本能够在任何支持 Perl 的系统中使用。Jake Cunningham 实现了一个具有类似功能的 Perl 脚本，名为 lnk-parse.pl，可以从他的 JAFAT[28]网站下载。

文件元数据

术语“元数据”是指“关于数据的数据”。在 Windows 系统中最为人所知的文件元数据是文件的 MAC 时间，MAC 表示修改、访问和创建。MAC 时间是文件曾经以某种方式修改（文件内容添加或者删除）、最后访问（文件最近被打开）和文件最初创建的时间戳。操作系统对这些时间的管理依赖于使用的文件系统。例如，在 FAT 文件系统中，时间存储基于计算机系统的本地时间，但是 NTFS 文件系统使用国际时间格式(UTC) 存储 MAC 时间，类似于格林尼治时间 (GMT)。当资源管理器等程序显示文件 MAC 时间时，需要同时考虑时区和夏令时设置。而且，FAT 系统中 MAC 时间的精度也各不相同，创建时间的精度为 10ms，修改时间为 2s，最后访问时间为 1 天（实际上是日期）。NTFS 文件系统中，最后访问时间的精度是 1h。

警 告

Windows 系统中，设置 NtfsDisableLastAccessUpdate 注册表值（位于 HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\FileSystem 键中）将允许用户禁用操作系统中文件最后访问时间的更新（*DWORD* 值为 1 时表示禁用功能），而且这是对大容量文件服务器的推荐设置（以优化性能和减少总体响应时间）。这会使取证分析更加困难，特别是文件访问时间对案件特别重要时。在

windwos XP 和 2003 中，该值可以通过 *fsutil* 命令设置，而 Vista 默认是禁止更新最后访问时间的。这意味着取证分析人员需要开发其他分析技术和方法，依赖其他的证据来源。

对文件和目录的 MAC 时间戳需要关注的另外一个方面是在执行文件移动或复制操作时，文件的这些时间所产生的变化[29]。对于 FAT 16 文件系统：

- **从 C:\subdir 下复制 myfile.txt 到 C:** myfile.txt 的修改时间保持不变，但是创建日期会更新为当前的日期和时间。
- **从 C:\subdir 下移动 myfile.txt 到 C:** myfile.txt 文件保持相同的创建和修改日期。
- **从 FAT 16 分区中复制 myfile.txt 文件到一个 NTFS 分区** myfile.txt 保持相同的修改时间，但是创建日期会更新到当前的日期和时间。
- **从 FAT 16 分区移动 myfile.txt 到一个 NTFS 分区** myfile.txt 文件保持相同的创建和修改日期。

对于 NTFS 文件系统：

- **从 C:\复制 myfile.txt 文件到 C:\subdir** myfile.txt 保持相同的修改时间，但是创建时间被更新到当前的日期和时间。
- **从 C:\下移动 myfile.txt 到 C:\subdir** myfile.txt 文件保持相同的创建和修改日期。

简单来说，不管使用的何种文件系统，如果文件被复制，文件的创建时间就被更新为当前的日期和时间；如果文件被移动，则创建时间保持一致。当文件改变时，修改时间被更新。

背景注释

修改 MAC 时间

由于文件的 MAC 时间戳在调查中经常使用，所以也要注意有人可能试图通过修改文件的 MAC 时间隐藏系统中的数据。我曾经在会议上演示怎样使用工具来修改文件的 MAC 时间戳，通过 Perl 脚本调用必须的 Windows 函数来先创建一个文件，然后把创建日期改为 6 年后而将修改日期设定为两年前。这种事会干扰调查，并且当你看到这种情况时，还能相信任何 MAC 时间戳吗？

但这还不是全部，MetaSploit 有一个反取证（Anti-Forensics）项目[30]，其中包括了一个名为 timestomp.exe 的工具，通过它不仅可以修改文件的 MAC 时间戳，还可以修改文件的“项目修改”（entry modified）的时间戳，这是文件属性修改的时间。

随带说一下，MetaSploit 反取证项目还发布了一个名为 slacker.exe 的工具，该工具可以在文件的闲置空间隐藏数据。

本节的后续部分讨论多种文件格式中的元数据。

Word 文档

Word 文档中的元数据问题存在已久。Word 文档是复合文档，基于对象链接嵌入技术（Object Linking and Embedding，OLE）技术，定义了一个文件中的文件系统结构。除了格式信息外，Word 文档中包含相当多用户不可见的附加信息，依赖于用户查看文档的方式。例如，Word 文档不仅能保存过去的修订，还能保存最后 10 个修改文件的作者信息。这会导致个人和组织信息的暴露。这其中最引人瞩目的就是 Richard M. Smith 于 2003 年中发表的公开文章，它与当时的英国首相布莱尔发布的一个文档[31]有关。2003 年 2 月，布莱尔政府在网站上以 Word 格式发布了一个有关伊拉克安全和情报组织的档案。一位剑桥大学政治学讲师认出了文档的部分内容是由美国在伊拉克的研究员写的。这带来了其他人对该文档的更多关注。在这个信息暴露问题的讨论中，该讲师展示了他从 Word 文档中获取的信息，包括最后 10 个修改文档的作者列表。这些信息的暴露使得布莱尔政府相当难堪。

讲师在个人网站上提到了本人编写的用于从 Word 文档中获取信息的工具，但是这个工具没有提供给其他人使用。作者写了一个 Perl 脚本叫 wmd.pl，包含在附送的光盘中，它能够通过处理 Word 文档的二进制头来获取一些信息。这个脚本使用了 Perl 的一些模块（脚本没有使用 MS Word API，因此该脚本能够在任何支持 Perl 并安装有必须的模块的系统中运行，脚本中 use 文法列出了需要的模块）来获取附加的信息。脚本处理布莱尔的文档的结果如下：

```
C:\Perl>wmd.pl g:\book2\ch5\blair.doc
.....

Statistics
.....

File = g:\book2\ch5\blair.doc
Size = 65024 bytes
Magic = 0xa5ec (Word 8.0)
Version = 193
LangID = English (US)

Document was created on Windows.

Magic Created:MS Word 97
Magic Revised:MS Word 97

.....

Last Author(s) Info
.....

1:cic22:C:\DOCUME~1\phamill\LOCALS~1\Temp\AutoRecovery save of Iraq-security.asd
2:cic22:C:\DOCUME~1\phamill\LOCALS~1\Temp\AutoRecovery save of Iraq-security.asd
```

```
3:cic22:C:\DOCUME~1\phamill\LOCALS~1\Temp\AutoRecovery save of Iraq-security.asd
4:JPratt:C:\TEMP\Iraq-security.doc
5:JPratt:A:\Iraq-security.doc
6:ablackshaw:C:\ABlackshaw\Iraq-security.doc
7:ablackshaw:C:\ABlackshaw\A;Iraq-security.doc
8:ablackshaw:A:\Iraq-security.doc
9:MKhan:C:\TEMP\Iraq-security.doc
10:MKhan:C:\WINNT\Profiles\mkhan\Desktop\Iraq.doc

.....
Summary Information
.....
Title:Iraq-ITS INFRASTRUCTURE OF CONCEALMENT, DECEPTION AND INTIMIDATION
Subject:
Authress:default
LastAuth:MKhan
RevNum:4
AppName:Microsoft Word 8.0
Created:03.02.2003, 09:31:00
Last Saved:03.02.2003, 11:18:00
Last Printed:30.01.2003, 21:33:00

.....
Document Summary Information
.....
Organization:default
```

如上所示，一些隐藏在 Word 文档中的信息会暴露并具有潜在的麻烦。除了最后 10 个作者外，脚本还能显示文档创建的平台（Windows 或者 Mac）、创建文档的 Word 版本和最后修订信息。这个脚本同样也能提取文档摘要信息（在本章“NTFS 分支数据流”小节中再深入讨论）。

附送光盘中还包括另一个小工具叫 oledmp.pl。这个工具使用与 wmd.pl 相同的模块，但功能有所不同。Oledmp.pl 会列出 OLE 流和 Word 文档内部的垃圾数据，同时列出与 wmd.pl 相同的摘要信息，如下显示了实例输出：

```
C:\Perl>oledmp.pl blair.doc
ListStreams
Stream:☺CompObj
Stream:Word Document
Stream:]DocumentSumaryInformation
Stream:OjbectPool
Stream:1Table
Stream:]SumaryInformation
```

```
Trash Bin           Size
BigBlocks           0
SystemSpace         940
SmallBlocks         0
FileEndSpace        1490

Sumary Information
Subject
lastauth            Mkhan
lastprinted         30.01.2003, 21:33:00
appname             Microsoft Word 8.0
created             03.02.2003, 09:31:00
lastsaved           03.02.2003, 11:18:00
revnum              4
title               Iraq-ITS INFRASTRUCTURE OF CONCEALMENT, DECEPTION AND INTIMIDATION
authress            default

1Table
1   cic22       C:\DOCUME~1\phamill\LOCALS~1\Temp\AutoRecovery save of Iraq-securi-
                ty.asd
2   cic22       C:\DOCUME~1\phamill\LOCALS~1\Temp\AutoRecovery save of Iraq-securi-
                ty.asd
3   cic22       C:\DOCUME~1\phamill\LOCALS~1\Temp\AutoRecovery save of Iraq-securi-
                ty.asd
4   JPratt      C:\TEMP\Iraq-security.doc
5   JPratt      A:\Iraq-security.doc
6   ablackshaw  C:\ABlackshaw\Iraq-security.doc
7   ablackshaw  C:\ABlackshaw\A;Iraq-security.doc
8   ablackshaw  A:\Iraq-security.doc
9   MKhan       C:\TEMP\Iraq-security.doc
10  MKhan       C:\WINNT\Profiles\mkhan\Desktop\Iraq.doc
```

ListStreams 信息显示了各种组成 Word 文档的 OLE 流名字。Microsoft 将 OLE 定义为“文件中的文件系统”，并且在文档中称这些流为“文件”。

警 告

有时人们会被 Word 文档中元数据包含的信息所震惊。做一个小实验：找到一个工作文件服务器，并且寻找其中存储的一些 Word 文档，如一些发给客户的文档，查看文档中隐藏的元数据。我做了一些尝试，只是使用 Google 来查找文档。根据得到的搜索结果，我将文档范围限制在 .mil 和 .gov 域名，但是仍然找到很多额外的信息，说明了这些文档的用途。

有趣的是，在写第一本书时，一位技术评论人员不想让我知道他的名字，特意

让出版商不要告诉我任何评论者的信息。为做到这点，这位评论人员在 Word 文档中完成评论表格，但是以 ASCII 文本文件保存内容，从而删除了所有元数据。我猜他是真的不想我知道他是谁！

这些元数据不仅会使得个人和公司承担信息暴露的风险，而且对查找特定文档信息的调查员也很有用。这在电子发现的案件中特别重要，尤其是查找的关键词和短语被限制于文档中可见文字的时候。

为了讨论的完整性，在开始下个话题前还要说几句。首先，Microsoft 提供了有关 Word 文档中元数据信息的介绍，以及最小化元数据的方法。第二，Office 文件中不仅是 Word 文档有元数据问题。对于这两点，Microsoft 提供了下面的知识库文档：

- 223 790：WD97：怎样最小化 Microsoft Word 文档中的元数据
- 223 396：OFF：怎样最小化 Microsoft Office 文档中的元数据
- 223 789：XL：怎样最小化 Microsoft Excel 工作簿中的元数据
- 223 793：PPT97：怎样最小化 Microsoft PowerPoint 演示文档中的元数据
- 290 945：怎样最小化 Word 2002 中的元数据
- 825 576：怎样最小化 Word 2003 中的元数据

在这些知识库文章之外，作为 Office 2003 和 XP 的插件，Microsoft 还提供了移除隐藏数据的工具[32]。文档作者可以使用这个工具移除文档中的大量元数据。这是一个很好的工具，确保了可用元数据被最小化，甚至在将文档保为其他格式时也很有用，如保存为 PDF。

背景注释

MergeStreams 工具

在 NTKERNEL.com 上提供了一个名为 MergeStreams[33] 的工具，实现了 Office OLE 文档的一个有趣功能。简单来说，它可以将一个 Excel 表格“合并”到 Word 文档中。这个工具有简单的图形用户界面，通过界面可以选择一个 Word 文档和一个 Excel 电子表格，并且将两个文件合并在一起。例如，目录中各有一个 Word 和 Excel 文档。如果运行了工具并对这两个文档进行合并，将获得一个 Word 文档，它比原始的 Word 文档和 Excel 电子表格都大。不过，如果删除 Excel 电子表格，并且将那个 Word 文档文件的扩展名改为 .xls，然后双击文件，就可以在桌面上看到打开的 Excel 电子表格，其中没有任何 Word 文档及其内容的证据。再将它的扩展名改回 .doc，双击打开 Word 文档，其中也没有 Excel 电子表格的痕迹。

在会议上讲述这个主题时，我也会介绍这个工具的演示。通常我利用从一个公司用户试图偷走财政预算电子表格或者公司重要投标合同信息的角度来演示。用户所要做的就是合并 Excel 电子表格到 Word 文档（一些平常的文档，如信件），

然后将 Word 文档复制到闪存设备。如果有人在路上拦住用户并查看闪存设备的内容，他们所能看到的是 Word 文档。

在与司法官员讨论这一问题时，我会采用不同的方法。设想公司用户有一些不正当的图片想与他的同伴分享。他将图片复制到一个 Word 文档中，然后在文件服务器上找到一个他们都有权限访问的 Excel 表格，并且将两者合并。然后将 Word 文档改为电子表格的原始名字和扩展名，完成后通知他的同伴。这样，就可以不露声色地散播图片。

检测 MergeStreams 同类工具的使用不是特别困难。使用包括 oledmp.pl 功能的脚本，可以列出组成 Word 文档的 OLE 流。如果看到任何流名称表明 Excel 电子表格的存在（Workbook，Worksheet 或其他名称），这个 Word 文档绝对值得检测。

PDF 文档

便携文档格式（PDF）文件也包括作者、文件创建时间和创建文件的应用程序名称等元数据信息。通常情况下，元数据会显示文档是在 Mac 上创建或者是从 Word 文档转换而来等，这些暴露的元数据信息也是一种风险。对调查者来说，则是一种有用的信息来源，可以显示某种特定应用安装在用户系统中并被使用。

随书附赠的光盘中包括两个 Perl 脚本（pdfmeta.pl 和 pdfdmp.pl），使用这两个工具可以提取 PDF 文件的元数据。两个脚本的惟一差别是使用不同的 Perl 模块与 PDF 文档交互。实际上，使用这两个脚本可以在不同的情况下成功地提取数据。一些情况下，两个脚本都可以用，而另外一些情况下则是其中一个会失败。作为测试，使用 Google 搜索样本 PDF 文件并找到了两个。一个来自联邦交易委员会（FTC），另外一个来自 IRS。FTC 的 PDF 文件名为 idtheft.pdf，pdfmeta.pl 返回如下信息：

```
C:\Perl>pdfmeta.pl d:\pdf\idtheft.pdf
Author          FTC
CreationDate    D:20050513135557Z
Creator         Adobe Indesign CS (3.0)
Keywords        identity theft, id theft, idtheft, credit
ModDate         D:20050513151619-04'00'
Producer        Adobe PDF Library 6.0
Subject         Identity Theft
Title           Take Charge:Fighting Back Against Identity Theft
```

IRS 网站下载的 PDF 文件是 2006 W-4 的表格，名为 fw4.pdf，pdfmeta.pl 返回如下信息：

```
C:\Perl>pdfmeta.pl d:\pdf\fw4.pdf
Author          SE:W:CAR:MP
CreationDate    D:20051208083254-05'00'
Creator         OneForm Designer Plus
Keywords        Fillable
```

ModDate	D:20060721144654-0400'
Producer	APJavaScript 2. 2. 1 Windows SPDF_1112 Oct 3 2005
Subject	Employee's Withholding Allowance Certificate
Title	2006 Form W-4

这两个文件都很普通，但是可以简单地显示 PDF 文件元数据信息在电子调查中的应用，至少可以作为关键词搜索的来源。如果不能使用这两个脚本提取元数据，则可以使用最原始的方式，用 Adobe Reader 打开文档，从菜单上单击“File（文件）| Document Properties（文档属性）”，“文档属性”对话框中 Description（描述）选项卡包括了元数据信息。图 5. 9 是 idtheft. pdf 文件的文档属性内容。

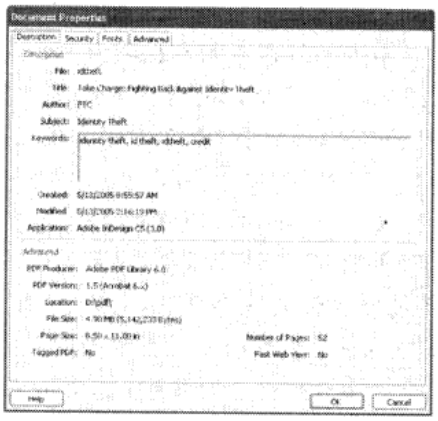


图 5. 9 Idtheft. pdf 文档属性

图像文件

不仅仅是字处理文件包含内部元数据。2006 年 2 月，华盛顿邮报杂志中的一篇关于僵尸操控者（使用蠕虫感染机器并管理僵尸网络的人，也包括租用僵尸网络的人）的在线文章包含了一个 JPEG 格式的图像。虽然文章作者尽力不暴露僵死操控者的隐私，但 JPEG 图片中的信息却显示了图片拍照的地点（城市和州）。

JPEG 图像中的元数据信息依赖于创建或修改图像文件的应用。例如，数码相机拍摄的照片嵌入了图像文件交换格式（EXIF）信息，其中包括相机的厂商和型号（可惜的是没有保存相机序列号）、缩略图和音频信息（EXIF 使用 TIFF 图像文件目录格式）。Adobe Photoshop 应用也添加了自己的元数据信息。

Exifer[34]，IrfanView[35]和 Image::MetaData::JPEG[36]Perl 模块等工具可以查看、提取，甚至修改 JPEG 图像文件中嵌入的元数据。ProDsicover 也可以显示 JPEG 图像文件的 EXIF 数据信息，Chris Brown（来自 Technology Pathways 公司）提供了一篇白皮书[37]解释 EXIF 数据，其中也包括一部分 JPEG 文件的格式信息。

文件特征分析

调查中有时会发现异常扩展名文件，或者是常见的扩展名文件出现在不寻常的位置。这种情况下，可以使用文件特征分析技术来确定文件实际用途，进而对攻击者的技

术能力做出评估。通过文件特征分析技术可以忽略文件的扩展名，分析文件的实际用途。

文件特征分析提取文件的前 20B，并且查找签名特征或者“幻数”来确定文件类型和功能。不同类型的文件具有不同的特征，这些特征不受文件扩展名的影响。事实上，坏人会修改文件的扩展名，使得 Windows 资源管理器不能正常显示文件。修改了扩展名的文件会在资源管理器中显示不同的图标，从而达到隐藏文件内容及其用途的目的。很久以前，在分析一个 russiantopz[38]僵尸程序的时候，发现这个 IRC 僵尸程序在被感染的机器上保存了若干个文件，并且文件扩展名都是 .drv 和 .dll。系统管理员看这些文件的时候，一般不会打开这样的文件。毕竟，大部分情况下，管理员使用十六进制查看程序打开这种扩展名的文件看到的都是一系列二进制数据。分析中打开了这些文件，发现它们包含的都是文本，也就是程序的配置和接收到命令时的行为定义信息。

类似 ProDiscover 的取证分析工具允许调查员方便地执行文件特征分析并查看结果。进行分析的时候，工具提取文件的扩展名并和文件中的特征进行比较。例如，Windows 便携执行文件（PE）使用 MZ（是 Microsoft 的架构师 Mark Zbikowski[39]名字的头字母）两个字母作为文件开头。在 ProDsicover 的文件扩展名和特征数据库 headersig. txt[40]中，可执行文件的扩展名可以是 .exe，.dll，.sys，.ocx，.drv 等。简而言之，如果文件的扩展名是可执行文件类型的，那么应该具有有效的可执行文件特征。如果文件不具有该类扩展名的有效特征，就需要进一步调查了。

类似 JPEG、GIF 等图像文件也有自己的特征。JPEG 文件的特征是 JFIF，GIF 文件的特征是 GIF87a 或 GIF89a。图 5.10 是 PDF 文档的特征%PDF，后面是 PDF 文件的版本。

```
00000000h: 25 50 44 46 2D 31 2E 35 0D 25 E2 E3 CF D3 0D 0A ; %PDF-1.5.%âãÏÓ..
```

图 5.10　PDF 文件特征

光盘上的 sigs. pl 脚本可以对开机系统进行特征分析。该脚本对单个文件、目录或者目录树中的文件进行检测，确定文件特征是否正确。脚本使用 Technology Pathways 公司的 headersig. txt 文件作为默认的特征数据库，也可以使用同样格式的其他特征库。脚本比较文件特征，并且在发现特征库不存在的扩展名时也会报警。此时脚本会提取扩展名和文件特征，供调查人员更新特征库。通常情况下，脚本采用 CSV 格式输出到控制台，也可以重定向输出到文件。

NTFS 分支数据流

分支数据流（Alternate Data Stream，ADS）是 NTFS 文件系统的一个特性，系统管理员们对此特性却很陌生。毕竟，为啥要使用 ADS 呢？Microsoft 的一些应用使用了这个特性，那就说明这种情况是存在且合理的，对吗？

换个角度来说，可以在 Windows 系统中创建一个合法文件，该文件不仅包括数据，还包括脚本和可执行代码，这些文件可以被创建和执行，但是操作系统系统本身却没有提供工具来检测可执行二进制代码的存在。情况就是这样，Windows 操作系统可以创

建、修改和操纵 ADS，但是没有提供工具来查看这些 ADS。但也不完全是这样，从 Vista 开始，dir 命令有一个选项可以显示 ADS。这部分内容随后讨论。

那么到底什么是分支数据流（ADS），它们是怎么来的，如何使用呢？从 Windows NT 3.1 开始，NTFS 文件系统提供了 ADS 特性，用来支持 Macintosh 使用的 HFS 文件系统。HFS 文件系统采用资源分支来维护文件的元数据信息，如图标、菜单、对话框等。这个功能添加到 NTFS 文件系统中，却没有引起广泛的注意。事实上，在很长一段时间内，包括 Microsoft 都没有这方面的信息和讨论。虽然 Microsoft 的应用和命令行功能可以创建特定的 ADS，但很少在实际工作中使用。坏人在了解这一特性后，用它来隐藏工具，甚至部分 rootkit。这是一个很有效的方法，反病毒软件默认情况下不扫描 ADS，甚至没有检测 ADS 的功能。因而，隐藏在 ADS 中的恶意软件不会被反病毒程序发现。

背景注释

使用分支数据流（ADS）

20 世纪 90 年代末，我作为顾问参与一系列的渗透测试和漏洞评估。渗透测试过程中，如果我被授权并获得了 Windows 系统访问权限，就在系统中创建一个 ADS。这只会占用系统很少的空间，因为只留下了文本消息。不过，这是给系统管理员的标识，证明我们成功做到了。有些渗透测试人员可能会将所有工具复制到被攻入系统的 ADS 流中。

创建 ADS

创建 ADS 很容易，并且一些 Microsoft 的应用会自动创建。只要具有创建文件的权限，任何用户都可以创建。例如，创建 ADS 的一种最简单的方式如下：

```
D:\ads>notepad myfile.txt:ads.txt
```

开始会弹出一个对话框询问是否创建一个新文件。单击 Yes 按钮，然后在窗口中输入一些文字，保存文件，关闭 Notepad 窗口。此时，如果输入 dir 命令，则显示 myfile.txt 文件大小为零，并且没有显示在 Notepad 中输入了文字。

另外一个创建 ADS 的方式是使用 echo 命令：

```
D:\ads>echo "This is another ADS test file" > myfile.txt:ads2.txt
```

好了，现在创建了两个 ADS，不管是 dir 命令还是 Windows 资源管理器，都只能在目录中看到一个文件，而且文件的长度是零。

还可以使用 type 命令将另外一个文件复制到 ADS：

```
D:\ads>type c:\windows\system32\sol.exe>myfile.txt:ads3.exe
```

这会将 sol.exe（一个纸牌游戏）程序复制到 ADS 中。这些命令也可以在 Vista 中创建 ADS，也许 Vista 中这个纸牌游戏程序的路径会不同。

使用如下命令还可将 ADS 添加到目录列表中：

```
D:\ads>echo "This is an ADS attached to a directory" >:ads.txt
```

注意命令中不包含文件名，这可以将 ADS 附加到目录列表文件，这里是 D:\ads。

ADS 还会在用户不注意的时候创建。右击一个文件，选择“属性”（Properties）选项，对话框中有一个选项卡为“摘要”（Summary）（奇怪的是，Vista 没有这个选项卡）。在文字输入框中可以输入任何字符，单击“确定”按钮保存后，输入的信息会保存到 ADS 中（Office 文档例外，它会将输入的信息保存到文件中）。

此外，Windows SP 2，IE 和 Outlook 会为下载的文件和附件添加一个分支数据流（ADS）。使用 IE 下载文件时，在保存文件的同时，会保存一个名为 Zone. Identifier 的 ADS。此数据流被附加到文件，下次用户执行或者打开文件时，系统会弹出一个警告对话框，提示文件可能不安全。

查看 ADS

现在已经创建了几个 ADS，那么怎样检测呢？前面已经说过，Windows 没有提供本地工具用以显示 ADS。不能使用 Windows 资源管理器，dir 命令同样没用。当然，Vista 中的 dir 命令可以使用/r 选项来显示 ADS，如图 5.11 所示。

```
C:\ads>dir /r
 Volume in drive C has no label.
 Volume Serial Number is 98A5-80D5

 Directory of C:\ads

11/20/2006  07:17 PM    <DIR>          .
11/20/2006  07:17 PM    <DIR>          ..
11/20/2006  07:33 PM                 0 myfile.txt
                                    23 myfile.txt:ads.txt:$DATA
                                    34 myfile.txt:ads2.txt:$DATA
                               982,528 myfile.txt:ads3.exe:$DATA
               1 File(s)              0 bytes
               2 Dir(s)  14,823,571,456 bytes free
```

图 5.11　Vista 中显示 ADS 信息的例子

图 5.11 中的 Vista 的 dir /r 命令显示了上节创建的几个 ADS 数据流。其他 Windows 操作系统（2000，XP 和 2003）则需要另外的工具协助。本人喜欢使用 Frank Heyne 编写的 lads. exe 工具。Lads. exe 是命令行工具，运行结果如下：

```
D:\tools>lads d:\ads
LADS -Freeware version 4.00

(C) Copyright 1998-2004 Frank Heyne Software (http://www.heysoft.de) This program
lists files with alternate data streams (ADS) Use LADS on your own risk!
Scanning directory d:\ads\
size ADS in file
      size  ADS in file
---------------  ------------------------------
         0  d:\ads\myfile.txt:ads.txt
        34  d:\ads\myfile.txt:ads2.txt
   1032192  d:\ads\myfile.txt:ads3.exe
```

```
1032226 bytes in 3 ADS listed
```

Lads. exe[41]只是可以显示 ADS 信息的工具之一。还有其他命令行和图形用户界面工具可以显示，一些工具可以实现为系统命令插件，在资源管理器窗口显示 ADS。

上节中提到文件摘要信息会添加 ADS 数据，这与实验中添加的 ADS 有所不同。如果为 myfile. txt 添加了摘要信息，lads. exe 显示信息如下：

```
·····size··ADS·in·file¶
----------·--------------------------------¶
·······120··d:\ads\myfile.txt:♣DocumentSummaryInformation¶
·······232··d:\ads\myfile.txt:♣SummaryInformation¶
·········0··d:\ads\myfile.txt:ads.txt¶
·····34··d:\ads\myfile.txt:ads2.txt¶
···1032192··d:\ads\myfile.txt:ads3.exe¶
·········0··d:\ads\myfile.txt:{4c8cc155-6c1e-11d1-8e41-00c04fb9386d}¶
```

从 lads. exe 的输出可以看到，myfile. txt 多了 3 个 ADS 数据流：一个类似于 GUID（大小为零），另外两个的名称使用梅花符号开头。后面两个保存了在“属性 | 摘要”选项卡中输入的信息。有时会看到 ADS 流的名称为 AFP _AfpInfo 或者 AFT _Resource，此时需要检查系统是否启动了 Macintosh 文件服务。如果服务启动，该数据流是使用 AppleTalk 协议从苹果系统中未命名流复制而来。这种情况下 HFS 文件系统中的数据分支会复制到文件中，如 myfile. txt。资源分支则保存到 myfile：AFP _Resource 数据流，而属性信息保存到 myfile. txt：AFP _AftInfo。

前面提到过，还有其他工具可以显示 ADS 信息。Streams. exe（来自 SysInternals. com），lns. exe（来自 NTSecurity. nu 的 Arne Vidstrom）和 sfind. exe（包含在 Found-Stone. com 取证工具箱中）都是与 lads. exe 类似的命令行工具。ADSDector 是由 CodeProject. com 提供的系统（也就是资源管理器）插件，通过它可以在图形化界面中实时看到非加密文件的分支数据流。最后，CrucialADS（来自 CrucialSecurity. com）和 ADSSpy（来自 SpyWareInfo. com）是可以查看 ADS 信息的图形用户界面工具。如图 5. 12 所示，ADSSpy 可以用来删除选中的 ADS。

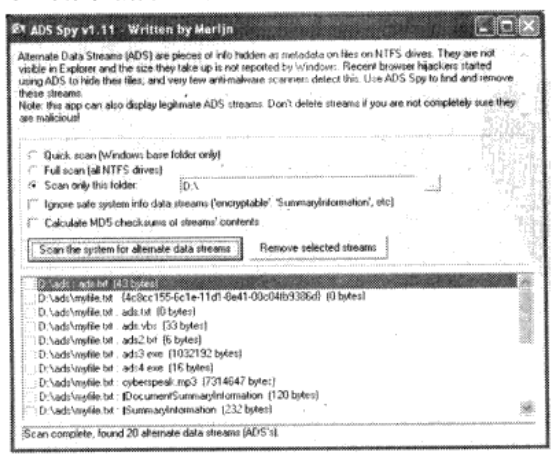


图 5. 12　ADSSpy 图形用户界面

定位 ADS 之后，可以使用 Notepad 或 cat 命令打开并查看文件内容，cat 是 SourceForge. net 上 UnxUtils 工具包的一部分。cat 可以将 ADS 内容输出到控制台或者重定向到独立文件。

警 告

2000 年，来自 29A（十六进制表示为 666）病毒小组的 Benny 和 Ratter 发布了一款使用 ADS 的病毒，名为 W2K. Stream。该病毒会感染文件，替换文件内容，并且将原先的文件内容复制到 ADS。如果病毒感染了 Notepad. exe，则会替换该文件的内容，原始的 Notepad 执行程序会复制到 Notepad. exe：STR。这只能在 NTFS 文件系统起作用。如果是 FAT 文件系统，由于没有 ADS 功能，那么就只留下了被感染的文件。

2006 年 6 月，反病毒公司 F-Secure 的 Blog 上有一篇文章描述 Mailbot. AZ（也就是 Rustock. A），这是一个内核级 rootkit 驱动（rootkit 相关内容参见第 7 章），该病毒使用 ADS 来隐藏自身，很难检测到，而且使用常用的 ADS 查看工具并不能看到隐藏的 ADS，因为它被 rootkit 所隐藏。很狡猾！

使用 ADS

那么，除了隐藏数据外，ADS 还能做什么？ADS 还有几项用途。例如，可以将可执行文件放到 ADS 中，然后执行。使用前面提到的 type 命令，如下所示：

```
D:\ads>type c:\windows\system32\sol. exe > myfile. txt:ads4. exe
```

通过上面的命令，Solitaire 游戏被放入 ADS。这是一个很好的例子，因为程序启动后会看到一个漂亮的图形用户界面。输入如下命令执行程序：

```
D:\ads>start . \myfile. txt:ads4. exe
```

可以看到 Solitaire 游戏的界面出现了。这不仅对可执行程序有效，因为脚本（Windows 脚本、Perl 等）也可以隐藏到 ADS 中并很容易启动。Windows 脚本工具（cscript. exe，wscript. exe）可以运行 ADS 中的脚本，Perl 也可以，甚至 IIS Web 服务器都可以运行隐藏在 ADS 中的脚本（如果用在“夺标”游戏中会很棒）。

如要在 Vista 中运行 ADS，则是另外一种情况，见图 5. 13。

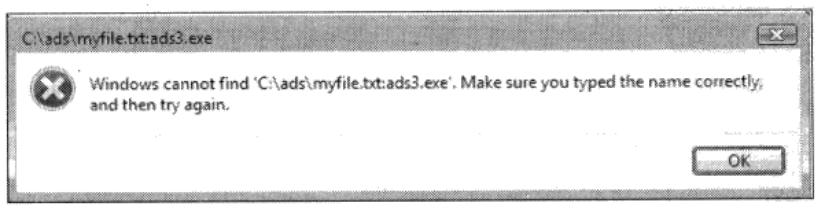


图 5. 13　在 Vista 中尝试执行 ADS 返回的对话框

尝试执行 ADS（例如，myfile. txt：ads3. exe 是 Vista 版本的 Solitaire）总是不能

成功，包括使用 Start 方式启动或者 Start | Run 方式启动都一样。但是，在 Vista 系统上启动 ADS 中的 WSH 脚本还是可以的。

ADS 的另外一个用途是隐藏媒体文件。电影和音频可以隐藏到 ADS 中，然后使用命令行方式启动 Widows 媒体播放器打开多媒体文件：

```
wmplayer d:\ads\myfile.txt:cyberspeak.mp3
```

例如，可以将 CyberSpeak 播客音频文件的内容放在 ADS 中，有趣的是，虽然 ADS 中的音频是从命令行启动的，文件名却出现在注册表键中：

```
HKEY_CURRENT_USER\Software\Microsoft\MediaPlayer\Player\RecentFileList
```

该项内容出现在注册表 File0 值的数据中，意味着任何时候一个添加到列表的新文件都会出现在列表的开头，列表中旧的文件名顺次序后退。文件数越小，该文件越先打开。从第 4 章可以看到，通过注册标键的最后写入时间（LastWriteTime）可以获知 Windows 媒体播放器何时访问该文件。

警 告

通过查看 ProDiscover 的样例案件，发现回收站中有几个 ADS。ProDsicover 使用红色字体突出显示 ADS。首先删除了几个工作文档，其中有一个文档是从因特网下载的。可以发现文件的 ZoneIndentifier ADS 数据还在，但是 INFO 2 文件的删除文件计数并不能反映 ADS 的存在。

删除 ADS

上面讨论了 ADS 的创建和使用，那么怎样删除它呢？有几种方式能达到这个目的，根据使用目的和选项的不同可以自由选择。

一种方式是直接删除 ADS 附着的文件。不过，显而易见的结果是如果该文件很重要，数据就丢失了。

如果要保存文件中的数据，可以使用 type 命令将数据内容复制到文件中，然后删除文件。另外一个选项是将文件复制到非 NTFS 文件系统介质。记住，ADS 是 NTFS 的特性，所以可将文件复制到软盘、闪存、其他 FAT、FAT 32 分区或其他分区类型（将文件 FTP 上传到 Linux 系统的 ext 2 分区），然后再复制回来。这样就可以有效地删除 ADS。

如果 ADS 附着于目录列表，如 C:\或者 C:\Windows \ System32，怎么办呢？不能删除目录，也不能复制到其他文件系统再复制回来。此时使用 echo 命令可以将 ADS 修改成无害的文本文件。用以前的 Solitaire 游戏的例子来演示，lads.exe 显示的信息如下：

```
56832 d:\ads\myfile.txt:ads4.exe
```

发现有一个 56 832 字节大小的 ADS，并且知道这是一个可执行文件。输入如下命令：

```
D:\ads>echo "deleted ADS" > myfile.txt:ads4.exe
```

再次运行 lads.exe，文件容量变了：

```
16 d:\ads\myfile.txt:ads4.exe
```

这样就有效“处理”了 ADS，虽然并没有删除，但是已经无害了。甚至可以在其中写入原先 ADS 的属性、处理人姓名和删除时间等信息。

最后，还有一种方法是使用前面提到的 ADSSpy 图形用户界面程序。

ADS 小结

上面讨论了 ADS 的很多内容，如何创建、使用和删除等。在应急响应和取证过程中记住这些信息很重要。ADS 是一种非正常状态，类似 ProDiscover 的商业取证分析工具会使用红色来突出显示。不过并非所有的 ADS 都是恶意的，也可以看到一些正常的应用在使用 ADS。

调查人员要记得查看 ADS 的内容。因为已知合法的 ADS 应用并不意味着 ADS 不会包含恶意内容。在看到名字为 AFP _ AfpInfo 的数据流时也不要简单忽略。坏人喜欢将恶意软件命令为普通名称，从而使管理员和取证分析人员麻痹大意。

其他分析方法

后期计算机取证分析过程中（已经获取了镜像），有时会碰到直接采用镜像分析很棘手的情况。例如，在需要对系统进行扫描，以识别木马、后门和间谍软件的时候。镜像只是一个文件（通常情况下，也可能是多个文件组成对原始硬盘的镜像），而现在需要对镜像中的文件进行扫描。此时，除了可以将镜像中所有的文件复制出来，还有几个工具可以将镜像转换为适合扫描的格式。

一个工具是通过 ProDiscover。从 4.85 版本开始，ProDsicover 可以将 ProDiscover 原始格式的镜像或者 DD 格式的镜像转换为 ISO 格式。ProDiscover 也可以创建 Vmware 从 DD 镜像启动所需要的文件。这个新的功能选项如图 5.14 所示。

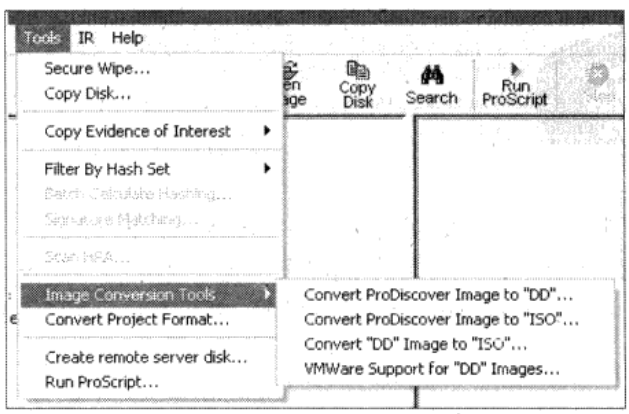


图 5.14 ProDiscover 新工具菜单

从图 5.14 可以看到，使用 ProDiscover 能够将本地 .eve 格式镜像转换为 DD 格式，或者将 ProDiscover，DD 格式转换 ISO 9660 Joliet 格式的镜像。还可以使用 ProDsicover 创建 Vmware 需要的文件，以便从 DD 镜像启动，这类似 Vmware P2V（意思是从物理到虚拟）工具的功能。使用这样的工具，可以启动系统进行更多的分析，如病毒扫描、恶意软件扫描，或者仅仅是查看系统运行时的“样子”。有时这种功能对调查很重要，因为在后期分析中确定运行系统的状况非常难（仅通过系统配置、安装的软件等信息）。

在 Technology Pathways 站点的资源中心[42]，有一个 Web 技术讲座文件，演示了如何使用 ProDiscoer 工具来尽可能在 Vmware 中启动镜像。这个讲座文件需要 WebEx. com 的商业客户端软件打开。

另外一个在 Vmware 中启动获取镜像的免费工具是 LiveView[43]，该工具由 CERT 开发提供。LiveView 使用简单易懂的用户界面（如图 5.15 所示）来定义启动镜像的配置选项，自动生成在 Vmware 中启动所需要的文件。

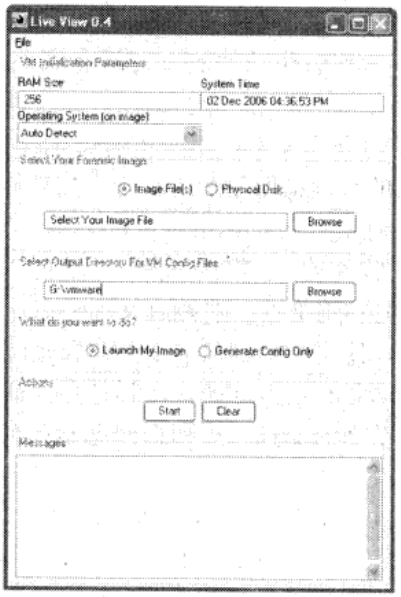


图 5.15 LiveView 图形用户界面

运行 LiveView 是一个简单直观的过程，LiveView 支持大部分版本的 Windows 和一些 Linux 系统。实际案件中，我曾数次使用它成功启动并进入获取的镜像系统。

注 释

在获取启动的镜像之后，可以做很多事情。如果使用的是 Vmware 工作站程序，可以配置桥接的网络，激活网络界面就可以对启动系统进行远程端口/漏洞扫描。也可以登录到系统中，如果口令不为空，就需要通过用户单位或其他各种方式获得系统登录口令。

另一个出色（但不免费）的工具是 Mount Image Pro（MIP）[44]。MIP 可以将镜像文件加载为只读的驱动器。图 5.16 显示的是将两个镜像加载为 F:\和 H:\，使用的是 MIP 30 天评估版本。

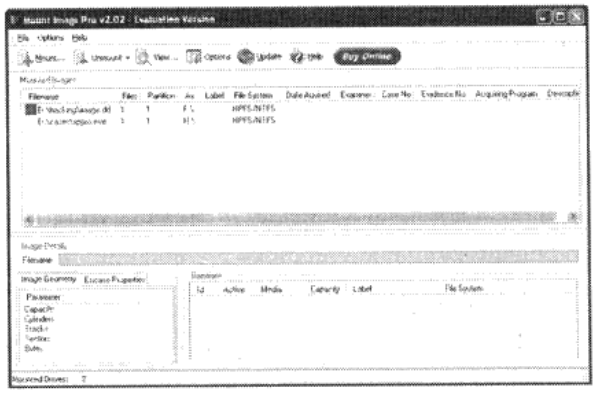


图 5.16　使用 MIP 版本 2.02 加载镜像为驱动器

通过 MIP 工具可以在不启动镜像的情况下访问镜像中的文件，此时不能获取运行进程、物理内存等信息。不过将镜像加载为驱动器后，可以如访问本地介质一样访问镜像中的文件。MIP 使用只读的方式加载，不会更改镜像中的任何数据。为了验证，使用 md5deep. exe[45]计算获取的系统镜像的加密哈希。然后使用 MIP 加载镜像为驱动器，从加载的驱动器上访问并复制几个文件到系统中的另外一个分区，包括运行几个 Perl 脚本分析其中的文件。然后卸载加载的镜像，彻底关闭 MIP 程序。再次运行 md5deep. exe 计算镜像文件的加密哈希，哈希值保持不变。证明镜像是采用只读方式加载的。读者可以自己做实验，使用已知的算法来创建和验证镜像的加密哈希值。

将镜像加载为只读的驱动器有很多好处，尤其是在数据推理和分析领域。例如，可以使用自动化的方式运行一系列的工具，如病毒和恶意软件检测工具、文件特征分析、列举 NTFS 分支数据流（ADS）等。这要比从镜像中复制文件进行详细分析要方便的多。这些检测和分析工具还以使用 Perl 脚本自动化处理。

分析工作中一个繁重的任务是查找系统中的恶意程序。有一次，用户报告系统中有可疑事件，我将系统做成镜像来分析，花费了不少时间才找到了产生这些事件的恶意程序。但是使用 MIP 加载，不仅会使查找恶意程序的任务更加快速，还可以同时自动扫描数个镜像。在这个案件中，在获取镜像两年之前系统就被感染了，如果使用 MIP 进行分析会很有帮助。

通过 MIP 可以像访问系统中普通文件一样访问镜像中的文件。因而，类似 Perl 的脚本语言可以打开文件、目录的句柄，使得还原点、Prefetch 目录分析更加直接。在开机取证中，这些功能的分析脚本只要简单修改、指向正确的位置就可以执行。

小结

大家都知道或者说过，没有两次调查是相同的。每次的调查都会与以前不同。不过，调查中的一些基本概念是相同的，知道到哪里查找证据就是很重要的一个方面。调

查员会被外界压力、时间所迫，知道在哪里寻找信息或者行为的证据要超过取证分析工具所能给予的，非常有用。很多调查员在时间和资源有限的时候，只能进行关键词和特定文件的搜索。如果根据问题所在知道到哪里查找信息，就可能有很多信息。在已知的特定文件之外（非法图片、恶意程序），有很多文档格式未知的文件可供分析，从而得知系统中何时、发生了什么。

对取证分析人员来说，基于操作系统和应用对用户行为的反应，从而了解到哪里查找信息及证据会在什么地方都很重要。知道日志文件会在哪里及其格式，可以为调查提供重要的线索——同样，如果这些痕迹很明显，则会是更好的线索。

缺少不同文件格式的清晰文档（或者一些文件不存在）是对取证调查的挑战。克服这一挑战的关键是全面地记录、调查文件格式并共享信息。这不仅包括当前调查的操作系统文件和文件格式，也包括新版本的 Vista 系统。

参考资料

1. http://msdn2.microsoft.com/en-us/library/aa380129.aspx
2. http://support.microsoft.com/kb/299475/
3. http://support.microsoft.com/kb/301677/EN-US/
4. http://blogs.msdn.com/ericfitz/default.aspx
5. www.microsoft.com/technet/support/ee/ee_advanced.aspx
6. http://support.microsoft.com/kb/329463/en-us
7. http://pyfiag.sourceforge.net/
8. http://128.175.24.251/forensics/repaireventlogfile.htm
9. http://computer.forensikblog.de/en/2007/02/a_common_misconception.html
10. http://www.ranum.com
11. http://support.microsoft.com/? id=318380
12. http://xforce.iss.net/xforce/xfdb/3682
13. www.cert.org/advisories/CA-2001-26.html
14. www.stevengould.org/software/indexdatspy/screenshots.html
15. www.systenance.com/indexdat.php
16. http://support.microsoft.com/kb/283673
17. http://support.microsoft.com/kb/308127
18. http://support.microsoft.com/kb/308538/
19. http://support.microsoft.com/kb/156280/
20. http://support.microsoft.com/kb/315271/
21. http://support.microsoft.com/kb/136517
22. www.datalifter.com/tutorial/bt/NProtect_Using_NProtect.htm
23. www.ediscovery.co.nz/wip/srp.html
24. http://support.microsoft.com/kb/301224
25. www.mitec.cz/wfa.htm

26. www. i2s-lab. com/Papers/ The_Windows_Shortcut_File_Format. pdf
27. forensicfocus. com/link-file-evidentiary-value
28. http://jafat. sourceforge. net/files. html
29. http://support. microsoft. com/? kbid=299648
30. www. metasploit. org/projects/antiforensics/
31. www. computerbytesman. com/privacy/blair. htm
32. http://support. microsoft. com/kb/834427
33. www. ntkernel. com/w&p. php? id=23
34. www. friedemann-schmidt. com/software/exifer/
35. www. irfanview. com/
36. http://search. cpan. org/~bettelli/Image-MetaDataJPEG-0. 15/
37. http://toorcon. techpathways. com/cs/ forums/storage/8/11/EXIF. pdf
38. www. securityfocus. com/infocus/1618
39. http://en. wikipedia. org/wiki/Mark_Zbikowski
40. www. techpathways. com/uploads/headersig. txt. 41. www. heysoft. de/Frames/f_sw _la_en. htm
42. www. techpathways. com/DesktopDefault. aspx? tabindex=8&tabid=14
43. http://liveview. sourceforge. net/
44. www. mountimage. com/
45. http://md5deep. sourceforge. net/
46. http://blogs. technet. com/robert_hensing/default. aspx
47. www. mandiant. com/webhistorian. htm
48. http://support. microsoft. com/? id=235162.

快速解决方案

事件日志

- √ 传统计算机取证分析主要围绕文件是否存在及文件碎片的分析。Windows 系统维护了一系列的文件，可以为传统方法提供更细级别的分析。
- √ Windows 系统中维护的大部分日志文件都包括时间戳信息，这可以融入对系统行为的时间分析中。

文件元数据

- √ “元数据”的字面意思是关于数据的数据。与文件中的内容信息不同，元数据提供了关于文件的更多信息。
- √ 很多应用在文件中维护关于文件或文档的元数据。

其他分析方法

- √ 在传统的文件取证分析方式之外，调查员还以采取更多的方式进行分析。

- √ 在虚拟的环境中启动获取的镜像为调查员分析系统、演示获取的数据（如给陪审团演示）提供了实用的方法。
- √ 用只读方式访问镜像文件系统，使扫描病毒、木马和其他恶意程序更加快速。

常见问题

下面的这些问答用来测试读者是否已经完全理解本章的内容，并可以帮助读者在实际案件调查过程中应用本章知识。如果还有其他问题，浏览：http://www.syngress.com/solutions 并单击 Ask the Author 一栏。

问：我在镜像中搜索因特网访问行为，发现“默认用户”有浏览历史。这是为什么？

答：虽然本章并不讨论因特网上网历史（这个问题已经通过其他方式彻底解决了），但这是我收到的问题，事实上，我在调查中也发现了。Robert Hensing（Microsoft 的员工）在 Blog 中[46]解决了这一问题。简而言之，默认用户默认情况下是没有因特网临时文件或上网历史记录的。如果账户中发现了上网历史，这表示有人用 SYSTEM 级别的用户调用了 WinInet API 函数。我曾在案件中发现攻击者获取了 SYSTEM 级别的权限，运行 wget.exe 下载工具到被入侵的系统。因为 wget.exe 使用 WinINet API，所以在默认用户因特网临时文件目录中的“上网历史”中留下了证据。Robert 使用了一个很好的例子来演示这一情况，他使用调度任务来启动因特网浏览器（IE），这样就具有了 SYSTEM 级别的权限。上网历史信息进入默认用户目录。对上网历史/index.dat 文件的分析可以使用 ProDiscover 的因特网历史查看器，或者 Mandiant.com 的 WebHistorian[47]。

问：我获取了硬盘的镜像，经过初步检查，系统中没有多少数据。我知道系统的使用者已经在单位好几年了，最近发现有点可疑。注册表中的安装时间大概是一个月以前，从分析方面来看我该采取哪些方法？

答：在分析系统的过程中这个问题会经常出现，调查员相信就在镜像获取之前可疑对象重装了操作系统。这可能使调查终止，但是在结束之前，调查员可以看几个地方，收集关于这一问题的更多信息。安装过程中操作系统在下列注册表键中写入 InstallDate 值的时间戳：

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion
```

InstallData 值的数据是 DWORD 类型，表示 1970 年 1 月 1 日 0 时 0 分 0 秒之后过去的秒数。Microsoft 知识库文章 235 162[48]显示这个值可能会被不正确记录。调查员还可以查看 setuplog.txt 文件中记录的时间戳信息、Service 注册表键的最后写入时间等，以便确认该时间信息。同时也要检查 SAM 文件中用户账户注册表键的最后写入时间。其他地方还包括 Windows XP 系统的 Prefetch 目录和用户目录 MAC 时间戳等。

问：我听说有反取证技术，用户采用特殊的工具和步骤来隐藏证据。针对这点我该怎么办？

答：取证分析人员永远不要由于一个理论或者一些发现而止步不前。相反，只要有可能，就要为发现找到支持的证据信息。很多情况下，尝试隐藏数据会留下隐藏行为的痕迹，对操作系统行为、不同情况和事件可能产生什么痕迹的理解，可以使调查员看到行为的迹象，而且要记住缺少痕迹本身也是一种痕迹。

Chapter 6

第 6 章
可执行文件分析

本章主要内容：

- ■ 静态分析
- ■ 动态分析

- √ 小结
- √ 快速解决方案
- √ 常见问题

引言

案件调查过程中，可能会发现一些可疑文件，并且需要分析这些文件以便了解该文件的功能。一般的情况下，入侵者可能会在被侵入的计算机上遗留下一些脚本或配置文件，这些文本文件可以直接打开查看。如果是脚本，完全理解该文件的功能可能需要一些编程知识。

第 5 章讨论了文件特征分析方法——根据文件内容来判断文件类型与其扩展名是否一致。修改文件名（包括扩展名）不过是黑客用来在被入侵计算机上隐藏文件实际用途、干扰用户视线最简单的方法之一。攻击者假设（往往是正确的）管理员在看到一个扩展名为 .dll 的文件时，是不会有兴趣做进一步分析的。

本章讨论可执行文件的分析方法，包括获取可执行文件信息、分析其用途的各种工具和技巧。本章内容不是特别针对恶意软件分析的，而是可执行文件分析的通用方法。在讨论分析技术时，不会涉及反汇编技术详细内容和 IDAPro[1] 之类工具的具体使用方法。因为逆向工程本身就是一个很大的课题，足够另外再写一本书了。这里仅仅讨论适用于大部分系统管理员和取证调查人员的可执行文件分析技巧。

静态分析

所谓“静态分析”就是在不运行被分析的可执行文件的前提下，对可执行文件进行分析，收集其中所包含信息的方法的统称。如果用文本编辑器（我在给客户演示时做过很多次）或者十六进制编辑器打开一个可执行文件，可以看到的仅仅是一堆二进制乱码！在这堆乱码中或许会有一两个认识的单词，但是没有上下文，文件的主体还是乱码。调查员一定要记住：可执行文件也有自己的格式，符合一定的规律，完全可以从 Windows 可执行文件中找到其信息。只要理解其规律，就可以分析其内容并从中提取出有用的信息。

在深入分析可执行文件之前，我们还要做一些准备工作。

记录文件信息

分析之前要做的第一件事就是要为这个文件建档，记录文件相关信息。一个普遍的现象是技术人员讨厌写文档（这都快成都市传奇了）。与其他传奇一样，这是事实，至少是部分事实。在事件处理过程中，我无数次碰到这样的场景：现场人员告诉我：“我们找到了一个文件。”“很好，”我接着问道“在哪里找到的?”于是，他一下子就变得目瞪口呆，顾左右而言它了。要知道，如果能知道文件所在的文件夹，再加上其他一些信息，就可以马上做出一些初步的推断了！所以，要做的第一件事就是记录文件所在的系统、文件的全路径及谁、在什么时候发现的等。

警 告

很多调查人员经常会忽视这点：在计算机系统中（不仅是 Windows 系统），文件名（包括扩展名）事实上是可以随意取的。跟踪邮件列表一段时间，就会看到这样的邮件“我在系统中发现这么一个文件，通过 Google 搜索发现这不过是个很正常的文件……”。基于文件名搜索文件能找到不少有趣且有用的信息，但这不能作为调查的最终信息。在一次事件中，IT 支持人员在被感染的机器上找到了几个文件，当他们 Google 其中一个文件名后，发现这个文件是 Microsoft 公司提供的合法文件，于是就放弃了对这个文件的进一步调查。但是，我用本章讨论的技术分析之后，确定这个文件正是要调查的恶意软件！

在记录文件的基本信息之外，还要记录可疑文件是如何找到的。例如，在对开机系统进行取证分析时，使用第 1 章介绍的一些技巧发现有关可疑文件。静态分析时也一样，例如，记录如何使用 ProDiscover 或者其他取证分析工具在系统镜像中找到可疑文件。

此外，需要记录的信息还包括发现可疑文件所在的操作系统和版本信息。Windows 的不同版本之间和相同版本的不同补丁包之间都会有所不同。恶意软件在不同目标系统上的执行效果可能会大不一样。例如，“泰迪熊”病毒，它会向其他用户发送一封恶作剧邮件，声称 jdbgmgr. exe（这是 Java 调试管理器，之所以被命名为“泰迪熊”病毒，是因为这个文件的图标是一个泰迪熊）是恶意程序，并且要求用户立即删除这个文件。在 Windows NT 4. 0 中，要是用户上了当，该文件会被立刻删除。不过要是在 Windows 2000 中，Windows 文件保护机制（WFP）就会起作用，删除文件会被立刻恢复。此外，Windows 2000 和 XP 中的 Windows 文件保护机制所保护的文件也不是完全一样的。又如 Benny 和 Ratter 公布的 W32. Stream 概念型病毒，该病毒会使用 NTFS 的可选数据流功能（ADS，参见第 5 章）。如果被感染的系统分区为 FAT/FAT 32 文件系统，这个病毒的行为表现就会不同，因为 FAT 文件系统不支持可选数据流！

除了上面需要记录的要点之外，还必须要记录文件的一些其他的信息，例如，文件的 MAC 时间、文件系统中对它的引用（如用户开始菜单中的快捷方式）及注册表中涉及它的键。

警 告

调查人员在刚开始接触需要检查的计算机时要特别小心，特别是还在运行的计算机。在第 1 章中讨论了诺卡德交换原理，调查人员在系统中进行的任何操作都会在系统中留下痕迹。所以，在开机取证时，即使发现了一个可疑的文件，也要尽量避免在开机状态下对这个文件做过多的分析工作，并且所做的每个动作都要留下详细的记录。另外，也不要迷信底层文本搜索（ASCII/unicode）方法对注册表的搜索就不会起作用，因为注册表中数据有自己的二进制编码方式。

记录的信息越完整越好。对于每个调查人员来说，应该养成随时记录有关信息的好习惯，免得以后费时费力。进一步讲，这也是取证调查“最佳实践”的一部分。

最后还有一条至关重要的信息必须记录——有关文件的加密哈希（Hash）。哈希在计算机安全领域和取证领域主要用途是确保文件的完整性，即保证提取的证据文件不被故意或者意外地修改。最常用的一个哈希算法是 MD-5，它接收任意长度的数据流，生成 128b 的哈希输出（通常使用 32b 的十六进制数表示）。如果输入数据被改动了，哪怕只是被改了一个比特，也会得出不同的 MD-5 哈希值。虽然 MD-5 算法中的漏洞导致哈希冲突的可能性增加[2]，这个算法在计算机取证中仍然有用。另外一个应用广泛的哈希算法是 SHA-1[3]。例如，NIST 的 NSRL（National Software Reference Library）库就用 SHA-1 算法来构造参考数据集（RDS）光盘①。使用 NSRL 哈希数据库使调查员能方便地过滤正常的操作系统的文件，并且挑出已知的病毒或者木马文件。

另外，还有一种很有意思的算法——Jesse Kornblum 在 Ssdeep[4]（基于 Andrews Tridgell 博士的 SpamSum 工具）里实现的“上下文触发的分段哈希”[5]。Ssdeep 不会计算机整个文件的哈希，而是采用分段的方式，每次计算随机长度（如 4KB）的一段数据的哈希。这种方法不仅可以校验文件的完整性，而且可以比较两个文件的相似度。例如，使用 ssdeep 计算一个 Word 文档的哈希，对这个文档做一点修改（如加/删文字或者改变文章的格式等），然后重新计算这个文档的哈希，ssdeep. exe 可以显示两个文件的相似度为多少。这一技术也适用于其他类型的文件，如图片、视频和音频文件等。

文件的所有信息都已经记录之后，就可以开始分析文件内的信息了。

分析可执行文件

调查人员在静态分析开始时倾向于首先使用杀毒软件对可疑文件进行扫描。这是个好注意，但如果杀毒软件没有任何报告也不要惊讶。新的恶意代码在不断发布。事实上，根据一家反病毒公司 2007 年 1 月发布的报告，该公司的产品在 2006 年检测出 207 684 种不同的威胁，其中 41 536 种属于新的恶意代码。扫描可疑文件可以使我们对文件有所了解，不过在得出“未发现病毒”的结果时也不要过分自信。另外，使用多种杀毒引擎进行扫描可以提高识别的准确度。

接下来调查员会用 strings. exe[6] 工具提取可疑文件中包含特定长度的字符串（ASCII/unicode 的）。通过检查这些字符串，调查人员可以对可疑文件的用途做一个大概的判断。最新版本的 strings. exe 可以同时搜索文件中 ASCII 和 unicode 字符串，同时给出字符串在文件中的偏移位置。偏移位置可以显示字符串所处的文件段上下文信息（段信息将在本章的稍后部分介绍）。strings. exe 还可以用来从多个文件中找出含有指定字符串的文件，有关的示例教程可以在 strings. exe 的官方网站找到。

---

① NSRL，这是一个计算所有“已知的合法文件和所有已知的非法文件的哈希值后得到的一个数据库，这个库一般被刻录在被称为 RDS 的光盘中——译注。

注 释

几年前，我协助调查从一台内网服务器上获取的文件，这台服务器会不断向因特网发送数据包。经查这个文件是IE0 199[7]病毒，感染了该病毒的系统会向保加利亚电信网络发送大量数据包。我们从文件中发现的字符串构成一个“声明”，幸好小组里有个成员在军队里接受过俄语培训。后来病毒作者被指控非法访问保加利亚因特网服务器，并且发起对网络设施的拒绝服务攻击。

另一个提取文件中字符串的出色工具是BinText，该工具由FoundStone公司提供（现在属于McAfee公司）。BinText可以定位二进制文件中的ASCII，unicode及资源段中的字符串，并且连同字符串在文件中的偏移位置显示在图形用户界面。图6.1显示的是记事本程序（notepad.exe）中的几个字符串。

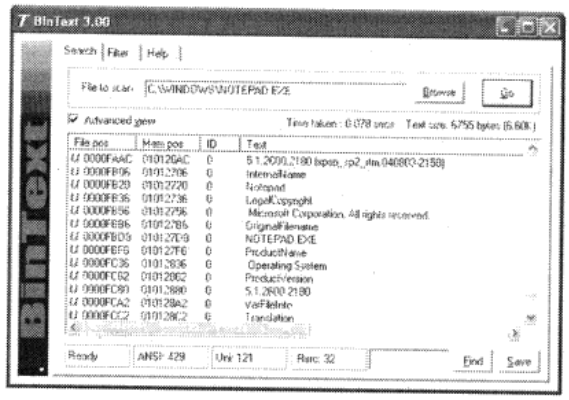


图6.1 使用BinText打开记事本（Notepad.exe）

文件中的字符串不可能完整揭示文件的用途，但是可以为调查员提供线索。字符串可能只有位置而没有上下文。由图6.1可以看出是unicode字符串（界面的左边是高亮的字母U），并且可以看出是文件版本信息的一部分（版本信息本章后面会详细讨论），但是其他字符串就可能没有这样的上下文可以分析。对文件中奇怪的惟一字符串，可以在其他文件或因特网上搜索。搜索的结果可以为下一步的分析（静态分析或是动态分析）提供线索（我曾在文件中发现过类似supercalifragilisticexpialidocious的字符串）。

网上有很多关于恶意软件逆向工程的网站，也有针对合法软件的逆向分析。奇怪的是，它们都用到一些类似的核心技术来获取可执行文件的信息，同时也使用一些非常类似的工具。下面介绍的pedump.exe和PEview.exe就是这样的两个工具。

2002年2月，Matt Pietrek发表了题为《Win32 PE可执行文件格式深入分析》[8]系列文章的第一部分。这些文章中，Matt不仅从多个角度深入探讨了PE文件格式，还提供了一个名为pedump.exe（可以在www.wheaty.net下载）的命令行工具，该工具可以从PE文件头中提取文件的详细信息[9]。提取出来的信息被发送到标准输出（STDOUT），所以可以直接从控制台读取相关信息，也可以重定向到文件中进一步分析。

另一个可用于分析 Windows PE 的工具是 Wayne Radburn 开发的 PEview. exe[10]。PEview. exe 是个图形界面的工具，通过它可以查看 PE 头部中包含的各种信息，而且是以漂亮的分层形式呈现出来的。在本书成文时，PEview. exe 的最高版本是 0. 96，这个版本不能保存有关 PE 文件的分析结果。

由于涉及版权问题，本书附送的光盘中没有提供上述两款软件，而且从官方网站下载可以保证得到最新版本工具。光盘中附带了一个用于访问 PE 文件结构 Perl 脚本。pedmp. pl 脚本使用 File::ReadPE 模块来访问 PE 文件头并从中提取出相关的信息。提供这个脚本的目的是为了提供一个参考，使读者进一步了解其他工具的工作原理。另外，由于 Perl 的平台无关性，实际上可以在任何一个操作系统上——Windows，Linux，甚至是 Mac OS X 操作系统——执行这段脚本，分析 PE 文件（这对调查人员来说是个好消息，在 Linux 或者 Mac OS X 平台上不会由于“不小心”运行了恶意软件，导致系统被感染）。

PE 头部

Microsoft 已经彻底公开了 PE 文件格式[11]（也包括 COFF 文件格式，COFF 用于 VAX/VMS 操作系统）。作为 ImageHlp[12] API 数据结构文档的一部分，Microsoft 也公开了 PE 头部中使用的大部分数据结构。通过上述文档，可以深入理解 PE 文件的格式，并且从中提取出调查取证中有用的信息。

.PE 文件可以分成几个部分（一般把这些部分称为“段”，但是因为这个术语在本章里有其他含义，所以这里尽量避免这个术语）。第一个，可能也是最重要的（即使不能算是最不重要的，也可以算是很复杂的）部分是 IMAGE _DOS _HEADER。这一部分的前两个字节是 PE 文件的文件签名 MZ（十六进制的 0x5A4D）。这两个字母是 Mark Zbikowski[13]（Microsoft 公司设计可执行文件格式的系统架构师）名字的缩写。不过，文件能够执行不是靠这两个字母或者 . exe 扩展名决定的。

整个 IMAGE _DOS _HEADER 结构共有 64B 组成。除了刚才讲的 PE 文件签名之外，这个结构的最后一个 *DWORD*（4B）也很重要，这个 *DWORD* 是 e _lfanew，在 ntimage. h 头文件中定义是指向真正的 PE 头部起始位置的指针——e _lfanew 中的值就是 IMAGE _NT _HEADERS 结构在 PE 文件中的偏移量。Windows 只有读取了 IMAGE _NT _HEADERS 中的信息，才能解析可执行文件中其余部分的含义，进而把可执行文件加载到内存中并执行。图 6. 2 是 PE 文件在十六进制编辑器中的显示。

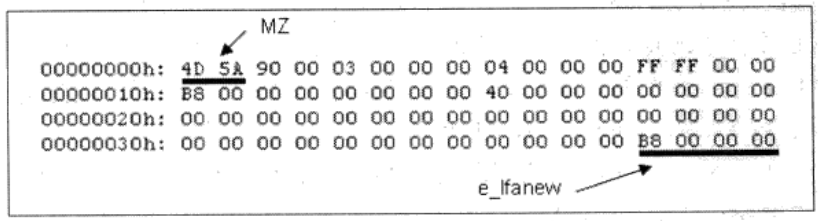


图 6. 2　使用十六进制编辑器查看 IMAGE _DOS _HEADER 结构

如图 6. 2 所示，IMAGE _NT _HEADERS 结构在文件中的偏移应该是 0xB8（也就是十进制的 184）。IMAGE _NT _HEADERS 包括一个签名和 IMAGE _FILE _HEAD-

ER 及 IMAGE _OPTIONAL _HEADER 两个结构。PE 签名很简单，就是字母 PE 再加上两个 0（是个 *DWORD*，共 4B，可以表示成 PE\00\00）。如图 6.3 所示。

```
000000b0h: 00 00 00 00 00 00 00 00 50 45 00 00 4C 01 03 00
000000c0h: C2 E1 C2 40 00 00 00 00 00 00 00 00 E0 00 0F 01
000000d0h: 0B 01 06 00 00 80 07 00 00 30 00 00 00 00 00 00
```

图 6.3　IMAGE _ NT _ HEADERS 签名

接下来是 IMAGE _FILE _HEADER[14]结构，紧接在 PE 签名之后，包含 20B。这个结构中有几个值对调查取证很有用处。在表 6.1 中列出了 IMAGE _FILE _HEADER 结构中各个成员的定义。

表 6.1　IMAGE _ FILE _ HEADER 结构

大小	名称	描述
2B	Machine	指定该可执行文件应该在哪类体系结构的计算机上运行
2B	Number of Sections	PE 文件中 Sections 的数量，实际上给出的就是 IMAGE _ SECTION _ HEADERS 结构的大小
4B	Time Date Stamp	可执行文件创建的时间和日期，这些时间和日期是以 UNIX 时间记录格式（即记录该时间离 1970 年 1 月 1 日有多少秒）记录的。基本上就是该程序的开发人员编译生成该可执行文件的时间
4B	Pointer to Symbol Table	符号表的偏移（如果没有 COFF 符号表，则为 0）
4B	Number of Symbols	符号表中符号的数量
2B	Size of Optional Header	IMAGE _ OPTIONAL _ HEADER 结构体的大小。通过这个成员确定目标系统是 32 位还是 64 位的
2B	Characteristics	记录了文件的多个属性

图 6.4 演示了使用 PEView 软件查看 PE 文件的 IMAGE _ FILE _ HEADER 结构的情况。

pFile	Data	Description	Value
000000BC	014C	Machine	IMAGE_FILE_MACHINE_I386
000000BE	0003	Number of Sections	
000000C0	40C2E1C2	Time Date Stamp	2004/06/06 Sun 09:20:02 UTC
000000C4	00000000	Pointer to Symbol Table	
000000C8	00000000	Number of Symbols	
000000CC	00E0	Size of Optional Header	
000000CE	010F	Characteristics	
		0001	IMAGE_FILE_RELOCS_STRIPPED
		0002	IMAGE_FILE_EXECUTABLE_IMAGE
		0004	IMAGE_FILE_LINE_NUMS_STRIPPED
		0008	IMAGE_FILE_LOCAL_SYMS_STRIPPED
		0100	IMAGE_FILE_32BIT_MACHINE

图 6.4　使用 PEView 查看 IMAGE _ FILE _ HEADER

对于调查取证来说，Time Date Stamp 显得特别有用，它是编译时链接器创建可执行文件的时间（当然也应该注意，该时间可以使用十六进制编译器修改，而且不会影响可执行文件的正常运行）。它通常指程序员编译生成该可执行文件时的系统时间，通过它可以大致推断这个程序大概是什么时候构建的。在对文件进行分析时，IMAGE _FILE _HEADER 中段的数量应该与 PE 文件中段的数目相吻合。同时，IMAGE _FILE _HEADER

结构中的特征（Characteristics）也提供了该文件的一些特性。例如，如果特征中设置了IMAGE _ FILE _ DLL 标志位（0x2000），该可执行文件就是一个动态链接库（DLL），不能被直接运行。浏览器帮助对象（BHO，参见第 3 章）就是一个例子。BHO 可以被 IE 加载到内存中提供 IE 扩展功能，也可以执行各种功能。一般情况下，这些 DLL 所提供的功能是合法的（例如，IE 就会使用 BHO 加载 Adobe 公司的 Acrobat Reader 软件打开 PDF 文件），但是在很多情况下 BHO 可能是间谍软件或者广告软件。

对于文件分析来说，IMAGE _FILE _HEADER 中说明 IMAGE _OPTIONAL _HEADER 的结构大小的值也很关键，因为这个值给出了 32b 还是 64b 的应用程序的定义。怎么判断呢？IMAGE _OPTIONAL _HEADER 的头 2B 称为是“幻数”[①]，如果这个幻数的值是 0x10b，则说明这是一个 32b 的应用程序；如果值为 0x20b，则说明这是一个 64b 的应用程序；如果该值为 0x107，则说明这是一个 ROM 镜像。本书只讨论 32b 应用程序的情况。图 6.5 是使用 PEView 软件查看样例文件 IMAGE _OPTIONAL _HEADER 结构的情况。

pFile	Data	Description	Value
00000108	0007C000	Size of Image	
0000010C	00001000	Size of Headers	
00000110	0007F430	Checksum	
00000114	0002	Subsystem	IMAGE_SUBSYSTEM_WINDOWS_GUI
00000116	0000	DLL Characteristics	
00000118	00100000	Size of Stack Reserve	
0000011C	00001000	Size of Stack Commit	
00000120	00100000	Size of Heap Reserve	
00000124	00001000	Size of Heap Commit	
00000128	00000000	Loader Flags	
0000012C	00000010	Number of Data Directories	

图 6.5　使用 PEView 查看 IMAGE _OPTIONAL _HEADER 结构

从图 6.5 可以看出，该样例程序是为 Windows 图形界面子系统设计的，DLL 特征值是 0000，表示该程序不是动态链接库。

由前文可知，IMAGE _OPTIONAL _HEADER 结构的大小保存在 IMAGE _FILE _HEADER 结构中。对于可执行文件的详细分析来说，IMAGE _OPTIONAL _HEADER 结构中存储的一些信息是很有用的。对它的详细分析已经超出了本章讨论的范围之外。

不过，IMAGE _OPTIONAL _HEADER 结构中的 SubSystem 值需要关注。这个值的作用是告诉操作系统运行该可执行文件需要哪个子系统。Microsoft 知识库文章中（KB90 493）中描述了如何确定应用程序的所属的子系统，并且包含了示例代码。在 MSDN 有关 IMAGE _OPTIONAL _HEADER 结构的文章中提供了该值更多可能性的说明。

IMAGE _OPTIONAL _HEADER 同样需要关注 Address of Entry Point 值。它是指向程序的入口点（entry point，也就是主线程的应该运行的第一条指令的地址）指针。这个指针加上可执行文件镜像加载的基地址，就得到可执行文件被加载到内存中之后程序入口点的地址。关于这个值的重要性，本章的后面还会说明。

紧接 IMAGE _OPTIONAL _HEADER 之后是多个 IMAGE _DATA _DIRECTO-

① magic number，是文件特征的另一种讲法，此处为应用程序类型标识——译注。

RY结构[16]。如图6.6所示，这些结构定义了PE文件中的信息结构目录，共有16个可能的目录，其中有导入表（列出该程序运行需要导入和使用的DLL函数）、导出表（对于DLL来说，需要导出供其他程序使用的函数位置）、调试目录[17]（如果有）的起始地址和大小及资源目录等。每个数据目录结构由一个相对虚地址（RVA）和有关数据长度两项数据构成，并且遵从严格的顺序排列。

00000138	00078004	RVA	IMPORT Table
0000013C	00000028	Size	
00000140	0007A000	RVA	RESOURCE Table
00000144	0000114C	Size	
00000148	00000000	RVA	EXCEPTION Table
0000014C	00000000	Size	
00000150	00000000	Offset	CERTIFICATE Table
00000154	00000000	Size	

图6.6　PEView查看IMAGE_DATA_DIRECTORYS的部分输出

图6.6给出样例程序16个数据目录中的4个，PEView给出每个结构在PE文件中的偏移量（如0x138）、有关数据加载的相对虚地址（如0x78004），以及每个结构体所代表的含义。从图6.6看出，这个应用程序只使用了导入表和资源表。

提示

在可执行文件被加载到内存中时，一般不能使用硬编码的地址，而要使用其中定义的相对虚拟地址（RVA），这是因为可执行文件不可能在每个系统中加载到同样的内存地址。RVA用来指定独立于文件的内存加载地址，也就是内存中文件加载地址的相对偏移地址。计算RVA的公式如下：

RVA=(变量在内存中的地址)－(可执行镜像加载的基地址)

要得到真实的内存地址，将RVA加上可执行镜像加载的基地址即可。

PE文件中最后一个需要关注的部分是IMAGE_SECTION_HEADER[18]结构。在IMAGE_FILE_HEADER结构中定义了PE段的数量，也就是PE文件中包含几个IMAGE_SECTION_HEADER结构。IMAGE_SECTION_HEADER结构大小为40B，它包括段名（8B）、硬盘和内存中段的长度及段的特性（是否可读、可写和可执行）。图6.7是使用PEview查看IMAGE_SECTION_HEADER结构的实例。

pFile	Data	Description	Value
000001B0	2E 74 65 78	Name	.text
000001B4	74 00 00 00		
000001B8	000776EC	Virtual Size	
000001BC	00001000	RVA	
000001C0	00078000	Size of Raw Data	
000001C4	00001000	Pointer to Raw Data	
000001C8	00000000	Pointer to Relocations	
000001CC	00000000	Pointer to Line Numbers	
000001D0	0000	Number of Relocations	
000001D2	0000	Number of Line Numbers	
000001D4	60000020	Characteristics	
		00000020	IMAGE_SCN_CNT_CODE
		20000000	IMAGE_SCN_MEM_EXECUTE
		40000000	IMAGE_SCN_MEM_READ

图6.7　使用PEView查看IMAGE_SECTION_HEADER结构

提示

查看段名时要记住，段名只不过是一个不超过 8B 的字符串而已，没有对段名的硬性规定。例如，一般应用程序的正文段的段名都是 .text，但是改成一个昵称也没问题。事实上，有些恶意软件的作者会使用故意修改段名的手段来欺骗缺乏经验的调查人员。大多数应用程序都包含 .code，.data，.rsrc 及 .text 段，而一些系统程序会有段名为 PAGE 或者 PAGEDATA 的段。有些恶意软件中，恶意软件的作者会故意用这种“正常”的段名来命名恶意代码所在的段，以期蒙混过关。而另一些段名，几乎直接就可以与加壳/加密联系在一起。例如，如果看见以 *UPX* 开头的段名，基本上可以肯定这个程序已经被加壳了。本章稍后部分会详细讨论这一问题。

PE 文件相关信息也可以使用 pedump. exe 工具查看，图 6.7 给出一个例子，使用 pedump. exe 查看 PE 可执行文件的显示结果如下：

```
01 .text    VirtSize:   000776EC  VirtAddr:       00001000
raw data offs:          00001000  raw data size:  00078000
relocation offs:        00000000  relocations:    00000000
line # offs:            00000000  line #'s:       00000000
characteristics:        60000020
CODE EXECUTE READ ALIGN_DEFAULT(16)
```

从上可以看出，这两个工具显示的信息基本相同。其中，VirtSize 是指当这个段被加载到内存中时所需要占用内存空间的大小，而“原始数据大小”（raw data size）则是指该段在磁盘上占用了多大的空间。正如第 3 章所讨论的，这个信息对于从内存镜像文件中提取可执行文件也很有帮助。

导入表

现在很少有程序是完全从无到有编写的。绝大多数应用程序中都会使用 Windows 系统 API（应用程序编程接口）中提供的服务，这些 API 是放在系统的许多个 DLL 中供程序员调用的。它们提供了很多现成的功能，从创建 Windows 窗口、菜单、对话框、网络套接字到系统内核的对象等，几乎无所不包。这就使程序员在编程时方便了很多。

在编译并链接生成可执行文件时，该可执行文件运行时需要调用的 API 及其所在的 DLL 都要被保存到可执行文件中。具体说，这些信息是保存在导入表中的。

注释

我以前参与过一个的项目，项目内容包括确定一个可执行文件是否有网络功能。如果具有，还要确定这个可执行文件扮演的是服务器角色（监听某个端口，等待黑客连上来，类似特洛伊后门）还是客户端的角色（主动连接到某个服务器

上，类似一个 IRCbot)。项目要求检测过程自动化。我们想到的办法就是把所有提供网络功能的 API 及其所属的 DLL 全部列出来，然后编程自动分析 PE 文件，检查可执行文件的导入表，看被检查的 PE 文件中是否使用了提供网络访问功能的 API。

Pedump. exe 工具可以自动定位 PE 文件的导入表目录并解析其结构，确定应用程序中使用了哪些 API 及其所属的 DLL。Pedump 输出样例如下：

```
KERNEL32. dll
OrigFirstThunk:   0000D114 (Unbound IAT)
TimeDateStamp:    00000000 -> Wed Dec 31 19:00:00 1969
ForwarderChain:   00000000
First thunk RVA:  0000B000
  Ordn   Name
  448    GetSystemTimeAsFileTime
  77     CreateFileA
  393    GetNumberOfConsoleInputEvents
  643    PeekConsoleInputA
  571    LCMapStringW
  570    LCMapStringA
  443    GetSystemInfo
```

从上可以看到，样本程序导入了 kernel32. dll 中的几个函数。kernel32. dll 提供了大量 API 函数供应用程序使用（可以查看其导出表），但是样本程序只使用了 *GetSystemTimeAsFileTime()* 和 *CreateFileA()* 等函数。Microsoft 提供了大量与 API 函数相关的文档（MSDN），可以在线阅读获取阅读参考信息。例如，通过在线查阅 MSDN，可以知道 *GetSystemTimeAsFileTime()* 函数的作用就是提取出系统当前的时间，以 UTC 格式（距 1601 年 1 月 1 日多少百纳秒表示时间的方法）放在一个 64b 的 FILETIME 对象中。

提 示

通过查阅 MSDN（Microsoft Developer's Network），可以查询 Microsoft 公司的 API 函数的详细信息。我习惯在 IE 的收藏夹里添加一个 MSDN 高级搜索页面[19]的链接，供遇到不熟悉的 API 函数，如 GetSystemTimeAsFileTime 时快速进行查阅。有时除了能查到这个 API 的信息之外，还能查到一些其他的辅助信息。

通过检查应用程序导入的函数可以得到程序功能的大致线索。例如，如果这个应用程序根本就没有导入提供网络访问功能的 API 函数（从底层的 socket 函数到高层的直接访问互联网的函数），那么这个应用程序就不太可能是一个后门程序，或者能把机器中的信息发送到互联网上去的“小偷”程序。这是一个很有用的技术，用它可以大致推

断可执行文件的功能。有一次，客户交给我一个可疑文件，要求分析这是不是一个后门程序。在记录文件基本信息之后，我检查了这个可执行文件的导入表，并且发现这个程序没有导入任何与网络有关的 API 函数。然后我又进一步分析了这个可执行文件导入的各个函数所提供的功能，结果发现除了有几个用于数学计算的 API 之外，这个可执行文件确实没有使用任何关于网络的 API[①]。

另一种分析可执行文件需要导入的函数及其所属的 DLL 的工具是 Dependency Walker，也就是 depends. exe。这个工具可以从同名的网站（http://www. dependency-walker. com/）下载。图 6. 8 是用 Dependency Walker 打开一个名为 DCode. exe 程序时的截图。

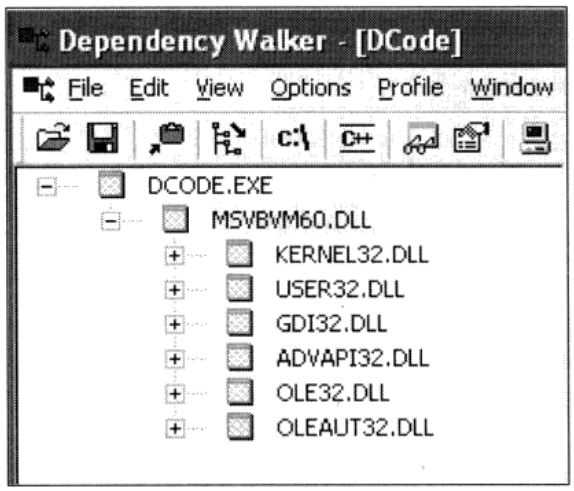


图 6. 8　Dependency Walker 界面

如图 6. 8 所示，dcode. exe 应用程序需要调用 MSVBVM60. DLL 动态链接库中导出的函数，而 MSVBVM60. DLL 动态链接库又需要调用其他 6 个动态链接库中的函数（这 6 个动态链接库在当前任何一个版本的 Windows 操作系统中都有）。

图 6. 9 给出的是用 Dependency Walker 读出的一部分 MSVBVM60. DLL 导出的函数。

Dependency Walker 不仅能查看 PE 可执行文件中导入的函数，还可以查看动态链接库中所有导出的函数。这点会在下一小节讨论。

Dependency Walker 还提供了一个很有用的“轮廓”（profile）分析功能。该功能就是运行可执行文件并监视可执行文件运行时加载了哪些动态链接库。可以在轮廓分析时设置参数过滤掉一些冗余的信息。通过这一功能，可以在程序运行时跟踪不同的 DLL 函数调用，函数的返回值是什么。这个功能的有用之处在于，有些 DLL 并不是在 PE

① 作者在这个地方的讲法是有问题的。一般来说，如果在一个单机程序中发现其使用了网络相关的 API，我们有理由认为这个程序很有可能就是一个病毒。但是要是分析一个可执行文件的导入表，发现其中并没有导入与网络有关的程序，于是就说这个程序不可能是一个病毒或者木马，这点应该说是证据不足的。因为在程序中调用 API 的方法有很多种，并不仅限于使用导入表——译注。

E	Ordinal ^	Hint	Function	Entry Point
C	100 (0x0064)	60 (0x003C)	ThunRTMain	0x0000DE3E
C	101 (0x0065)	73 (0x0049)	VBDllUnRegisterServer	0x0001BCFC
C	102 (0x0066)	70 (0x0046)	VBDllCanUnloadNow	0x0002C692
C	103 (0x0067)	72 (0x0048)	VBDllRegisterServer	0x000A4A8A
C	104 (0x0068)	71 (0x0047)	VBDllGetClassObject	0x00028FCA
C	105 (0x0069)	69 (0x0045)	UserDllMain	0x0001BBA7
C	106 (0x006A)	13 (0x000D)	DllRegisterServer	0x000D3AD5
C	107 (0x006B)	14 (0x000E)	DllUnregisterServer	0x000D3CB3
C	108 (0x006C)	94 (0x005E)	__vbaAryLock	0x000E24D0

图 6.9　MSVBVM60.DLL 导出的函数

导入表中指定，在应用程序初始化时被装入内存的，而是在程序运行时动态装入内存的(如使用 LoadLibrary()函数)。另外，在分析“应用程序初始化失败”错误时，轮廓分析功能也很好用。但是不管怎么说，要使用轮廓分析功能必须要先运行可执行文件，显然这部分内容已经超出静态分析的范畴了。

导出表

有导入表就有导出表，不然导入表中的数据又该从哪里来呢？每个动态链接库中都会有一张导出表，这张表中列出了所有该动态链接库中可能会被其他可执行文件(DLL、EXE 等）调用的函数。使用 DLL 中导出的函数，就大大简化了程序员的工作。当他们要实现某个功能时，只要 DLL 中有，直接调用 DLL 相关函数就可以了，而不必再去实现了。DLL 就像是一个函数库，或者就是为系统可能会经常使用的功能编写的预先写好的代码集一样。

Pedump.exe 可以读出动态链接库中的导出表。例如，ws2_32.dll 的部分导出表如下：

```
exports table:
Name:                WS2_32.dll
Characteristics:     00000000
TimeDateStamp:       41107EDA->Wed Aug 04 02:14:50 2004
Version:             0.00
Ordinal base:        00000001
# of functions:      000001F4
# of Names:          00000075
Entry Pt Ordn Name
00011028    1  accept
00003E00    2  bind
00009639    3  closesocket
0000406A    4  connect
00010B50    5  getpeername
0000951E    6  getsockname
000046C9    7  getsockopt
00002BC0    8  htonl
00002B66    9  htons
00004519    10 ioctlsocket
```

```
00002BF4     11  inet_addr
```

如果有 UNIX 或者 Perl 网络编程经验，可以一眼看出上面列出的是一些用于网络通信的基本函数。例如，*bind()* 和 *accept()* 就是服务器或者后台程序用来监听网络端口的（后门程序等）；*connect()* 函数则是客户端程序连接服务器时使用的，例如，IE，firefox 之类的浏览器或者 IRC 病毒。

另外必须指出的是，动态链接库也可以导入其他动态链接库的函数。例如，使用 Pedump. exe 分析上面 ws2_32. dll，就会发现 ws2_32. dll 从 kernel32. dll，ws2help. dll，ntdll. dll 等多个动态链接库中导入了函数。一些 DLL 会从其他 DLLs 中导入函数来构建其导出函数的基本功能。从这点上说，Dependency Walker 是个很好的工具，采用友好的图形界面显示了 DLL 库之间的链式或线性依赖关系。

资源段

在 PE 文件中还有一个资源段，这个段一般被命名为 . rsrc 段。在这个段里，包含了程序使用的对话框、图标等资源信息。这些信息对识别文件很有帮助，其中对可执行文件分析最有用的可能就是文件版本信息。

本书附带光盘中的 fvi. pl 脚本使用 Win32::File::VersionInfo 模块提取 PE 文件的版本信息。fvi. pl 要求输入文件名（带全路径），返回从文件中得到的信息：

```
C:\Perl>fvi. pl c :\windows\system32\svchost. exe
Filename:c          :\windows\system32\svchost. exe
Type                :Application
OS                  :NT/Win32
Orig Filename       :svchost. exe
File Descriptoin    :Generic Host Process for Win32 Services
File Version        :5. 1. 2600. 2180 (xpsp_sp2_rtm. 040803-2158)
Internal Name       :svchost. exe
Company Name        :Microsoft Corporation
Copyright           :Microsoft Corporation. All rights reserved.
Product Name        :Microsoft «Windows« Operating System
Product Version     :5. 1. 2600. 2180
Trademarks          :
```

使用这个脚本时必须要记住，首先，Win32::File::VersionInfo 模块只能在 Windows 平台下使用。其次，Win32::File::VersionInfo 模块和 fvi. pl 不会验证 PE 文件的有效性。也就是说，要是 fvi. pl 不能返回信息，这也不能证明被分析的文件是恶意程序。事实上，有些恶意软件的作者会很小心，以确保生成的恶意程序中不带任何版本信息，另一些则会包含虚假版本信息，以期误导调查人员。甚至有些文件的版本信息中包含的只是一句笑话。

背景注释

RussianTopz 僵尸病毒

有一次我分析了一个名为 russiantopz 的 IRCbot 僵尸病毒。有意思的是，我发现这个病毒（文件名为 statistics. exe）根本就不是一个俄罗斯黑客编写的病毒！忽略文件的命名来分析其中的版本信息，我发现这个文件实际上就是 mIRC32. exe[20] IRC 客户端程序。其图形客户端被 teamscan. exe 程序所隐藏，而这个隐藏工具实际上是 Adrian Lopez 的 hidewndw. exe[21]。

虽然文件版本信息不会对分析起决定性作用，但至少是分析可执行文件时应该收集的信息之一。

反分析技术

到目前为止，分析都还只是编译器正常生成的可执行文件。虽然这些工具和技术可以用来识别可执行文件，但是恶意软件的作者往往尽力伪装或“混淆”（obfuscate）文件，以期避开管理员的检测并逃避杀毒软件/安全工具的查杀。通常恶意软件的作者会对恶意软件进行加壳甚至加密处理，或者通过发布新版本来达到这一目的。

有很多工具，如捆绑机、加壳工具和加密机，能用来在可执行文件上加入反分析功能。下面逐一分析这些工具。

捆绑机

捆绑机，顾名思义就是把一个应用程序和另一个应用程序捆绑在一起，其实质就是创建一个木马程序。其思路是使用载体程序引诱用户运行，载体程序可能是游戏或者其他可执行程序。当受害者启动载体程序时，程序可以正常使用，没有任何问题。但与此同时，木马程序也开始执行，用户在不知不觉中就中招了。

目前已知的第一个捆绑机是 EliteWrap[22]，但是在“死牛党”（the Cult of the Dead Cow）发布 Back Orifice 后门程序之后，Silk Rope 和 SaranWrap[23]这两个捆绑机则更加流行。不过现在浏览反病毒网站对恶意软件的介绍，会发现捆绑机已经风光不再，大多数黑客已经不认为使用捆绑机很酷了。因为使用捆绑机会留下很明显的痕迹，容易被杀毒软件发现。

虽然现在市面上有很多捆绑机，但是这些捆绑机执行的基本操作都是一样的，即将一个程序和另外一个程序绑定。EliteWrap 有些特别，允许用户输入脚本，在捆绑软件运行时加入额外功能。

警告

下载 EliteWrap 1.04 版本到 Windows XP Pro SP 2 系统之后，几次测试生成捆绑程序都失败了。然后我又试图写 EliteWrap 的脚本，但是每次 EliteWrap 捆绑后的程序竟然都比原来的程序还要小，当我试图执行 EliteWrap 生成的捆绑程序时，系统都会弹出一个对话框，报告 Error #57 reading package 错误。

加壳程序（压缩壳）

加壳程序是指能够压缩可执行文件大小的工具。但是加壳的主要目的却并不是为了节约硬盘空间，一方面加壳的程序在网络上传输会更快，另外就是加壳的程序可以躲避主机和网络防病毒/入侵检测系统的查杀。加壳也使得对可执行文件的分析更加困难。一些合法公司也会对程序加壳，使得程序运行起来更快（由于程序小，从磁盘加载到内存中的速度更快），同时也为了保护其商业秘密。市面上有很多种加壳程序，其中比较流行的有 ASPack[24]，UPX[25]和 FSG。

ASPack 加壳程序的工作原理是压缩可执行文件镜像，并且在文件末尾附加一小段解压缩代码。程序的入口点（entry point）被修改为那段解压程序的起始位置，原始入口点被保存其中。当可执行文件的镜像解压缩之后，解压程序再把系统的控制权交给原来的程序。ASPack 壳的一个标志是文件中存在 .adata，.udata 和 .aspack 这 3 个段（记住，前面已经说过了段的名字可以是可以随意更改的，这 3 个段名也不例外）。当然，现在也有能脱 ASPack 壳的专门工具。

UPX 是另一款比较流行的加壳程序，同时也可以用来脱 UPX 壳，所以也是自身的脱壳机。UPX 加壳后的文件会出现 UPX0 和 UPX1 两个段，当然这两个段名也可以使用十六进制编辑器修改。

这只是几个恶意软件用到的压缩壳工具，其他还有很多。在脱壳时，根据软件壳的不同，可以找到一些可以用来脱这种壳的程序或者插件。分析时需要到因特网上搜索脱壳是否可行，是否有工具可以自动完成。

ProcDump32[26]可以脱多种常见的壳。图 6.10 给出的是 ProcDump32 的脱壳对话框，从中可以选择各种脱壳算法。

ProcDump32 也有一些其他的功能，如转储一个正在运行中的进程到硬盘上，脱壳、解密 PE 文件及编辑 PE 文件头等。虽然之前介绍的一些工具也能做到这些，但 ProcDump32 还是一款相当有用的工具，它应该成为恶意软件分析工具包中的一员。

加密程序（加密壳）

加密程序是可以执行程序进行加密的工具。加密是恶意软件逃避主机、网络杀毒软件和入侵检测系统查杀的另一种方法。这种手段在恶意软件中使用的相当普遍，有些加密算法和代码已被破解或者至少可以被发现（根据其特征码），另外一些则处于完全未知状态。

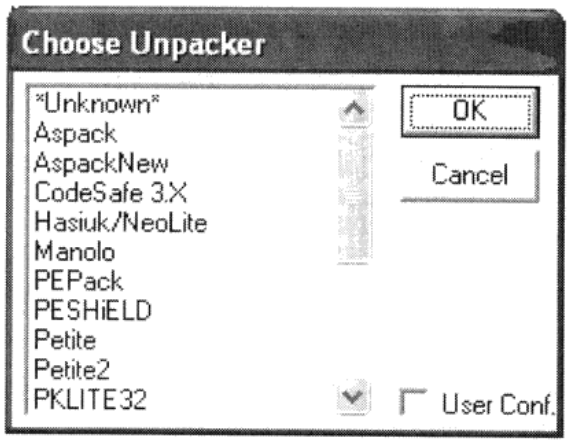


图 6.10　ProcDump32 脱壳对话框

作为恶意软件混淆自身结构的一种方式，首先要找到一个已知的样本文件来分析。HoneyNet 项目提供每月挑战（SotM[27]）已经有很长时间了，其中提供了不少数据和场景供研究人员分析和解密。这一系列的挑战中有不少是要求脱加密壳的例子，可供有兴趣的人研究。特别是在 SotM 挑战公布一段时间后，网站上还会给出提交的解决方案、评论，这样就可以详细得知这些挑战是怎么解决的。Ed Skoudis 在自己的网站（CounterHack. net）上也提供了类似的挑战。

HoneyNet SotM 第 32 个挑战是分析一个名为 RaDa. exe 的恶意软件。这个恶意二进制程序的图标如图 6. 11 所示。

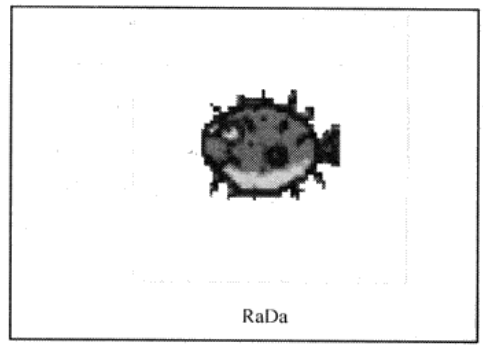


图 6.11　RaDa. exe 恶意二进制程序的图标

使用 pedump. exe 和 PEview 查看 RaDa. exe 文件，发现它的 PE 头很正常，所有的信息都被正确解析。我说的这个“正常”是指工具可以解析 PE 头部信息，而且信息也很合理。如果不正常，可能会出现指针指向文件中奇怪的段或者指向文件之外的情况。

可能最奇怪的就是该文件有 3 个段：JDR0，JDR1 和 . rsrc，. rsrc 大家都很熟悉，但是另外两个段在 PE 文件中从来没看到过。另外一点就是，导入表中只有两个 DLL——KERNEL32. DLL 和 MSVBVM60. DLL，如下所示：

```
Imports Table:
```

```
KERNEL32. DLL
OrigFirstThunk:   00000000 (Unbound IAT)
TimeDateStamp:    00000000 -> Wed Dec 31 19:00:00 1969
ForwarderChain:   00000000
First thunk RVA:  00010BE0
Ordn Name
  0 LoadLibraryA
  0 GetProcAddress
  0 ExitProcess
MSVBVM60. DLL
OrigFirstThunk:   00000000 (Unbound IAT)
TimeDateStamp:    00000000-> Wed Dec 31 19:00:00 1969
ForwarderChain:   00000000
First thunk RVA:  00010BF0
Ordn Name
  618
```

对于一个恶意软件来说，太奇怪了，任何恶意软件都不会只导入两个动态链接库，也不会只从 KERNEL32. DLL 中导入 3 个函数。

提 示

这也是快速判断软件恶意软件的好方法。当导入表显示只有 KERNEL32. DLL 被导入，并且只包括 LoadLibraryA 和 GetProcAddress 等很少几个导入函数时，基本上就能做出对象文件被加壳的判断了。

另一个被导入的模块 MSVBVM60. dll 是 Visual Basic 运行时函数库。fv. pl 的分析结果显示该文件资源段的文件描述是“VB 虚拟机”。由此可以推断这个 RaDa. exe 是用 VB 编写的程序。网站上提交的挑战答卷也得出了这个结论。

由于 RaDa. exe 有资源段，所以可以运行 fvi. pl 分析，并且读出如下数据：

```
Filename          :d:\tools\rada. exe
Type              :Application
OS                :Unknown/Win32
Orig Filename     :RaDa
File Description  :
File Version      :1. 00
Internal Name     :RaDa
Company Name      :Malware
Copyright         :
Product Name      :RaDa
Product Version   :1. 00
Trademarks        :
```

很好，作者在这里说明这是一个恶意软件。呵呵，别大惊小怪的，这种事也不是不可能发生啊。

现在已经有明确的标识表明这个恶意程序被加壳了（当然，我们选择的就是一个加壳的程序），但是加了什么壳呢？压缩壳还是加密壳？可以使用更强大的 PEiD[28] 工具来分析文件，以确定加壳方法。PEiD 的分析结果如图 6.12 所示。

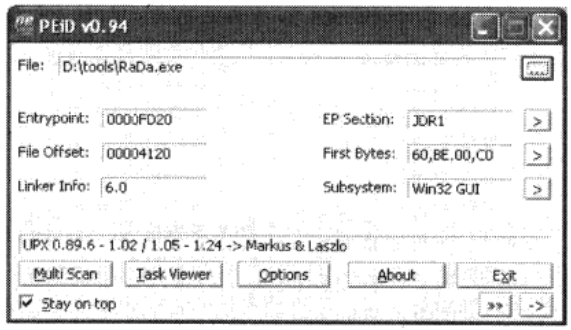


图 6.12　用 PEiD 加载 RaDa.exe

注意，PEiD 显示这个软件是加了 UPX 壳的。但是，为什么 PE 文件中的 2 个段是 JDR0 和 JDR1，而不是 UPX0 和 UPX1 呢？之前讲过 UPSX 加壳文件的特征段名字是 UPX0 和 UPX1。如果 PEiD 显示的信息是正确的，那么程序的作者肯定用编辑器修改了段名称。

PEiD 通过获取 PE 格式中定义的文件入口点，然后分析该处的代码，从而检测执行文件特性。PEiD 的作者收集了很多加壳工具/编译器在文件入口点附近的特征码，然后把这些成果都加到 PEiD 中，所以 PEiD 对常见壳的识别正确率非常高。另外，在 PEiD 中还包括了不少其他有用的工具，包括显示运行进程及其使用模块的任务查看工具、显示文件更多信息的对话框（图 6.13）、查看 PE 头结构的对话框，甚至还包括一个反汇编二进制程序的对话框。

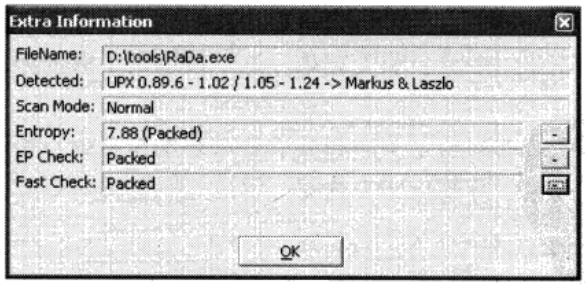


图 6.13　PEiD 显示 RaDa.exe 更多信息的对话框

如果读者能够下载 PEiD，用它分析 RaDa.exe 可执行文件，通过单击 First Bytes 文本框右边的>按钮即可运行反汇编工具。如果读者熟悉 x86 汇编语言（我毕业后没有用过，读书时是用 Motorla 68 000 微处理器编程的），就会立刻注意到 jmp 指令和很多加法指令。如果对这个二进制程序分析很好奇，可以看 HoneyNet 站点提交的答卷，特别是 Chris Eagle 提交的。Chris 是 BlackHat 大会知名讲师和演讲者[29]，现任加利福尼

亚州 Monterey 的海军研究生院计算机科学系高级讲师。

提示

如果你对恶意软件分析或者逆向工程感兴趣，通读 HoneyNet 网站 SotM 第 32 和 33 个挑战会很有帮助。从中不仅可以看到提交的各种分析答卷的共同点，还可以学到各种工具的用法，提高分析技巧。

动态分析

所谓的动态分析就是在一个可以控制和检测的环境下运行可执行文件，然后观察并记录其对系统的影响。动态分析是一种非常有用的分析手段，通过动态分析可以更详细地了解恶意软件对系统有何影响，特别是可以看到其执行顺序。如果恶意软件被加壳或加密则更有用，因为软件在运行之前必须先被解压或者解密到内存中。所以进行动态分析时，不仅可以看到雪和折断的树枝，通过获取和分析其内存，还可以看到活生生的雪怪。

测试环境

如果要对恶意软件进行动态分析，就必须要认真考虑测试/宿主环境搭建问题。毕竟，在工作网络/计算机上执行恶意软件不是一个好主意。不管是有意还是无意，只能说这是一场灾难，应该确保不会发生类似的事情。

搭建测试环境的一种方法是在一个独立的网络里放上一台专用的机器专供测试，注意这里说的“独立”是指该网络与其他网络没有物理连接（不是逻辑连接，也不是 VLAN 交换网络连接）！必须是物理隔离的，连硬件隔离开关都不要用，因为在你忙于测试的时候，有人可能过来观看并有可能忘记切断网络。另外，要是你正在受正规的客户委托对恶意程序进行审查，也必须防止恶意软件侵入内网，盗取敏感的私人信息。如果你所在的实验室已经通过了认证机构的认证，发生一次这样的失误很可能就把实验室的声誉毁了，个人的职业生涯也会蒙受灾难。这不仅适用于取证分析实验室，对常规的调查事务所也是这样。

使用“一次性”系统的问题是每次测试后要重装操作系统。否则如何证明收集的数据是干净的，没有被其他恶意软件所干扰呢？解决此问题的一个方法是使用虚拟机。

虚拟机

如果使用“一次性”的独立系统，那就要在每次测试完毕之后重装操作系统，并且还要装上需要的软件，谁不嫌烦呢？虚拟机是另外一个可用选项，现在市场上有很多免费和商用的虚拟机软件：

- **Bochs**[30] 可以运行在 Windows，Linux，甚至是 Xbox 上。有些用户抱怨这个软件很难安装。Bochs 是个开源的免费软件，但是很难安装和管理。
- **Parallels**[31] 可以运行在 Mac 平台上，另外也兼容 Windows 和 Linux。
- **Microsoft 的 Virtual PC**[32] 运行在 Windows 环境下，客户机可以是 DOS，Win-

dows 和 OS/2 操作系统，也是免费的。

- **VirtualIron**[33] 采用“裸机安装”（意思是不需要装在宿主操作系统上）技术，可以接近本地速率运行 Windows 和 Linux 操作系统。
- **Xen**[34] 运行于 NetBSD 和 Linux 操作系统，可以接近本地计算机的速度运行 Windows 客户机。
- **Win4Lin**[35] 运行在 Linux 操作系统上，通过它运行 Windows 下的应用程序。
- **VMware** 可以运行在 Windows，Linux 等操作系统环境下，通过它能在一台计算机上虚拟多台客户端计算机。其中，VMware Server 和 VMPlayer 是两款免费产品。VMware 被许多人认为是事实上的虚拟机工业标准，在下面部分里会详细讨论。

当然，上面这些并不全。具体选用哪个虚拟机很大程度上取决于需求、环境（运行平台、预算等）及个人喜好。如果实在拿不定主意，可以去看维基网上的“虚拟机之比较”（the Comparison of Virtual Machines）文章[36]。根据其中客观描述，选择一款适合的虚拟机产品。

使用虚拟机对恶意软件进行分析的好处是可以事先创建一个系统“快照”，然后“感染”系统并进行测试和分析。数据收集完成后，返回系统快照点，也就是感染前的状态。使用虚拟机不仅可以轻易恢复系统，还可以在同一平台上对多个恶意软件进行比较分析。

所有的虚拟机平台中最出名的要算 VMware[37]了。VMware 公司提供了几个免费的虚拟机软件，如 VMPlayer 和 VMware Server。VMPlayer 可以运行已创建的虚拟机（但不能创建）。现在网上有很多安装好的虚拟机和应用框架供免费下载使用。截止本书编写的时候，就有安装好 ISA Server 和 MS SQL Server 的虚拟应用框架供下载。

使用 VMware 进行分析也有问题，这个问题对其他虚拟机运行环境也适用。不久之前，网上曾经有过如何检测软件自身是否运行在虚拟环境下的讨论。这次讨论之后很快就有反病毒公司的分析人员发现有恶意软件不仅可以检测，而且其行为模式在不同的环境中会有所变化，或者就干脆不运行。2006 年 11 月 19 日，Lenny Zeltser 在 ISC 的分析员日记[38]里写了一篇文章，讨论了恶意软件如何使用一些商用的工具（实际上是一个商业的加壳工具）检测自身是否运行于虚拟机环境的方法。在对恶意软件进行动态分析时一定要记住这点！在把恶意软件样本带回实验室分析之前，必须确保对所有“中招”的受害者进行了调查取证，把恶意软件所有可能的行为都列出来。如果在虚拟机中分析时发现恶意软件的行为模式有所不同，很有可能这个恶意软件可以检测虚拟机的存在。

一次性系统

如果由于价格、经验、使用问题等不能使用虚拟机来分析，可以考虑使用基于镜像还原和重建的“一次性”系统。有些公司使用 Symantec 公司的 Norton Ghost[39]工具为具有相同硬件配置的系统创建镜像，这样就可以对系统进行统一地安装和管理。一些公司在培训环境中使用类似的技术，系统管理员可以快速将系统还原到已知状态。例如，我曾经对一家具有培训环境的公司进行漏洞评估，系统管理员得意他告诉我，使用 Ghost 启动盘，可以很轻松地重新安装所有 68 台培训机器的操作系统。

还是要再说明：无论选择何种动态分析环境，一定要确保测试环境与公司网络或生产环境物理隔离！有人可能认为这是废话，但是有太多由于管理员时间仓促或配置错误，导致质保和测试网络出问题的案例。测试环境必须保证与任何其他网络相隔绝，甚至逻辑断开都不够，最好是物理隔离。

在分析平台确定以后，虚拟机、一次性系统等分析环境中数据收集和分析过程都是一样的。在一次性系统中需要包括一些获取物理内存的工具（VMware 可以通过挂起会话的方式得到内存镜像），这点在对加壳可疑软件进行分析的时候很重要。

工具

有很多工具可以在测试恶意软件时监控系统的变化。大多数情况下，在运行恶意软件之前，就应该运行监控工具。另外，也要在事前充分了解工具的功能和操作方法。

恶意软件分析和应急处理之间的最大区别之一就是，在执行恶意软件分析时，可以在恶意软件感染之前安装和配置系统。虽然理论上系统管理员也有这个机会，但是大部分服务器系统在安全配置方面都不会很全面，特别是考虑到应急响应事件处理的相关配置。

对恶意软件进行分析的时候，必须意识到可能面临的挑战。例如，分析之前并不知道恶意软件到底会做什么。这听上去如废话，但有些调查员就是不够小心。作者的意思是在分析之前，不知道恶意软件运行后会直接开始工作，等你去慢慢分析它，还是会飞快的在系统中做一些手脚，然后就退出，消失的无影无踪。恶意软件可能的行为包括运行并打开监听网络端口（后门）、试图连接因特网（IRCbot）、快速注入到其他运行进程然后退出。进行动态分析时，可以让恶意软件一遍遍重复其“犯罪过程”，以了解其细节。应急响应时，获取的只是现场系统快照，而且得到的是离散的系统信息。这就好比拿着照相机拍摄一个奔跑中的运动员，得到的只是一张张静态的画面。动态分析就不同了，就好比在用摄像机拍摄，所获得的信息都是连续的。在这种情况下，更有把握揭示出恶意软件整个运行过程中的所作所为。

那么，应该使用哪些工具呢？首先，应该记录所有的网络流量，因为恶意软件可能会试图与远程计算机通信，也有可能打开侦听端口，或者二者兼有。要记录网络数据流，一种方式是使用类似 WireShark[40]（以前叫 Ethereal）的网络嗅探器。如果分析使用的是独立主机，那么需要在另一个系统上运行嗅探器；如果使用的是 VMware，可以在宿主机上运行嗅探器。后面会解释这样做的原因。

另一款需要使用的工具是 Port Reporter[41]，这是 Microsoft 提供的免费软件。Port Reporter 以服务方式运行于 Windows 系统，记录 TCP 和 UDP 端口活动情况。在 Windows XP 和 2003 系统中，Port Reporter 会记录使用的网络端口、使用端口的进程或服务、进程加载了哪些模块及运行进程的用户账号。Windows 2000 下记录的信息就要少得多。Port Reporter 具有多个配置选项，如日志文件在系统中的位置、服务是否自动启动等。配置参数可以通过启动时设置的命令行参数来控制。在安装 Port Reporter 之前，应该认真阅读 Microsoft 知识库中的相关文章，以确保掌握其工作原理，并且明确了解可以得到哪些信息。

Port Reporter 会创建 3 种类型的日志文件：初始化日志（文件名形如 PRINITIAL-*

.log[42]），记录服务启动时的系统状态信息；端口日志（文件名形如 PR-PORTS-*.log），记录网络连接和端口使用信息，类似 netstat.exe；进程 ID 日志（文件名形如 PR-PIDS-*.log），记录有关进程的信息。

Microsoft 公司还提供了一个介绍 Port Reporter 工具使用的网络视频[43]。另外，Microsoft 也提供了 Port Reporter Parser[44]工具，用于分析 Port Reporter 产生的大量日志。

安装这些网络监控工具之后，读者可能会感到很奇怪：为什么要在另一个系统中运行嗅探器呢？为什么嗅探器不能和其他网络监视工具一样，也在测试系统中运行呢？答案是，恶意软件中可能带有 rootkit，这个问题在第 7 章讨论。这里简单提一下，rootkit 会通过阻止操作系统“看见”恶意软件的进程、使用的端口等信息，达到隐藏恶意进程的目的。由于要保证分析中收集的数据不会受到 rootkit 的影响，所以需要在另一个系统上运行网络嗅探器。

提 示

恶意软件动态分析过程中从另一个系统上对被“感染”的系统进行一次扫描也许会有意外的发现。扫描可能会发现使用某种方法隐藏的后门（如 rootkit，第 7 章详细讨论）。使用 nmap[45]或者 PortQry[46]之类的工具对被“感染”系统进行一次快速扫描，扫描器甚至可以检测端口提供的服务属性。虽然 TCP/IP 连通性及端口扫描已经超出了本书的范围，但是利用这个办法总能找到恶意软件隐藏自身时留下的一些蛛丝马迹。

记住，对于案件调查人员来说，“缺少的痕迹”同样是一种痕迹。具体到恶意软件的动态分析，如果发现测试系统确实在向外发出数据包试图连接互联网（或者查找本地网络中的其他系统），但是测试系统内安装的监控工具又找不到任何相关进程和网络数据流的迹象，那就可以肯定系统中存在 rootkit。

另外要强调，对恶意软件进行彻底地分析并完整详细地记录整个分析过程是十分重要的。一些恶意软件本身并不具有 rootkit 功能，但却可以将恶意代码注入其他进程的内存空间里并执行。这也是一种需要明确的情形，如果此时假设存在 rootkit，就会导致错误的报告，也会导致后续判断和措施的偏差。如果报告中有详细的过程和工具记录，其他的调查人员就可以验证你的结论。毕竟，使用同样的工具、同样的过程对同一个恶意软件进行分析会得到同样的结果，不是吗？或者其他调查人员也可以检查你的分析过程，对特定工具的使用提出进一步的意见，使得对恶意软件分析更加彻底。

提 示

执行恶意软件动态分析时，必须拟定详细的计划，但是同时也不要太过。工具装多了会拖垮系统，花费太多的时间管理工具反而会使注意力偏离分析目标。

有一次我受客户邀请分析一个特别的文件。初始异常迹象出现在注册表中，

运行后该文件会在用户的 Run 键和 RunOnce 键下添加内容。有意思的是，RunOnce 键下添加的值，其文件名前有一个“*”，这是让操作系统即使以安全模式启动也要自动运行该文件（相当狡猾吧!）。静态分析发现这个恶意软件被加密了，PEiD 也不能确定加密方式，所以只能求助于动态分析。

通过在测试平台上运行恶意软件并分析获取的数据，我们发现，这个恶意软件以隐藏方式启动 Web 浏览器（浏览器进程在运行，却没有界面显示）并注入浏览器进程空间。由此确定，这个恶意软件运行之后，可以分析浏览器进程来获取更多信息。这也解释了为什么在易失数据分析时发现浏览器进程有不正常的网络连接，并且没有恶意进程运行的痕迹。

激活事件日志程序对“进程跟踪事件”的审计也是一个好的分析方法，审计应同时包括成功和失败事件。事件日志程序会记录系统中各种不同的活动，包括用户权限的使用、登录、对象访问（这个设置需要配置对象访问控制列表，可能监控的对象包括文件、目录、注册表中的键）等。由于动态分析时进程特别重要，所以打开“进程跟踪”审计并记录其成功、失败日志会得到很多有用的数据。使用 Windows 资源工具包（第 1 章介绍过）中的 auditpol. exe 工具可以设置分析平台的审计策略，也可以在测试之前对配置进行确认。例如，使用如下命令确认需要的审计已开启：

```
C:\tools>auditpol /enable /process:all
```

要确认测试之前有关的审计策略已开启，可以不带任何参数运行 auditpol. exe。

提示

调查员也许想开启对“系统”事件的审计，但是要确保不要开启太多的审计。问题是太多的数据会拖慢分析进度，特别是有很多无关数据的时候。一些调查员认为应该监控系统中的每个事件，这样才能确保不会错过任何信息，但是能够有效利用和分析的数据是有限的。认真考虑分析的目的所在，为分析平台建立标准配置并坚持使用，理由充足的情况下才去改变。数据太多和数据太少对调查人员来说都不见得是件好事。

前面提到监控文件和注册表键访问的一种方法是打开“对象访问”审计，在所有关心的对象上设置访问控制列表，然后运行恶意软件，从事件日志中理出头绪。除此之外，还有两种方法能做到这点：一种是在事前和事后建立快照并相互比较；另一种方法是实时监控。分析恶意软件时，最好是同时使用这两种方法，当然这需要工具的协助。可以访问 SysInternals 网站，下载 FileMon[47] 和 RegMon[48] 工具（分别是文件系统和注册表实时监控工具），或者下载 Process Monitor[49] 工具。FileMon 和 RegMon 已被 Process Monitor 所替代，而且 ProcMon 还提供了很多新功能。当然可以使用分开的监控工具，这只是个人喜好问题。实时监控工具优于快照工具的地方是，不仅能看到创建、修改的文件/注册表项，还能看到被搜索但不存在的注册表项。另外，还能看到所有活动的时间线，这在恶意软件分析中很重要。

下一节讨论基于快照的工具。

流程

建立恶意软件动态分析平台的过程是很简单的，关键是要有流程和检查表。与开机取证和后期分析一样，没人愿意每次都凭记忆完成动态分析，因为有时会很匆忙，或者忘记一个重要的步骤。人们都会犯这样的错误，我就有过这样深刻的教训。有一次分析时，我忘了打开一个工具，使所有的过程从头再来。重装机器、重装所有的软件，再把计算机配置到一个正确状态下，不用说也知道这个过程有多枯燥了。

第一件事就是确保找到、下载并安装所有要使用的工具。本章介绍了不少工具，未来还会有更多的工具需要使用。建立一个动态分析的工具清单，保持更新，与同行共享，随时加入新的工具，去除不用的工具。

所有工具准备就绪之后，还要确保理解所有工具的用法，确保明了必须的配置选项。大多数的工具需要手工启动，最好能有一个工具启动顺序的清单。例如，RegShot[50]（如图 6.14 所示）和 Incontrol5[51]（如图 6.15 所示）是用来制作系统快照的，所以要最先启动（收集快照基准数据），然后再启动实时监控软件。

图 6.14 RegShot 图形用户界面

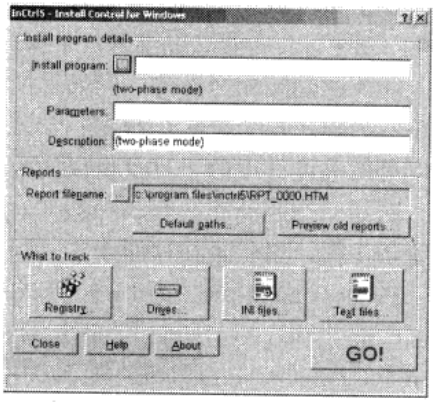


图 6.15 InControl5 图形用户界面

RegShot 的输出结果保存为纯文本或者 HTML 格式。在 ParaGlider 的 PEBuilder 插件[52]页面上有一个修改版本的 Regshot，可以将输出结果保存为 .reg 文件格式。另外，使用 Regshot 之类的快照和监视工具时，必须要记住大部分工具只能监控当前用户环境或其权限下的信息。换句话说，以系统管理员账户运行的工具只能监控管理员及其以下权限的改变，检测不到 SYSTEM 权限账号对系统所做的修改。

Incontrol5 提供关于文件和注册表键添加、改动和删除信息的漂亮报告（HTML、电子表格或纯文本格式）。它还能监控特定文件的修改，不过监控的文件数量十分有限。另外，可以选择一个安装程序，如 MSI 文件，监控其安装过程给系统带来的影响。不过，我还没看到过利用 Microsoft 安装程序传播的木马或蠕虫。

运行恶意软件并完成数据搜集之后，就要关掉实时监控工具，并且给系统拍第二次快照。这时出现了一个问题：应该在系统“快照”之前还是之后保存实时监控工具日志呢？毕竟测试系统只是对恶意软件进行分析的手段，它不是证据，所以可以根据个人喜

好来决定。就我个人而言，我会先保存实时监控系统的日志，然后再执行“快照”。这样做的一个副作用是快照比较时包括实时监控工具创建的输出文件，不过由于我对这些文件很清楚，所以很容易排除这部分内容。

更进一步，所有的日志文件应该存放在不同的目录中。这不仅使分析的时候更容易区分，在监控完成以后提取数据也会更加方便。实际上，可以考虑在测试系统中附加一个 USB 移动存储设备，将所有的日志文件保存在这个设备上。

概括整个恶意软件动态分析流程如下：

- 根据工作清单，确保所有监控软件安装/更新完毕 。
- 确保所有的监控软件正确配置。
- 创建日志存储位置（本地硬盘、USB 移动存储等）
- 准备待分析的恶意软件（将恶意软件文件复制到测试系统中，记录其在文件系统中的位置）。
- 使用“快照”工具创建基线数据。
- 运行实时监控工具。
- 运行恶意软件（记录启动方式：调度任务、双击运行还是从命令行启动等）。
- 停止实时监控工具，保存日志到指定位置。
- “快照”工具第二阶段运行，保存数据到指定位置。

上述流程确实相当简单，但如果没有流程或者没有遵照流程执行将会遗漏很多重要而有用的数据。换个说法，制定一个流程并按照流程来做，使得分析方法更加规范，并且可以更加关注其中的工具及生成的数据。当然，其中的工具会随着时间的流逝而发生变化。例如，按照流程，很长一段时间内 RegMon 和 FileMon 被用来监控注册表和文件系统的访问，虽然现在这两个工具仍然可用，但是作者将它们的功能集成到一个工具中——Process Monitor。Process Monitor 的截图如图 6.16 所示。

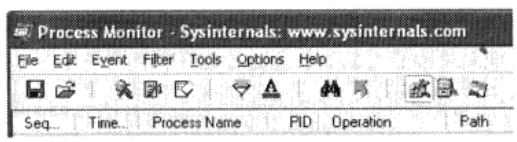


图 6.16 Process Monitor 工具栏

如果读者以前使用过 RegMon 或者 FileMon，可以看出 Process Monitor 的工具栏与其很相似，其中大部分图标和排列顺序都一样。

使用 Process Monitor 监控文件系统和注册表访问信息时，必须意识到所有的访问信息都会被捕获，其中很大一部分是系统产生的，应该被过滤掉。例如，单击工具栏上那个带叉的放大镜，然后什么也别做，鼠标和键盘都不要碰，只要坐在屏幕前看就行了。此时窗口中会出现了大量事件，但是你什么也没有做啊！显然，Windows 中每秒都会发生许多事件，只不过之前看不到罢了。如果要查看 Process Monitor 中捕获的数据，可以单击一个事件，然后选择 Event | Exclude | Process Name 来排除不必要的进程和冗余数据。

提示

还记得第4章讨论过的 Image File Execution Options 注册表键吗？Process Monitor 可以轻易展示 Windows 系统是如何访问这个键的。做个实验，打开命令行窗口，输入 net use，先不要按 Enter 键。打开 Process Monitor，开始捕获注册表访问信息。切换回命令行窗口按 Enter 键。命令完成以后，停止 Process Monitor 数据捕获（就是单击那个带叉的放大镜）。图 6.17 显示的是捕获的部分数据，数据显示 net.exe 在使用 DLL 时确实会试图访问 Image File Execution Options 中的相关键。

\Windows NT\CurrentVersion\Image File Execution Options\ntdll.dll	NAME NOT FOUND
\Windows NT\CurrentVersion\Image File Execution Options\kernel32.dll	NAME NOT FOUND
\Windows NT\CurrentVersion\Image File Execution Options\msvcrt.dll	NAME NOT FOUND
\Windows NT\CurrentVersion\Image File Execution Options\RPCRT4.dll	NAME NOT FOUND
\Windows NT\CurrentVersion\Image File Execution Options\ADVAPI32.dll	NAME NOT FOUND
\Windows NT\CurrentVersion\Image File Execution Options\NETAPI32.dll	NAME NOT FOUND

图 6.17　Process Monitor 捕获对 Image File Excution Options 注册表键的访问

不要忘了动态分析的最后一步，使用第3章讨论的方法获取整个物理内存。这样做不仅可以得到动态分析的所有数据，分析恶意软件在测试系统中做了什么，而且可以从物理内存中提取出加壳恶意软件的可执行文件镜像，从而对其代码进行逆向工程，使得分析进一步深入。

小结

本章介绍了两种可执行文件分析方法。通过了解可执行文件格式，可以知道文件的结构，从而找出这些文件采用了何种反分析技术。使用本章讨论的分析方法可以确定软件（包括恶意软件）对系统的影响及其留下的痕迹。这对调查来说很有用，因为反病毒软件的检测范围有限或者提供的信息不够详细，调查员必须获得进一步的信息。对应急响应人员来说，这些痕迹可以协助查找网络中其他被入侵的系统。通过分析，调查人员可以详细了解入侵的情况及恶意软件的行为模式。对涉及特洛伊木马/后门或者远程访问控制的案件，通过动态分析可建立其系统中活动的时间线。

和其他工具一样，本章介绍的分析方法各有优缺点，每种方法都要经过彻底地验证和记录。静态分析可以得出恶意软件大致的类型，并且可以提供动态分析可能碰到问题的线索。但是，静态分析所能提供的信息还是十分有限的。动态测试又称“行为”分析，因为通过在一个可控的环境下运行恶意软件的，可以看到恶意软件对目标系统的影响、其执行顺序如何。当然，动态分析时必须非常小心，毕竟是在运行一个恶意软件，稍不留神就对分析的网络环境造成影响。

即使不准备对恶意软件进行任何分析，也要对记录其详细情况：在哪里发现的这个可疑软件？有哪些文件与之相关联？计算并记录哈希等。恶意软件的作者一般不会采用

“系统杀手”这类明显的文件名。相反，恶意软件的名字会很普通，有时甚至会和系统文件相混淆，所以记录它们的细节特别重要。

参考资料

1. www. datarescue. com/idabase/index. htm
2. MD-5 的定义参见：http://en. wikipedia. org/wiki/Md5
3. SHA-1 的定义参见：http://en. wikipedia. org/wiki/Sha-1
4. ssdeep，http://ssdeep. sourceforge. net/
5. www. dfrws. org/2006/proceedings/12-Kornblum. pdf
6. www. microsoft. com/technet/sysinternals/Miscellaneous/Strings. mspx
7. IE 0199 病毒，www. mycert. org. my/virusinfo/ie0199. htm
8. Win 32 可执行文件格式第一部分，Matt Pietrek，http://msdn. microsoft. com/msdn-mag/issues/02/02/PE/
9. Win 32 可执行文件格式第二部分，Matt Pietrek，http://msdn. microsoft. com/msdn-mag/issues/02/03/PE2/
10. PEView. exe，www. magma. ca/~wjr/
11. www. microsoft. com/whdc/system/platform/firmware/PECOFF. mspx
12. http://msdn2. microsoft. com/engb/library/ms680198. aspx
13. http://en. wikipedia. org/wiki/Mark_Zbikowski
14. http://msdn2. microsoft. com/engb/library/ms680313. aspx
15. http://msdn2. microsoft. com/engb/library/ms680339. aspx
16. http://msdn2. microsoft. com/enus/library/ms680305. aspx
17. http://msdn2. microsoft. com/enus/library/ms680305. aspx
18. http://msdn2. microsoft. com/enus/library/ms680341. aspx
19. http://search. microsoft. com/advancedsearch. aspx? mkt=en-US&-setlang=en-US
20. www. mirc. com/get. html
21. http://premium. caribe. net/~adrian2/creations. html
22. http://homepage. ntlworld. com/chawmp/elitewrap/
23. http://packetstormsecurity. org/trojans/bo/index3. html
24. www. aspack. com/
25. http://upx. sourceforge. net/
26. www. fortunecity. com/millenium/firemansam/962/html/procdump. html
27. www. honeynet. org/scans/index. html
28. http://peid. has. it/
29. www. blackhat. com
30. http://bochs. sourceforge. net/
31. www. parallels. com/
32. www. microsoft. com/windows/virtualpc/default. mspx

33. www. virtualiron. com/
34. www. xensource. com/xen/
35. www. win4lin. com/
36. 维基百科虚拟机比较文章，http://en. wikipedia. org/wiki/Comparison_of_virtual_machines
37. www. vmware. com
38. http://isc. sans. org/diary. php? storyid=1871
39. www. symantec. com/home_homeoffice/products/overview. jsp?pcid=br&pvid=ghost10
40. www. wireshark. org/
41. http://support. microsoft. com/kb/837243
42. 日志文件创建时，* 会被 24 小时格式的日期和时间信息所代替
43. http://support. microsoft. com/kb/840832/
44. http://support. microsoft. com/kb/884289
45. http://insecure. org/nmap/index. html
46. http://support. microsoft. com/kb/832919
47. www. microsoft. com/technet/sysinternals/FileAndDisk/Filemon. mspx
48. www. microsoft. com/technet/sysinternals/SystemInformation/Regmon. mspx
49. www. microsoft. com/technet/sysinternals/ProcessesAndThreads/processmonitor. mspx
50. http://regshot. blog. googlepages. com/regshot. html
51. http://home. planet. nl/~pa0joz/pc_util. html
52. www. paraglidernc. com/6901. html

快速解决方案

静态分析

- √ 在调查过程中发现的任何可疑程序、文件都要做好相关的记录，这是分析其行为和功能的第一步。
- √ 虽然大部分人明确了解可执行文件的内容，但是如果能了解可执行文件的格式，调查中还是可以从这些二进制文件的结构进行初步分析的。
- √ 对可疑文件进行分析时不能轻信其文件名。即使有经验的恶意软件分析人员也可能被这种简单的“技巧”所蒙骗。

动态分析

- √ 通过动态分析可以了解恶意软件对系统的影响。
- √ 同时使用快照工具和实时监控工具不仅可以分析恶意软件入侵后遗留的痕迹，还可以知道其执行顺序。
- √ 动态分析时，尽量在测试平台之外运行监控工具，这样收集的信息不会受到恶意软件的影响。

√ 对恶意软件的动态分析完成后，可以对测试平台进行后期取证分析。这不但可以提高分析人员的技能，还可以对恶意软件的痕迹进行验证。

常见问题

下面的这些问答用来测试读者是否已经完全理解本章的内容，并可以帮助读者在实际案件调查过程中应用本章知识。如果还有其他问题，浏览 http://www.syngress.com/solutions 并单击 Ask the Author 一栏。

问：应急响应时，我发现系统中一个名为 svchost.exe 的程序发起了好几个网络连接。这个系统感染恶意程序了吗？

答：这个问题不是系统是否被感染，而是 svchost.exe 是否是一个恶意软件？针对你的情况，我的第一个问题是：你是怎么查看网络连接的？特别是，连接处于什么状态？是在侦听端口、等待连接，还是已经与其他系统建立了连接。第二个问题是：这些网络连接中使用了什么端口？连接和 svchost.exe 的关联看起来正常吗？最后一个问题是：是在哪里发现的这个 svchost.exe？是不是在 system32 目录里？是否有 WFP（Windows 文件保护）保护？如果 WFP 正常，是否计算了这个文件的哈希，并且已知文件哈希进行过比较？在应急响应的过程中，由于对系统缺乏熟悉和了解，经常会误导调查的方向，并且使调查人员得出错误的结论。

问：我在调查中发现了一个文件，用十六进制编辑器打开文件后发现它有 MZ 签名和 PE 头，但是找不到常见的 .text，.idata 和 .rsrc 段。这是怎么回事啊？

答：PE 文件自身并不需要特别使用 PE 段名称，所以段名被修改并不会影响 PE 文件的正常运行。虽然“正常的”PE 文件和一些被加壳的 PE 文件会有“标准的”段名，但是这些段名也是很容易被修改的。段名仅仅是分析 PE 文件的属性的一小部分，千万不要过于依赖它。

问：我已经完成了对一个恶意软件的静态和动态分析，并且已经知道这个恶意软件能干什么及在系统中会留下什么痕迹。现在该怎么验证我的结论呢？

答：在完成分析之后，可以用杀毒软件对这个恶意软件进行扫描。大多数情况下，调查员会在分析之前做，但这也不能保证检测出什么结果。很多应急响应人员在安装了最新的杀毒软件系统中发现过蠕虫。如果手头的工具检测没有效果，可以尝试将文件上传到类似 www.virustotal.com 的网站，这个网站会用至少 20 多种杀毒引擎进行扫描。如果还是没有结果，可以把可疑文件样本连同分析过程记录发送给反病毒厂商分析。

问：我对可执行文件和恶意软件分析很感兴趣，能给我推荐一些参考资料吗？

答：根据投入的时间和精力，应该阅读不同深度的关于软件逆向工程方面的资料。其中讨论的很多技术与恶意软件分析是一样的。这样的站点包括 REblog（http://blogs.msdn.com/geffner/default.aspx）和 OpenRCE（www.openrce.org/articles/）。

Chapter 7

第 7 章
Rootkits 及其检测

本章主要内容：

- Rootkits
- Rootkit 检测

√ 小结

√ 快速解决方案

√ 常见问题

引言

在2005年2月的RSA峰会上，来自Microsoft公司的Mike Danseglio和Kurt Dillard提出了一个术语——rootkit。在随后的几个月中，不断有“专家”就此概念发表评论和文章，而许多软件公司则相继开发了不少用于检测rootkit的工具。虽然实际上rootkit在这之前已经“出生”好几年了，先是在UNIX世界中，而后又被移植到Windows系统，但是这一问题却被长期误解或者忽略，颇有一些掩耳盗铃的味道。在RSA 2005峰会上提出了rootkit这个概念，引起了人们对rootkit的广泛关注，不久商业化的rootkit检测工具也问世了（以前只有几种免费软件和方法用于检测rootkit）。随着检测工具的日益进步，rootkit的作者也在寻找新的破坏操作系统及其内核的方法，以使其编写的rootkit避免被检测出来。

rootkit可以用来隐藏其他的黑客工具，如键盘记录器、网络嗅探器及远程访问被入侵计算机的后门等。毫无疑问，rootkit的线程自身也应该在系统中隐密地运行，而不能被发现。不仅使利用计算机的用户无法察觉，就算是计算机操作系统本身对此也应该是毫无察觉的。因为rootkit的这一特性，在进行计算机安全事件应急响应过程中，缺乏与rootkit相关知识和相应训练的调查人员所得到的结果就会有问题。如果被调查的系统中存在某个rootkit（没有对数据进行硬件备份），调查人员就会被误导从而做出错误的估计，这样相当大量的资源就会被投入到错误的方向。在不能确定问题根源的情况下，系统可能会被擦除然后重新安装，而后在投入使用的时候重新被感染。

Rootkits

那么，什么是rootkit呢？根据Sophos公司[1]2006年8月24日在其Blog上公布的调查统计显示：37%的受访者根本不知道什么是rootkit。在维基百科中rootkit是这样被定义的[2]：“在计算机操作系统中用以隐藏运行中的进程、文件或者其他系统中的数据的所有软件的集合”。Jaime Butler，一个广为接受的rootkit技术专家，与Sherri Sparks一起在发表于《安全焦点》（SecurityFocus）系列文章的第一部分给出如下定义：

> ……rootkit是黑客用来在被入侵计算机中隐藏自己的踪迹并使自己能够在将来继续控制被入侵计算机的一个或一组程序。为了能做到这点，这些工具会修改操作系统中相关组件的执行流程，或者操控操作系统用以内部审计和记录的数据集[3]。

从另一个角度来看，rootkit就是一个能修改操作系统，从而把自己及其他一些对象隐藏起来的软件，该隐藏不仅对普通用户和系统管理员有效，甚至操作系统本身也无法察觉。

rootkit可以用来隐藏操作系统中的进程、网络连接、注册表键、文件。rootkit这个术语来自于UNIX世界，常用来获取/维持对系统root级（就是Windows中的系统管理员）的访问权限。后来Windows下的恶意软件发展出类似的功能，rootkit名字也

随之被带入 Windows 世界。

NTRootkit 是 Windows 下最早被开发出来的 rootkit 之一，作者是 Greg Hoglund，发布于 1999 年。NTRootkit 包括一个驱动程序，它的源代码现在仍可下载，如图 7.1 所示。

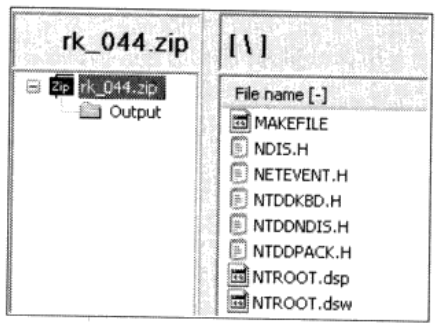


图 7.1 NTRootkit 0.44 源代码包

此后，rootkit 的开发和研究日益深入，Hoglund 和其他人在 BlackHat[4] 和其他大会上讲授如何编写 rootkit 的课程，而他的网站 http://www.rootkit.com 则成了汇集 Windows 下 rootkit 有关知识、开发和共享信息的知名网站。近几年来，其他 rootkit 也不断涌现，编写 rootkit 的技巧则更是日新月异。另外还有一本有关 rootkit 工作原理和设计思路的书籍——*"Rootkits: Subverting the Windows Kernel"*，该书由 Greg Hoglund 和 Jamie Butler 合著，亚马逊有售。

2005 年 2 月 RSA 峰会之后，引发了 rootkit 及其检测技术的研究热潮，检测技术的发展迫使 rootkit 技术也不断更新。接下来的形势变得如同一个不断升温的战场，一方取得的任何一点进展都会激发起另一方更深入的研究热情。

rootkit 可以分成几种。早期 rootkit 的工作原理是把系统中的一些程序替换成恶意的版本，当这些替换的程序运行时，就会自动隐藏特定对象。例如，替换版本的 netstat 不会显示攻击者使用的网络连接，只会显示正常网络连接列表。

在这之后演变出 DLL 注入型的 rootkit，又名"用户模式"的 rootkit。这种 rootkit 是安装在系统当前登录用户的安全上下文环境，替换、挂钩（hook）或者修改某些操作系统调用和 DLL 中的函数。还是以 netstat 程序为例，用户模式下的 rootkit 不会替换 netstat 程序本身，而是通过挂钩负责列出网络连接的 Windows 函数调用，使得函数不会返回完整的网络连接列表。Netstat 命令只会显示函数的返回结果，并不知道该信息已经被替换。这样做还可以欺骗其他使用相同 API 函数的程序。隐藏文件的用户模式 rootkit 会挂钩 *FindFirstFile()* 和 *FindNextFile()* 两个 API 函数调用，使得所有使用这两个 API 查看、浏览文件的程序，包括 Windows 的 shell 程序，都不能发现 rootkit 所隐藏的文件。

下面给出部分用户模式下的 rootkit 实例：

- **AFX Rootkit 2005** 这是一个使用 Delphi 编写的开源 rootkit（作者是 Aphex）。该 rootkit 使用 DLL 注入和 API 挂钩方式隐藏文件、注册表键、进程等对象。
- **Hacker Defender** 作者是 holy_father，来自 hxdef.org。这可能是目前所知最流

行、使用最广泛的一个 rootkit。F-Secure 网站把 Hacker Defender 称为实际黑客攻击案例中使用最多的 rootkit。Hacker Defender 使用了端口重定向技术，这样就能防止使用诸如远程端口扫描之类的传统意义上的 rootkit 检测技术。Hacker Defender 使用的配置文件不会被写入计算机硬盘，其配置文件只存在于系统物理内存。只有使用物理内存分析方法才能发现 Hacker Defender 及被这个 rootkit 所隐藏的进程。调查人员必须将物理内存中发现的进程列表和操作系统中列出的进程相比较，才能发现 rootkit 隐藏的进程。

- **NTIllusion**[5] 这个 rootkit 运行时所需要的权限最低，可以改变当前用户的进程信息。
- **Vanquish** 一个来自罗马尼亚的 DLL 注入型 rootkit，可用来隐藏文件、进程、注册表键等系统对象。Vanquish 包括一个自动加载器（.exe 文件）和一个 DLL，该 DLL 包括 6 个子模块。Vanquish 安装时需要管理员权限，根据 Vanquish 的文档描述，当系统中存在其他 rootkit 时，Vanquish 不能正常工作。
- **Gromozon**[6] 这是一个 BHO 感染系统的用户模式 rootkit，使用了多种手段防止自身被清除（在 EFS 文件和 NTFS 分支数据流中隐藏代码、创建服务、在注册表的 AppInit _DLLs 键中创建引用等）。此外，该 rootkit 还会关闭当前用户的调试权限，使得 rootkit 检测工具不能正常运行。Symantec[7]将其比作“意大利式细面条”，以形容这个 rootkit 的难缠。

提示

这几种 rootkit 都可以从 rootkit.com 网站下载。

潜伏最深的是“内核模式” rootkit，因为这些 rootkit 破坏的是操作系统内核。内核模式 rootkit 不仅会拦截底层的 API 调用，还会操纵内核数据结构。Jamie Butler 编写的 FU 就是这样一个内核模式 rootkit 实例，它使用直接内核对象操纵（DKOM）技术来实现隐藏。DKOM 是不使用 Windows API 直接操纵内核数据结构的进程。例如，Windows 内核维护了一个当前系统运行进程的双向链表，而 FU 将需要的进程从这个双向链表中移除。该进程依然存在，但是内核再也“看”不到这个进程。由于 Windows 系统中调度的基本单位是线程而不是进程，所以 FU 的线程仍可继续运行，进程则被隐藏。FU 使用默认名为 msdirectx.sys 的驱动程序以获得对系统的访问和控制权限，而 FU 程序自身（fu.exe）在加载驱动程序之后就会终止运行。

内核模式的 rootkit 也会修改其他内核数据结构。作为 FU 的后续版本，FUTo[8]在“*Uniformed Journal*”[9]杂志的第三卷（2006 年 1 月发表）引发了长篇大论。通过使用汇编语言操纵 *PspCidTable* 变量，FUTo 进一步扩展了 FU 的 DKOM 功能，*PspCidTable* 是指向进程和线程句柄表结构的指针。这个句柄表结构用来跟踪所有进程标识符①。

① 可以参考《基于 PspCidTable 的进程检测技术》一文——译注。

Shadow Walker是一个概念型的内核模式rootkit，于2005年BlackHat大会上提出讨论。Shadow Walker基于FU，包括一个额外的操纵内存管理器的驱动程序用于隐藏rootkit文件。简而言之，Shadow Walker将隐藏的内存页全部放在不分页内存池中，并且截获系统对这些内存页的访问。如果操作系统要求读这些页中的内容时，返回内容全为0，当操作系统请求执行这些页中的代码时，就返回恶意代码。看过《星球大战》吗？Obi-Wan Kenobi对风暴军团司令官说："这儿没你要找的东西"。这就是这个rootkit的绝妙注解。

内核模式rootkit如果写得不好，则会导致系统"蓝屏"，这是它的一个缺点。Microsoft的技术支持人员曾经帮助许多客户跟踪不断出现的"蓝屏死机"（BSoD）问题，最后发现问题出在内核模式的rootkit。在第3章提到过，系统崩溃或者"蓝屏死机"会在硬盘上留下一个崩溃转储文件，技术支持人员就是利用这个文件来诊断问题所在。通常这也是发现系统中rootkit（已知或未知）的一种方法。

有时rootkit这个术语会被随意使用。例如，在Symantec安全响应网站日志上有过一篇题为《菜鸟的rootkit》的文章。作者描述了一个名为Trojan.Zonebac的恶意软件，它使用了一种简单的技术在系统中实现隐藏。简单地说，在木马安装的时候，它会扫描注册表中自动运行键（Run）中的一个常用程序，把原来的可执行文件备份到其他地方，然后将自身写到原始文件名所在的位置。当系统启动时，木马程序就会自动运行，同时也会运行备份的程序，系统表面上不会出现异常。同时，注册表键的最后写入时间也会保持不变，因为根本就不需要更改注册表键。

虽然这不是一个新颖和巧妙的隐藏方法，也不是一个rootkit。但是，通过将恶意软件命名为普通名称的方式还是很有效的。这种方法很常见，这类技术更多时候被归类为"欺骗管理员"或者"欺骗检查人员"，而不是"精通系统内核"。

警告

仅仅根据文件名诊断问题所在很容易被误导，甚至导致调查员错误地判断事件问题所在。管理员在发现可疑文件时经常会使用Google搜索文件名，而Google返回的结果会说该文件是一个合法的Microsoft系统文件，事情也就此了结。这种错误不仅系统管理员会犯，有些恶意软件分析人员也常犯同样的错误。实际上，很多恶意软件都会把自己命名成诸如alg.exe，svchost.exe等合法的Windows系统文件。在我调查的一些案件中，很多先期调查人员轻易地放过了这些"合法"文件！甚至没人注意到这些文件根本就不是位于system32目录中。所以在调查案件时应该切记，不能只靠文件名来判别文件的作用！

rootkit也会被用于商业用途。这不仅是指一些rookit作者会开发定制的rootkit提供给付费用户，而且商业公司也会利用rootkit保护自己发布的产品。2005年10月31日，Mark Russinovich（SysInternals作者之一，现在Microsoft工作）在他的个人Blog上宣布：他发现索尼公司使用rootkit以保护其电子出版物不受盗版侵权之苦[10]。Mark指出索尼公司不仅是在消费者毫不知情的情况下（既没有明确告知消费者，也没有在软件的服务条款中提起）向消费者的电脑中安装了一个rootkit，而且知情的攻击者还

可以利用这个 rootkit 隐藏自己的工具。由于公布了这一发现，以及随后发生的一些事，Mark 加入了 Microsoft 公司。他的站点日志内容可以在病毒公告站点[11]找到。

背景注释

信息共享

在披露了索尼 rootkit 的问题之后，Mark 发表了如下声明：

在几年前我们公开了 Regmon 的源代码，这使得在 www. rootkit. com 发布的 NTRootkit 中使用了与我们相同的挂钩函数和支持程序。而 Aries 病毒代码的结构又显示，这个病毒很可能源于 NTRootkit。

这又一次说明不同的技术来源会促进应用程序的开发，包括恶意软件的开发。本例中，挂钩函数的使用经过一轮循环后回到了起点。

Mark 和其他研究者还发现在另外一些公司的软件产品中也使用了 rootkit，2006 年 1 月 10 日，Symantec 公司宣称[12]Norton 回收站保护程序中也使用了类似 rootkit 的功能。

Rootkit 检测

现在已经初步了解 rootkit 的定义和用途了，那么如何检测系统中是否有 rootkit 存在呢？为解答这个问题，我们考察两种检测模式——开机检测和后期检测。开机检测是指对一个运行中的系统进行检测，分析其中是否存在 rootkit，而后期检测分析的是系统的镜像。

开机检测

rootkit 的开机检测是一件极具技术含量的工作，如何能指望一个根本就没有 rootkit 相关知识的调查员在系统中找出 rootkit 呢？这种情况下，rootkit 通常都会被漏掉，从而导致之后的调查过程全部被误导。

2006 年秋，Jesse Kornblum 在国际数字证据期刊上发表了题为《使用 Windows 内存分析技术揭开 rootkit 谜团》[13]的文章。在这篇文章中，Jesse 提出了两个 rootkit 必定遵循的基本原理，那就是它们需要隐藏，同时也要运行。本质上来讲，rootki 必须在保持与系统交互的情况下最小化自身的踪迹。系统自身，尤其是操作系统，必须可以运行 rootkit，而 rootkit 在谋求隐藏并在系统重启后仍能运行。因而 Jesse 提出，既然操作系统能找到 rootkit，调查人员也应该能够找到。不过我想补充一点，这里的“调查人员”应该是指“有足够经验的调查人员”，我想 Jesse 的原意也是如此。

rootkit 开机检测的主要方法是“行为分析”或者“差异分析”。该方法基本思路是使用不同的方法对同一信息进行查询，然后比较查询结果之间的差异，从而检测出 rootkit 的存在或者 rootkit 所隐藏的东西。例如，早期有一个 VB Script 编写的 rootkit 检测工具名为 rkdetect. vbs[14]，它可以检测出流行的 Hacker Defender rootkit。该工具先

用 sc. exe 远程枚举系统所有服务，同时使用 sc. exe 或者 psexec. exe 本地枚举系统所有服务，然后比较两次输出结果的差异。在我的第一本书中[15]，包括一个名为 rkd. pl 的 Perl 脚本。该脚本会用远程/本地两种方法枚举出系统中的进程、服务及一些注册表键，并且标记两种查询方法结果之间的差异，同时 rkd. pl 还会进行一些 rootkit 特征检查。书中演示了应用该工具检测 AFX Rootkit 2003 的实例。

提 示

2006 年 7 月 16 日，SANS 因特网风暴中心（ISC）事件处理员 Lenny Zeltser[16]发表了一篇题为“rootkit 恶意程序的行为分析”的日志[17]。文章中，Lenny 介绍了几种 rootkit 检测工具及其运行截图，分析对象就是本章前面提到的一些 rootkits。

随着时间的推进，rootkit 也在不断进化，使用的隐藏技术也越加复杂，所以 rootkit 检测技术也要不断更新。“差异分析”还是 rootkit 检测的最好方法，不过查询的粒度要更细才行。例如，一些 rootkit 检测工具会执行诸如 dir /s，dir /s /ah 之类的命令，然后将输出与主文件表（MFT）中的内容相比较。其思路是执行一个高级查询和一个底层查询（尽可能的底层），然后比较两者查询结果的差异。

现在有不少免费和商业 rootkit 检测工具，但是这些工具都没有提供工作原理细节。这也是为了不让 rookit 作者轻易了解检测方法，从而可以避开工具的检测。但这并不能避免 rookit 作者下载检测工具，分析这些工具的功能。

RootkitRevealer

RootkitRevealer[18]（如图 7.2 所示）工具发布于 2005 年春季（2005 年 2 月 23 日，Slashdot. org 站点介绍了[19]这款 rootkit 检测工具，并且引发了广泛的讨论）。该工具自发布以来，已随 rootkit 技术的发展数次更新。

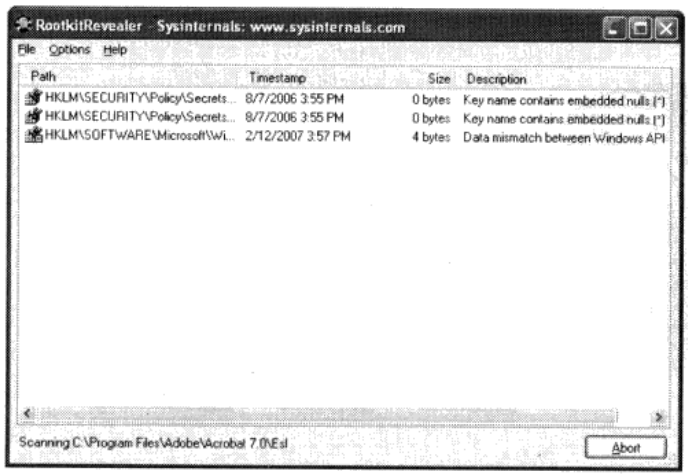


图 7.2 RootkitRevealer 图形界面

RootkitRevealer 的作者特别声明，尽管该工具专门用于检测 rootkit，并且可以检测 rootkit 隐藏的文件和注册表键，但是它不能检测修改内核对象的内核模式 rootkits（如 FU）。

警 告

运行任何工具时，必须要对工具的功能和原理有所了解，rootkit 检测工具也不例外。有一次，我为一个应急响应力量薄弱的客户提供服务。当时，几个管理员都是各行其事，没有协调。有人就在系统中运行防病毒扫描，也包括 RootkitRevealer。扫描时，RootkitRevealer 会安装成服务，并且可执行文件设置成随机名称，而且可执行镜像中加入了一些随机填充值（所以每次文件的哈希值都不同），这也是木马防护程序的常用技巧。过了一段时间，一位系统管理员打电话给我，说是发现了一个“感染严重”的系统，其中有 8 个奇怪的服务，RootkitRevealer 检测不到 rootkit。那么事实如何呢？首先，如果在系统中能“看到”那些服务，这些服务肯定就没有被 rootkit 隐藏起来。其次，那 8 个服务执行文件的图标是一样的。最后，这 8 个服务其实都是 RootkitRevealer！由于缺少协调和对工具的了解，本来很正常的应急响应活动，却被当成是严重感染。

GMER

GMER[20]是一款图形用户界面的免费 rootkit 检查工具。这个工具能检查出如下内容：

- 隐藏的进程、文件、服务、注册表键及驱动程序。
- 挂钩系统服务描述表（SSDT）、中断描述表（IDT）及 IRP 调用的驱动程序。

另外，GMER 还能够显示 NTFS 分支数据流，如图 7.3 所示。

图 7.3 GMER 图形用户界面

在 GMER 的官方网站上还有一个叫做 catchme 的小型命令行工具，该工具可以检测包括 Gromozon，Hacker Defender，AFX 及 Vanquish 在内的用户模式 rootkit。

GMER官方网站上除了能免费下载这些rootkit检测工具之外，还有一些使用GMER检测rootkit的视频教程和扫描日志文件可供下载。

Helios

Helios[21]被描述为“高级恶意软件检测系统”，使用行为分析方式，而不是特征检测方式。虽然Helios被称为恶意软件检测系统，但也能检测rootkit。Helios并不公开源代码，但是可以免费下载，而且还提供调用其核心功能的API函数（详见其官方网站）。Helios不仅能检测rootkit，还能防止系统安装rootkit。其界面如图7.4所示。

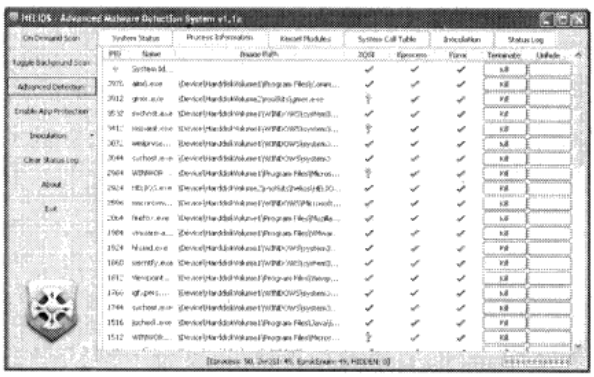


图7.4 Helios图形用户界面

提示

如果要下载和使用Helios，必须在系统中安装.NET 2.0应用框架。Helios下载页面中有相关链接。

Helios官方网站上还有一些演示如何使用这款软件的视频（可以下载或在线观看）。

提示

本章介绍的许多免费rootkit检测工具都是“绿色软件”。在应急响应过程中，只要把这些工具复制到一个U盘中，打开写保护（有些U盘可能不支持写保护功能），将U盘插入需要扫描的系统中即可。不过，要记住工具运行需要具备的条件，如Helios需要.NET 2.0。

MS Strider GhostBuster

Microsoft研究中心[22]提供了很多研究Windows系统中rootkit检测的资源，其结果之一就Strider GhostBuster[23]项目，一款用于检测挂钩或者篡改Windows API函数

的 rootkit 的工具。GhostBuster 使用“交叉视图差异比较”技术（其基本思路与行为分析/差异分析类似）。首先在被“感染”的系统中进行一次查询，随后用“干净的”介质（启动光盘）引导系统再做一次同样的查询，然后比较两次查询结果的差异，找出被隐藏的东西。这一技术在检测隐藏文件时特别有效，但是由于很难在启动光盘中完全复制对象操作系统（包括应用程序和补丁），所以不能检测隐藏的进程。因为要做到这点，在“干净的”系统启动之后，管理员要完全根据原来的系统对“干净的”系统进行配置，任何一点细微的差别都有可能影响检测的结果。对于大多数情况而言，这个过程实在是太繁琐了。

虽然 GhostBuster 官方网站上有不少关于 rootkit 检测技术方面的信息和文章链接，但是到本书写作时还没有提供工具下载。不过该站点还是有不少非常有用、值得一读的文章的。例如，一篇发表于 Usenix LISA 2004 的题为“Gatekeeper：Monitoring Auto-Start Extensibility Points（ASEPs）for Spyware Management”的文章中就提供了有关注册表中自动启动位置的非常好的资料。

ProDiscover

来自 Pathways 技术公司[24]的 ProDiscover/IR 产品中包含帮助调查人员检测系统中 rootkit 的功能。在安装 ProDiscover 服务应用程序（PDServer. exe）之后（可以通过 U 盘、光盘或网络远程安装），调查人员可以连接到这个服务应用程序，执行多种操作。如图 7.5 所示。

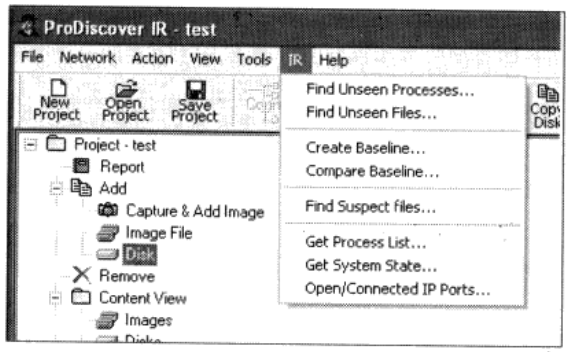


图 7.5 ProDiscover Rootkit 检测功能

如图 7.5 所示，调查人员可以通过 ProDiscover/IR 的菜单查找隐藏进程和文件，也可以获取系统中活动进程列表和系统状态。ProScript API 提供了对信息的更细粒度的管理和灵活性。隐藏进程和文件的显示可以协助调查人员定位系统中的 rootkit。

F-Secure BlackLight

F-Secure 是一家位于芬兰的防病毒公司（根据该公司的网志[25]信息，最近他们的防病毒产品可以通过 VirusTotal. com 提供在线扫描），也提供了一款 rootkit 移除产品——BlackLight[26]。BlackLight 能够检测使用 rootkit 技术隐藏的对象，并且让用户选择是否把有关程序删除。

BlackLight 有试用版本可以下载，同时提供了图形界面和命令行界面的版本，其图形界面如图 7.6 所示。

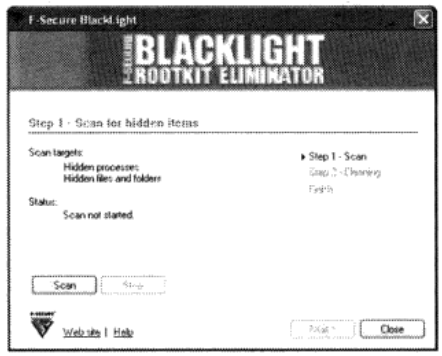


图 7.6　BlackLight 图形界面

与其他 rootkit 检测工具一样，BlackLight 也是“绿色软件”，不需安装。下载选定的版本之后，立刻就可以运行。

Sophos Anti-Rootkit

Sophos[27]也是一家反病毒公司，同时也提供一款 rootkit 的检测工具——Anti-Rootkit，而且可以免费下载和使用。与 F-Secure 的 BlackLight 一样，Anti-Rootkit 同时提供了命令行版本和图形界面版本（如图 7.7 所示）。Sophos 公司的产品不仅可以扫描单机，还可以扫描网络并删除 rootkit。Anti-Rootkit 可以扫描系统中隐藏的进程、注册表键和本地硬盘中隐藏的文件。

图 7.7　Sophos Anti-Rootkit 图形用户界面

提 示

还记得前面提到的 Jamie Butler 和 Sherri Sparks 合写的那篇“Windows Rootkits of 2005”吗？文章讨论了 5 种 rootkit 检测技巧和 9 种 rootkit，很值得一读。

AntiRootkit. com

反病毒公司中并不是只有 F-Secure 和 Sophos 提供 rootkit 检测和删除产品，其他厂商的反病毒产品中也用组件或独立的方式提供了 rootkit 检测功能。McAfee 的 rootkit-检测产品和 Trend Micro 的 RootkitBuster 产品都可以搜索目标系统中隐藏的文件、进程和注册表项。

AntiRootkit. com 可能是提供 rootkit 检测技术和产品信息最好的网站。该网站提供了一个网志，同时列出了可用的免费或商业的 rootkit 检测（主要是 Windows 下的，也包括 Linux，BSD 和 Mac OS X 下的产品）和 rootkit 预防工具（用来预防系统被安装 rootkit）。另外，该站点还提供了很多与 rootkit 相关的新闻和文章链接。

后期检测

rootkit 的后期检测的侧重点会有所不同，那么到底后期分析的难点在哪里呢？毕竟，后期分析面对的是系统镜像，而不是处于运行中的系统……此时该从哪里入手呢？由于恶意软件的作者使用了很多花招，包括广为流传的反取证技术（MetaSploit 项目网站上有一个完整的板块专门用于介绍反取证技术），要在系统镜像中查找恶意软件确实很难。不过，如果知道应该到哪里找及怎样找，成功的概率会大为增加。

后期检测的一种方法是使用类似 Mount Image Pro（MIP）[28] 的工具将镜像文件加载成虚拟文件系统，这样可以在不与镜像操作系统交互的情况下访问其中的文件。MIP 使用只读方式加载镜像文件，所以操作过程中不会对其中文件造成改动。加载之后，可以运行各种反病毒工具对镜像中的文件进行扫描。由于被分析的系统并未运行，镜像中的进程和服务就不会启动，rootkit 也不可能运行，所以用于分析的系统也不会受到影响。

警 告

使用 MIP 加载镜像，然后用反病毒软件扫描其中的文件和目录是个不错的选择，但这种方式不适用于 rootkit 检测工具。这是因为这些工具需要查找隐藏的文件、注册表键和进程等对象——但是现在，分析的镜像被加载后，没有文件是隐藏的。

我在检测获取的镜像文件中的 rootkit 时，常使用 ProDiscover 的注册表查看器检查每个 ControlSet 的 Services 键，如图 7.8 所示。

提 示

要确定“当前”或系统启动时加载为 CurrentControlSet 的 ControlSet，可以检查 System\Select 键下的 Current 值。该值的数据是 DWORD 类型，定义了“当前”ControlSet。

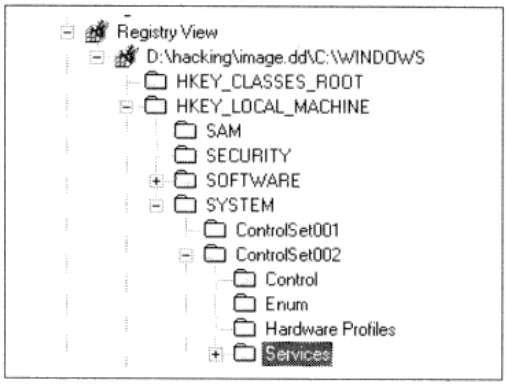


图 7.8　ProDiscover 注册表查看器部分输出

找到相应的键之后，我会按照最后修改（LastWrite）时间对列表中的键进行排序。列表中大多数项目会与系统安装时间吻合。有时会有几个键的最后修改时间一样，这一般是某个软件升级造成。内核模式 rootkit 安装时，很可能就是孤零零的一两个子键，其最后修改时间是在同一天，最后修改时间不一定就是事件报告提供的时间，但大多数情况下很容易发现。另外，使用这个方法得到的时间也有助于对时间线分析中的时间进行校准。由于 Microsoft 公司并没有公开发布用户模式下修改注册表键最后修改时间的 API，可以确定该时间就是 rootkit 及其驱动安装的时间。

另一种介于开机和后期检测之间的 rootkit 检测方式就是第 3 章介绍的内存镜像分析。调查员可以在开机时获取系统物理内存镜像，然后马上对镜像进行分析，此时系统可能还在继续运行，但是分析的是内存快照。就像 Jesse 在那篇“谜团”文章所讲的那样，“聪明的”rootkit 不会干扰生成内存镜像的过程，因为这会暴露 rootkit 的存在。毕竟，导致系统崩溃（蓝屏死机）对谁也没有好处，包括攻击者自己。镜像系统物理内存的一个主要工具是 dd. exe，由 George M. Garner 定制修改，大多数情况下该工具都可稳定运行。如果 rootkit 导致系统崩溃，崩溃转储可以用来分析 rootkit 的存在。获取内存镜像之后，搜索所有的 EPROCESS 块（详见第 3 章），将镜像中的活动进程信息与开机系统的进程列表做对比，就可以找到 rootkit 隐藏的进程。

另外还可以使用 LiveView[30] 在 VMWare[29] 中启动镜像，然后用开机检测的方法查找可能的 rootkit。在第 6 章中，曾经提到过一些恶意软件能够检测虚拟环境的存在，如果发现自己运行在 VMWare 之类的虚拟机中，它会改变自身行为模式以逃避检测。毋庸置疑，一些 rootkit 也会这招，改变的行为可能会导致系统出现问题。此时可以用 nmap[31] 对扫描系统开放的端口，然后将结果与 netstat. exe 或 openports. exe 的输出进行比较。如果 namp 扫描的开放端口没有出现在 netstat. exe 的输出中，基本可以确定系统中存在 rootkit。

预防

前面介绍了 rootkit 及其检测技术，但没有涉及 rookit 预防的问题。实际上 rootkit 检测的第一步就是要通过系统配置预防 rootkit 对系统的感染，不过这一主题的详细论述不属本书的范畴。简而言之就是使用最小权限的原则（即只给用户完成所需任务所需

要的最小的权限，只在完成特定任务的时候授予管理员权限）配置计算机，这样能比较好地防止计算机被安装 rootkit。只要能防止 rootkit 的安装，rootkit 就不能正常运行，我们可以轻松地就找到它。事实上，rootkit 安装的时候也是最容易发现的时候，因为此时系统会有很多报错或者出现不寻常的行为。

警 告

有些人就是不喜欢最小权限原则。使用管理员权限运行的最大问题是可以安装任何软件。在一次案件调查中，我发现系统运行了 4 个远程桌面服务，而且我确定黑客正在通过其中一个服务访问系统。一开始我以为是黑客安装了远程控制软件，但是系统管理员后来告诉我这 4 个应用都是合法的，是由 IT 部门安装的，其中的每个远程管理应用都是其他应用的备份。没有哪个系统管理员会有时间和精力同时管理这么多应用，而黑客入侵利用的恰恰就是这点。

小结

虽然 rootkit 的存在由来已久，但是直到 2005 年 2 月的 RSA 峰会它才引起人们的广泛关注。到现在，包含 rootkit 开发技巧的书（Hoglund 编写的“Rootkits：Subverting the Windows Kernel”和一本名为“Rootkits for Dummies”的书都可以在 Amazon. com 买到）、课程（Hoglund 在 BlackHat 上的演讲和课程）及能实际运行的 rootkit（虽然有些还只是概念模型）都在广泛流传。

rootkit 对系统和网络构成了严重威胁，对于对 rootkit 知之甚少的系统管理员和调查人员来说更是如此。如果能够深入了解 rootkiet 的概念、功能和工作原理，调查人员就可以在开机和后期分析过程中有效解决 rootkit 问题。

参考资料

1. www. sophos. com
2. http://en. wikipedia. org/wiki/Rootkit
3. www. securityfocus. com/infocus/1850
4. www. blackhat. com
5. www. securiteam. com/securityreviews/5FP0E0AGAC. html
6. http://pcalsicuro. phpsoft. it/gromozon. pdf
7. www. symantec. com/enterprise/security_response/weblog/2006/08/gromo-zoncom_and_italian_spaghe. html
8. www. uninformed. org/?v=3&a=7
9. www. uniformed. org
10. www. symantec. com/avcenter/security/Content/2006. 01. 10. htm

11. www. virusbtn. com/virusbulletin/archive/2005/12/vb200512-sonys-rootkit
12. www. symantec. com/avcenter/security/Content/2006. 01. 10. html
13. www. utica. edu/academic/institutes/ecii/publica-tions/articles/EFE2FC4D-0B11-B-C08-AD2958256F5E68F1. pdf
14. www. security. nnov. ru/files/rkdetect. zip
15. *Windows Forensics and Incident Recovery*, Addison-Wesley, July 2004
16. www. zeltser. com/
17. http://isc. sans. org/diary. html? storyid=1487
18. www. microsoft. com/technet/sysinternals/utilities/RootkitRevealer. mspx
19. http://it. slashdot. org/it/05/02/23/1353258. shtml? tid=172&tid=218
20. www. gmer. net/index. php
21. http://helios. miel-labs. com/
22. http://research. microsoft. com/
23. http://research. microsoft. com/rootkit/
24. www. techpathways. com/
25. www. f-secure. com/weblog/archives/archive022007. html#00001106
26. www. f-secure. com/blacklight/
27. www. sophos. com/products/free-tools/sophos-anti-rootkit. html
28. www. mountimage. com
29. www. vmware. com
30. http://liveview. sourceforge. net/
31. http://insecure. org/nmap/index. html

快速解决方案

Rootkits

- √ rootkit 能够隐藏文件、注册表键、进程、网络连接及其他对象，管理员甚至是操作系统本身都很难发现。
- √ 恶意软件和网络犯罪对 rootkit 及其相关技术的使用在日益增长。
- √ 只有很好地了解 rootkit 的功能和工作原理，计算机犯罪调查人员才能在实际工作中解决 rootkit 问题。

Rootkit 检测

- √ 对开机系统中的 rootkit 检测经常使用差异分析的方法。
- √ 对获取镜像的 rootkit 检测，可以使用反病毒工具扫描 Mount Image Pro 加载的镜像，或者基于注册表 Services 键“最后修改时间”的排序快速分析。
- √ 获取和分析系统物理内存，将获取内存中进程列表与开机系统中的进程列表进行比较，也可以有效检测 rootkit 的存在。

常见问题

下面的这些问答用来测试读者是否已经完全理解本章的内容，并可以帮助读者在实际案件调查过程中应用本章知识。如果还有其他问题，浏览 http://www.syngress.com/solutions 并单击 Ask the Author 一栏。

问：我发现防火墙记录了一些异常流量，时间从 4h 以前开始。日志显示网络中的一个系统使用了一个奇怪的端口访问互联网，但是系统上没有找到相应的活动连接。这是 rootkit 在做怪吗？

答：也许不是。源自系统的活动，尤其是网络数据流，实际上都是某个进程或者线程运行的结果。服务会随系统一直运行，但是进程是有生命周期的。如果不能在防火墙日志中发现持续的网络活动，这可能是进程已经退出的缘故，这也是在系统中找不到活动连接的原因。

问：怎样才能防止 rootkit 感染系统？

答：良好的配置管理可以有效地阻止 rootkit 的感染。如果采用最小化权限管理方法，如提供最小化的服务和访问权限，可以有效减少攻击点。例如，如果用户不能安装任何软件，也就不会安装间谍软件、rootkit 等。减少系统中提供的服务数量也就减少了黑客攻击时所能采用的方法，同时也减少了系统感染 rootkit 的可能性。

问：我的服务器上被安装了一个 rootkit，怎么办？有人告诉我系统已经失去掌控，惟一的办法就是擦除硬盘，重装一个干净的系统，然后恢复未感染的数据。

答：大多数系统管理员在遇到 rootkit 时都会选择这么做，不过这种做法有些问题。首先，应该对系统进行彻底地检查（也可请专家来做），搞清楚系统到底发生了什么问题（例如，数据被盗窃）。其次，还应该搞清楚 rootkit 最初是怎么进入的。如果不解决这个问题，返回网络中的系统还会再度被入侵、感染。最后，如果公司有正规的信息安全监督体系（PCI，HIPAA，FISMA 等），则要提供问题调查报告（明确规定或间接规定），尽可能多地提供与事件有关的信息。